忍

SHINOBI AI (NARUTO)

دليل تعليمي كامل بالعامية المصرية: كل ملف، كل جزء، وإزاي السيستم كله بيشتغل

التعرف على حركات إيد ناروتو لحظياً وتشغيل الـ jutsu بالتأثيرات

FastAPI + WebSocket · React + TypeScript · MediaPipe · YOLOX · Landmark classifier

2026-11-01

الفهرس

الجزء الأول: محرك التعرف على الحركات (shinobi)

مقدمة: الجزء ده بيشرح إيه؟

نظرة عامة على المعمارية (Architecture)

الملف `shinobi/__init__.py`

الملف `shinobi/config.py`

الملف `shinobi/camera.py`

الملف `shinobi/hands.py`

الملف `shinobi/features.py`

الملف `shinobi/classifier.py`

الملف `shinobi/seal_detector.py`

الملف `shinobi/fusion.py`

الملف `shinobi/controller.py`

الملف `shinobi/engine.py`

الملف `shinobi/events.py`

الملف `shinobi/filters.py`

الملف `shinobi/tracking.py`

الملف `shinobi/segmentation.py`

الملف `shinobi/hud.py`

الملف `shinobi/main.py`

الملف `shinobi/catalog.py`

الخلاصة: إزاي العلامة بتبقى جوتسو (مثال Fire Style كامل)

ملخص القواعد المهمة (للمراجعة السريعة)

ملاحظات: الفروق بين النسخة الإنجليزية والكود الحالي

قاموس صغير للمصطلحات

الجزء التاني: السيرفر والتأثيرات والصوت

مقدمة: إحنا هنتكلم عن إيه في الجزء ده؟

مسار البيانات الكامل بين الواجهة والسيرفر

الملف `shinobi/server.py`

إزاي التفعيل النهائي للـ jutsu بيحصل (من آخر حركة لحد الـ effect)

حزمة التأثيرات (effects package): نظرة عامة

الملف `shinobi/effects/__init__.py`

الملف `shinobi/effects/base.py`

الملف `shinobi/effects/fx.py`

الملف `shinobi/effects/fire.py`

الملف `shinobi/effects/chidori.py`

الملف `shinobi/effects/rasengan.py`

الملف `shinobi/effects/shadow_clone.py`

الصوت: الفولدر `assets/audio/` وملفات `frontend/src/audio/`

الفولدر `gesture_references/`
فروقات بين النسخة الإنجليزية والكود الحالي
خلاصة: لو حد سألك تشرحي الجزء ده في دقيقتين

الجزء التالت: الواجهة (React Frontend)

قبل ما نبدأ: أساسيات React و TypeScript اللي هتحتاجيها
نظرة عامة: الصفحات وطريقة مرور البيانات
ملفات المشروع في جذر `frontend/`
الملف `frontend/package.json`
الملف `frontend/vite.config.ts`
الملف `frontend/index.html`
ملفات تانية في الجذر
نقطة البداية والـ Routing
الملف `frontend/src/main.tsx`
الملف `frontend/src/App.tsx`
الأنواع (Types): العقد بين الفرونت إند والباك إند
الملف `frontend/src/types/protocol.ts`
الملف `frontend/src/types/catalog.ts`
الخدمات (Services): الكلام مع السيرفر
الملف `frontend/src/services/api.ts`
الملف `frontend/src/services/socket/framing.ts` (ومعاه `framing.test.ts`)
الملف `frontend/src/services/socket/client.ts` (ومعاه `client.test.ts`)
الملف `frontend/src/services/socket/index.ts`
الـ Hooks: الكاميرا والجلسة
الملف `frontend/src/hooks/useCamera.ts`
الملف `frontend/src/hooks/useShinobiSession.ts`
الـ Contexts (الحالة المشتركة على مستوى التطبيق كله)
الملف `frontend/src/contexts/ThemeContext.tsx`
الملف `frontend/src/contexts/SettingsContext.tsx`
الملف `frontend/src/contexts/SoundContext.tsx`
الملف `frontend/src/contexts/CatalogContext.tsx`
الملف `frontend/src/lib/storage.ts`
واجهة Konoha: الإطار والقطع المشتركة
الملف `frontend/src/konoha/konoha.css`
الملف `frontend/src/konoha/KonohaShell.tsx` (ومعاه `KonohaShell.module.css`)
الملف `frontend/src/konoha/shell/KonohaBackdrop.tsx` (ومعاه `KonohaBackdrop.module.css`)
الملف `frontend/src/konoha/shell/FallingLeaves.tsx`
الملفات `frontend/src/konoha/shell/KonohaHeader.tsx` (ومعاه `KonohaHeader.module.css`) و `frontend/src/konoha/shell/BrushSwash.tsx` و `frontend/src/konoha/shell/nav.ts`
الملف `frontend/src/konoha/shell/MusicControl.tsx` (ومعاه `MusicControl.module.css`)
الملف `frontend/src/konoha/shell/LegacyFrame.tsx` (ومعاه `LegacyFrame.module.css`)
الملف `frontend/src/konoha/LeafEmblem.tsx`
الملف `frontend/src/konoha/BrushButton.tsx` (ومعاه `BrushButton.module.css`)

الملف `frontend/src/konoha/BrushTag.tsx` (ومعاه `BrushTag.module.css`)

الملف `frontend/src/konoha/ParchmentCard.tsx` (ومعاه `ParchmentCard.module.css`)

الملف `frontend/src/konoha/jutsuLook.ts`

الدوجو (الصفحة الرئيسية Home وصفحة Live)

الملف `frontend/src/konoha/dojo/useDojo.ts`

الملف `frontend/src/konoha/dojo/DojoView.tsx` (ومعاه `DojoView.module.css`)

الملف `frontend/src/konoha/dojo/CameraCard.tsx` (ومعاه `CameraCard.module.css`)

الملف `frontend/src/konoha/dojo/JutsuListCard.tsx` (ومعاه `JutsuListCard.module.css`)

الملف `frontend/src/konoha/dojo/sequenceSteps.ts` (ومعاه `sequenceSteps.test.ts`)

الملف `frontend/src/konoha/dojo/SequenceCard.tsx` (ومعاه `SequenceCard.module.css`)

صفحة Live (الوضع الحي)

الملف `frontend/src/konoha/live/LiveView.tsx` (ومعاه `LiveView.module.css`)

الملفات `frontend/src/pages/LiveDemoPage.tsx` و `frontend/src/pages/HomePage.tsx`

صفحة الجوتسو (المكتبة Library)

الملفات `frontend/src/konoha/pages/JutsuPage.tsx` و `frontend/src/konoha/pages/PracticePage.tsx`

الملف `frontend/src/konoha/jutsu/JutsuView.tsx` (ومعاه `JutsuView.module.css`)

الملف `frontend/src/konoha/jutsu/pageState.ts`

الملف `frontend/src/konoha/jutsu/sequenceKind.ts`

الملف `frontend/src/konoha/jutsu/JutsuCard.tsx` (ومعاه `JutsuCard.module.css`)

الملف `frontend/src/konoha/jutsu/JutsuDetail.tsx` (ومعاه `JutsuDetail.module.css`)

الملف `frontend/src/konoha/jutsu/SignSteps.tsx` (ومعاه `SignSteps.module.css`)

الـ Carousel

الملف `frontend/src/konoha/carousel/Carousel.tsx` (ومعاه `Carousel.module.css`)

صفحة Practice (التدريب)

الملف `frontend/src/konoha/practice/PracticeView.tsx`

الملف `frontend/src/konoha/practice/PracticeSession.tsx` (ومعاه `PracticeSession.module.css`)

صفحة Settings (الإعدادات)

الملف `frontend/src/konoha/pages/SettingsPage.tsx`

الملفات `frontend/src/konoha/settings/controls.tsx` (ومعاه `controls.module.css`) و `frontend/src/konoha/settings/SettingsPage.module.css`

موسيقى الخلفية

الملف `frontend/src/konoha/music/player.ts`

الملف `frontend/src/konoha/music/MusicContext.tsx`

محرك الصوت (المؤثرات، أصوات الشخصيات، موسيقى الجوتسو)

الملف `frontend/src/audio/cues.ts`

الملف `frontend/src/audio/gains.ts`

الملف `frontend/src/audio/engine.ts`

الملف `frontend/src/audio/eventCues.ts` (ومعاه `eventCues.test.ts`)

الملف `frontend/src/audio/voiceCues.ts` (ومعاه `voiceCues.test.ts`)

الملف `frontend/src/audio/voices.ts`

الملف `frontend/src/audio/useVoiceManifest.ts`

الملف `frontend/src/audio/jutsuMusic.ts` (ومعاه `jutsuMusic.test.ts`)

الملف `frontend/src/audio/jutsuMusicPlayer.ts`

الملف `frontend/src/audio/useSessionSounds.ts`
الملف `frontend/src/audio/voicePlayback.test.ts`
رحلة الإشارة: من الإيد للكارت والصوت (خطوة بخطوة)
كومبوننتس الطبقة الحية (`src/components/live`)
الملف `frontend/src/components/live/useLiveEvents.ts`
الملف `frontend/src/components/live/labels.ts` (ومعاه `labels.test.ts`)
الملف `frontend/src/components/live/StageOverlay.tsx` (ومعاه `StageOverlay.module.css`)
كومبوننتس Live القديمة (موجودة؛ مستخدمة بس في الصفحة غير المتوصلة `pages/PracticePage.tsx`)
كومبوننتس الفن (`src/components/art`)
الملف `frontend/src/components/art/JutsuIcon.tsx` (ومعاه `JutsuIcon.module.css`)
الملف `frontend/src/components/art/ShadowCloneIcon.tsx` (ومعاه `ShadowCloneIcon.module.css`)
الملف `frontend/src/components/art/JutsuMotif.tsx` (ومعاه `JutsuMotif.module.css`)
الملف `frontend/src/components/art/jutsuVisuals.ts`
ملفات فن تانية (مستخدمة بس في AppShell القديم، أو مش مستخدمة خالص)
الـ Layout القديم و UI kit (`src/components/layout` و `src/components/ui`)
الملف `frontend/src/components/layout/useDocumentTitle.ts`
الملفات `frontend/src/components/layout/AppShell.tsx` و `MobileMenu.tsx` و `ServerStatus.tsx` و `ThemeToggle.tsx` و `nav.ts` (ومعاهم ملفات الـ `.module.css` بتاعتهم)
الفولدر `frontend/src/components/ui/` (الملف `index.ts` وكل كومبوننت مع الـ CSS module بتاعه)
الصفحات القديمة (`src/pages`)
الـ Styles والـ Assets
اختبارات الوحدة (Unit tests): إزاي تشغليها
اختبارات End-to-End في المتصفح (`frontend/e2e`)
الملف `frontend/e2e/make_fixtures.py`
الملف `frontend/e2e/lib.mjs`
سكريبتات الاختبار
حالة الملفات القديمة وغير المستخدمة (ملخص واحد)
فروق بين الكود والوصف الإنجليزي القديم
فهرس كل ملفات `frontend/src` (مرجع سريع)
خلاصة الجزء التالت

الجزء الرابع: البيانات والتدريب والتقييم والأدوات والاختبارات

قبل ما نبدأ: أساسيات Machine Learning اللي هتحتاجيها
الصورة الكبيرة في صفحة واحدة
فولدر `data/`: شكل البيانات وتنظيمها
فولدر `models/`: كل ملف فيه
الملف `requirements.txt`
الملف `README.md`
الإعدادات (Configuration) اللي ليها علاقة بالجزء ده
الملف `collect_data.py`
الربط بين البيانات الخام والـ features (Preprocessing و Feature Extraction)
الملف `record_sequences.py`
الملف `splits.py`

الملف `train.py`

الملف `evaluate.py`

أدوات التسجيل والتقسيم في فولدر `tools/`

الملف `tools/recording_review.py`

الملف `tools/record_training_session.py`

الملف `tools/record_test_session.py`

الملف `tools/assign_recordings.py`

الملف `tools/make_test_set.py`

الملف `tools/grouped_cv.py`

الملف `tools/evaluate_pipeline.py`

الملف `tools/check_voices.py`

الملف `tools/build_bold_signs.py`

الملف `tools/build_gesture_references.py`

فولدر `reports/`: كل فولدر فيه إيه

فولدر `tests/`: كل ملف اختبار بيتأكد من إيه

الملف `tests/test_core.py`

الملف `tests/test_splits.py`

الملف `tests/test_evaluate.py`

الملف `tests/test_recording_tools.py`

الملف `tests/test_sequences.py`

الملف `tests/test_fusion.py`

الملف `tests/test_engine.py`

الملف `tests/test_events.py`

الملف `tests/test_activation.py`

الملف `tests/test_shadow_clone.py`

الملف `tests/test_catalog.py`

الملف `tests/test_server.py`

اختبارات المتصفح (Browser tests) باختصار

الحالة الحالية للموديل والتقييم (بصراحة)

إزاي تشغلي كل حاجة

الفروقات اللي لقيناها بين الكلام والكود

ملخص الجزء في سطور

تحديث: نتايج تجربة الموديلات (01-11-2026)، والـ MLP اتركب بعدها

الجزء الأول: محرك التعرف على الحركات (shinobi)

مقدمة: الجزء ده بيشرح إيه؟

أهلاً بيكي في الجزء الأول من دليل SHINOBI AI. الجزء ده هو "قلب" المشروع: الكود اللي بيشوف إيدك في الكاميرا، ويفهم إنتي عاملة أنهي علامة (sign)، ويتابع ترتيب العلامات، ولما تخلّصي جوتسو (jutsu) كامل يقول للمؤثرات البصرية "يلا اشتغلي".

هنشرح كل ملف Python جوه الفولدر `shinobi/` ما عدا حاجتين: `shinobi/server.py` (سيرفر الويب) والفولدر `shinobi/effects/` (المؤثرات البصرية)، ودول ليهم أجزاء تانية في الدليل. كل اللي مكتوب هنا متراجع على الكود الحالي نفسه، فلو في أي وقت لقيتي رقم في الشرح مختلف عن رقم في الكود، **الكود هو الصح**. وفي الأماكن اللي النسخة الإنجليزية من الدليل كانت قديمة أو مش دقيقة، هتلاقي ملاحظة صريحة بتقول كده.

قبل ما نبدأ، شوية مصطلحات هتتكرر كتير:

- **Python**: لغة البرمجة اللي الجزء ده كله مكتوب بيها.
- **frame**: يعني صورة واحدة من الفيديو. الكاميرا بتبعت حوالي 30 frame في الثانية، والبرنامج بيعالج كل frame لوحده.
- **model**: يعني برنامج ذكاء اصطناعي اتدرّب على أمثلة كتير، وبياخد مدخل (صورة أو أرقام) ويطلّع إجابة (مثلاً "دي علامة tiger").
- **landmark**: يعني نقطة مهمة على الإيد، زي المفصل أو طرف الصباع. MediaPipe بيطلع 21 landmark لكل إيد.
- **threshold**: يعني حد أدنى أو "عتبة". مثلاً لو الـ threshold بتاع الثقة 0.70، يبقى أي إجابة ثقتها أقل من 0.70 بنتجاهلها.
- **probability**: يعني احتمال من 0 لـ 1. الـ model بيقول مثلاً "tiger باحتمال 0.9، و snake باحتمال 0.05".
- **thread**: يعني خيط تنفيذ. البرنامج ممكن يشغّل كذا حاجة في نفس الوقت، كل حاجة في thread لوحدها، عشان حاجة بطيئة متعطّلش الباقي.
- **config**: يعني الإعدادات. كل الأرقام اللي ممكن نغيرها موجودة في ملف واحد اسمه `config.py`.

الملفات اللي الجزء ده بيغطيها (17 ملف):

- `shinobi/__init__.py`
- `shinobi/config.py`
- `shinobi/camera.py`
- `shinobi/hands.py`
- `shinobi/features.py`
- `shinobi/classifier.py`
- `shinobi/seal_detector.py`
- `shinobi/fusion.py`

- `shinobi/controller.py`
- `shinobi/engine.py`
- `shinobi/events.py`
- `shinobi/filters.py`
- `shinobi/tracking.py`
- `shinobi/segmentation.py`
- `shinobi/hud.py`
- `shinobi/main.py`
- `shinobi/catalog.py`

نظرة عامة على المعمارية (Architecture)

Architecture: يعني "التصميم العام" للسيستم، يعني إيه الأجزاء الكبيرة وإزاي بتكلم بعض. قبل ما ندخل في تفاصيل كل ملف، لازم تبقي شايفة الصورة الكبيرة.

البرنامج بيعمل إيه في فقرة واحدة

SHINOBI AI بيتفرج عليكي من خلال الـ webcam. لما تعملي الأربع علامات (hand seals) بتوعة جوتسو معين بالترتيب الصح (مثلاً Monkey بعدين Boar بعدين Horse بعدين Tiger عشان الـ Fire Style)، البرنامج بيشغّل المؤثر البصري بتاع الجوتسو ده فوق الفيديو الحي: كرة نار طالعة من إيدك، أو برق (Chidori)، أو كرة بتلف على كف إيدك (Rasengan)، أو عشر نسخ منك (Shadow Clone).

البرنامج يقدر يشتغل بطريقتين:

- **كتطبيق Desktop**: بالأمر `python -m shinobi.main`، وبيفتح شباك OpenCV. (OpenCV: مكتبة مشهورة جداً للتعامل مع الصور والفيديو والكاميرا.)
- **كتطبيق ويب (Web app)**: بالأمر `python -m shinobi.server`، وده بيشغّل سيرفر FastAPI/WebSocket، وصفحة React في المتصفح بتكلمه. (FastAPI: مكتبة Python لعمل سيرفرات ويب بسرعة. WebSocket: طريقة اتصال بتفضل مفتوحة بين المتصفح والسيرفر عشان يبعتوا لبعض بيانات بسرعة وباستمرار، زي frames الفيديو. React: مكتبة JavaScript/TypeScript لبناء واجهات المواقع.)

الطريقتين بيستخدموا **نفس محرك التعرف بالظبط**، وهو الكلاس `Engine` في الملف `shinobi/engine.py`. ده معناه إن أي تحسين في التعرف بيفيد الاتنين مع بعض.

الاتنين "المتعرّفين" (The two recognisers)

المشروع بيستخدم **اتنين** models مختلفين للتعرف على علامات الإيد، لأن ولا واحد فيهم لوحده يقدر يتعرف على العشر علامات كلها:

- **YOLOX-Nano seal detector** (الملف `seal_detector.py`). ده model جاهز ومتدرّب قبل كده (pretrained) من نوع object detector. Object detector: يعني model بيبص على الصورة كلها ويرسم مربع (box) حوالين الحاجة اللي بيدور عليها، هنا علامة الإيد. الـ model ده يعرف الـ 12 علامة الرسمية بتاعة Naruto وكام

class زيادة، والمشروع بيستخدم 7 منهم بس: tiger, snake, horse, ram, monkey, dog, boar. هو **ميعرفش** العلامتين المخصوصين بتوع المشروع.

- **Landmark classifier** (الملفات `hands.py` + `features.py` + `classifier.py`). هنا MediaPipe بيلاقي الـ 21 مفصل (landmark) في كل إيد، وبعدين `features.py` بيحوّل النقط دي لـ 206 رقم، وبعدين model دربته صاحبة المشروع بنفسها (Random Forest أو شبكة عصبية صغيرة MLP، ومتخزن في `models/gesture_model.joblib`) بيحوّل الـ 206 رقم لـ probability لكل واحد من الـ 10 labels. الـ model ده هو المسؤول عن العلامتين المخصوصين: `clone_seal` و `rasengan_palm`.

مصطلحات: **MediaPipe** مكتبة من Google بتلاقي الإيد والجسم في الصورة. **Classifier** يعني model بيصنّف المدخل لواحد من كذا "فئة" (label). **Random Forest** نوع classifier عبارة عن مجموعة كبيرة من "أشجار القرار" (decision trees) وكل شجرة بتصوّت، والنتيجة متوسط الأصوات. **MLP** (Multi-Layer Perceptron) شبكة عصبية بسيطة من كام طبقة. **label** يعني اسم الفئة، زي `"tiger"` أو `"idle"`.

الملف `fusion.py` بيجمع رأي الاتنين في **قرار واحد** لكل frame، عشان العلامة متتحسبش مرتين أبداً. (Fusion: يعني "دمج".)

الـ Pipeline الكامل: من frame الكاميرا لحد الجوتسو

Pipeline: يعني سلسلة مراحل، كل مرحلة بتاخد ناتج اللي قبلها وتشتغل عليه، زي خط إنتاج في مصنع.

ده الرسم النصي للسيستم كله:

```
webcam / browser frame (BGR image)
       |
       |---- copy ---> SealWorker thread (YOLOX, max 10x per second) ----+
       |                                                                 |
       v                                                                 | latest SealResult
HandTracker.detect  (MediaPipe, 21 landmarks per hand, up to 2 hands)    |
       |                                                                 |
       v                                                                 |
features.extract  (206 numbers)                                          |
       |                                                                 |
       v                                                                 |
GestureClassifier.predict  (probability per label, ~0.3 ms fast forest)  |
       |                                                                 |
       +------------------------> SignFusion.update <--------------------+
                                  |  one-hot: "tiger" (or "idle")
                                  v
                     ActionController.update
                       |- GestureStabilizer  (EMA smoothing, CONF_ON/OFF, 0.3 s hold)
                       |      -> "confirmed: tiger" (once)
                       |- SequenceMatcher    (steps per jutsu, 7 s gap, wrong attempts)
                       |      -> "fire_style completed"
                       |- cooldown (1.5 s)
                                  |  effect name, e.g. "fire"
                                  v
AnchorTracker (One Euro filtered hand position) --> EffectManager.render
PersonSegmenter (only for Shadow Clone, worker thread) ----^
                                  |
                                  v
                  output image (+ HUD on desktop, + events on the web)
```

نقرا الرسم ده بالعربي، خطوة بخطوة:

1. بييجي frame من الـ webcam (في الـ Desktop) أو من المتصفح (في الويب). الصورة بتكون بصيغة BGR (يعني ترتيب الألوان أزرق-أخضر-أحمر، وده الترتيب اللي OpenCV بيستخدمه).
2. نسخة صغيرة من الـ frame بتروح لـ thread منفصل اسمه `SealWorker` بيشغّل YOLOX بحد أقصى 10 مرات في الثانية. الـ thread ده بيحط آخر نتيجة عنده في `SealResult`.
3. في نفس الوقت، الـ frame بيروح لـ `HandTracker.detect` اللي بيستخدم MediaPipe ويطلّع 21 landmark لكل إيد (لحد إيدين).
4. النقط دي بتروح لـ `features.extract` اللي بيحوّلها لـ 206 رقم.
5. الـ 206 رقم بيروحوا لـ `GestureClassifier.predict` اللي بيطلّع probability لكل label، وده بياخد حوالي 0.3 ملي ثانية بفضل الـ "fast forest".
6. `SignFusion.update` بياخد رأي الـ classifier وآخر نتيجة من YOLOX، ويطلّع قرار واحد "one-hot": يعني علامة واحدة بس (مثلاً `"tiger"`) أو `"idle"` (مفيش علامة).
7. `ActionController.update` بياخد القرار ده ويعدّيه على تلات مراحل: `GestureStabilizer` (بيهدّي القرار مع الوقت وبيأكد العلامة بعد ما تتمسك 0.3 ثانية)، وبعدين `SequenceMatcher` (بيتابع خطوات كل جوتسو، وحد 7 ثواني بين الخطوات، والمحاولات الغلط)، وبعدين الـ cooldown (فترة راحة 1.5 ثانية بعد أي جوتسو).
8. لو جوتسو اكتمل، بيطلع اسم المؤثر، مثلاً `"fire"`.
9. `AnchorTracker` بيحدد مكان الإيد اللي المؤثر هيتبعها ويهدّي حركتها بفلتر اسمه One Euro.
10. `PersonSegmenter` (بس مع Shadow Clone، وفي worker thread) بيفصل الشخص عن الخلفية.
11. `EffectManager.render` بيرسم المؤثر على الصورة.
12. الناتج: صورة نهائية، ومعاها HUD في الـ Desktop، أو events بتتبعت للمتصفح في الويب.

الترتيب جوه `Engine.process` لكل frame

دي نفس الخطوات بس من وجهة نظر الدالة `Engine.process` اللي بتتنادى مرة لكل frame:

1. أحدث frame بيتسلّم لـ thread بتاع YOLOX (وعمره ما بيخلي الـ loop الأساسي يستنى).
2. لو المؤثر الشغال محتاج "قناع" (mask) للشخص (ده بس Shadow Clone)، الـ segmentation بتبدأ على worker thread تاني.
3. MediaPipe بيلاقي الإيدين في الـ frame.
4. لو في إيد واحدة على الأقل ظاهرة، وفي landmark model متحمّل، بيتبنى الـ 206 feature والـ classifier بيدي probabilities.
5. `SignFusion` بيجمع probabilities الـ classifier مع آخر نتيجة YOLOX في علامة واحدة (أو idle).
6. `ActionController` بيهدّي القرار ده مع الوقت، ويأكد العلامة بعد ما تتمسك، ويتابع كل تسلسلات الجوتسو، ويطلق جوتسو لما يكتمل (إلا لو لسه في فترة الـ cooldown).
7. `AnchorTracker` بيختار الإيد اللي المؤثرات هتتبعها ويهدّي مكانها.
8. الـ effect manager بيرسم المؤثر الشغال على الـ frame.
9. النتيجة (الصورة، التوقيتات، العلامة، التقدم، الوقت الباقي، مرحلة المؤثر) بترجع جوه object من نوع `EngineResult`.

بعد كده، تطبيق الـ Desktop (`main.py`) بيرسم HUD ويعرض الصورة. أما سيرفر الويب (`server.py`) فبيدّي ملخص لـ `EventTracker` (في `events.py`) عشان يطلّع events البروتوكول، وبيحوّل الصورة لـ JPEG، ويبعت الاتنين للمتصفح. (HUD: يعني المعلومات المكتوبة فوق الصورة، زي الـ FPS والاحتمالات، زي شاشة الطيار. event: يعني رسالة صغيرة بتقول "حصل كذا"، زي "اتأكدت علامة tiger".)

أفكار تصميم ماشية في الـ core كله

- **الذكاء الاصطناعي منفصل عن الرسم.** الـ models اللي بتتعرف على العلامات عمرها ما بتعرف أنهي مؤثر هيشتغل، والمؤثرات عمرها ما بتعرف أنهي model اتستخدم. بيتقابلوا بس من خلال حاجتين: اسم المؤثر، ومكان الإيد. ده بيخلي أي جزء ممكن يتغير من غير ما يبوّظ التاني.
- **الـ models بتبص على frame واحد بس، والوقت بتتعامل معاه "قواعد".** بدل ما نستخدم model بيتفرج على فيديو (زي LSTM، وده نوع شبكة عصبية بتفتكر اللي فات)، كل frame بيتصنّف لوحده، والـ controller هو اللي بيضيف قواعد الوقت (امسكي العلامة 0.3 ثانية، امشي بالترتيب، حد 7 ثواني بين الخطوات). ده أسهل بكتير في تتبع الأخطاء (debug) ومحتاج بيانات تدريب أقل بكتير.
- **كل الأرقام القابلة للضبط موجودة في `config.py`.** الـ thresholds، الأوقات، وتعريف كل جوتسو.
- **مقاس، مش متخمّن.** تعليقات كتير في الكود بتذكر قياسات اتعملت على اللابتوب اللي اتطور عليه المشروع (Intel i7-8550U)، زي وقت فتح الكاميرا أو سرعة الـ classifier.

الملف `shinobi/__init__.py`

مسؤول عن إيه؟

الملف ده بيخلي الفولدر `shinobi` يبقى Python package. Package: يعني فولدر فيه ملفات Python تقدري تستوردي منها، فتقدري تكتبي `from shinobi import config`. ومعاه كمان إعداد واحد مهم جداً لويندوز وقت التشغيل.

بيعمل إيه؟

```
import os

# Must be set before OpenCV opens a camera: without it the Windows Media Foundation
# backend takes ~15 s to open the webcam. (Measured on this laptop: 14.4 s -> 0.6 s.)
os.environ.setdefault("OPENCV_VIDEOIO_MSMF_ENABLE_HW_TRANSFORMS", "0")
```

الكود ده بيعمل إيه؟ بيحط متغير بيئة (environment variable) اسمه `OPENCV_VIDEOIO_MSMF_ENABLE_HW_TRANSFORMS` بالقيمة `"0"`. متغير البيئة: يعني إعداد بيتحط للبرنامج كله من بره، والمكتبات (زي OpenCV) بتقراه.

ليه محتاجينه؟ المتغير ده بيقفل حاجة اسمها "hardware transforms" في Windows Media Foundation، وده الـ backend اللي المشروع بيستخدمه للكاميرا (backend: يعني الطبقة اللي تحت اللي بتكلم الهاردوير). لما الحاجة دي كانت شغالة، فتح الـ webcam كان بياخد حوالي 14.4 ثانية على لابتوب التطوير، ومن غيرها حوالي 0.6 ثانية بس.

إزاي بيشتغل خطوة خطوة؟

1. Python بيشغّل `__init__.py` بتاع أي package أول مرة أي حاجة من الـ package تتستورد.
2. يعني أول ما أي ملف يكتب `from shinobi import ...`، السطر ده بيتنفذ.
3. ده بيضمن إن المتغير اتحط قبل ما أي module تاني (وبالأخص `camera.py`) يفتح الكاميرا.
4. `setdefault` معناها: حط القيمة دي **بس لو** المستخدمة مكانتش حاطّاها بنفسها قبل كده. فلو حد عايز يغيّرها من بره، يقدر.

علاقته بباقي السيستم

كل module تاني بيستورد حاجة من `shinobi`، فالملف ده دايماً بيشتغل الأول. وفيه كمان الـ docstring بتاع المشروع: `"""SHINOBI AI — real-time hand-sign recognition with anime-style visual effects."""`.

الملف `shinobi/config.py`

مسؤول عن إيه؟

ده المكان الوحيد لكل مسار (path)، وكل threshold، وكل حد زمني، وكل تعريف جوتسو. ولا ملف تاني في الـ core بيكتب الأرقام دي جواه مباشرة (hard-code)؛ كلهم بيقروها بالشكل `config.NAME`. لو عايزة تغيري سلوك (مثلاً تخلي وقت مسك العلامة أطول)، بتغيري رقم واحد هنا. الملف مفيهوش أي منطق (logic)، فيه ثوابت (constants) بس.

ليه اتعمل كده؟

تخيلي لو الرقم 0.75 مكتوب في خمس ملفات مختلفة، وعايزة تغيريه: هتنسي واحد أكيد. لما كل الأرقام في مكان واحد، التعديل أسهل وأأمن، وكمان تقدري تشوفي "شخصية" السيستم كلها في ملف واحد.

المسارات (Paths)

الاسم	القيمة	المعنى
`ROOT`	فولدر المشروع (الأب بتاع `shinobi/`)	بيتحسب من مكان الملف نفسه (`Path(__file__).resolve().parent.parent`)، فالمشروع يشتغل من أي فولدر.
`MODELS_DIR`	`ROOT / "models"`	المكان اللي فيه ملفات الـ models.
`DATA_DIR`	`ROOT / "data"`	بيانات التدريب المسجّلة.
`LANDMARKS_DIR`	`DATA_DIR / "landmarks"`	الإيد المسجّلة (بيانات تدريب).
`VIDEOS_DIR`	`DATA_DIR / "videos"`	الفيديوهات المسجّلة.
`HAND_MODEL_PATH`	`models/hand_landmarker.task`	model بتاع MediaPipe للـ hand landmarks.
`SEGMENTER_MODEL_PATH`	`models/selfie_segmenter.tflite`	model بتاع MediaPipe لفصل الشخص عن الخلفية (Shadow Clone).
`GESTURE_MODEL_PATH`	`models/gesture_model.joblib`	الـ landmark classifier المتدرّب (بيعمله `train.py`).

الاسم	القيمة	المعنى
`REFERENCES_DIR`	`ROOT / "gesture_references"`	صور وشرح لكل علامة.
`SEAL_MODEL_PATH`	`models/yolox_nano.onnx`	الـ YOLOX-Nano seal detector.

(ONNX: صيغة ملفات عامة للـ models، أي برنامج يقدر يشغّلها بمكتبة زي onnxruntime. joblib: طريقة في Python لحفظ objects كبيرة في ملف وتحميلها تاني.)

الكاميرا وتتبع الإيد

الاسم	القيمة	المعنى
`CAMERA_INDEX`	`None`	`None` = جرّب الكاميرات من 0 لـ 3 واستخدم أول واحدة بتطلّع صورة حقيقية (مش سودا). أنهي webcam نستخدم.
`CAMERA_WIDTH`	`1280`	العرض المطلوب من الكاميرا بالبيكسل.
`CAMERA_HEIGHT`	`720`	الارتفاع المطلوب بالبيكسل.
`MIRROR`	`True`	اقلب الصورة يمين-شمال، عشان الشاشة تتصرف زي المراية (selfie view). سيرفر الويب بيستخدمها كمان.
`MAX_HANDS`	`2`	MediaPipe بيدور على إيدين بالكتير.
`MIN_DETECTION_CONFIDENCE`	`0.5`	MediaPipe: أقل درجة ثقة عشان يكتشف إيد جديدة.
`MIN_PRESENCE_CONFIDENCE`	`0.5`	MediaPipe: أقل درجة إن الإيد اللي بيتتبعها لسه موجودة.
`MIN_TRACKING_CONFIDENCE`	`0.5`	MediaPipe: أقل درجة عشان يكمل تتبع بدل ما يكتشف من الأول.

العلامات (Gestures)

`GESTURES` هي قايمة الـ 10 labels اللي السيستم يعرفها:

```
GESTURES = ["idle", "ram", "snake", "tiger", "dog", "monkey", "boar", "horse", "clone_seal",
"rasengan_palm"]
```

ده إيه؟ قايمة بأسماء كل الفئات. أول عنصر (index 0) **لازم** يكون `"idle"`، ومعناه "مفيش علامة": الإيدين تحت، أو بتلوّحي، أو علامة نص مكتملة. **الترتيب مهم** لأنه بيتخزن مع الـ model المتدرّب؛ لو غيّرتيه من غير ما تدرّبي تاني، الأرقام هتتلخبط.

`GESTURE_HINTS` بيدي وصف من سطر واحد لكل علامة (أدوات التسجيل بتستخدمه). أمثلة من الكود:

- `"rasengan_palm": "one hand: open palm toward camera, fingers spread"` يعني إيد واحدة، كفها مفتوح ناحية الكاميرا، والصوابع مفرودة ومتفرقة.
- `"tiger": "palms together, index fingers straight up, other fingers interlocked"` يعني الكفين لازقين، السبابتين لفوق، وباقي الصوابع متشابكة.
- `"clone_seal": "both hands: index+middle up, crossed like a plus sign"` يعني الإيدين، كل إيد السبابة والوسطى مفرودين، ومتقاطعين زي علامة +.
- `"idle": "anything else: hands down, waving, touching face, half-formed signs"`.

`LEGACY_LABELS = {"rasengan": "rasengan_palm"}` بيترجم label التسجيلات القديمة اللي اتعملت قبل ما مجموعة العلامات تتغير، وقت تحميل البيانات. يعني أي تسجيل قديم اسمه `rasengan` بيتقرا كأنه `rasengan_palm`. الـ labels القديمة اللي مش موجودة لا هنا ولا في `GESTURES` (علامة "chidori" القديمة اللي كانت زي المخلب، وعلامة "fire" القديمة اللي كانت دايرة زي OK) فاضلة على الديسك بس مش بتستخدم في التدريب.

الـ YOLOX hand-seal detector

الاسم	القيمة	المعنى
`SEAL_INPUT_SIZE`	`416`	ملف الـ ONNX بيقبل صور 416 x 416 بس.
`SEAL_THREADS`	`2`	عدد CPU threads لـ onnxruntime (مقاس: أحسن سرعة لكل core مستخدم).
`SEAL_RATE_HZ`	`10`	بالكتير 10 اكتشافات YOLOX في الثانية، في thread لوحده.
`SEAL_CLASSES`	`{2: "tiger", 5: "snake", 6: "horse", 7: "ram", 8: "monkey", 10: "dog", 11: "boar"}`	بيحوّل رقم الـ class في الـ model لاسم العلامة في المشروع. أي class تاني (rat, ox, hare, dragon, bird، والـ classes الزيادة زي gassho, unknown, mizunoe) بيتجاهل.

في تعليق بيوضح مصدر الـ model (Kazuhito00/NARUTO-HandSignDetection): الكود نفسه رخصته MIT، لكن الـ weights (الأوزان المتدرّبة) وبيانات تدريبها **مش واضحة الرخصة**، فالـ model للاستخدام الشخصي والتعليمي بس. (التفاصيل في `models/THIRD_PARTY_YOLOX.md`.)

دمج التعرف (Recognition fusion)

دي الإعدادات اللي `fusion.py` بيستخدمها، وهي من أهم الأرقام في المشروع:

الاسم	القيمة	المعنى
`LANDMARK_SIGNS`	`["clone_seal", "rasengan_palm"]`	العلامات اللي "ملك" الـ landmark classifier (YOLOX) معندوش classes ليها).
`SEAL_MIN_SCORE`	`0.70`	درجة YOLOX المطلوبة لما المربع بتاعه يكون فوق إيد MediaPipe شايفها.
`SEAL_NO_HAND_SCORE`	`0.85`	درجة YOLOX المطلوبة لما MediaPipe مش شايف ولا إيد خالص (بعض العلامات بتخبي الصوابع لدرجة إن MediaPipe بيتوه عن الإيدين).
`SEAL_CONFIRM`	`2`	نفس العلامة لازم تكسب العدد ده من نتايج YOLOX ورا بعض.
`SEAL_MAX_AGE`	`0.35`	عدد الثواني اللي نتيجة YOLOX بتفضل صالحة فيها.
`LANDMARK_MIN_PROB`	`0.60`	الـ probability المطلوبة من الـ classifier عشان `clone_seal` / `rasengan_palm`.
`LANDMARK_BACKUP_PROB`	`0.90`	الـ classifier مسموحله يبلّغ عن علامة رسمية بس لو YOLOX مش شغال خالص، وبالـ probability دي بس.

الاسم	القيمة	المعنى
`LANDMARK_MIN_MARGIN`	`0.30`	`clone_seal` / `rasengan_palm` محتاجين كمان إن الفرق بين أعلى probability وتاني أعلى (top1 - top2) يكون على الأقل كده. التعليق بيسجّل إن على تسجيلات `rasengan_palm` اللي متستخدمتش في التدريب (held-out)، الـ 5th percentile بتاع الفرق كان 0.84 (مقاس 2026-01-11)، يعني الكفوف الحقيقية بتعدّي 0.30 بسهولة جداً.
`PALM_MIN_THUMB_GAP`	`0.60`	`rasengan_palm` محتاج طرف الإبهام يكون بعيد عن طرف السبابة على الأقل 0.60 x حجم الإيد. علامة OK (الإبهام لامس السبابة) المسافة فيها حوالي 0.3-0.5، والكفوف المفتوحة الحقيقية 0.8-1.2.

(percentile: يعني لو رتبتي كل القيم من الصغير للكبير، الـ 5th percentile هو القيمة اللي 5% بس من القيم أقل منها. فلو حتى أضعف 5% من الكفوف الحقيقية فرقها 0.84، يبقى الحد 0.30 مريح جداً.)

من التعرف للفعل (stabilizer، التسلسلات، الجاهزية)

الاسم	القيمة	المعنى
`PROB_SMOOTHING`	`0.4`	وزن الـ EMA لأحدث frame عند الـ frame rate المرجعي.
`PROB_SMOOTHING_FPS`	`15`	الـ frame rate اللي الوزن ده اتضبط عليه؛ عند أي frame rate تاني الوزن بيتعدّل عشان التهدئة في الثانية تفضل زي ما هي.
`CONF_ON`	`0.75`	الـ probability المهدّية المطلوبة عشان نبدأ نأكد علامة.
`CONF_OFF`	`0.45`	لو الـ probability المهدّية نزلت تحت كده، العلامة المتأكدة بتتسحب (release).
`MIN_MARGIN`	`0.25`	الفرق المهدّي top1 - top2 لازم يكون على الأقل كده.
`HOLD_SECONDS`	`0.30`	العلامة لازم تفضل قوية المدة دي قبل ما تتأكد.
`COOLDOWN_SECONDS`	`1.5`	بعد ما جوتسو يتطلق، أي جوتسو جديد بيتجاهل المدة دي.
`SEQUENCE_MAX_GAP`	`7.0`	أقصى عدد ثواني من خطوة متأكدة (أو محاولة غلط) للخطوة اللي بعدها في نفس التسلسل. ده **حد أقصى، مش وقت انتظار**: الخطوة الجاية بتتحسب أول ما تتأكد. اتعلّى من 3.5 ثانية لأن تغيير الإيدين بين علامتين كان بياخد وقت أطول في الاستخدام الحقيقي.
`SEQUENCE_CLOCK_MAX_STEP`	`0.4`	الـ frame الواحد المتعالج بيزوّد ساعة التسلسل بحد أقصى 0.4 ثانية، فلو حصل تهنيج (stall) مياكلش من الـ 7 ثواني.
`READY_FRAMES`	`5`	العلامات بتتحسب بس بعد العدد ده من الـ frames ورا بعض اشتغلوا بسرعة كفاية.
`READY_MAX_MS`	`500`	"بسرعة كفاية" = الـ frame اتعالج في أقل من 500 ملي ثانية.

(EMA: يعني Exponential Moving Average، "متوسط متحرك أُسّي"، طريقة بنخلط بيها القيمة الجديدة مع القديمة عشان نهدّي التذبذب. هنشرحها بالتفصيل في `controller.py`.)

ملاحظة على تعليق جوه الكود نفسه: التعليق اللي فوق `SEQUENCE_MAX_GAP` في `config.py` بيقول إن التايمر بيبدأ من جديد مع كل علامة متأكدة "(also when the same sign is re-confirmed while held)"، يعني حتى لو نفس العلامة اتأكدت تاني وإنتي ماسكاها. بس الكود الحقيقي في `controller.py` (`SequenceMatcher.push`) بيعمل **العكس**: التكرار (repeat) عمره ما بيجدد الساعة. يعني التعليق ده قديم، والسلوك الصح هو اللي في `controller.py` (وده نفس اللي النسخة الإنجليزية من الدليل بتقوله). هنشرح ده بالتفصيل تحت.

جدول الجوتسو (The jutsu table)

```
JUTSU = {
    "fire_style": {"sequence": ["monkey", "boar", "horse", "tiger"], "effect": "fire", "official":
True},
    "chidori": {"sequence": ["snake", "dog", "tiger", "monkey"], "effect": "chidori", "official":
True},
    "shadow_clone": {"sequence": ["ram", "snake", "tiger", "clone_seal"], "effect": "shadow_clone",
                     "official": False},
    "rasengan": {"sequence": ["clone_seal", "dog", "horse", "rasengan_palm"], "effect": "rasengan",
                 "official": False},
}
```

ده بيعمل إيه؟ ده dictionary (قاموس: مفتاح وقيمة) بيعرّف كل جوتسو. كل جوتسو ليه:

- `sequence`: قايمة مرتبة من أربع علامات لازم تتعمل بالترتيب ده.
- `effect`: اسم المؤثر اللي هيشتغل.
- `official`: `True` لو الأربع علامات دول علامات متتالية رسمية من الأنمي، و `False` لو تسلسل مخصوص عمله SHINOBI AI.

نفس الجدول بالعربي:

الجوتسو	العلامات بالترتيب	المؤثر	رسمي؟
`fire_style`	monkey ثم boar ثم horse ثم tiger	`fire`	أيوه (`True`)
`chidori`	snake ثم dog ثم tiger ثم monkey	`chidori`	أيوه (`True`)
`shadow_clone`	ram ثم snake ثم tiger ثم clone_seal	`shadow_clone`	لأ (`False`)
`rasengan`	clone_seal ثم dog ثم horse ثم rasengan_palm	`rasengan`	لأ (`False`)

في قاعدة بيتأكد منها وقت التشغيل الدالة `SequenceMatcher._check_conflicts`: مينفعش أي جوتسو يظهر (بالترتيب) جوه جوتسو تاني قبل آخر علامة في التاني. (هنشرح ليه في `controller.py`.) وإضافة جوتسو جديد = سطر واحد جديد في الجدول ده.

لاحظي حاجة مهمة: `tiger` موجودة في تلات تسلسلات (fire_style و chidori و shadow_clone)، و `clone_seal` موجودة في اتنين. ده معناه إن علامة واحدة ممكن تقدّم أكتر من جوتسو في نفس الوقت، وده اللي `SequenceMatcher` بيتعامل معاه.

مين بيستخدمه؟ `controller.py` (الـ `SequenceMatcher` والـ `ActionController`)، و `engine.py`، و `events.py`، و `catalog.py`.

المؤثرات، والتتبع، والـ segmentation (الجزء بتاع المؤثرات بيستخدمهم)

الاسم	القيمة	المعنى
`EFFECT_DURATION`	shadow_clone 10.0، rasengan 8.0، chidori 8.0، fire 6.0	عدد ثواني تشغيل كل مؤثر أساسي.
`EFFECT_KEYS`	المفاتيح 1، 2، 3، 4	مفاتيح في الـ Desktop بتشغّل shadow_clone، rasengan، chidori، fire مباشرة.

الاسم	القيمة	المعنى
`HAND_LOST_GRACE`	`0.6`	عدد الثواني اللي الـ anchor بتاع الإيد بيفضل يتحسب "ظاهر" بعد ما MediaPipe يتوه عن الإيد.
`HAND_EFFECT_MIN_SECONDS`	`None`	مؤثرات الإيد بتكمل في آخر مكان للإيد لما الإيد تختفي (`None` = عمره ما بيخلص بسبب اختفاء الإيد، بيكمل كل الـ `EFFECT_DURATION`).
`ACTIVATION_CHARGE`	1.2 ثانية لكل جوتسو	طول مرحلة "الشحن" (charge).
`ACTIVATION_CHARGE_DEFAULT`	`1.2`	طول الشحن لجوتسو مش موجود في الجدول.
`VOICE_DEFAULT`	`0.4`	طول مرحلتي "voice" و "name" لما مفيش ملف صوت متركّب.
`VOICE_GAP`	`0.05`	بيتزاد بعد طول ملف الصوت نفسه.
`CLONE_COUNT`	`10`	عدد الـ shadow clones (11 شخص على الشاشة).
`CLONE_OFFSETS` ، `CLONE_SCALES` ، `CLONE_LIFTS` ، `CLONE_DELAYS`	10 قيم لكل واحد	المكان، والحجم، والرفع لفوق، والتأخير الزمني لكل clone (تلات صفوف: 2، 4، 4).
`CLONE_PARALLEL`	`True`	ابني طبقة الـ clones على worker thread.
`SEGMENT_WIDTH`	`640`	الـ segmentation بتشتغل على frame متصغّر لعرض 640 بيكسل (للسرعة).
`MASK_SMOOTHING`	`0.5`	وزن الـ EMA لأحدث person mask.
`FILTER_MIN_CUTOFF`	`1.5`	فلتر One Euro: كل ما يصغر = أنعم بس فيه تأخير أكتر.
`FILTER_BETA`	`0.02`	فلتر One Euro: قد إيه التنعيم بيخف بسرعة لما الإيد تتحرك بسرعة.

القيم بالظبط في الكود:

```
CLONE_COUNT = 10
CLONE_OFFSETS = [-0.36, 0.36, -0.46, -0.27, 0.27, 0.46, -0.19, 0.19, -0.37, 0.37]
CLONE_SCALES = [0.68, 0.68, 0.56, 0.56, 0.56, 0.56, 0.45, 0.45, 0.45, 0.45]
CLONE_LIFTS = [0.00, 0.00, 0.22, 0.22, 0.22, 0.22, 0.40, 0.40, 0.40, 0.40]
CLONE_DELAYS = [0.25, 0.40, 0.55, 0.70, 0.85, 1.00, 1.15, 1.30, 1.45, 1.60]
```

مهم: عدد الـ clones هو 10 بالظبط (11 شخص على الشاشة) وده قرار نهائي في المشروع، ومش بيتقلّل عشان الـ FPS. جزء المؤثرات في الدليل بيشرح إزاي قيم الـ clones والـ activation بتتستخدم.

جمع البيانات وسيرفر الويب

الاسم	القيمة	المعنى
`RECORD_SECONDS`	`20`	طول جلسة التسجيل الواحدة (`collect_data.py`).
`COUNTDOWN_SECONDS`	`3`	العد التنازلي قبل التسجيل.
`SEQUENCES_DIR` ، `SEQUENCE_VIDEOS_DIR`	`data/sequences` ، `data/videos/sequences`	تسجيلات التسلسلات الموجّهة.

الاسم	القيمة	المعنى
`SEQ_HOLD_SECONDS`، `SEQ_TRANSITION_SECONDS`، `SEQ_LEAD_SECONDS`	1.5 لكل واحد	وقت المسك، ووقت تغيير الإيدين، ووقت البداية والنهاية (الإيدين تحت) في التسجيلات الموجّهة (`record_sequences.py`).
`APP_VERSION`، `PROTOCOL_VERSION`	`"0.1.0"`، `1`	بيتبلّغوا لتطبيق React.
`SERVER_HOST`، `SERVER_PORT`	`"127.0.0.1"`، `8000`	المكان اللي السيرفر بيسمع عليه. (`0.0.0.0` = يتشاف من الشبكة المحلية، والموبايلات محتاجة HTTPS عشان الكاميرا.)
`SERVER_MAX_SESSIONS`	`2`	عدد جلسات المتصفح في نفس الوقت (كل واحدة ليها `Engine` خاص بيها).
`FRAME_MAX_WIDTH`، `FRAME_MAX_HEIGHT`	1280، 1280	الـ frames الأكبر من كده من المتصفح بتتصغّر (مع الحفاظ على النسبة).
`FRAME_MIN_SIZE`	`32`	الـ frames الأصغر من كده بترفض.
`FRAME_JPEG_QUALITY`، `OUTPUT_JPEG_QUALITY`	80، 90	جودة JPEG للـ frames الداخلة والخارجة.
`OUTPUT_JPEG_444`	`True`	دقة ألوان كاملة في الـ JPEG الخارج (الخطوط الملونة الرفيعة تفضل حادة).
`FRAME_MAX_IN_FLIGHT`	`2`	عدد الـ frames اللي الـ client يقدر يبعتها قبل ما ياخد رد.
`FRAME_MAX_BYTES`	4 MiB (`4 * 1024 * 1024`)	الرسايل الأكبر من كده بترفض.
`FRONTEND_DIST`، `AUDIO_DIR`	`frontend/dist`، `assets/audio`	تطبيق React المتبني (`npm run build`) وملفات الصوت.

علاقته بباقي السيستم

بيتستورد من كل الملفات تقريباً. `catalog.py` بيقرا `EFFECT_DURATION` و `CLONE_COUNT` و `AUDIO_DIR`؛ السيرفر بيقرا بلوك السيرفر؛ و `train.py` وسكربتات التسجيل بيقروا المسارات و `GESTURES`.

مثال عملي: لو لاحظتي إن المستخدمين مش بيلحقوا يغيروا إيديهم بين علامتين، بتغيري `SEQUENCE_MAX_GAP` من 7.0 لرقم أكبر، وخلاص؛ مفيش أي ملف تاني محتاج يتلمس. ولو عايزة جوتسو جديد، بتضيفي سطر في `JUTSU`، و `_check_conflicts` هيقولك فوراً وقت التشغيل لو التسلسل الجديد بيتعارض مع واحد موجود.

الملف `shinobi/camera.py`

مسؤول عن إيه؟

بيقرا الـ webcam في thread في الخلفية، ودايماً بيدّي أحدث frame. ده مستخدم **بس** في تطبيق الـ Desktop (`main.py`)؛ في تطبيق الويب، المتصفح هو اللي ماسك الكاميرا وبيبعت الـ frames للسيرفر.

ليه thread؟

الدالة `cap.read()` بتاعة OpenCV "بتقفل" (block) لحد ما الكاميرا تبعت الـ frame اللي بعده (حوالي 33 ملي ثانية عند 30 FPS). Block: يعني البرنامج بيقف مستني ومبيعملش أي حاجة تانية. لو الـ loop الأساسي ناداها مباشرة، هيضيّع الوقت ده في الانتظار بدل ما يشغّل الذكاء الاصطناعي.

مع الـ thread، الكاميرا بتفضل تملا "خانة أحدث frame"، والـ loop الأساسي بياخد أحدث حاجة موجودة من غير ما يستنى. الـ frames اللي الـ loop كان أبطأ من إنه يلحقها بتتكتب فوقها ببساطة، وده بيخلي العرض "حالي" بدل ما يتأخر ورا الواقع.

(FPS: يعني Frames Per Second، عدد الصور في الثانية.)

الدوال (Functions)

- `_open(index, width, height)`: بتفتح الكاميرا رقم `index` باستخدام الـ backend بتاع Media Foundation (`cv2.CAP_MSMF`) وبتطلب الدقة المطلوبة. التعليق بيشرح الاختيار: MSMF بيدّي 30 FPS عند 720p على لابتوب التطوير، و DirectShow بيدي حوالي 10 بس.
- `_is_real(cap, tries=10)`: بتقرا لحد 10 frames وبترجع `True` لو واحد منهم متوسط الإضاءة بتاعه (`frame.mean()`) أكبر من 5. الكاميرات الوهمية (virtual cameras، زي Camo من غير موبايل متوصّل) بتفتح عادي من غير error بس بتبعت frames سودا؛ الدالة دي بتفلترهم.
- `find_camera(width, height, max_index=4)`: بتجرب أرقام الكاميرات من 0 لـ 3 وبترجع `(index, capture)` لأول واحدة حقيقية. لو مفيش ولا واحدة شغالة بترمي `RuntimeError("No working camera found. Check that the webcam is not disabled or in use.")`.

```
def _is_real(cap, tries=10):
    """Virtual cameras (e.g. Camo with no phone attached) open fine but send black frames."""
    for _ in range(tries):
        ok, frame = cap.read()
        if ok and frame is not None and frame.mean() > 5:
            return True
    return False
```

بيعمل إيه خطوة خطوة؟ بيلف 10 مرات. كل مرة بيقرا frame. لو القراءة نجحت (`ok`)، والـ frame موجود، ومتوسط قيم البيكسلات أكبر من 5 (يعني الصورة مش سودا تماماً؛ البيكسل الأسود قيمته 0 والأبيض 255)، يبقى دي كاميرا حقيقية فيرجع `True`. لو خلص العشر محاولات من غير ما يلاقي صورة حقيقية، يرجع `False`.

الكلاس `Camera`

- `__init__(index, width, height, mirror)`: القيم الافتراضية جاية من config. لو `index=None` بتنادي `find_camera`؛ غير كده بتفتح الرقم ده وبترمي error لو مقدرتش (`"Cannot open camera {index}. Set CAMERA_INDEX = None in config.py"`). بتطبع الدقة الفعلية (مثلاً `camera 0: 1280x720`)، وبعدين بتشغّل الـ reader thread (من نوع daemon، يعني مش هيمنع Python إنه يقفل لما البرنامج يخلص).
- `_loop()`: جسم الـ thread. بيقرا frame؛ لو القراءة فشلت بينام 10 ملي ثانية ويحاول تاني. بيقلب الـ frame أفقياً لو `mirror` شغال (`cv2.flip(frame, 1)`). بيخزن الـ frame ويزوّد `_frame_id` بواحد، وده كله تحت lock.

- `read()`: بترجع `(frame_id, frame)` تحت الـ lock. الـ id بيسمح للي بينادي إنه يتخطى frame اتعالج قبل كده (الـ loop الأساسي بيقارنه بآخر id اتعامل معاه).
- `release()`: بتوقف الـ loop، وتستنى الـ thread لحد ثانية واحدة، وتقفل الكاميرا.

```
def _loop(self):
    while self._running:
        ok, frame = self.cap.read()
        if not ok:
            time.sleep(0.01)
            continue
        if self.mirror:
            frame = cv2.flip(frame, 1)
        with self._lock:
            self._frame = frame
            self._frame_id += 1
```

الـ lock (`threading.Lock`) بيضمن إن `read()` عمرها ما ترجع frame و id من لحظتين مختلفتين. Lock: يعني "قفل"؛ لما thread ماسك القفل، أي thread تاني عايز نفس القفل لازم يستنى. فلو الـ main loop بيقرا في نفس اللحظة اللي الكاميرا بتكتب فيها، واحد منهم هيستنى التاني يخلص، ومش هيحصل إن الـ id يبقى الجديد والـ frame القديم.

المدخلات والمخرجات

- المدخل: الكاميرا نفسها (هاردوير).
- المخرج: `(frame_id, frame)` من `read()`، والـ frame صورة BGR (مصفوفة numpy بشكل `(720, 1280, 3)` عادةً).

علاقته بباقي السيستم

بيستخدمه `main.py`. بيقرا `CAMERA_INDEX` و `CAMERA_WIDTH` و `CAMERA_HEIGHT` و `MIRROR` من config.

الملف `shinobi/hands.py`

مسؤول عن إيه؟

غلاف (wrapper) صغير حوالين الـ Hand Landmarker بتاع MediaPipe (Tasks API). Wrapper: يعني كود بسيط بيلف حوالين مكتبة تانية عشان يخلي استخدامها أسهل ومناسب للمشروع. بيحوّل frame الكاميرا لقايمة objects من نوع `Hand`، كل واحد فيه 21 landmark بتلات أنظمة إحداثيات. وفيه كمان دالة مساعدة بترسم هيكل الإيد.

أرقام الـ landmarks

MediaPipe بيرقّم نقط الإيد الـ 21 كده:

الرقم	النقطة
0	الرسغ (wrist)
1-4	الإبهام (4 = طرف الإبهام)

الرقم	النقطة
5-8	السبابة (5 = العقلة/الـ knuckle، 8 = الطرف)
9-12	الوسطى (9 = عقلة الوسطى)
13-16	البنصر
17-20	الخنصر

الملف بيدّي أسماء للنقط اللي محتاجها:

```
WRIST, THUMB_TIP, INDEX_MCP, INDEX_TIP, MIDDLE_MCP, MIDDLE_TIP = 0, 4, 5, 8, 9, 12
RING_MCP, PINKY_MCP = 13, 17
FINGERTIPS = [4, 8, 12, 16, 20]
PALM = [0, 5, 9, 13, 17]
```

(MCP: اختصار طبي للمفصل اللي بين الصباع وكف الإيد، اللي بنسميه "العقلة".) `FINGERTIPS` هي أطراف الخمس صوابع، و `PALM` هي خمس نقط بتحدد كف الإيد. `CONNECTIONS` قايمة بأزواج النقط اللي بيتوصل بينها خط لما نرسم الهيكل (مثلاً `(0, 1)`، `(1, 2)`... للإبهام).

الـ dataclass `Hand`

Dataclass: يعني كلاس بسيط وظيفته الأساسية إنه يشيل بيانات (زي "استمارة" فيها خانات).

الخانة (Field)	الشكل (Shape)	المعنى
`landmarks`	(21, 3)	إحداثيات الصورة المطبّعة (normalized): x و y من 0 لـ 1 على عرض وطول الصورة، و z عمق نسبي.
`world`	(21, 3)	إحداثيات 3D حقيقية بالمتر، ونقطة الأصل قريبة من نص الإيد. مش بتتأثر ببعد الإيد عن الكاميرا.
`handedness`	str	`"Left"` أو `"Right"` (زي ما MediaPipe بيقولها للصورة المقلوبة كمراية).
`score`	float	درجة الثقة في الـ handedness.
`px`	(21, 2)	إحداثيات بالبيكسل في الـ frame (x و y المطبّعين مضروبين في العرض والطول).

في خاصيتين (properties) المؤثرات بتستخدمهم:

- `palm_center`: متوسط مكان الخمس نقط بتوع الكف بالبيكسل، يعني "نص الكف".
- `size_px`: المسافة بالبيكسل من الرسغ لعقلة الوسطى. دي "مسطرة" للحجم، عشان المؤثرات تكبر مع الإيد لما تقربي من الكاميرا.

الكلاس `HandTracker`

- `__init__()`: بيعمل الـ landmarker في وضع `VIDEO` مع `num_hands=MAX_HANDS` وتلات thresholds الثقة من config. وضع VIDEO معناه إن MediaPipe بيتتبع الإيد من frame للي بعده بدل ما يدوّر عليها من الصفر كل مرة، وده أسرع وأثبت.
- `_timestamp()`: وضع VIDEO لازم ياخد timestamps بالملي ثانية **بتزيد على طول** (strictly increasing). بيستخدم `time.monotonic()`، ولو ندائين وقعوا في نفس الملي ثانية، بيزود 1 على القيمة اللي قبلها.

- `detect(frame_bgr)`: بيحوّل BGR (ترتيب ألوان OpenCV) لـ RGB (ترتيب MediaPipe)، ويشغّل `detect_for_video`، ويبني `Hand` لكل إيد اتلقت. وبعدين بيرتب الإيدين حسب x بتاع الرسغ، من الشمال لليمين على الشاشة.
- `close()`: بيقفل object الـ MediaPipe.

```
def _timestamp(self):
    # VIDEO mode requires strictly increasing timestamps in milliseconds.
    ts = int(time.monotonic() * 1000)
    self._last_ts = max(ts, self._last_ts + 1)
    return self._last_ts
```

ليه محتاجينه؟ لو اتنادى `detect` مرتين في نفس الملي ثانية، MediaPipe هيرمي error لأن الـ timestamp متكرر. فالسطر `max(ts, self._last_ts + 1)` بيضمن إن الرقم دايماً أكبر من اللي قبله ولو بواحد.

```
# Order by wrist x, not by handedness: handedness flips when hands overlap.
hands.sort(key=lambda hand: hand.landmarks[WRIST, 0])
```

الترتيب ده مهم جداً للـ features: الخانة (slot) 0 دايماً هي الإيد اللي على الشمال في الشاشة، والخانة 1 اللي على اليمين. ليه مش بنستخدم "label "Left/Right بتاع MediaPipe؟ لأنه مش موثوق لما الإيدين يتداخلوا (وده بيحصل في أغلب العلامات)، وممكن يقلب من frame للتاني. فبنستخدم المكان بدله، وده ثابت.

الدالة `draw_hands(frame, hands)`

بترسم خطوط بيضا للـ `CONNECTIONS`، ونقط برتقالي مصفر لأطراف الصوابع، ونقط برتقالي للمفاصل التانية، و label الـ handedness تحت الرسغ. بتتستخدم لما "رسم الـ landmarks" يكون شغال (مفتاح L في الـ Desktop).

المدخلات والمخرجات

- المدخل: frame بصيغة BGR.
- المخرج: `list` من `Hand` مترتبة شمال لليمين (ممكن تكون فاضية لو مفيش إيد).

علاقته بباقي السيستم

`engine.py` بيعمل `HandTracker` وبينادي `detect` كل frame. الـ `Hand` objects بتروح لـ `features.hands_to_arrays`، و `fusion.py` (مربعات الإيد والـ open-palm gate)، و `tracking.py` (الـ anchor)، والمؤثرات.

الملف `shinobi/features.py`

مسؤول عن إيه؟

Feature Extraction: يعني بنستخرج المعلومات المهمة من شكل الإيد عشان الـ model يقدر يتعرف عليها. الملف ده بيحوّل الـ landmarks الخام لـ vector (قايمة أرقام) طوله ثابت = 206 رقم، ودي مدخلات الـ landmark classifier.

نفس الدالة بتستخدم في التدريب (`train.py`) وفي التطبيق الحي، فالـ model دايماً بيشوف features محسوبة بنفس الطريقة بالظبط. `FEATURE_VERSION = 1` بيتحفظ مع الـ model المتدرّب؛ لو كود الـ features اتغير، لازم الـ version يتغير، و `classifier.py` بيرفض أي model اتدرّب على version تاني.

ليه features بدل الإحداثيات الخام؟

أماكن البيكسلات الخام بتتغير لما تحركي إيدك في الصورة، أو تقربي، أو تبعدي، بس العلامة نفسها هي هي. الـ features بتشيل الفروق دي: الأماكن بتتقاس من الرسغ، وبتتقسم على حجم الإيد، ومعاها زوايا الصوابع والمسافات بين أطرافها، ودول بيوصفوا **شكل** الإيد مباشرة.

مثال: علامة tiger في ركن الشاشة الشمال وإيدك قريبة، وعلامة tiger في النص وإيدك بعيدة: الإحداثيات الخام مختلفة خالص، بس الـ features تقريباً متطابقة. وده اللي بيخلي الـ model يتعلم بأمثلة أقل.

توزيع الـ 206 رقم

الجزء	العدد	المحتوى
خانة الإيد 0 (الشمال على الشاشة)	89	علم وجود واحد (presence flag) + 63 إحداثي 3D مطبّع + 15 زاوية مفصل + 10 مسافات بين أطراف الصوابع
خانة الإيد 1 (اليمين على الشاشة)	89	نفس الكلام
بين الإيدين	28	علم "الإيدين موجودين" واحد + إزاحة الرسغ (x, y) اتنين + 25 مسافة من طرف صباع لطرف صباع (5 x 5)
المجموع	206	`FEATURE_SIZE = 2 * PER_HAND_SIZE + INTER_HAND_SIZE`

في الكود:

```
PER_HAND_SIZE = 1 + 63 + 15 + 10      # present flag + coords + angles + tip distances
INTER_HAND_SIZE = 1 + 2 + 25          # both flag + wrist offset + 5x5 tip distances
FEATURE_SIZE = 2 * PER_HAND_SIZE + INTER_HAND_SIZE
```

لو إيد مش موجودة، بتدّي 89 صفر (والـ presence flag بتاعها 0)؛ ولو مفيش إيدين الاتنين، الـ 28 رقم بتوع "بين الإيدين" كلهم أصفار. (مثلاً `rasengan_palm` إيد واحدة، فنص الـ vector تقريباً أصفار، والـ model بيتعلم ده.)

الدالة `hands_to_arrays(hands)`

بتحوّل قايمة `Hand` (بتستخدم أول اتنين بس) لصيغة التخزين الخام: `image_lm` بشكل (2, 21, 3)، و `world_lm` بشكل (2, 21, 3)، و `present` بشكل (2,) فيها 1.0 لكل خانة فيها إيد. ودي كمان نفس الصيغة اللي أدوات التسجيل بتحفظها على الديسك، فالتدريب يقدر يعيد حساب الـ features بعدين (مثلاً لو الـ features اتغيرت).

الدالة `_hand_features(world)`: إيد واحدة

بتشتغل على الـ 3D world landmarks بتوع إيد واحدة:

```
rel = world - world[0]                                  # wrist becomes the origin
scale = np.linalg.norm(rel[9]) + 1e-6                   # wrist -> middle knuckle
rel = rel / scale                                       # same size for near/far hands
```

بيعمل إيه؟

- السطر الأول: بيطرح مكان الرسغ (`world[0]`) من كل النقط، فالرسغ يبقى نقطة الأصل (0, 0, 0).
- السطر التاني: بيحسب طول المسافة من الرسغ لعقلة الوسطى (نقطة 9). `np.linalg.norm` بتحسب طول المتجه. الـ `+ 1e-6` رقم صغير جداً عشان منقسمش على صفر أبداً.
- السطر التالت: بيقسم كل حاجة على الطول ده، فالإيد الكبيرة والصغيرة يبقوا نفس الحجم.

الأجزاء التلاتة:

1. **الإحداثيات (63 رقم).** كل نقطة ناقص الرسغ، مقسومة على مسافة الرسغ لعقلة الوسطى. فالمكان والحجم ميفرقوش، الشكل بس اللي بيفرق. (21 نقطة 3 x أبعاد = 63.)
2. **الزوايا (15 رقم).** لكل واحدة من الـ 15 ثلاثية في `ANGLE_TRIPLETS` (تلاتة لكل صباع، مثلاً `(5, 6, 7)` = الزاوية عند مفصل السبابة 6)، بتتحسب الزاوية بين المتجهين b->a و b->c بمعادلة الضرب النقطي (dot product): `cos = (v1 . v2) / (|v1| |v2|)`، وبعدين `arccos`، وبعدين قسمة على pi. النتيجة من 0 لـ 1، والـ 1 معناه صباع مفرود على الآخر، والقيم الصغيرة معناها صباع مثني. `np.clip` بيخلي الـ cosine بين -1 و 1 عشان أخطاء التقريب متخليش `arccos` تفشل.
3. **مسافات أطراف الصوابع (10 أرقام).** المسافة بين كل زوج من الخمس أطراف (`TIP_PAIRS`، 10 أزواج)، مقسومة على نفس الـ scale. دي بتمسك حاجات زي "الإبهام لامس السبابة".

والـ vector بيبدأ بـ `1.0`، وده الـ presence flag.

```
ANGLE_TRIPLETS = [
    (0, 1, 2), (1, 2, 3), (2, 3, 4),          # thumb
    (0, 5, 6), (5, 6, 7), (6, 7, 8),          # index
    (0, 9, 10), (9, 10, 11), (10, 11, 12),    # middle
    (0, 13, 14), (13, 14, 15), (14, 15, 16),  # ring
    (0, 17, 18), (17, 18, 19), (18, 19, 20),  # pinky
]
TIP_PAIRS = [(i, j) for i in range(5) for j in range(i + 1, 5)]   # 10 pairs
```

ملاحظة صغيرة من الكود: الزوايا بتتحسب على `world` الأصلي (مش `rel`)، وده مش بيفرق لأن الزاوية مش بتتأثر بالإزاحة ولا بالتكبير.

الدالة `extract(image_lm, world_lm, present, aspect=16/9)` : الـ vector الكامل

بتبني الـ 89 رقم لكل خانة (أو أصفار)، وبعدين جزء "بين الإيدين" لما الإيدين الاتنين موجودين. جزء بين الإيدين بيستخدم إحداثيات الصورة 2D، لأن الـ world coordinates بتاعة كل إيد متمركزة على الإيد نفسها، فمتقدرش تقولك الإيدين فين بالنسبة لبعض:

```
pts = image_lm[:, :, :2] * np.array([aspect, 1.0])
palm = np.mean([np.linalg.norm(pts[s, 9] - pts[s, 0]) for s in range(2)]) + 1e-6
offset = (pts[1, 0] - pts[0, 0]) / palm
cross = np.linalg.norm(pts[0, TIPS][:, None] - pts[1, TIPS][None], axis=2) / palm
```

خطوة خطوة:

1. x و y في الصورة الاتنين من 0 لـ 1، بس الصورة أعرض من طولها، فبنضرب x في الـ aspect ratio (العرض / الطول) عشان المسافات تبقى نسب حقيقية. من غير كده، مسافة أفقية ومسافة رأسية متساويتين على

الشاشة هيبانوا مختلفين.

2. `palm` هو متوسط مسافة الرسغ لعقلة الوسطى في الإيدين، ويستخدم كمقياس.
3. `offset` هو المتجه من رسغ الإيد الشمال لرسغ الإيد اليمين (رقمين: x و y).
4. `cross` جدول 5 5 x فيه المسافة بين كل طرف صباع في إيد 0 وكل طرف صباع في إيد 1 (بيتحسب في خطوة واحدة بحاجة اسمها numpy broadcasting، يعني numpy بيكرر المصفوفات تلقائياً عشان يحسب كل الأزواج مرة واحدة).
5. النتيجة بترجع بنوع `float32`.

مين بينادي الدالة دي؟ `engine.py`، بالشكل: `extract(*hands_to_arrays(hands), aspect=w / h)`. الـ `*` معناها "فك" الـ tuple اللي راجع من `hands_to_arrays` لتلات مدخلات منفصلة.

الدالتين `mirror(...)` و `order_slots(...)`

بيستخدموا في الـ data augmentation وقت التدريب. Data augmentation: يعني بنعمل أمثلة تدريب زيادة من الأمثلة الموجودة بتعديلات بسيطة (هنا: صورة مراية)، عشان الـ model يتعلم أحسن من غير ما نسجّل أكتر.

- `mirror` بتعمل صورة المراية (يمين-شمال) لـ frame متسجّل: x في الصورة بيبقى `1 - x` (بس للإيدين الموجودين)، و x في الـ world بيتعكس إشارته.
- لأن المراية بتبدّل مين اللي على الشمال، `order_slots` بترتب الخانات تاني: الإيدين الموجودين الأول، وبعدين حسب x بتاع الرسغ، نفس قاعدة `HandTracker.detect`.

ده بيخلي العلامة اللي اتسجلت بترتيب إيدين معين تعلّم الـ model كمان الترتيب المعكوس.

علاقته بباقي السيستم

`engine.py` بينادي `extract(*hands_to_arrays(hands), aspect=w / h)`. `classifier.py` بيستورد `FEATURE_VERSION`. `train.py` بيستخدم `extract` و `mirror` و `order_slots`.

الملف `shinobi/classifier.py`

مسؤول عن إيه؟

بيحمّل الـ landmark model المتدرّب من `models/gesture_model.joblib`، وبيحوّل vector الـ 206 رقم لـ probability لكل label. وفيه كمان تحسين سرعة: كود مكتوب باليد و vectorized بيحسب الـ Random Forest بنفس الأرقام بالظبط اللي بيطلعها scikit-learn، بس أسرع حوالي 100 مرة.

(scikit-learn: مكتبة Python مشهورة للـ machine learning، فيها Random Forest و MLP وغيرهم. vectorized: يعني بدل ما نلف على العناصر واحد واحد في Python، بنخلي numpy يشتغل على مصفوفة كاملة مرة واحدة، وده أسرع بكتير.)

الـ model bundle المحفوظ

`train.py` بيحفظ dictionary بـ `joblib` فيه:

- `"model"`: model من scikit-learn. التدريب بيقارن Random Forest فيه 300 شجرة (`n_estimators=300`) مع شبكة عصبية MLP (`hidden_layer_sizes=(128, 64)`) وبيحتفظ بالأحسن.
- `"model_name"`: اسم الـ model اللي كسب.
- `"labels"`: أسماء الـ labels بترتيب مخرجات الـ model.
- `"feature_version"`: نسخة الـ features اللي اتدرب عليها.

(إضافة من الكود: `train.py` بيحفظ كمان `"cv_macro_f1"`، نتايج المقارنة، والنسخة الإنجليزية مذكرتهاش. الـ classifier نفسه مش بيستخدمها.)

الكلاس `GestureClassifier`

- `__init__(path=None)`: بيحمّل الـ bundle. لو `feature_version` مختلف عن `features.FEATURE_VERSION` بيرمي error بيقولك تشغّلي `train.py` تاني، لأن model اتدرّب على features مختلفة هيدي إجابات ملهاش معنى. بيخزن `model` و `labels` و `name`. بيحط كل بارامتر `n_jobs` بـ 1: وقت التدريب `n_jobs=-1` بيستخدم كل الـ CPU cores، بس لـ frame واحد، تشغيل thread pool في كل نداء بيضيّع وقت بس. وفي الآخر `self.fast = fast_forest(self.model)`.
- `predict(features)`: بترجع مصفوفة probabilities نوعها float32 بنفس ترتيب `self.labels`. لو الـ fast forest موجود والـ features مفيهاش NaN أو infinity، بيستخدمه؛ غير كده بينادي `predict_proba` بتاع scikit-learn (scikit-learn بيتعامل مع NaN بطريقة خاصة، فالحالات دي بنسيبهاله).
- `exists(path=None)`: static method، بترجع `True` لو ملف الـ model موجود. `engine.py` بيستخدمها عشان يقرر يحمّل model أصلاً ولا لأ.

```
def predict(self, features):
    """Return a probability array aligned with self.labels."""
    if self.fast is not None and np.isfinite(features).all():   # sklearn treats NaN specially: leave
it to it
        return self.fast(features).astype(np.float32)
    return self.model.predict_proba(features.reshape(1, -1))[0].astype(np.float32)
```

بيعمل إيه؟ بيختار الطريق السريع لو ينفع، وإلا الطريق العادي. `reshape(1, -1)` معناها "خلّي الـ vector ده صف واحد" لأن scikit-learn مستني جدول (عدة أمثلة)، و `[0]` بياخد نتيجة أول (والوحيد) مثال. (NaN: يعني "Not a Number"، قيمة غلط بتطلع من عمليات زي 0/0.)

مثال: لو المدخل features بتاعة علامة tiger، الناتج ممكن يكون حاجة زي `[0.02, 0.01, 0.03, 0.90, ...]` بترتيب `self.labels`، يعني tiger بـ 0.90.

الطريق السريع للـ Random Forest: `ForestOneFrame`

المشكلة. الـ Random Forest عبارة عن أشجار قرار كتير. كل شجرة بتسأل سلسلة أسئلة زي "هل feature رقم 37 أقل من أو يساوي 0.42؟ لو أيوه روح شمال، لو لأ روح يمين" لحد ما توصل لورقة (leaf)، والورقة فيها نسب الفئات. إجابة الـ forest هي متوسط نسب الأوراق في كل الأشجار. scikit-learn بيمشي في الـ 300 شجرة واحدة واحدة، وبيتشيك على المدخل في كل نداء، و (مع `n_jobs=-1`) بيشغّل 40-50 thread pool: ملي ثانية لكل frame (مقاس 31-10-2026)، في حين إن ميزانية الـ frame كله 70-100 ملي ثانية بس.

الفكرة. نحط الـ 300 شجرة كلهم في مصفوفات numpy مسطّحة (flat)، ونمشي فيهم كلهم في نفس الوقت، مستوى واحد من الشجرة في كل خطوة numpy. بدل 300 مشوار منفصل، يبقى عندنا بس `depth` خطوة (عمق أطول شجرة)، كل خطوة بتعالج 300 node مرة واحدة.

إزاي المصفوفات بتتبني في `__init__(forest)`:

1. `counts` = عدد الـ nodes في كل شجرة. `roots` = كل شجرة بتبدأ فين في المصفوفات المسطحة (0، وبعدين المجموع التراكمي).
2. `shift` بيكرر مكان بداية كل شجرة لكل الـ nodes بتاعتها، فأرقام الأبناء الخاصة بالشجرة (اللي بتبدأ من 0 في كل شجرة) تتحول لأرقام في المصفوفة المسطحة بإننا نزود الـ shift.
3. `left` / `right` = أبناء كل node. في scikit-learn الورقة ابنها -1. هنا الورقة بتشاور على نفسها:

```
leaf = left < 0
here = np.arange(len(left))
self.left = np.where(leaf, here, left + shift)          # a leaf points to itself, so walking on
keeps it
self.right = np.where(leaf, here, right + shift)
self.feature = np.where(leaf, 0, np.concatenate([t.feature for t in trees]))
```

الحيلة دي معناها إن الشجرة اللي توصل لورقة بدري بتفضل واقفة مكانها، والأشجار الأعمق تكمل مشي، فكل شجرة ممكن نمشيها `depth` خطوة بالظبط من غير حالات خاصة. والـ feature بتاع الورقة بيتحط 0 (أي رقم صحيح ينفع) لأن نتيجة المقارنة هناك مش فارقة: الشمال واليمين الاتنين بيرجعوا لنفس الورقة.

1. `threshold` = قيمة التقسيم لكل node. `proba` = قيم الورقة مقسومة على مجموعها، ودي `predict_proba` بتاع كل شجرة عند الورقة دي. `depth` = عمق أعمق شجرة.

حساب frame واحد:

```
def __call__(self, features):
    x = np.asarray(features, np.float32).ravel()           # sklearn also compares float32 features
    node = self.roots
    for _ in range(self.depth):
        node = np.where(x[self.feature[node]] <= self.threshold[node], self.left[node],
self.right[node])
    return self.proba[node].mean(axis=0)                    # the forest = mean of its trees
```

خطوة خطوة:

1. `node` مصفوفة فيها 300 مكان حالي (واحد لكل شجرة)، بتبدأ من جذور الأشجار.
2. كل لفة في الـ loop بتنزّل الـ 300 كلهم مستوى واحد: لكل شجرة بنبص على الـ feature اللي الـ node بتاعها بيسأل عنه، ونقارنه بالـ threshold، ونروح شمال أو يمين.
3. بعد `depth` خطوة كل شجرة بتكون في ورقة.
4. متوسط probabilities الأوراق = إجابة الـ forest.

الـ features بتتحول لـ float32 لأن scikit-learn كمان بيقارن بـ float32؛ ده بيخلي قرارات شمال/يمين متطابقة. الوقت المقاس: حوالي 0.3 ملي ثانية.

فحص الأمان: `fast_forest(model)` بترجع `ForestOneFrame` بس لو الـ model من نوع `RandomForestClassifier` بمخرج واحد، وإجاباته مطابقة لـ scikit-learn: 16 vector اختبار عشوائي (من seed ثابت،

`np.random.default_rng(0)`، وأول واحد كله أصفار) بيتحسبوا بالطريقتين ويتقارنوا بـ `np.allclose(..., atol=1e-9)`. لو اختلفوا في أي يوم، بتطبع رسالة (`"landmark model: the fast forest does not match sklearn (here, using sklearn (slower)"`) وترجع `None`، و `GestureClassifier` يرجع لـ scikit-learn. ولو الـ model من نوع MLP بترجع `None` على طول (الـ MLP سريع كفاية لوحده).

علاقته بباقي السيستم

`engine.py` بيعمل `GestureClassifier` (لو ملف الـ model موجود) وبينادي `predict` كل frame فيه إيد؛ `fusion.py` بيستلم الـ probabilities و `classifier.labels`. السيرفر يقدر يشارك classifier واحد متحمّل بين كذا جلسة لأنه بيتقري بس (مفيش حاجة بتتكتب فيه).

الملف `shinobi/seal_detector.py`

مسؤول عن إيه؟

بيشغّل الـ YOLOX-Nano hand-seal detector الجاهز (من مشروع Kazuhito00/NARUTO-HandSignDetection، كملف ONNX) بـ onnxruntime على الـ CPU. فيه طبقتين:

- `SealDetector`: اكتشاف واحد على صورة واحدة.
- `SealWorker`: بيشغّل الـ detector على طول في thread لوحده على أحدث frame، بحد أقصى 10 مرات في الثانية.

الـ pre-processing والـ post-processing (التجهيز قبل الـ model والمعالجة بعده) بيعيدوا تنفيذ `yolox_onnx.py` بتاع الـ repository الأصلي.

الـ Data classes

- `Detection`: `sign` (اسم العلامة في المشروع، أو `None` لـ class مفيش جوتسو بيستخدمه)، `class_id`، `score`، `box` = (x0, y0, x1, y1) بالبيكسل.
- `SealResult`: `id` (بيزيد واحد مع كل تشغيل اكتشاف، عشان الـ fusion يفرّق بين نتيجة جديدة ونتيجة حسبها قبل كده)، `time` (وقت التقاط الـ frame اللي اتحسبت عليه)، `latency_ms` (الاكتشاف أخد قد إيه)، `detections` (الأحسن الأول).

الدالة `nms(boxes, scores, iou)`

Greedy non-maximum suppression. الـ detector غالباً بيطلّع كذا مربع متداخلين لنفس الحاجة. الـ NMS بياخد المربع صاحب أعلى درجة، ويشيل كل مربع باقي متداخل معاه بأكتر من `iou`، ويكرر مع اللي بعده. بترجع أرقام المربعات اللي اتحفظت، الأحسن الأول. (IoU: يعني Intersection over Union، مساحة التقاطع بين مربعين مقسومة على مساحة اتحادهم؛ 0 = مش متداخلين خالص، 1 = متطابقين.)

هي class-agnostic: يعني المربعات المتداخلة من classes مختلفة بتلغي بعض برضه، فإيدين واحدة بتطلّع علامة واحدة بس، مش "tiger" و "ram" فوق بعض.

الكلاس `SealDetector`

- `__init__(path, size=416, threads=2)`: بيعمل onnxruntime session على الـ CPU بـ 2 intra-op threads، و 1 inter-op thread، و "allow_spinning" مقفول (الـ threads الفاضية بتنام بدل ما تفضل تلف مستنية وتضيّع CPU cores باقي التطبيق محتاجها). وبيحسب مسبقاً شبكة الـ anchors. YOLOX بيتوقع على تلات شبكات بـ 8 strides و 16 و 32 بيكسل: 52 x 52، و 26 x 26، و 13 x 13 خلية، المجموع 3549 خلية. `grids` فيها (x, y) لكل خلية، و `strides` فيها الـ stride بتاعها.
- `detect(frame, min_score=0.1, iou=0.45)`: الاكتشاف الكامل.

(stride: يعني كل خلية في الشبكة بتغطي كام بيكسل. شبكة stride 8 فيها خلايا صغيرة كتير للحاجات الصغيرة، وstride 32 فيها خلايا كبيرة قليلة للحاجات الكبيرة. 416/8 = 52، 416/16 = 26، 416/32 = 13، و 2704 + 676 + 169 = 3549.)

خطوات `detect`:

1. **Letterbox.** معامل التكبير `r = min(416 / h, 416 / w)` بيحافظ على النسبة. الصورة المتصغّرة بتتحط في الركن الشمال الفوقاني لكانفاس 416 x 416 مليان رمادي بقيمة 114 (زي الكود الأصلي). البيكسلات بتفضل BGR من 0 لـ 255 من غير normalization، لأن ده اللي الـ model اتدرّب عليه.
2. **التشغيل.** الكانفاس بيتحول لـ (3, 416, 416) channels-first، وبياخد بُعد batch، ويتشغّل. المخرج: 3549 صف كل صف 21 رقم: objectness، dx، dy، log w، log h، و 16 درجة class (معدّية على sigmoid خلاص).
3. **الدرجة.** `score = objectness x class score` لكل class؛ بنحتفظ بأحسن class في كل خلية، والخلايا اللي أقل من `min_score` بتتشال.
4. **فك المربعات (decode).** المركز = (dx, dy) + مكان الخلية) x stride؛ الحجم = exp(log w, log h) x stride. بيتحولوا لإحداثيات أركان وبيتقسموا على `r` عشان يرجعوا لبيكسلات الصورة الأصلية.
5. **NMS** عند IoU 0.45، وبعدين `Detection` لكل مربع اتحفظ، والـ class بتاعه بيتحوّل عن طريق `config.SEAL_CLASSES` (`None` للـ classes اللي مش مستخدمة)، والمربع بيتقص على حدود الصورة.

```
xy = (out[keep, :2] + self.grids[keep]) * self.strides[keep]
wh = np.exp(out[keep, 2:4]) * self.strides[keep]
boxes = np.concatenate([xy - wh / 2, xy + wh / 2], 1) / r
```

بيعمل إيه؟ السطر الأول بيحسب المركز: الـ model بيطلع إزاحة صغيرة (dx, dy) جوه الخلية، فبنزود مكان الخلية ونضرب في الـ stride عشان نوصل لبيكسلات. السطر التاني بيحسب العرض والطول (الـ model بيطلع اللوغاريتم بتاعهم، فبنعمل exp). السطر التالت بيحوّل (مركز، حجم) لـ (أقصى شمال فوق، أقصى يمين تحت) ويقسم على `r` يرجع للمقاس الأصلي.

(objectness: يعني درجة "في حاجة هنا أصلاً"، و class score: "لو في حاجة، هي أنهي نوع". sigmoid: دالة بتحوّل أي رقم لرقم بين 0 و 1.)

الكلاس `SealWorker`: الـ thread اللي في الخلفية

الـ loop الأساسي لازم عمره ما يستنى YOLOX (بياخد عشرات الملي ثواني). `SealWorker` بيشغله في thread منفصل:

- `submit(frame, now)` : الـ engine بينـاديها مع كل frame. بتصغّر الـ frame فوراً لعرض 416 بيكسل (وده كمان بيعمل نسخة خاصة، فالـ main thread يقدر يعيد استخدام الـ frame بتاعه)، وبتخزن `(small, w / 416, now)` كشغلانة مستنية (pending job)، وبتبدّل أي شغلانة أقدم لسه متعالجتش. وبعدين بتصحّي الـ thread بـ `notify()`.
- `_run()` : loop الـ thread. الأول بينام لحد `next_start` (تحديد المعدل: بالكتير `SEAL_RATE_HZ` = 10 مرات في الثانية، يعني فترة 0.1 ثانية)، وبعدين بيستنى على الـ condition variable لحد ما يبقى في شغلانة، وياخد أحدث شغلانة، ويشغّل `detector.detect(small)`، ويكبّر المربعات تاني لبيكسلات الـ frame الكامل بالـ scale المتخزن، وينشر `self.result = SealResult(run_id, stamp, latency, detections)`. الـ `stamp` المتخزن هو وقت التقاط الـ frame، **مش** وقت ما الاكتشاف خلص؛ الـ fusion بيستخدمه عشان يحكم على "طزاجة" النتيجة.
- `close()` : بيحط علامة الوقوف، ويصحّي الـ thread، ويستناه (`join`) لحد 2 ثانية.

```
def submit(self, frame, now):
    """Hand over the newest frame (an older one that was not processed yet is dropped)."""
    h, w = frame.shape[:2]
    size = self.detector.size
    small = cv2.resize(frame, (size, round(h * size / w)), interpolation=cv2.INTER_LINEAR)   # also a
copy
    with self._cond:
        self._pending = (small, w / size, now)
        self._cond.notify()
```

لأن `submit` بيصغّر لعرض 416 بالظبط، الـ frame العرضي (landscape) طوله بيبقى أقل من 416، فجوه `detect` المعامل `r` بيطلع 1 ومفيش تصغير تاني (الكود بيتشيك `(nw, nh) == (w, h)` وبينسخ مباشرة). مثال: frame 720 x 1280 بيبقى 416 x 234.

بيستخدم `threading.Condition` عشان الـ thread ينام من غير ما يستهلك CPU طول ما مفيش frame جديد، ويصحى في نفس اللحظة اللي واحد يوصل. (Condition variable: أداة بتخلي thread "يستنى إشارة" من thread تاني بدل ما يفضل يسأل كل شوية.)

علاقته بباقي السيستم

`engine.start_seal_worker()` بيعمل `SealWorker(SealDetector())` لو `models/yolox_nano.onnx` موجود و onnxruntime يتستورد؛ غير كده YOLOX بيبقى مقفول والتطبيق برضه بيشتغل. `engine.py` بينادي `submit` كل frame وبيدّي `seal_worker.result` لـ `SignFusion.update`.

الملف `shinobi/fusion.py`

مسؤول عن إيه؟

بيجمع الاتنين "المتعرّفين" في قرار واحد لكل frame. المخرج بتاعه "vector probabilities": one-hot فيه 1.0 عند العلامة المختارة (أو عند idle) وصفر في كل مكان تاني. (One-hot: يعني خانة واحدة بس "مولّعة" والباقي مطفي.)

كل thresholds الثقة بتتطبق هنا، فالـ stabilizer اللي بعده بيضيف بس قواعد الوقت، وعلامة عمرها ما تتبلّغ مرتين من two models.

ليه اتعمل؟

تخيلي لو YOLOX قال "tiger" والـ classifier قال "tiger" برضه، وكل واحد بعت رأيه للـ controller لوحده: ممكن العلامة تتحسب خطوتين. أو لو واحد قال "tiger" والتاني "clone_seal" في نفس الـ frame: مين نصدّق؟ الـ fusion بيحل المشكلة دي: **قرار واحد بس لكل frame**، وقواعد واضحة مين مسؤول عن إيه.

مين "مالك" أنهي علامات

- YOLOX مالك الـ 7 علامات الرسمية: tiger، snake، horse، ram، monkey، dog، boar.
- الـ landmark classifier مالك `clone_seal` و `rasengan_palm` (`LANDMARK_SIGNS`)، لأن YOLOX معندوش classes ليهم.
- احتياطي (Backup): لو YOLOX مش شغال خالص، الـ classifier مسموحله يبلّغ عن علامة رسمية، بس بـ probability 0.90 أو أكتر (`LANDMARK_BACKUP_PROB`).

الدوال المساعدة

- `hand_boxes(hands, margin=0.25)`: مربع بالبيكسل حوالين كل إيد MediaPipe لقاها (أقل وأكبر قيمة للـ 21 نقطة)، متكبّر بـ 25% من حجمه من كل ناحية.
- `overlaps(a, b)`: بترجع `True` لو مربعين بيتقاطعوا.
- `open_palm(hands, min_gap=0.60)`: بوابة الكف المفتوح (open-palm gate)، هتتشرح تحت.

```
def hand_boxes(hands, margin=0.25):
    """Pixel boxes around the hands MediaPipe found, grown by `margin` of their size."""
    boxes = []
    for hand in hands:
        (x0, y0), (x1, y1) = hand.px.min(axis=0), hand.px.max(axis=0)
        mx, my = (x1 - x0) * margin, (y1 - y0) * margin
        boxes.append((x0 - mx, y0 - my, x1 + mx, y1 + my))
    return boxes


def overlaps(a, b):
    return min(a[2], b[2]) > max(a[0], b[0]) and min(a[3], b[3]) > max(a[1], b[1])
```

`hand_boxes` بتعمل إيه؟ لكل إيد، بتجيب أصغر x و y (الركن الشمال الفوقاني) وأكبر x و y (الركن اليمين التحتاني) من الـ 21 نقطة، وبعدين بتكبّر المربع ربع عرضه وربع طوله من كل ناحية. ليه بنكبّر؟ لأن مربع YOLOX حوالين العلامة ممكن يكون أوسع شوية من نقط MediaPipe، فبنسيب "هامش أمان".

`overlaps` بتعمل إيه؟ مربعين بيتقاطعوا لو في نفس الوقت: الحافة اليمين الأقرب أكبر من الحافة الشمال الأبعد (تقاطع أفقي)، والحافة التحتانية الأقرب أكبر من الفوقانية الأبعد (تقاطع رأسي).

الـ `FusionResult`

الخانة	المعنى
`probs`	one-hot على الـ labels (العلامة المختارة، أو idle).
`sign`	العلامة المختارة أو `"idle"`.

الخانة	المعنى
`source`	`"yolox"` أو `"landmarks"` أو `"landmarks-backup"` أو `"none"`.
`score`	درجة الـ model اللي كسب نفسه (0 لـ idle).
`conflict`	الـ two models الاتنين ادّعوا علامة في الـ frame ده.

الكلاس `SignFusion` والإعدادات بتاعته

الـ constructor بياخد كل الأرقام من config كقيم افتراضية: `labels=config.GESTURES`، `landmark_signs=config.LANDMARK_SIGNS`، `min_score=config.SEAL_MIN_SCORE`، `no_hand_score=config.SEAL_NO_HAND_SCORE`، `confirm=config.SEAL_CONFIRM`، `max_age=config.SEAL_MAX_AGE`، `landmark_min=config.LANDMARK_MIN_PROB`، `landmark_backup=config.LANDMARK_BACKUP_PROB`، `landmark_margin=config.LANDMARK_MIN_MARGIN`، `palm_gap=config.PALM_MIN_THUMB_GAP`.

والحالة (state) اللي بيحتفظ بيها:

- `rejected`: عداد للعلامات اللي اترفضت لأن الـ classifier مش متأكد أو مش كف مفتوح (للتقييم).
- `conflicts`: عداد لعدد المرات اللي الاتنين ادّعوا علامة (للـ HUD والتقييم).
- `_last_id`: آخر نتيجة YOLOX اتحسبت.
- `_streak`: `(seal, results in a row, latest score)`، يعني العلامة الحالية، كام نتيجة ورا بعض، وآخر درجة.

تأكيد علامة YOLOX

علامة YOLOX بتتقبل بس لو التلات قواعد دول اتحققوا:

1. **طازة (Fresh).** النتيجة اتحسبت على frame عمره بالكتير `SEAL_MAX_AGE` = 0.35 ثانية (`now - result.time`). النتيجة القديمة (stale) بتصفّر سلسلة التأكيد. ده بيحمي من إننا نعرض علامة بعد ما المستخدمة خلاص غيرت إيدها، مثلاً لو thread بتاع YOLOX اتأخر.
2. **فوق إيد.** `_seal_from` بتلف على الاكتشافات من الأحسن للأسوأ وتتخطى الـ classes اللي مفيش جوتسو بيستخدمها. الاكتشاف بيتحسب لو المربع بتاعه متقاطع مع مربع إيد من MediaPipe والدرجة على الأقل `SEAL_MIN_SCORE` = 0.70، **أو**، لو MediaPipe مش شايف ولا إيد خالص، لو الدرجة على الأقل `SEAL_NO_HAND_SCORE` = 0.85. الحالة التانية موجودة لأن بعض العلامات (Snake مثلاً) بتخبي الصوابع لدرجة إن MediaPipe بيتوه عن الإيدين؛ ساعتها بنطلب درجة أعلى لأن مفيش إيد تأكد المكان. المربع البعيد عن أي إيد ظاهرة (بوستر على الحيطة، أو وش) عمره ما يتقبل طول ما في إيدين ظاهرين.
3. **مرتين ورا بعض.** نفس العلامة لازم تكسب `SEAL_CONFIRM` = 2 نتايج YOLOX متتالية.

```
def _seal_from(self, result, hands):
    """The best usable seal in one YOLOX result: (sign, score) or None."""
    boxes = hand_boxes(hands)
    for d in result.detections:                          # best first
        if d.sign is None:
            continue                                     # a seal no jutsu uses
        if boxes and d.score >= self.min_score and any(overlaps(d.box, b) for b in boxes):
            return d.sign, d.score
        if not boxes and d.score >= self.no_hand_score:
            return d.sign, d.score
    return None
```

منطق السلسلة (streak) في `_seal` بيتحدّث **بس** لما نتيجة YOLOX **جديدة** توصل (الـ `id` بتاعها مختلف عن `_last_id`)، لأن الـ main loop أسرع من YOLOX، ولولا كده كان هيعد نفس النتيجة كذا مرة:

```
if result.id != self._last_id:                       # a new YOLOX result: update the streak
    self._last_id = result.id
    seal = self._seal_from(result, hands)
    if seal is None:
        self._streak = (None, 0, 0.0)
    elif seal[0] == self._streak[0]:
        self._streak = (seal[0], self._streak[1] + 1, seal[1])
    else:
        self._streak = (seal[0], 1, seal[1])
sign, count, score = self._streak
return (sign, score) if sign is not None and count >= self.confirm else None
```

خطوة خطوة:

1. لو النتيجة جديدة: نسجّل الـ id بتاعها.
2. نجيب أحسن علامة صالحة فيها بـ `_seal_from`.
3. لو مفيش: السلسلة بتتصفّر.
4. لو نفس العلامة اللي في السلسلة: العداد يزيد واحد، والدرجة تتحدث.
5. لو علامة مختلفة: سلسلة جديدة بعداد 1.
6. في كل frame (حتى لو النتيجة مش جديدة): لو العداد وصل `confirm` (2)، نرجّع العلامة؛ غير كده `None`.

مثال: الـ main loop شغال 15 frame في الثانية و YOLOX 10 مرات. نتيجة YOLOX رقم 41 قالت "monkey": العداد = 1، لسه مش متأكدة. الـ frames اللي بعدها قبل النتيجة 42 بيشوفوا نفس الـ id 41، فالعداد مبيزيدش. النتيجة 42 قالت "monkey" برضه: العداد = 2، تأكدت. من دلوقتي كل frame بيرجّع "monkey" لحد ما السلسلة تنكسر.

بمجرد ما تتأكد، العلامة بتتبلّغ في كل frame لحد ما السلسلة تنكسر (نتيجة من غير العلامة، أو علامة مختلفة، أو نتيجة قديمة).

قواعد علامات الـ landmarks: `LANDMARK_MIN_PROB` و `LANDMARK_MIN_MARGIN` وبوابة الكف المفتوح

بنبص على أعلى label عند الـ classifier، `sign`، والـ probability بتاعه `p`:

- لو `sign` هو `clone_seal` أو `rasengan_palm` و `p >= LANDMARK_MIN_PROB` (0.60)، لازم كمان يعدّي اختبار `_sure`:

- **الفرق (Margin).** الفرق بين أعلى probability وتاني أعلى لازم يكون على الأقل `LANDMARK_MIN_MARGIN` (0.30). مثال: 0.62 قصاد 0.35 (فرق 0.27) بيترفض: الـ model متردد بين إجابتين.
- **بوابة الكف المفتوح (لـ `rasengan_palm` بس).** على الأقل إيد واحدة ظاهرة لازم تبيّن كف مفتوح: في الـ 3D world landmarks، المسافة من طرف الإبهام (4) لطرف السبابة (8) مقسومة على حجم الإيد (الرسغ 0 لعقلة الوسطى 9) لازم تكون على الأقل `PALM_MIN_THUMB_GAP` (0.60). البوابة دي اتضافت لأن الـ classifier عمره ما شاف علامة OK (الإبهام لامس السبابة) فكان بيسميها كف. علامات OK بتقيس حوالي 0.3-0.5، والكفوف الحقيقية 0.8-1.2. الإيد اللي معندهاش 3D landmarks صالحة (حجمها قريب من الصفر، أقل من `1e-4`) بتفشل.
- علامة landmark بتفشل في `_sure` بتدّي **ولا علامة** من الـ landmarks في الـ frame ده (وبتزود `self.rejected`، عداد للتقييم). ده مهم لأن الـ frame المتردد بعد كده ميقدرش يتمسك، ولا يتأكد، ولا يقدّم جوتسو.
- لو الـ probability أقل من 0.60، علامة الـ landmark بتتجاهل ببساطة.

```
def open_palm(hands, min_gap=config.PALM_MIN_THUMB_GAP):
    for hand in hands:
        w = np.asarray(hand.world, np.float32)
        size = float(np.linalg.norm(w[9] - w[0]))
        if size > 1e-4 and float(np.linalg.norm(w[4] - w[8])) / size >= min_gap:
            return True
    return False


def _sure(self, landmark_probs, sign, hands):
    top2 = np.sort(np.asarray(landmark_probs, np.float32))[-2:]
    if float(top2[1] - top2[0]) < self.landmark_margin:
        return False
    return sign != "rasengan_palm" or open_palm(hands, self.palm_gap)
```

`open_palm` خطوة خطوة: لكل إيد: بنجيب نقطها الـ 3D، نحسب حجم الإيد (المسافة من الرسغ لعقلة الوسطى)، ولو الحجم صالح ونسبة (مسافة الإبهام للسبابة / الحجم) على الأقل 0.60، يبقى في كف مفتوح فنرجّع `True`. لو ولا إيد عدّت: `False`.

`_sure` خطوة خطوة: `np.sort(...)[-2:]` بيرتب الـ probabilities ويجيب آخر اتنين (يعني أكبر اتنين): `top2[1]` الأكبر و `top2[0]` التاني. لو الفرق أقل من 0.30: مش متأكد. غير كده: لو العلامة مش `rasengan_palm` يبقى تمام، ولو `rasengan_palm` لازم كمان `open_palm` ترجع `True`.

مثال عملي من SHINOBI AI: مستخدمة عملت علامة OK بالغلط. الـ classifier قال `rasengan_palm` بـ 0.80 والتاني idle بـ 0.10، يعني الفرق 0.70 وعدّى. بس `open_palm` حسبت المسافة بين الإبهام والسبابة / حجم الإيد = 0.4، أقل من 0.60، ففشلت. النتيجة: مفيش علامة، و `rejected` زاد واحد، والـ Rasengan مش هيكمل غلط. ده متوافق مع تعريف المشروع: `rasengan_palm` = إيد **واحدة**، كف مفتوح، الصوابع متفرقة.

القرار في `update()`

`update(landmark_probs, landmark_labels, seal_result, hands, now, yolox_running=True)`:

1. لو YOLOX شغال، علامة YOLOX المتأكدة (لو في) بتبقى المرشح (candidate) بالمصدر `"yolox"`.

2. لو الـ classifier طلّع probabilities وأعلى label عنده علامة landmark متأكدة منها: لو لسه مفيش مرشح، بتبقى هي المرشح (`"landmarks"`). لو YOLOX عنده مرشح برضه، ده **تعارض (conflict)**: `conflict = True`، و `self.conflicts` بيزيد (وبيظهر في نص الـ status في الـ HUD)، والدرجة الأعلى بتكسب. (تفصيلة من الكود: الشرط `p > candidate[1]` بعلامة "أكبر من" مش "أكبر من أو يساوي"، يعني لو الدرجتين متساويتين بالظبط، YOLOX هو اللي بيكسب.)
3. احتياطي: لو لسه مفيش مرشح، و YOLOX **مش** شغال، وأعلى label عند الـ classifier علامة رسمية (مش idle، ومش علامة landmark) بـ probability على الأقل 0.90، بتبقى هي المرشح (`"landmarks-backup"`). خلي بالك: لما YOLOX يكون شغال بس ببساطة مش شايف حاجة، الـ classifier عمره ما يقدر يبلّغ عن علامة رسمية.
4. الـ one-hot output بيتبني. لو مفيش مرشح (أو اسمه مش في قايمة الـ labels)، المخرج idle بالمصدر `"none"`.

```
probs = np.zeros(len(self.labels), np.float32)
if candidate is None or candidate[0] not in self.labels:
    probs[self.idle] = 1.0
    return FusionResult(probs, "idle", "none", 0.0, conflict)
probs[self.labels.index(candidate[0])] = 1.0
return FusionResult(probs, candidate[0], candidate[2], candidate[1], conflict)
```

تفصيلة تانية من الكود: لو علامة landmark اترفضت في `_sure`، الكود بيدخل في فرع `rejected` وما بيوصلش لفرع الـ backup خالص (لأنهم `if / elif`). يعني الـ frame ده ممكن يطلع فيه علامة YOLOX بس، أو idle.

ملخص قواعد الـ fusion في جدول

الحالة	النتيجة
YOLOX متأكد (طازة + فوق إيد + مرتين) والـ classifier مش بيدّعي علامة landmark	علامة YOLOX، المصدر `"yolox"`
الـ classifier: `clone_seal` أو `rasengan_palm` بـ p >= 0.60 وفرق >= 0.30 (والكف مفتوح لـ rasengan_palm)	علامة landmark، المصدر `"landmarks"`
الاتنين ادّعوا علامة	conflict، الأعلى درجة يكسب، `conflicts` يزيد
علامة landmark بـ p >= 0.60 بس فرق أقل من 0.30، أو rasengan_palm من غير كف مفتوح	مفيش علامة من الـ landmarks، `rejected` يزيد
YOLOX مش شغال خالص، والـ classifier بيقول علامة رسمية بـ p >= 0.90	العلامة، المصدر `"landmarks-backup"`
أي حاجة تانية	`"idle"`، المصدر `"none"`

علاقته بباقي السيستم

بيتعمل في `engine.py` (بس لو في "متعرّف" واحد على الأقل) وبيتنادى مرة كل frame. الـ `probs` بتاعته بتروح لـ `ActionController.update`. والـ `sign` و `source` و `score` و `conflicts` بيكوّنوا نص الـ status (مثلاً `"sign: tiger [yolox 0.93] conflicts 0"`).

الملف `shinobi/controller.py`

مسؤول عن إيه؟

ده "العقل" اللي بيحوّل القرارات المهزوزة اللي بتيجي frame بـ frame لعلامات متأكدة، وبعدين لتفعيل جوتسو:

```
probabilities -> GestureStabilizer -> confirmed sign events
              -> SequenceMatcher -> jutsu name -> ActionController (cooldown) -> effect name
```

يعني: الـ probabilities بتدخل الـ `GestureStabilizer` فيطلع "علامة متأكدة"، والعلامة دي بتدخل الـ `SequenceMatcher` فيطلع "اسم جوتسو" لما التسلسل يكمل، والـ `ActionController` بيطبق الـ cooldown ويطلع "اسم المؤثر".

الملف فيه تلات كلاسات و data class واحد. هو ميعرفش أي حاجة عن الكاميرا، أو الـ models، أو الرسم، وده بيخليه سهل جداً في الاختبار (`tests/test_sequences.py` وغيره بيختبروه بمدخلات متألّفة، من غير كاميرا خالص).

الكلاس `GestureStabilizer`

الهدف. حتى بعد الـ fusion، الـ frames لوحدها ممكن "ترعش": frame يقول tiger، واللي بعده idle، واللي بعده tiger. الـ stabilizer بيأكد العلامة بس بعد ما تفضل قوية فترة، وبيبلّغ بيها **مرة واحدة**، ومش بيبلّغ بيها تاني لحد ما تختفي بوضوح.

الحالة (State).

- `smoothed`: الـ probability المهدّية لكل label، وبتبدأ 100% idle.
- `candidate`: العلامة اللي بتتمسك دلوقتي (المرشحة).
- `candidate_since`: إمتى بقت مرشحة.
- `latched`: العلامة اللي اتأكدت خلاص ولسه متسابتش. (latch: يعني "ترباس"؛ العلامة "اتقفلت" ومش هتتبلّغ تاني لحد ما تتفك.)

تهدئة EMA مظبوطة على الوقت. الـ EMA (Exponential Moving Average) بيخلط الـ frame الجديد مع القيمة الجارية: `smoothed = w x new + (1 - w) x smoothed`. لو الوزن ثابت لكل frame، سرعة التهدئة هتعتمد على الـ frame rate: عند 15 FPS العلامة محتاجة 3 frames (0.2 ثانية) عشان تعدّي 0.75، بس عند 6 FPS نفس الـ 3 frames بياخدوا 0.5 ثانية، والعلامات القصيرة (حوالي ثانية) كانت بتخلص قبل ما تتأكد. فالوزن بيتحسب من الوقت الحقيقي بين الـ frames:

```
def weight(self, now):
    dt = self.ref_dt if self.last_time is None else min(max(now - self.last_time, 1e-3), 1.0)
    self.last_time = now
    return 1.0 - (1.0 - self.alpha) ** (dt / self.ref_dt)
```

بيعمل إيه خطوة خطوة؟

1. `dt` = الوقت من الـ frame اللي فات. أول مرة (مفيش frame قبله) بياخد الوقت المرجعي `ref_dt`. وبيتحدد بين 1 ملي ثانية و 1 ثانية (`min(max(...), 1.0)`) عشان أي رقم شاذ ميبوّظش الحساب.
2. `alpha` = 0.4 هو الوزن عند المعدل المرجعي 15 FPS (`ref_dt` = 1/15 ثانية).

3. المعادلة `1 - (1 - alpha) ** (dt / ref_dt)`: لو اتنين frames بينهم ضعف المسافة، جزء "الاحتفاظ" (1 - 0.4) بيتطبق مرتين: (0.6)^2 = 0.36، فالوزن الجديد = 1 - 0.36 = 0.64. عند 15 FPS بالظبط المعادلة بتدّي 0.4 بالظبط، نفس السلوك القديم.

ملاحظة: الـ docstring بتاع الكلاس في الكود بيوصف نفس المشكلة بأرقام مختلفة شوية: بيقول إن الـ 3 frames عند 6 FPS بتكلف "0.33 s instead of 0.13 s"، والنسخة الإنجليزية من الدليل بتقول 0.5 ثانية بدل 0.2. الفرق بس في طريقة العد (هل بنعد الفترات بين الـ frames ولا الـ frames نفسها)؛ الفكرة واحدة والكود هو هو.

مثال عند 15 FPS: الـ fusion بيقلب من idle لـ "tiger" (one-hot). قيمة tiger المهدّية بتمشي كده: 0.40، 0.64، 0.784. في الـ frame التالت بتعدّي `CONF_ON` (0.75)، و idle بيكون نزل لـ 0.216.

الـ frame	tiger المهدّية	idle المهدّية	قوية؟
1	0.40	0.60	لأ
2	0.64	0.36	لأ
3	0.784	0.216	أيوه (>= 0.75 والفرق 0.568 >= 0.25)

`update(probs, now)` خطوة خطوة:

1. لو `probs is None` (مفيش إيدين، أو الـ engine لسه مش جاهز) بيتعامل معاها كأنها 100% idle.
2. بيطبق الـ EMA بالوزن المحسوب من الوقت.
3. **الفك بالـ hysteresis.** لو في علامة latched وقيمتها المهدّية نزلت تحت `CONF_OFF` (0.45)، الـ latch بيتفك. استخدام threshold للـ "off" أقل من الـ "on" بيمنع الرعشة حوالين threshold واحد. (Hysteresis: يعني "تأخر الرجوع"؛ زي الترموستات اللي بيفتح التكييف عند 26 ويقفله عند 24، مش عند 25 في الحالتين، عشان ميفضلش يفتح ويقفل كل ثانية.)
4. بيلاقي الـ label الأول والتاني. الأول بيبقى **قوي (strong)** لو مش idle، وقيمته المهدّية على الأقل `CONF_ON` (0.75)، وسابق التاني بفرق على الأقل `MIN_MARGIN` (0.25).
5. مش قوي: المرشح بيتمسح، ويرجّع `None`.
6. قوي بس علامة مختلفة عن المرشح الحالي: بتبقى المرشح الجديد و `candidate_since = now`، ويرجّع `None`.
7. قوي، ونفس المرشح، ومش latched خلاص، واتمسك `HOLD_SECONDS` (0.30 ثانية): بيعمله latch ويرجّع اسمه. **دي اللحظة الوحيدة اللي علامة متأكدة بتتبلّغ فيها.**

```
order = np.argsort(self.smoothed)[::-1]
top, second = order[0], order[1]
strong = (top != self.idle and self.smoothed[top] >= self.conf_on
          and self.smoothed[top] - self.smoothed[second] >= self.margin)

if not strong:
    self.candidate = None
    return None
if top != self.candidate:
    self.candidate, self.candidate_since = top, now
    return None
if top != self.latched and now - self.candidate_since >= self.hold:
    self.latched = top
    return self.labels[top]
return None
```

(`np.argsort(...)[::-1]` بيرتب أرقام الـ labels من الأكبر قيمة للأصغر.)

طول ما المستخدمة ماسكة العلامة، بتفضل latched ومفيش حاجة تانية بترجع. بعد ما تنزل تحت 0.45 وترجع قوية لمدة 0.3 ثانية، بتتأكد تاني؛ و `SequenceMatcher` بيعاملها كتكرار (repeat)، هنشرح تحت.

`hold_progress(now)` بترجع رقم من 0 لـ 1: المرشح الحالي وصل لفين في طريقه للتأكيد (0 لو مفيش مرشح أو latched خلاص). الـ HUD في الـ Desktop بيرسمها كشريط تقدم.

الدالة `_in_order(small, big)`

بترجع `True` لو كل عنصر في `small` موجود في `big` بنفس الترتيب، مع السماح بفجوات بينهم.

```
def _in_order(small, big):
    rest = iter(big)
    return all(item in rest for item in small)
```

بتستخدم حيلة Python حلوة: `item in rest` على iterator بتستهلك الـ iterator لحد العنصر اللي لقته، فكل بحث بعد كده بيكمل من بعد آخر تطابق. (iterator: يعني حاجة بتمشي على العناصر واحد واحد ومبترجعش لورا.) مثال: `_in_order(["a", "c"], ["a", "b", "c"])` = `True`، بس `_in_order(["c", "a"], ["a", "b", "c"])` = `False`.

الكلاس `SequenceMatcher`

الهدف. يتابع كل تسلسلات الجوتسو في نفس الوقت ويقول إمتى واحد يكمل.

الحالة، لكل جوتسو:

- `steps[name]`: عدد الخطوات اللي اتعملت (0 لـ 3).
- `last_step[name]`: وقت آخر خطوة ليه، أو وقت آخر محاولة غلط للـ leader (هنشرح تحت).
- `first_step[name]`: وقت أول خطوة في المحاولة الحالية.

حالة مشتركة:

- `history`: آخر العلامات المتأكدة (للـ HUD في الـ Desktop)، بحد أقصى طول أطول تسلسل (4).
- `last_sign`: آخر علامة متأكدة (عشان نعرف التكرار).
- `repeated`: الـ push الأخير كان تكرار.
- `advanced`: الجوتسو اللي اتقدموا في الـ push الأخير.

فحص وقت التشغيل: `_check_conflicts()`. بيرفض الإعدادات اللي عمرها ما هتشتغل، بإنه يرمي `ValueError`:

- تسلسل فيه نفس العلامة مرتين ورا بعض (العلامة الممسوكة بتتحسب مرة واحدة، فعمره ما هيتعمل).
- جوتسوين بنفس التسلسل بالظبط.
- جوتسو التسلسل بتاعه كله ظاهر بالترتيب جوه تسلسل جوتسو تاني قبل آخر علامة فيه (بيتشيك بـ `_in_order(inner, seq[:-1])`). الجوتسو الداخلي كان هيتطلق في النص ويصفّر كل حاجة، فالخارجي عمره ما كان هيكمل.

مثال تخيلي: لو كان في جوتسو `["ram", "tiger"]` وجوتسو تاني `["ram", "snake", "tiger", "clone_seal"]`، الأول موجود بالترتيب جوه أول تلات علامات من التاني، فهيتطلق في النص و shadow_clone مش هيكمل أبداً. `_check_conflicts` كان هيرفض ده فوراً.

قواعد `push(gesture, now)` الأساسية

`push` بتستلم كل علامة متأكدة:

1. **الانتهاء الأول (Expire first).** `expire(now)` بيصفّر أي جوتسو آخر خطوة ليه أقدم من `max_gap` (7 ثواني في الوضع الحر). بيرجّع `{jutsu: steps it had}` مترتبة من الأبعد للأقرب، وبيمسح الـ history لما ميبقاش في حاجة شغالة خالص.
2. **مين بيتقدم.** `nexts` = كل جوتسو العلامة الجاية بتاعته هي العلامة دي. العلامة الواحدة بتقدّم كل جوتسو زي ده في نفس الوقت، فممكن كذا جوتسو يبقوا شغالين مع بعض (مثلاً "tiger" خطوة في تلات تسلسلات).
3. **التكرار (Repeat).** لو العلامة هي هي `last_sign` ومش العلامة الجاية لأي جوتسو، يبقى تكرار: مفيش حاجة بتتغير (مفيش خطوة، والساعة **مش** بتتجدد)؛ بس وقت آخر عنصر في الـ history بيتحدث. التكرار بيحصل طبيعي: العلامة الممسوكة بتتأكد تاني كل ما المتعرّف يقع لحظة (نتيجة YOLOX واحدة فايتة كفاية)، و "ram, snake, tiger" لازم تتحسب "snake, snake, tiger". نفس العلامة **بتتحسب** خطوة جديدة لو كانت العلامة الجاية لجوتسو ما (مثلاً بعد timeout أو retry، لما الجوتسو ده يبدأ من الأول).
4. **العلامة الغلط عمرها ما بتصفّر.** العلامة اللي مش العلامة الجاية لجوتسو بتسيب الجوتسو ده مكانه بالظبط. المستخدمة ببساطة تجرب الخطوة تاني. الوقت بس (أو إكمال جوتسو) هو اللي بيصفّر التقدم.
5. **التقدم (Advance).** لكل جوتسو في `nexts`: يسجّل `first_step` لو كان عند 0، ويزود خطوة، ويحط `last_step = now`. لو وصل لطوله الكامل، يبقى "خلص".
6. **المحاولة الغلط بتعيد ساعة الـ leaders** (قاعدة اتضافت 2026-11-01، تحت).
7. **الإطلاق (Fire).** لو جوتسو واحد أو أكتر خلصوا، أطول تسلسل بيكسب (لو متساويين، بالترتيب في `config.JUTSU`؛ `max` بتحتفظ بأول واحد من القيم المتساوية). وبعدين **كل** التقدم بيتصفّر، فأداء واحد عمره ما يطلق مرتين أو يتحسب بداية جوتسو تاني. `last_sign` بيتحفظ بعد التصفير، فلو فضلتي ماسكة آخر علامة يبقى تكرار، مش أول خطوة في حاجة جديدة.

```
def push(self, gesture, now):
    """Add a confirmed gesture. Returns the jutsu it completes (fires) or None."""
    self.expire(now)
    self.repeated, self.advanced = False, []
    nexts = [name for name in self.jutsu if self._next(name) == gesture]
    if gesture == self.last_sign and not nexts:
        if self.history and self.history[-1][0] == gesture:
            self.history[-1] = (gesture, now)
        self.repeated = True
        return None
    prog = self.progress(now)
    top = max(prog.values())
    leaders = [name for name in self.jutsu if top and prog[name] == top]   # furthest along, before
this sign
    self.last_sign = gesture
    self.history = (self.history + [(gesture, now)])[-self.max_len:]
    done = []
    for name in nexts:
        if self.steps[name] == 0:
            self.first_step[name] = now
        self.steps[name] += 1
        self.last_step[name] = now
        if self.steps[name] == len(self.jutsu[name]["sequence"]):
            done.append(name)
    self.advanced = nexts
    if leaders and not set(leaders) & set(nexts):
        for name in leaders:        # a wrong attempt at their next step: max_gap again to make it
            self.last_step[name] = now
    if not done:
        return None
    best = max(done, key=lambda name: len(self.jutsu[name]["sequence"]))   # ties: config order
    last = self.last_sign
    self.reset()                    # one performance fires once and starts nothing else
    self.last_sign = last           # (holding the last sign is then a repeat)
    return best
```

قاعدة 01-11-2026: المحاولة الغلط بتدّي الـ leaders ساعة 7 ثواني جديدة

قبل القاعدة دي، المستخدمة اللي غلطت في الخطوة 3 ومحتاجة كام ثانية تلاقي العلامة الصح كان ممكن الوقت يخلص عليها رغم إنها بتحاول فعلاً. دلوقتي:

```
prog = self.progress(now)
top = max(prog.values())
leaders = [name for name in self.jutsu if top and prog[name] == top]   # furthest along, before this
sign
...
if leaders and not set(leaders) & set(nexts):
    for name in leaders:        # a wrong attempt at their next step: max_gap again to make it
        self.last_step[name] = now
```

- الـ **leaders** هم كل الجوتسو اللي واصلين لأبعد نقطة **قبل** العلامة دي (أكتر عدد خطوات، وعلى الأقل 1).
- لو العلامة الجديدة مقدّمتش ولا واحد من الـ leaders، يبقى دي محاولة غلط في خطوتهم الجاية، وكل leader بياخد `max_gap` كامل تاني من دلوقتي.
- **كل** الـ leaders بيتجددوا، مش واحد بس، فلو في محاولة قديمة فاضلة ومتساوية مع المحاولة الحقيقية، متقدرش تاخد التجديد منها.

- **مفيش استثناء (تعديل آخر يوم 2026-11-01 بناءً على طلبك):** نسخة قديمة من القاعدة كانت بتستثني الحالة اللي العلامة الغلط فيها بتجيب جوتسو تاني متساوي مع الـ leaders (زي إنك تكوني عند خطوة 1 وتعملي بالغلط أول علامة في جوتسو تاني). في الحالة دي ما كانش فيه وقت جديد، فبعد 7 ثواني التسلسل كان بيرجع من الأول، وده بالظبط اللي كنتي بتشتكي منه. الشرط ده اتشال، ودلوقتي **أي** محاولة غلط بتدي الخطوة اللي عليها الدور 7 ثواني جديدة. ملاحظة: النسخة الإنجليزية من الـ PDF (اللي ما اتغيرتش بناءً على طلبك) لسه بتوصف الاستثناء القديم.
- **التكرار عمره ما بيتحسب محاولة** (بيرجع قبل ما يوصل للكود ده)، فمسك علامة غلط مش بيفضل يمدد الوقت. الحاجة الوحيدة اللي بتضيّع الخطوات هي **التوقف** (7 ثواني من غير علامة جديدة).

مثال (الوضع الحر): عند t = 0 المستخدمة عملت Ram، فـ shadow_clone بقى عنده خطوة واحدة. عند t = 5 عملت Dog بالغلط. Dog مش العلامة الجاية لحد (fire_style مستني chidori، monkey مستني snake، shadow_clone مستني snake، rasengan مستني clone_seal)، فمفيش حاجة اتقدمت، بس shadow_clone هو الـ leader ومتقدمش: الساعة بتاعته بتبدأ من جديد، والمستخدمة دلوقتي معاها لحد t = 12 (مش t = 7) عشان تعمل Snake.

ولو بدل كده عند t = 5 عملت Monkey: fire_style هيروح لخطوة 1، بس shadow_clone (اللي كان الـ leader ومتقدمش) برضه بياخد ساعة جديدة لحد t = 12. وعلى الكارت بيفضل shadow_clone هو اللي ظاهر، لأن التعادل بيروح للجوتسو اللي بدأ الأول. (قبل التعديل الأخير، الحالة دي ما كانش فيها تجديد، و shadow_clone كان بيرجع من الأول عند t = 7.)

الدوال التانية في `SequenceMatcher`

- **`progress(now=None)`** بترجع `{jutsu: steps}`، والجوتسو اللي وقته خلص عند `now` بيتحسب 0. عمرها ما بتوري الطول الكامل، لأن التسلسل الكامل بيتطلق ويتصفّر.
- **`leader(now=None)`** بترجع الجوتسو الأبعد: أكتر خطوات؛ لو متساويين، اللي شغال من أطول وقت (أقدم `first_step`)، وبعدين ترتيب config. قاعدة التعادل دي معناها إن علامة غلط بالصدفة بدأت جوتسو تاني عمرها ما تاخد القيادة من التسلسل اللي المستخدمة بتعمله فعلاً.
- **`time_left(now)`** بترجع عدد الثواني الباقية للـ leader عشان علامته الجاية (`max_gap - (now - last_step)`، وعلى الأقل 0)، أو `None` لو مفيش حاجة شغالة أو في وضع الـ practice.
- **`time_left_all(now)`** بترجع نفس الحاجة لكل جوتسو شغال، كل واحد على ساعته (فاضية `{}` في الـ practice).
- **`expire(now)`** اتشرحت فوق: بتصفّر اللي وقته خلص وترجع `{jutsu: steps it had}`، مترتبة (أكتر خطوات، وبعدين أحدث خطوة) من الأول.
- **`_expired(name, now)`**: `True` لو `now` موجود، و `max_gap` مش `None`، والجوتسو عنده خطوة على الأقل، و `now - last_step > max_gap`.
- **`reset()`** بتنسى كل التسلسلات الشغالة (زراير Retry / Reset، وتغيير الوضع).

```
def leader(self, now=None):
    prog = self.progress(now)
    alive = [name for name in self.jutsu if prog[name] > 0]
    # max() keeps the first of equal keys: alive is in config order
    return max(alive, key=lambda name: (prog[name], -self.first_step[name])) if alive else None
```

إزاي بتشتغل؟ المفتاح `(prog[name], -self.first_step[name])`: الأول عدد الخطوات (الأكتر يكسب)، ولو متساويين، سالب وقت أول خطوة (الأقدم = رقم سالب أكبر = يكسب). ولو لسه متساويين، `max` بتاخد أول واحد في الترتيب، وده ترتيب config.

وضع الـ Practice

`max_gap=None` بيقفل كل حدود الوقت (`_expired` دايماً `False`). تطبيق الويب بيستخدم ده لما المستخدمة تتمرن على جوتسو واحد مختار. **في الـ Practice مفيش تايمر خالص**، مهما المستخدمة أخدت وقت.

الـ `ControllerState`

Data class فيه اللي حصل في frame واحد:

الخانة	المعنى
`confirmed`	العلامة اللي اتأكدت في الـ frame ده (أو `None`).
`repeat`	كانت نفس العلامة تاني (مفيش خطوة جديدة).
`jutsu`	الجوتسو اللي اتطلق (أو `None`).
`effect`	المؤثر اللي المفروض يبدأ (أو `None`).
`history`	آخر العلامات المتأكدة.
`progress`	`{jutsu: steps done}` للـ HUD ووضع الـ practice.
`timed_out`	`{jutsu: steps}` اللي وقتهم خلص في الـ frame ده.
`timeout`	الجوتسو اللي كان الأبعد، لو هو واحد من اللي وقتهم خلص (بيتبلّغ للمستخدمة).
`leader`	الأبعد قبل علامة الـ frame ده (العلامات الغلط بتتبلّغ بالنسبة له).

الكلاس `ActionController`

بيربط الـ stabilizer والـ matcher مع بعض وبيضيف الـ cooldown.

- `__init__(labels, cooldown=1.5, jutsu=config.JUTSU, max_gap=7.0)`: بيعمل `GestureStabilizer` و `SequenceMatcher`. `max_gap=None` = وضع الـ practice. وبيحط `last_fire = -1e9` (يعني "من زمان جداً"، فأول جوتسو مش هيتمنع بالـ cooldown).
- `update(probs, now, seq_now=None)`: frame واحد. `now` هو ساعة الحيطة (wall clock)، وبتستخدم لوقت المسك في الـ stabilizer وللـ cooldown. `seq_now` هي الساعة اللي حد التسلسل بيمشي عليها (الـ engine بيبعت "ساعة التسلسل" بتاعته، شوفي `engine.py`)؛ افتراضياً نفس `now`.

```
furthest = self.matcher.leader()
state.timed_out = self.matcher.expire(t)
state.timeout = furthest if furthest in state.timed_out else None
state.leader = self.matcher.leader()
state.confirmed = self.stabilizer.update(probs, now)
if state.confirmed is not None:
    state.jutsu = self.matcher.push(state.confirmed, t)
    state.repeat = self.matcher.repeated
    if state.jutsu is not None:
        if now - self.last_fire >= self.cooldown:
            self.last_fire = now
            state.effect = self.jutsu[state.jutsu]["effect"]
        else:
            state.jutsu = None   # still cooling down: ignore
```

خطوة خطوة (و `t` هنا = `seq_now` لو موجود، وإلا `now`):

1. يسجّل مين الـ leader **قبل** الانتهاء.
2. يشغّل `expire(t)` ويحفظ اللي وقتهم خلص.
3. لو الـ leader القديم من ضمنهم، يبقى ده الـ `timeout` اللي هيتبلّغ.
4. يسجّل الـ leader الحالي (بعد الانتهاء، وقبل العلامة الجديدة).
5. يدّي الـ probabilities للـ stabilizer.
6. لو في علامة اتأكدت: يدّيها للـ matcher. لو جوتسو اكتمل: لو الـ cooldown خلص، يسجّل وقت الإطلاق ويحدد المؤثر؛ غير كده يتجاهل الجوتسو.

الترتيب مهم: الـ leader بيتسجل **قبل** الانتهاء، عشان الـ timeout يتبلّغ للتسلسل اللي المستخدمة كانت بتعمله. والانتهاء بيشتغل **كل frame** (مش بس لما علامة توصل)، فرسالة "التسلسل فشل" بتظهر أول ما الوقت يخلص. لو جوتسو اكتمل جوه الـ 1.5 ثانية بتوع الـ cooldown، الـ matcher بيكون صفّر تقدمه خلاص بس مفيش مؤثر بيبدأ.

- `cooldown_left(now)`: عدد الثواني الباقية لحد ما جوتسو تاني يقدر يتطلق.

علاقته بباقي السيستم

`engine.py` بيعمل الـ `ActionController` وبينادي `update` كل frame. `main.py` بيقرا الـ stabilizer عشان الـ HUD. خانات الـ `ControllerState` بتروح لـ `events.py` عن طريق السيرفر.

الملف `shinobi/engine.py`

مسؤول عن إيه؟

ده الـ pipeline الكامل بتاع SHINOBI AI لـ stream كاميرا واحد، من غير شباك، ولا كيبورد، ولا HUD. تطبيق الـ Desktop بيعمل `Engine` واحد؛ وسيرفر الويب بيعمل واحد لكل جلسة متصفح، فمستخدمين اتنين عمرهم ما بيشاركوا حالة التعرف.

ليه اتعمل كده؟

لو منطق التعرف كان مكتوب جوه `main.py`، كنا هنضطر نكرره في `server.py`، وأي تحسين في واحد هيتنسي في التاني. الـ `Engine` بيجمع كل حاجة في مكان واحد، والـ Desktop والويب الاتنين مجرد "غلاف" حواليه.

تحذير مهم: الـ `Engine` **مش thread-safe**: كل الدوال بتاعته لازم تتنادى من thread واحد. (thread-safe: يعني ينفع أكتر من thread يستخدموه في نفس الوقت من غير مشاكل.) السبب: objects بتاعة MediaPipe مينفعش two threads يستخدموها في نفس الوقت. الـ segmenter بيشتغل بس على الـ worker thread بتاعه، والشغلانة دي دايماً بتخلص قبل ما `process` ترجع.

الدالة `start_seal_worker()`

بترجع `SealWorker(SealDetector())` شغال، أو `None` مع رسالة مطبوعة بالسبب لما `models/yolox_nano.onnx` يكون مش موجود (`"YOLOX off: ... not found"`) أو onnxruntime ميتستوردش. الـ import جوه الدالة نفسها عشان التطبيق يفضل يشتغل حتى لو onnxruntime مش متسطّب.

```
def start_seal_worker():
    if not config.SEAL_MODEL_PATH.exists():
        print(f"YOLOX off: {config.SEAL_MODEL_PATH} not found")
        return None
    try:
        from shinobi.seal_detector import SealDetector, SealWorker
    except ImportError as e:
        print(f"YOLOX off: {e}")
        return None
    return SealWorker(SealDetector())
```

الـ `EngineResult`

كل اللي frame واحد اتعالج طلّعه:

الخانة	المعنى
`frame`	الصورة الناتجة بالمؤثر، ومعاها الـ landmarks لو مفعّلة (من غير HUD).
`now`	الوقت اللي اتدّى لـ `process()`.
`dt`	الثواني من الـ frame اللي فات، بحد أقصى 0.1 (للمؤثرات والجزيئات).
`fps`	الـ frame rate المهدّي.
`timings`	ملي ثواني لكل مرحلة: `hands`، `model`، `yolox`، `seg`، `fx` (المرحلة اللي مشتغلتش مش بتظهر).
`hands`	objects الـ `Hand`، من الشمال لليمين.
`tracking`	ناتج `AnchorTracker` (الإيد اللي المؤثرات بتتبعها).
`probs`	probabilities الـ classifier (`None`: مفيش model أو مفيش إيد).
`fused`	`FusionResult`: قرار العلامة في الـ frame ده.
`state`	`ControllerState`.
`stabilizer`	الـ `GestureStabilizer` بتاع الـ controller.
`status`	نص زي `"sign: tiger [yolox 0.93] conflicts 0"`.

الخانة	المعنى
`history`	آخر العلامات المتأكدة.
`progress`	`{jutsu: steps done}`.
`time_left`	الثواني الباقية للعلامة الجاية (`None`: مفيش حاجة شغالة، أو practice).
`time_left_per_jutsu`	`{jutsu: seconds left}` لكل جوتسو شغال.
`ready`	العلامات بتتعرف (False طول ما الذكاء الاصطناعي لسه بيجهز).
`cooldown`	الثواني قبل ما جوتسو تاني يقدر يتطلق.
`effect`	المؤثر الشغال بعد الرسم.
`effect_id`	رقم التفعيل (1، 2، ...): تفعيل جديد = رقم جديد.
`effect_jutsu`	الجوتسو اللي بدأ المؤثر.
`effect_phase`	`"voice"` أو `"charge"` أو `"name"` (مراحل التحضير) أو `"release"`.
`effect_phase_left`	الثواني الباقية في مرحلة التحضير دي (`None` في `"release"`).

(`fused` و `state` و `stabilizer` و `status` بيبقوا `None` لو مفيش ولا "متعرّف" متحمّل.)

`Engine.__init__(classifier=AUTO, yolox=True, jutsu=None, draw_landmarks=False)`

بيعمل الـ `HandTracker`، و `PersonSegmenter`، و `ThreadPoolExecutor` بـ thread واحد للـ segmentation، و `AnchorTracker`، و `EffectManager`. وبعدين "المتعرّفين":

- `classifier=AUTO` بيحمّل `GestureClassifier()` لو ملف الـ model موجود، وإلا `None`. السيرفر يقدر يبعت classifier متحمّل خلاص عشان يشاركه، أو `None` عشان يقفله.
- `yolox=True` بيشغّل YOLOX worker خاص بالـ engine ده.
- `SignFusion` و `ActionController` بيتعملوا بس لو في "متعرّف" واحد على الأقل. من غير ولا واحد، التطبيق بيشتغل في "وضع الكيبورد" (المؤثرات بالمفاتيح بس).
- `jutsu`: الجوتسو اللي ممكن يتطلقوا (`None` = كل `config.JUTSU`).

```
if classifier is AUTO:
    classifier = GestureClassifier() if GestureClassifier.exists() else None
self.classifier = classifier
self.seal_worker = start_seal_worker() if yolox else None
self.fusion = SignFusion() if (self.classifier or self.seal_worker) else None
self.jutsu = dict(config.JUTSU if jutsu is None else jutsu)
self.controller = ActionController(config.GESTURES, jutsu=self.jutsu) if self.fusion else None
```

وبيجهز كمان حالة الجاهزية (`ready = False`)، وساعة التسلسل (`seq_clock = 0.0`)، والحسابات الخاصة بالتفعيلات وحجم الـ frame.

`warm_up(timeout=15.0)`

أول تشغيل لـ MediaPipe و YOLOX والـ segmenter بيكون بطيء (مقاس: 1.8 ثانية لأول frame في الجلسة)، والعلامات اللي هتتعمل في الوقت ده كانت هتضيع. `warm_up` بتشغّل كل model مرة على صورة رمادي 640 x 480 قبل أول frame من الكاميرا:

1. اكتشاف إيد مرتين.
2. تشغيل YOLOX مرة (وبتستنى نتيجته لحد 15 ثانية، وبعدين بتمسحها).
3. segmentation مرة (وبعدين بتصفّر ذاكرة الـ segmenter).
4. بعد كده بتعمل `SignFusion` و `ActionController` جداد (مع الحفاظ على حد الوقت الحالي، practice أو حر) و `AnchorTracker` جديد، فمفيش أي حالة من الصور الفاضية بتفضل.

الـ engine لسه **مش** `ready` بعد ده؛ الجاهزية محتاجة frames حقيقية.

دوال التحكم

- `start_effect(name, now, jutsu=None, staged=False)`: بتبدأ مؤثر وبتبدّل اللي شغال. `staged=True` (جوتسو اتطلق من تسلسله) بيضيف مراحل التحضير من `activation_stages(jutsu)`؛ بالمفتاح (Desktop) المؤثر الأساسي بيبدأ على طول. وبتزود رقم التفعيل.
- `stop_effect(now)`: بتخلي المؤثر الشغال يختفي تدريجياً (fade out) (مفتاح 0 في الـ Desktop).
- `set_jutsu(jutsu, practice=False)`: بتختار أنهي جوتسو يقدر يتطلق (الكل = الوضع الحر؛ واحد = practice). بتعمل `ActionController` جديد بـ `max_gap=None` في الـ practice، و `SEQUENCE_MAX_GAP` غير كده. المؤثر الشغال بيكمل.
- `reset_sequence()`: بتنسى التسلسلات الشغالة؛ الـ cooldown والمؤثر بيفضلوا.
- `_new_frame_size(size)`: بتتنادى لما حجم الـ frame يتغير (متصفح غيّر الكاميرا أو لف لوضع طولي). أماكن البيكسلات القديمة ملهاش معنى، فالـ anchor tracker والـ segmenter بيتصفّروا، والـ Shadow Clone الشغال بيخلص (الطبقة بتاعته بالحجم القديم)؛ المؤثرات التانية بتكمل.

```
def set_jutsu(self, jutsu, practice=False):
    self.jutsu = dict(jutsu)
    if self.fusion:
        self.controller = ActionController(config.GESTURES, jutsu=self.jutsu,
                                           max_gap=None if practice else config.SEQUENCE_MAX_GAP)
```

مثال من SHINOBI AI: لما المستخدمة تختار في تطبيق الويب "اتمرني على Fire Style"، السيرفر (الدالة `_set_mode` في `server.py`) بينادي `self.engine.set_jutsu({"fire_style": config.JUTSU["fire_style"]}, practice=True)`، فمبقاش في غير جوتسو واحد يقدر يتطلق ومفيش تايمر.

`process(frame, now)`: frame واحد

```
dt = min(now - self.last_time, 0.1)                # clamp: a hiccup must not explode particles
if self._seq_last is not None:
    self.seq_clock += min(max(now - self._seq_last, 0.0), config.SEQUENCE_CLOCK_MAX_STEP)
self._seq_last = now
self.last_time = now
self.fps = 0.9 * self.fps + 0.1 / max(dt, 1e-3)
```

ساعة التسلسل (The sequence clock). حد الـ 7 ثواني المفروض يعد بس الوقت اللي الذكاء الاصطناعي كان فعلاً شايف المستخدمة فيه. لو السيرفر أو المتصفح هنّج 3 ثواني، الوقت ده مينفعش يضيع منها. فالـ engine بيحتفظ بساعته الخاصة، `seq_clock`، اللي بتتقدم بالوقت الحقيقي بين الـ frames المتعالجة بس عمرها ما بتزيد أكتر من `SEQUENCE_CLOCK_MAX_STEP` (0.4 ثانية) في الـ frame الواحد. عند 2.5 frame في الثانية أو أكتر بتساوي

ساعة الحيطة (لأن الفرق بين كل frame والتاني أقل من 0.4)؛ وأثناء التهنيج تقريباً بتقف. الساعة دي بتتبعت كـ `seq_now` للـ controller، وبتستخدم لـ `time_left`. وقت المسك في الـ stabilizer والـ cooldown لسه بيستخدموا الوقت الحقيقي `now`.

مثال: المستخدمة عملت Monkey، وبعدها السيرفر هنّج 3 ثواني (frame واحد وصل بعد 3 ثواني). ساعة الحيطة زادت 3 ثواني، بس `seq_clock` زادت 0.4 بس. فبدل ما يفضلها 4 ثواني من الـ 7، يفضلها 6.6.

`dt` بيتحدد بـ 0.1 ثانية للمؤثرات، فالتهنيج ميخليش الجزيئات (particles) تنط. والـ FPS عبارة عن EMA (90% قديم، 10% جديد).

وبعدين الأربع مراحل:

1. **الإدراك (Perception).** يبعت الـ frame لـ YOLOX. لو المؤثر الشغال محتاج segmentation، يبعته لـ worker الـ segmentation (فيشتغل بالتوازي مع تتبع الإيد على CPU core تاني). يشغّل `hands_tracker.detect(frame)`.
2. **التعرف (Recognition).** لو في classifier والإيدين ظاهرين: `probs = classifier.predict(extract(*hands_to_arrays(hands), aspect=w / h))`. وبعدين `fusion.update(...)` مع آخر نتيجة YOLOX. وبعدين الـ controller:

```
# not ready yet: the frame is processed (models, smoothing) but counts as no sign
state = self.controller.update(fused.probs if self.ready else None, now, seq_now=self.seq_clock)
if state.effect:
    self.start_effect(state.effect, now, jutsu=state.jutsu, staged=True)
```

السطر ده بيعمل إيه؟ لو الـ engine جاهز، بيدّي الـ controller قرار الـ fusion؛ لو لسه مش جاهز، بيدّيه `None` (يعني idle). وبيدّيه `seq_clock` كساعة للتسلسل. ولو الـ controller قال "شغّل مؤثر"، الـ engine بيبدأه بمراحل التحضير (`staged=True`).

1. **المؤثرات (Effects).** `anchor.update(hands, now)` بيدّي مكان الإيد المهدّي؛ نتيجة الـ segmentation بتتجمع (`seg_job.result()` بتستنى لو لسه مخلصتش)؛ `effects.render(frame, tracking, dt, segmentation)` بترسم المؤثر. لما مفيش مؤثر شغال، ذاكرة التهدئة بتاعة الـ segmenter بتتصفّر.
2. **الطبقة الفوقانية (Overlay).** الـ landmarks بتترسم لو `draw_landmarks` شغال.

في الآخر الـ `EngineResult` بيتملا: قيم الـ controller (الوقت الباقي باستخدام `seq_clock`، الـ history، الـ progress، الـ cooldown) وقيم المؤثر. `effect_id` و `effect_jutsu` بيتملوا بس لو المؤثر الشغال هو نفس الـ object اللي آخر `start_effect` عمله. ولو مفيش controller، الـ `progress` بيبقى `{name: 0}` لكل جوتسو.

بوابة الجاهزية (The readiness gate). في آخر كل frame، طول ما لسه مش جاهز:

```
took = (time.perf_counter() - frame_start) * 1000
self._ready_run = self._ready_run + 1 if took < config.READY_MAX_MS else 0
self.ready = self._ready_run >= config.READY_FRAMES
```

خطوة خطوة: بيحسب الـ frame أخد كام ملي ثانية. لو أقل من 500، العداد بيزيد واحد؛ لو أكتر، العداد بيرجع صفر. لو العداد وصل 5، يبقى جاهز.

الـ engine بيبقى `ready` بعد `READY_FRAMES` (5) frames ورا بعض، كل واحد اتعالج في أقل من `READY_MAX_MS` (500 ملي ثانية)؛ أي frame بطيء بيرجّع العد من الأول. لحد كده الـ controller بياخد `None` (يعني idle)، فمفيش

علامة تتأكد، ولا خطوة تبدأ، ولا حد وقت يشتغل طول ما الكاميرا والـ models لسه بيسخنوا. **بمجرد ما يبقى جاهز، بيفضل جاهز.** تطبيق الويب بيعرض علامة `ready` للمستخدمة.

`close()`

بتوقف YOLOX worker والـ thread pool بتاع الـ segmentation، وتقفل objects الـ MediaPipe، وتشيل المؤثر الشغال (الـ worker thread بتاع الـ Shadow Clone بيخلص معاه). آمن إنها تتنادى مرتين (عن طريق `_closed`).

علاقته بباقي السيستم

بيستورد من `classifier` و `controller` و `effects` و `features` و `fusion` و `hands` و `segmentation` و `tracking`. وبيستخدمه `main.py` (Desktop) و `server.py` (ويب).

الملف `shinobi/events.py`

مسؤول عن إيه؟

بيحوّل اللي الـ engine بيبلّغ عنه frame ورا frame لـ events بروتوكول الويب (`docs/web_protocol.md`، section 3.6). هو منطق صافي: مفيش models، ومفيش صور. السيرفر بيدّي `EventTracker` واحد `FrameView` لكل frame اتعالج، وبيبعت قايمة الـ events اللي رجعت للمتصفح، والمتصفح بيشغّل الأصوات والأنيميشن **بس** على الـ events دي. قواعد التسلسل نفسها موجودة في `SequenceMatcher`؛ الكلاس ده بس "بيحكي" اللي حصل.

ليه اتعمل؟

المتصفح محتاج يعرف "إيه اللي حصل دلوقتي" (اتأكدت علامة؟ خطوة اتعملت؟ جوتسو اتطلق؟) عشان يشغّل صوت مرة واحدة بس. لو السيرفر بعت الحالة الكاملة بس، كان المتصفح هيضطر يقارن كل frame باللي قبله بنفسه، وده ممكن يغلط. `events.py` بيعمل المقارنة دي مرة واحدة في Python، وبيطلع رسايل واضحة.

الـ Data classes

- `EffectView` (frozen، يعني مينفعش يتعدّل بعد ما يتعمل): `key` (رقم التفعيل؛ جوتسو جديد بياخد key جديد حتى لو نفس المؤثر)، `name`، `stopping` (بيختفي تدريجياً)، `clones` (Shadow Clone: عدد الـ clones الظاهرين)، `jutsu`، `phase`، `staged` (ليه مراحل تحضير).
- `FrameView` (frozen): `now`، `confirmed`، `repeat`، `fired`، `fired_effect`، `progress` (بعد الـ frame ده)، `effect` (`EffectView` أو `None`)، `timed_out`، `timeout`، `leader`، `fired_stages` (أطوال مراحل التفعيل اللي اتطلق).

`PHASE_ORDER = ("voice", "charge", "name", "release")`.

الـ events، بالترتيب اللي بتظهر بيه جوه frame واحد

الـ Event	إمتى
`sequence_failed` (reason `"timeout"`)	في الوضع الحر بس: الجوتسو الأبعد وقته خلص ورجع لخطوة 0. فيه الجوتسو، والخطوة اللي وصلها، والعلامة المتوقعة.
`sign`	الـ stabilizer أكد علامة. التكرار بياخد الـ event ده بس.
`wrong_sign`	العلامة مش اللي جاية في التسلسل الشغال. الوضع الحر: للـ leader بس (خطوات >= 1). الـ practice: للجوتسو اللي بيتمرن عليه في أي وقت العلامة مش هي الجاية، حتى عند خطوة 0. مش بيتبعت لو العلامة أطلقت جوتسو.
`step`	واحد لكل جوتسو خطواته زادت مع العلامة دي، ما عدا الجوتسو اللي اتطلق (التسلسل الكامل بيصفّر كل حاجة). فيه رقم الخطوة، والإجمالي، والعلامة.
`effect_end`	مؤثر الـ frame اللي فات مشي (اتبدّل أو اختفى).
`jutsu`	تسلسل كامل اتطلق؛ فيه `stages` (ثواني voice و charge و name) لما يكون في تحضير.
`effect_phase`	التفعيل دخل `"charge"` أو `"name"` أو `"release"` (كل واحدة مرة، بالترتيب؛ المرحلة اللي frame بطيء نطّها بتتبلّغ برضه).
`effect_stopping`	المؤثر بدأ يختفي (دايماً قبل الـ `effect_end` بتاعه).
`clones`	Shadow Clone: عدد الـ clones الظاهرين أكبر من أي وقت قبل كده في التفعيل ده.

ملاحظة دقيقة على الـ `step` من الكود: في الـ frame اللي جوتسو بيتطلق فيه، الـ `SequenceMatcher` بيصفّر **كل** التقدم، فالـ `progress` بعد الـ frame ده كله أصفار، فعملياً **مفيش ولا event** `step` بيطلع في frame الإطلاق، لا للجوتسو اللي اتطلق ولا لأي جوتسو تاني اتقدم بنفس العلامة. النسخة الإنجليزية بتقول "ما عدا الجوتسو اللي اتطلق"، وده صح بس مش كامل.

أمثلة شكل الـ events زي ما الكود بيبنيها:

```
{"type": "sequence_failed", "reason": "timeout", "jutsu": name, "step": steps,
 "expected": self.jutsu[name]["sequence"][steps], "got": None}
{"type": "wrong_sign", "jutsu": name, "step": steps, "expected": seq[steps], "got": view.confirmed}
{"type": "step", "jutsu": name, "step": steps, "total": len(spec["sequence"]), "sign": view.confirmed}
```

الكلاس `EventTracker`

- `__init__(jutsu, practice=None)` و `set_mode(jutsu, practice)`: بيخزنوا الجوتسو اللي ممكن يتطلقوا و id الجوتسو اللي بيتمرن عليه (`None` = الوضع الحر)، وبيصفّروا التقدم.
- `reset()`: بينسى التقدم المتخزن (Retry / Reset).
- `_leader(view, before)`: بيستخدم الـ leader بتاع الـ controller لو كان عنده خطوة على الأقل؛ من غيره (callers قديمة) بياخد الجوتسو اللي عنده أكتر خطوات، وترتيب config لو متساويين.
- `_timeout(view)`: بيبني event الـ `sequence_failed` من `view.timeout` و `view.timed_out` (عمره ما بيحصل في الـ practice، وعمره ما بيحصل لـ 0 خطوات).
- `_wrong_sign(view, before)`: بيقارن العلامة المتأكدة بالعلامة الجاية المتوقعة للـ leader (أو للجوتسو اللي بيتمرن عليه).

- `_effect_events(view)`: بيقارن `EffectView` بتاع الـ frame اللي فات بالجديد (بالـ `key`) عشان يطلع `effect_end`، و `jutsu`، و `effect_phase`، و `effect_stopping`، و `clones`. لو مؤثر اختفى من غير ما يتشاف وهو بيختفي، بيبعت `effect_stopping` الأول عشان المتصفح دايماً ياخد الاتنين.
- `update(view)`: الدالة الأساسية. بتحسب `before` (الخطوات قبل علامة الـ frame ده: التقدم اللي فات، ناقص الجوتسو اللي وقتهم لسه خالص)، وبعدين بتضيف الـ events بترتيب الجدول. التكرار بيطلّع `sign` بس.

```
def update(self, view):
    events = self._timeout(view)
    before = {name: 0 if name in view.timed_out else self.progress.get(name, 0) for name in
self.jutsu}
    if view.confirmed is not None:
        events.append({"type": "sign", "sign": view.confirmed})
        if not view.repeat:         # a repeat is neutral: nothing changes
            events += self._wrong_sign(view, before)
            for name, spec in self.jutsu.items():
                steps = view.progress.get(name, 0)
                if steps > before[name] and name != view.fired:
                    events.append({"type": "step", "jutsu": name, "step": steps,
                                   "total": len(spec["sequence"]), "sign": view.confirmed})
    self.progress = {name: view.progress.get(name, 0) for name in self.jutsu}
    events += self._effect_events(view)
    return events
```

خطوة خطوة: (1) لو في timeout، أول event هو `sequence_failed`. (2) بيحسب `before`. (3) لو في علامة متأكدة: `sign`، وبعدين لو مش تكرار: `wrong_sign` (لو ينطبق) و `step` لكل جوتسو زاد. (4) بيحفظ التقدم الجديد. (5) بيضيف events المؤثرات.

علاقته باقي السيستم

بيستورده `server.py` بس، اللي بيبني `FrameView` من `EngineResult` و `ControllerState`.

الملف `shinobi/filters.py`

مسؤول عن إيه؟

فلتر One Euro (Casiez et al., CHI 2012, https://gery.casiez.net/1euro/). بيهدّي الأماكن اللي بترعش من غير ما يضيف تأخير ملحوظ. أماكن الإيد اللي MediaPipe بيطلعها بتهتز كام بيكسل حتى والإيد ثابتة؛ المؤثرات اللي بتترسم في المكان الخام كانت هترعش.

الفكرة

الفلتر البسيط (low-pass filter)، يعني فلتر بيعدّي التغيرات البطيئة ويقلل السريعة) عنده مشكلة: التهدئة القوية بتقتل الرعشة بس بتخلي المؤثر يتأخر ورا الحركات السريعة؛ والتهدئة الخفيفة بتتبع الحركات السريعة بس بتسيب الرعشة تعدّي. فلتر One Euro بيغيّر كمية التهدئة حسب السرعة: حركة بطيئة = تهدئة قوية، حركة سريعة = تهدئة خفيفة.

الكود

`_alpha(cutoff, dt)` بتحوّل تردد قطع (cutoff frequency، بالـ Hz) والخطوة الزمنية لوزن تهدئة: `tau = 1 / (2 * pi * cutoff)`، `alpha = 1 / (1 + tau / dt)`. الـ cutoff الأعلى بيدّي alpha أعلى، يعني وزن أكبر للقيمة الجديدة (تهدئة أقل).

`OneEuroFilter(min_cutoff=1.5, beta=0.02, d_cutoff=1.0)`؛ لما تناديه بـ `(value, now)`:

```
speed = (value - self.x) / dt
a_d = _alpha(self.d_cutoff, dt)
self.dx = a_d * speed + (1 - a_d) * self.dx

cutoff = self.min_cutoff + self.beta * float(np.linalg.norm(self.dx))
a = _alpha(cutoff, dt)
self.x = a * value + (1 - a) * self.x
```

1. يقدّر السرعة ويهدّيها (بـ cutoff ثابت 1 Hz).
2. الـ cutoff بيكبر مع السرعة: `min_cutoff + beta x |speed|`.
3. يهدّي القيمة بالـ cutoff ده.

أول نداء بيخزن القيمة بس. بيشتغل على أي شكل numpy (نقطة 2D، حجم، اتجاه). `dt` بيتحدد بحد أدنى 1 ملي ثانية. `reset()` بتنسى كل حاجة، فالقيمة الجاية تبدأ من جديد.

مثال: إيدك ثابتة ومكانها بيرعش ±3 بيكسل: السرعة المهدّية صغيرة، فالـ cutoff قريب من 1.5 Hz، والتهدئة قوية والرعشة بتختفي. حركتي إيدك بسرعة 1000 بيكسل في الثانية: الـ cutoff = 1.5 + 0.02 x 1000 = 21.5 Hz، فالتهدئة خفيفة جداً والمؤثر بيلحق إيدك.

علاقته بباقي السيستم

بيستخدمه `tracking.py` (تلات فلاتر لكل tracker). بيقرا `FILTER_MIN_CUTOFF` و `FILTER_BETA`.

الملف `shinobi/tracking.py`

مسؤول عن إيه؟

بيختار الإيد اللي المؤثرات المفروض تتبعها (الـ "anchor"، يعني "المرساة") وبيهدّي مكانها وحجمها واتجاهها. مؤثرات زي الـ Rasengan والـ Chidori والـ Fire Style بتترسم عند الـ anchor ده.

الـ `Tracking` (data class)

الخانة	المعنى
`hands`	كل الإيدين في الـ frame ده.
`now`	الوقت.
`visible`	إيد الـ anchor ظاهرة دلوقتي، أو اختفت من أقل من فترة السماح.
`anchor`	نص الكف المهدّي بالبيكسل.

الخانة	المعنى
`size`	حجم الإيد المهدّي بالبيكسل.
`direction`	متجه وحدة (طوله 1) من الرسغ لعقلة الوسطى: الإيد بتشاور فين.

الكلاس `AnchorTracker`

- `__init__(grace=0.6)`: تلات فلاتر One Euro: المكان (قيم config الافتراضية)، الحجم (`min_cutoff=0.8, beta=0`: ثابت)، الاتجاه (`min_cutoff=1.0, beta=0`).
- `_pick(hands)`: يفضل على الإيد الأقرب للـ anchor اللي فات، لو بعدها أقل من 3 x حجم الإيد اللي فات (وعلى الأقل 40 بيكسل)، عشان المؤثر مينطش بين الإيدين. غير كده ياخد الإيد الأكبر (`size_px`)، واللي غالباً الأقرب للكاميرا.
- `update(hands, now)`: لو في إيدين: يختار واحدة، يحسب اتجاه الرسغ لعقلة الوسطى، يطبّعه (يخلي طوله 1)، يفلتره (ويطبّعه تاني لأن الفلترة بتغير الطول)، يفلتر نص الكف والحجم، ويسجّل `last_seen`. من غير إيدين: يحتفظ بالـ anchor والحجم والاتجاه القديمين، و `visible` بتفضل `True` لمدة `HAND_LOST_GRACE` (0.6) ثانية؛ بعد كده كل الفلاتر بتتصفّر عشان الإيد الجاية تبدأ من جديد بدل ما "تتزحلق" من المكان القديم.

```
def _pick(self, hands):
    if self.last.anchor is not None:
        dists = [np.linalg.norm(h.palm_center - self.last.anchor) for h in hands]
        best = int(np.argmin(dists))
        if dists[best] < 3 * max(self.last.size, 40):
            return hands[best]
    return max(hands, key=lambda h: h.size_px)   # otherwise the biggest (closest) hand
```

مثال: المستخدمة عاملة Rasengan على إيدها اليمين، وإيدها الشمال دخلت الكادر. الإيد اليمين لسه قريبة من الـ anchor اللي فات، فالمؤثر بيفضل عليها ومش بينط للشمال حتى لو الشمال أكبر.

علاقته بباقي السيستم

`engine.py` بينادي `anchor.update` كل frame وبيدّي الـ `Tracking` للمؤثرات.

الملف `shinobi/segmentation.py`

مسؤول عن إيه؟

بيفصل الشخص عن الخلفية باستخدام الـ selfie segmenter بتاع MediaPipe. (Segmentation: يعني تقسيم الصورة لأجزاء؛ هنا "شخص" و "خلفية".) مؤثر الـ Shadow Clone بس اللي محتاجه (عشان يقص الشخص من الصورة ويلزق نسخ منه). الـ engine بيشغّله بس طول ما المؤثر ده شغال، على worker thread.

الكلاس `PersonSegmenter`

- `__init__(work_width=640, smoothing=0.5)`: بيعمل `ImageSegmenter` بتاع MediaPipe في وضع VIDEO، وبيطلب confidence mask (قيمة من 0 لـ 1 لكل بيكسل: قد إيه البيكسل ده "شخص").
- `_timestamp()`: timestamps بالملي ثانية بتزيد على طول، نفس حيلة `HandTracker`.

- `segment(frame_bgr)` : بترجع `(small_frame, mask)` ، الاتنين بعرض 640 بيكسل، والـ mask نوعه float32 من 0 لـ 1 (1 = شخص).

خطوات `segment` :

1. تصغير الـ frame لعرض 640 بيكسل (للسرعة) وتحويله لـ RGB؛ تشغيل الـ segmenter؛ نسخ الـ mask.
2. **EMA زمني.** `mask = 0.5 x new + 0.5 x previous` (بس لو الـ mask اللي فات بنفس الشكل). بيشيل الرعشة بين الـ frames.
3. **Guided filter.** `cv2.ximgproc.guidedFilter(gray, mask, 6, 1e-3)` بيلزق حواف الـ mask على الحواف الحقيقية في الصورة (الشعر، الكتاف). الدليل (guide) الرمادي أسرع حوالي الضعف من الملون. الـ guide لازم يكون متقسّم لـ 0..1: بقيم 0..255، الـ local variance بتاع المناطق السادة كان بيفقد الدقة في float32 والـ mask كان بيتملي NaN (مقاس على 82 من 180 frame متسجّل؛ 0 مع الـ guide اللي 0..1).
4. **منحنى تباين ناعم.** `clip((mask - 0.3) / 0.4, 0, 1)` : اللي تحت 0.3 بيبقى 0، واللي فوق 0.7 بيبقى 1، وحافة ناعمة في النص.

```
gray = cv2.cvtColor(small, cv2.COLOR_BGR2GRAY).astype(np.float32) / 255.0
mask = cv2.ximgproc.guidedFilter(gray, mask, 6, 1e-3)
mask = np.clip((mask - 0.3) / 0.4, 0.0, 1.0)
```

(Guided filter: فلتر بيهدّي صورة (هنا الـ mask) بس بيحترم الحواف اللي موجودة في صورة تانية "دليل" (هنا الصورة الرمادية)، فالـ mask بيلزق على حدود الشخص الحقيقية. `6` هو نصف قطر الفلتر و `1e-3` هو الـ eps، يعني قد إيه بيهدّي.)

- `reset()` : بتنسى الـ mask اللي فات (بتتنادى لما مفيش مؤثر شغال أو حجم الـ frame اتغير).
- `close()` : بتقفل MediaPipe.

علاقته بباقي السيستم

بيعمله `engine.py` ، وبيستخدمه `effects/shadow_clone.py` عن طريق الـ engine.

الملف `shinobi/hud.py`

مسؤول عن إيه؟

بيرسم معلومات الـ debug على شباك الـ Desktop: الـ FPS، والتوقيتات، والـ probability المهدّية لكل label، وتقدم المسك، وتاريخ العلامات، والمؤثر الشغال، والـ cooldown، والرسايل. تطبيق الويب **مش** بيستخدمه (صفحة React بترسم الواجهة بتاعتها بنفسها).

الدالة `text(frame, msg, org, scale, color, thick)`

بترسم نص بحدود غامقة (outline) عشان يتقري على أي خلفية. في OpenCV 5 الـ `thickness` بتاع `putText` بقى بيختار بس بين حروف عادية أو bold ومبقاش بيعرّض الخط، فالحيلة القديمة (نص أسود تخين تحت النص الملون) بطّلت تعمل حدود. الطريقة الجديدة:

- `_rendered(msg, scale, thick)` (متخزنة في cache بـ `lru_cache`، لأن أغلب نصوص الـ HUD بتتكرر كل frame) بترسم النص في mask، وتكبّر الـ mask بـ 2-3 بيكسل (`cv2.dilate`، وده الحد)، وترجّع mask النص، و mask "احتفظ" مقلوب، والـ padding، والارتفاع.
- `text` بتقص المنطقة عند حدود الـ frame، وتغمّق الصورة تحت النص والحد (`multiply` بالـ keep mask)، وبعدين تضيف اللون من خلال mask النص الناعم (anti-aliased)، وكله بعمليات OpenCV بـ 8-bit.

(cache: يعني تخزين نتيجة حسبة عشان لو احتجناها تاني منحسبهاش من الأول. `lru_cache(maxsize=512)` بيحتفظ بآخر 512 نتيجة.)

```
roi[:] = cv2.add(cv2.multiply(roi, keep[part], scale=1 / 255), cv2.multiply(paint, fill[part], scale=1 / 255))
```

السطر ده بيعمل إيه؟ الجزء الأول (`roi x keep / 255`) بيخلي البيكسلات تحت النص والحد سودا وبيسيب الباقي زي ما هو. الجزء التاني (`paint x fill / 255`) بيحط لون النص بس مكان الحروف. وبعدين بيجمعهم.

الدالة `draw_hud(frame, fps, timings, labels, probs, candidate, latched, hold, history, effect, cooldown, message)`

- فوق على الشمال: الـ FPS (أخضر) وتوقيت كل مرحلة بالملي ثانية.
- رسم أعمدة (bar chart) للـ probabilities المهدّية: العلامة الـ latched بالأصفر، والمرشحة بالأخضر، والباقي رمادي؛ وتحتها شريط تقدم المسك.
- تحت: `"seals: ram > snake > ..."` (الـ history)، وسطر رسالة، ومساعدة المفاتيح (`"1-4 effects 0 stop L landmarks H hud F fullscreen Q quit"`).
- فوق في النص: اسم المؤثر الشغال بحروف كبيرة؛ فوق على اليمين: الـ cooldown.

علاقته بباقي السيستم

`main.py` بس اللي بينادي `draw_hud`.

الملف `shinobi/main.py`

مسؤول عن إيه؟

تطبيق الـ Desktop: الكاميرا، والشباك، والكيبورد، والـ HUD حوالين الـ `Engine`. بيشتغل بـ `python -m shinobi.main`. (الهدف النهائي للمشروع هو تطبيق الويب؛ شباك OpenCV متساب كأداة تطوير و debug، ومش هيتشال لحد ما الذكاء الاصطناعي يبقى مستقر.)

الدالة `main()`

1. بتعمل الـ `Camera` و `Engine(draw_landmarks=True)`. بتحط رسالة بداية حسب اللي اتحمّل: مفيش رسالة لو الـ landmark model موجود؛ `"No landmark model yet: only the 7 official seals (YOLOX) + keys 1-4"` لو YOLOX بس؛ `"No trained model yet: keyboard mode (press 1-4)"` لو ولا واحد.

2. بتفتح شباك بيتغير حجمه 1280 x 720.
3. الـ loop:

- **الكيبورد في كل لفة.** `cv2.waitKey(1)` بيتشيك حتى لو مفيش frame جديد وصل، وإلا ضغطات المفاتيح اللي في النص كانت هتضيع. Q أو Esc بيقفل؛ 1-4 بيشغّلوا مؤثر مباشرة (`EFFECT_KEYS`)؛ 0 بيوقفه؛ L بيشغّل/يقفل الـ landmarks؛ H بيشغّل/يقفل الـ HUD؛ F بيشغّل/يقفل الـ fullscreen. قفل الشباك بزرار X برضه بينهي الـ loop.
- **frame جديد بس.** `camera.read()` بتدّي `(frame_id, frame)`؛ لو الـ id زي آخر واحد اتعالج، الـ loop بيكمل من غير معالجة، فنفس الـ frame عمره ما يتعالج مرتين.
- **المعالجة.** `r = engine.process(frame, time.perf_counter())`.
- **الـ HUD.** لو ظاهر، `draw_hud` بتاخد الـ labels بتاعة الـ stabilizer، والـ probabilities المهدّية، والمرشح، والعلامة الـ latched، وتقدم المسك، والـ history، واسم المؤثر (ومعاه مرحلة التحضير بين قوسين، مثلاً `"rasengan (charge)"`)، والـ cooldown، ورسالة بتجمع رسالة البداية، والـ status بتاع الـ engine، و (للـ Shadow Clone) تحذير "not enough room … step back" لما المؤثر يقول إنه زحمة.
- `cv2.imshow` بيعرض الـ frame.

1. `finally`: دايماً بتقفل الـ engine، وتقفل الكاميرا، وتقفل الشبابيك، حتى لو حصل error.

```
frame_id, frame = camera.read()
if frame is None or frame_id == last_id:     # no new frame yet
    continue
last_id = frame_id
now = time.perf_counter()
r = engine.process(frame, now)
```

بيعمل إيه؟ بياخد أحدث frame من thread الكاميرا. لو لسه مفيش frame خالص، أو هو نفس اللي اتعالج، يرجع لأول الـ loop (ويتشيك على الكيبورد تاني). غير كده، يسجّل الـ id ويبعت الـ frame للـ engine بالوقت الحالي.

(`time.perf_counter()`: ساعة دقيقة جداً في Python مخصوصة لقياس الأوقات القصيرة.)

علاقته بباقي السيستم

بيستخدم `camera.Camera` و `engine.Engine` و `hud.draw_hud` و config. هو الملف الوحيد اللي بيستخدم الكاميرا والـ HUD.

الملف `shinobi/catalog.py`

مسؤول عن إيه؟

نصوص العرض للعلامات والجوتسو، والـ credits (حقوق المصادر)، ودوال مساعدة لملفات الصوت. بيانات صافية، مفيش صور ولا models. نسخة واحدة بيستخدمها برنامجين:

- `tools/build_gesture_references.py`: بيرسم الكروت المرجعية في `gesture_references/`.
- `server.py`: بيبعتها لتطبيق الويب عن طريق `/api/jutsu` و `/api/credits`.

لو غيرتي نص هنا وشغلتي الأداة تاني، الكروت وتطبيق الويب بيفضلوا متطابقين. **ترتيب الخطوات مش متخزن هنا**؛ ببيجي من `config.JUTSU`. (endpoint: يعني عنوان في السيرفر المتصفح يقدر يطلب منه بيانات، زي `/api/jutsu`.)

المحتوى

- `COMMONS_CREDIT`، `PALM_CREDIT`: سطور الـ credit المطبوعة على الكروت.
- `SIGNS`: لكل علامة من الـ 9 اللي التسلسلات بتستخدمها: `name` للعرض، والاسم الياباني `jp`، وعدد `hands` (2، ما عدا `rasengan_palm` بـ 1)، و `official`، و `cues` (تعليمات قصيرة بتظهر على كارت الخطوة). مثلاً `rasengan_palm`: `"ONE hand, palm facing the camera"`، `"Fingers straight and spread apart"`، `"In front of your chest / shoulder"`، `"Other hand down, out of the frame"`.
- `JUTSU`: لكل جوتسو: `title` الكارت، و `color` الهيدر، و `tag`، و `status` (التسلسل جاي منين؛ مثلاً Fire Style بيستخدم آخر 4 من الـ 6 علامات الرسمية Tiger، Horse، Boar، Monkey، Ram، Snake، و Chidori آخر 4 من نسخة الأنمي اللي فيها 9 علامات)، ولتطبيق الويب `name` و `kanji` و `description`. الأوصاف بتحط المدد من `config.EFFECT_DURATION` وعدد الـ clones من `config.CLONE_COUNT`، فبتفضل صح لو اتغيروا.
- `CREDITS`: قايمة أقسام (رسومات العلامات، المؤثرات الصوتية، الموسيقى، model الـ YOLOX، إخلاء المسؤولية)، كل قسم فيه عناصر متبنية بـ `_item(text, url, license, license_url)`. الثوابت `CC_BY_SA_4` و `CC_BY_4` و `CC_BY_3` و `CC0` فيها روابط الرخص. عنصر YOLOX بيوضح إن الـ weights ملهاش رخصة صريحة، فالـ model بيستخدم لأغراض شخصية وتعليمية بس.

مثال من الكود لوصف بيستخدم config:

```
description=(f"Ten copies of you pop up one after another, each in a puff of smoke: "
             f"{config.CLONE_COUNT + 1} people on screen. ...")
```

هنا `config.CLONE_COUNT + 1` = 11، فالنص بيقول "11 people on screen" تلقائياً.

ملفات الصوت (Voice clips)

مفيش ملفات صوت مرفقة مع المشروع؛ المستخدمة ممكن تحط تسجيلات مرخّصة في `assets/audio/voice/<jutsu>/activation.wav` و `name.wav` (PCM WAV بس).

- `VOICE_PARTS = ("activation", "name")`، `VOICE_FORMATS = (".wav",)`، `VOICE_SPEAKER` (Naruto لـ fire_style و shadow_clone و rasengan؛ Sasuke لـ chidori).
- `wav_seconds(path)`: طول ملف PCM WAV بالثواني باستخدام module `wave` في Python، أو `None` لو مش موجود أو مش بيتقري.
- `voice_seconds(jutsu, audio_dir=None)`: `{part: seconds or None}` لجوتسو واحد. `effects/__init__.py` (`activation_stages`) بيستخدمها عشان مرحلتي "voice" و "name" ياخدوا بالظبط طول الملف زائد `VOICE_GAP`، أو `VOICE_DEFAULT` لو مفيش ملف.
- `voice_clips(audio_dir=None)`: لكل جوتسو: المتكلم، ورابط كل ملف متركّب (أو `None`) وطوله؛ بيتبعت لتطبيق الويب عشان يشغّل بس الملفات الموجودة.

```
def wav_seconds(path):
    import wave
    try:
        with wave.open(str(path), "rb") as f:
            rate = f.getframerate()
            return f.getnframes() / rate if rate > 0 else None
    except (OSError, EOFError, wave.Error):
        return None
```

بيعمل إيه؟ بيفتح الملف، يجيب عدد العينات في الثانية (`rate`) وعدد العينات الكلي، ويقسمهم = الطول بالثواني. لو أي حاجة غلط (الملف مش موجود، أو مش WAV سليم) يرجّع `None` بدل ما البرنامج يقع.

مثال: لو حطيتي `assets/audio/voice/fire_style/activation.wav` طوله 1.2 ثانية، مرحلة "voice" في Fire Style هتاخد 1.2 + 0.05 = 1.25 ثانية. من غير الملف: 0.4 ثانية.

علاقته بباقي السيستم

بيقراه `server.py` و `effects/__init__.py` و `tools/build_gesture_references.py`. وبيقرا `config.EFFECT_DURATION` و `config.CLONE_COUNT` و `config.AUDIO_DIR`.

الخلاصة: إزاي العلامة بتبقى جوتسو (مثال Fire Style كامل)

خلينا نمشي مع مثال كامل من أول علامة لحد المؤثر النهائي. افترضي إن المستخدمة بتعمل Fire Style (Monkey، Boar، Horse، Tiger) في الوضع الحر، بحوالي 15 frame متعالج في الثانية، بعد ما الـ engine بقى جاهز (`ready`).

قبل أي علامة: الجاهزية

لما الجلسة بتبدأ، `warm_up` بتسخّن الـ models على صورة رمادي. بعدها أول 5 frames حقيقية اتعالجوا كل واحد في أقل من 500 ملي ثانية، فالـ engine بقى `ready = True`. قبل كده، أي علامة كانت هتتعامل كـ idle، ومفيش تايمر كان ممكن يبدأ.

الخطوات

1. **المستخدمة بتعمل Monkey.** MediaPipe بيلاقي الإيدين؛ YOLOX (10 مرات في الثانية) بيرجّع `"monkey"` بدرجة 0.91، والمربع بتاعه متقاطع مع مربع إيد. أول نتيجة زي دي بتبدأ سلسلة (streak) بـ 1؛ التانية المتتالية بتأكدها (`SEAL_CONFIRM = 2`). من ساعتها الـ fusion بيطلّع one-hot `"monkey"` (المصدر `"yolox"`) كل frame.
2. **الـ stabilizer.** قيمة monkey المهدّية بتطلع 0.40، 0.64، 0.78. عند 0.78 بتبقى قوية (فوق 0.75، والفرق فوق 0.25) وبتبقى المرشحة. بعد 0.30 ثانية بتتعمل latch و `"monkey"` بتتأكد **مرة واحدة**.
3. **`SequenceMatcher.push("monkey")`** monkey هي العلامة الجاية لـ fire_style (عند خطوة 0) بس chidori مستني snake، shadow_clone مستني ram، rasengan مستني clone_seal)، فـ fire_style بيروح لخطوة 1 والساعة بتاعته (7 ثواني على `seq_clock`) بتبدأ. تطبيق الويب بياخد event `sign` و event `step` (`"jutsu":"fire_style", "step": 1, "total": 4`).
4. **المستخدمة بتغير إيديها.** وهي بتغير، الـ fusion بيطلع idle؛ قيمة monkey بتنزل تحت 0.45 والـ latch بيتفك.

5. **Boar و Horse** بيمشوا بنفس الطريقة: fire_style بيروح لخطوة 2، وبعدين 3، وكل خطوة بتعيد ساعة الـ 7 ثواني بتاعته. لو المستخدمة كانت عملت Dog بالغلط وهي بتحاول الخطوة التالتة (Horse)، مكانش حاجة اتصفّرت، وكان event `wrong_sign` هيتبعت (`"expected": "horse", "got": "dog"`)، وساعة fire_style كانت هتبدأ من جديد، فتاخد 7 ثواني زيادة عشان تعمل Horse (قاعدة 2026-11-01).
6. **Tiger بتتأكد.** fire_style بيوصل 4 خطوات ويخلص. لأنه الجوتسو الوحيد اللي خلص، بيتطلق؛ وكل التقدم بيتصفّر. الـ cooldown عدّى، فـ `state.effect = "fire"`.
7. **الـ engine بينادي** `start_effect("fire", now, jutsu="fire_style", staged=True)`: المؤثر بيتحضّر (voice، name، charge) وبعدين بيطلق النار عند الـ anchor المهدّي بتاع الإيد لمدة 6 ثواني (`EFFECT_DURATION["fire"]`).

الـ events اللي المتصفح بياخدها في الإطلاق وبعده

- في frame الـ Tiger: `sign` (`"sign": "tiger"`)، ومفيش `step` (لأن التقدم اتصفّر)، ومفيش `wrong_sign` (العلامة أطلقت جوتسو)، وبعدين `jutsu` (`"jutsu": "fire_style", "effect": "fire"`) ومعاه `stages`، مثلاً `{"voice": 0.4, "charge": 1.2, "name": 0.4}` لو مفيش ملفات صوت.
- في الـ frames اللي بعدها: `effect_phase` بـ `"charge"`، وبعدين `"name"`، وبعدين `"release"` (المؤثر الأساسي بدأ).
- بعد حوالي 6 ثواني: `effect_stopping`، وبعدين `effect_end`.

الرسم الزمني للمثال

```
t (seq_clock)   event                        fire_style steps   deadline
0.0             monkey confirmed              1                  7.0
2.5             boar confirmed                2                  9.5
4.0             dog (wrong) -> wrong_sign     2                  11.0  (clock restarted)
6.0             horse confirmed               3                  13.0
7.5             tiger confirmed -> FIRE       0 (reset)          -
7.5 .. 9.5      build-up: voice 0.4, charge 1.2, name 0.4
9.5 .. 15.5     release: flames for 6 s at the hand anchor
```

(الأرقام في الرسم ده مثال توضيحي للترتيب والقواعد، مش قياس حقيقي.)

ملخص القواعد المهمة (للمراجعة السريعة)

قواعد التسلسل (Sequence rules)

- **حد الـ 7 ثواني في الوضع الحر (Live):** من كل خطوة متأكدة (أو محاولة غلط للـ leader) للخطوة اللي بعدها، بحد أقصى `SEQUENCE_MAX_GAP` = 7.0 ثانية. ده حد، مش انتظار: الخطوة الجاية بتتحسب أول ما تتأكد.
- **الساعة اللي الحد ده بيمشي عليها** هي `seq_clock` بتاعة الـ engine، مش ساعة الحيطة؛ كل frame بيزودها بحد أقصى 0.4 ثانية، فالتهنيج مش بيضيّع وقت.
- **لما الوقت يخلص:** الجوتسو بيرجع لخطوة 0، و `sequence_failed` event بيتبعت (reason `"timeout"`) للجوتسو اللي كان الأبعد.
- **العلامة الغلط عمرها ما بتصفّر خطوات.** المستخدمة بتجرب نفس الخطوة تاني.

- **قاعدة 2026-11-01:** المحاولة الغلط (علامة جديدة مقدّمتش ولا leader) بتدّي **كل** الـ 7 leaders ثواني جديدة، **إلا لو** العلامة جابت جوتسو تاني متساوي معاهم في عدد الخطوات (ساعتها كل جوتسو بيحتفظ بساعته).
- **التكرار عمره ما بيمدد الوقت:** نفس العلامة تاني (مش العلامة الجاية لحد) = repeat، مفيش خطوة ومفيش تجديد للساعة.
- **وضع الـ Practice مفيهوش تايمر خالص** (`max_gap=None`).
- **الإطلاق:** أطول تسلسل يكسب لو أكتر من واحد خلص، والتعادل بترتيب config، وبعدين كل التقدم بيتصفّر.
- **الـ cooldown:** 1.5 ثانية بعد أي إطلاق؛ جوتسو بيكمل جواها بيتصفّر من غير مؤثر.

قواعد الـ fusion

- YOLOX مالك الـ 7 علامات الرسمية؛ الـ landmark model مالك `clone_seal` و `rasengan_palm`.
- YOLOX: طازة (<= 0.35 ثانية)، فوق إيد (>= 0.70) أو مفيش إيد خالص (>= 0.85)، ومرتين ورا بعض.
- Landmarks: `LANDMARK_MIN_PROB` 0.60، والفرق `LANDMARK_MIN_MARGIN` 0.30، وبوابة الكف المفتوح لـ `rasengan_palm` (`PALM_MIN_THUMB_GAP` 0.60).
- الـ backup: لو YOLOX مش شغال خالص بس، والـ classifier عند 0.90 أو أكتر.
- التعارض: الأعلى درجة يكسب، ولو متساويين YOLOX.

قواعد الـ stabilizer

- EMA بوزن 0.4 عند 15 FPS، ومظبوط على الوقت الحقيقي.
- قوية = مش 0.75 =>، idle، وفرق >= 0.25.
- تتمسك 0.30 ثانية قبل التأكيد، وبتتبلّغ مرة واحدة.
- بتتفك لما تنزل تحت 0.45 (hysteresis).

بوابة الجاهزية

- 5 frames ورا بعض كل واحد أقل من 500 ملي ثانية. قبل كده: كل حاجة بتتعالج بس العلامات بتتحسب idle. بعدها: جاهز على طول.

ملاحظات: الفروق بين النسخة الإنجليزية والكود الحالي

راجعنا كل ملف في الجزء ده على الكود. النسخة الإنجليزية دقيقة في الغالب جداً، بس دي النقط اللي لقينا فيها اختلاف أو نقص:

- **ملاحظة: تعليق `SEQUENCE_MAX_GAP` في `config.py`** بيقول إن التايمر بيبدأ من جديد "also when the same sign is re-confirmed while held". بس الكود الحالي في `SequenceMatcher.push` بيعامل العلامة المتكررة كـ repeat ومش بيجدد الساعة. النسخة الإنجليزية من الدليل ماشية صح مع الكود؛ التعليق نفسه في `config.py` هو اللي قديم.
- **ملاحظة: الـ docstring بتاع `GestureStabilizer`** بيقول إن 3 frames عند 6 FPS بتكلف "0.33 s instead of 0.13 s"، والنسخة الإنجليزية بتقول "0.5 s" بدل "0.2 s". الفرق بس في العد (فترات بين frames ولا frames)؛

الكود واحد.

- **ملاحظة: الـ event** `step` **.** النسخة الإنجليزية بتقول إنه بيتبعت لكل جوتسو اتقدم "except the jutsu that fired". بس الكود الحالي بيصفّر كل التقدم عند الإطلاق، فمفيش `step` خالص في frame الإطلاق لأي جوتسو.
- **ملاحظة: التعارض في الـ fusion.** النسخة الإنجليزية بتقول "the higher score wins"؛ الكود بيستخدم `p >` `candidate[1]`، يعني لو الدرجتين متساويتين بالظبط YOLOX هو اللي بيكسب.
- **ملاحظة: الـ model bundle.** النسخة الإنجليزية بتذكر أربع مفاتيح؛ `train.py` بيحفظ كمان `"cv_macro_f1"` (مش مستخدم في `classifier.py`).
- **ملاحظة: الـ backup بعد الرفض.** الكود بيستخدم `if / elif` ، فلو علامة landmark اترفضت في `_sure` ، فرع الـ `"landmarks-backup"` مش بيتجرب في نفس الـ frame. ده تفصيلة مش مذكورة في الإنجليزي.

غير كده: كل الثوابت، والجداول، والدوال، والقواعد (ومنها قاعدة 01-11-2026، و `LANDMARK_MIN_PROB` 0.60، و `LANDMARK_MIN_MARGIN` 0.30، وبوابة الكف المفتوح، وساعة التسلسل، وبوابة الجاهزية) متطابقة مع الكود الحالي.

قاموس صغير للمصطلحات

المصطلح	معناه ببساطة
frame	صورة واحدة من الفيديو.
landmark	نقطة مهمة على الإيد (21 نقطة لكل إيد).
feature	رقم محسوب بيوصف حاجة مهمة (زاوية صباع، مسافة).
model	برنامج ذكاء اصطناعي متدرّب.
classifier	model بيختار label من كذا label.
detector	model بيلاقي حاجة في الصورة ويرسم حواليها مربع.
threshold	الحد الأدنى المطلوب.
probability	احتمال من 0 لـ 1.
one-hot	vector فيه 1 في خانة واحدة والباقي أصفار.
EMA	متوسط متحرك بيخلط الجديد بالقديم بوزن.
hysteresis	حد للتشغيل أعلى من حد الإيقاف عشان نمنع الرعشة.
latch	العلامة اتأكدت ومش هتتبدّغ تاني لحد ما تتفك.
leader	الجوتسو اللي واصل لأبعد خطوة.
cooldown	فترة راحة بعد الإطلاق.
thread	خيط تنفيذ بيشتغل بالتوازي مع الباقي.
NMS	شيل المربعات المتداخلة والاحتفاظ بالأحسن.
anchor	مكان الإيد اللي المؤثر بيترسم عنده.
segmentation	فصل الشخص عن الخلفية.

الجزء التاني: السيرفر والتأثيرات والصوت

مقدمة: إحنا هنتكلم عن إيه في الجزء ده؟

الجزء ده بيشرح الكود اللي قاعد في النص بين المتصفح وبين الذكاء الاصطناعي، والكود اللي بيرسم تأثيرات الـ jutsu على صورة الكاميرا، والأصوات اللي بتشتغل مع كل حاجة. هنمشي في تلات مناطق كبيرة:

- `shinobi/server.py` : السيرفر بتاع الويب المبني بـ FastAPI، والـ REST endpoints بتاعته، وجلسة الـ WebSocket اللي بتعالج فريمات الكاميرا، وشكل الرسايل الـ binary اللي بتتبعت في الاتجاهين.
- `shinobi/effects/` : الخمس ملفات Python اللي بترسم Fire Style و Chidori و Rasengan و Shadow Clone على كل فريم، ومعاهم الكلاس الأساسي المشترك وأدوات الرسم المشتركة.
- فولدرات الـ assets: `assets/audio/` (كل الأصوات، تراخيصها، وإمتى الويب أب بيشغّلها)، و `gesture_references/` (صور حركات الإيد اللي بتظهر في الأبلكيشن). ومعاهم من ناحية الواجهة ملفات `frontend/src/audio/` (`cues.ts` و `eventCues.ts` و `gains.ts` وغيرهم) اللي بتقرر إمتى كل صوت يشتغل وبأي علو.

قبل ما نبدأ، خلّينا نتفق على شوية مصطلحات هتتكرر كتير. هسيبها بالإنجليزي زي ما هي في الكود، بس هشرح معناها أول مرة:

- **FastAPI**: مكتبة Python بنعمل بيها سيرفر ويب بسهولة. بتعرّف فيها "عناوين" (routes) وكل عنوان بيرجّع بيانات.
- **endpoint**: عنوان معيّن على السيرفر (زي `/api/info`) لما الواجهة تطلبه بيرجّع لها حاجة محددة.
- **REST**: الطريقة "العادية" للتعامل مع السيرفر: الواجهة تبعت طلب (request) وتستنى رد (response) واحد وخلاص.
- **WebSocket**: طريقة اتصال بتسمح للواجهة والسيرفر يتبادلوا البيانات بشكل لحظي ومستمر في الاتجاهين، من غير ما كل مرة نفتح اتصال جديد. ده اللي بنستخدمه عشان نبعت فريمات الكاميرا ونستقبلها.
- **frame (فريم)**: صورة واحدة من الكاميرا. الفيديو ما هو إلا فريمات ورا بعض بسرعة.
- **JPEG**: طريقة لضغط الصور عشان حجمها يصغر قبل ما تتبعت على الشبكة.
- **payload**: "الحمولة"، يعني البيانات الفعلية اللي جوه الرسالة (زي بايتات الصورة أو محتوى الـ JSON).
- **JSON**: شكل نصي لكتابة البيانات المنظمة، زي `{"type": "ping", "t": 12}` .
- **event (حدث)**: رسالة صغيرة بتقول "حصل كذا في الفريم ده"، زي "اتأكدت حركة" أو "الـ jutsu اشتغلت".
- **effect (تأثير)**: الرسم البصري للـ jutsu على الصورة (النار، البرق، الكرة الزرقا، النسخ).
- **session (جلسة)**: اتصال واحد من متصفح واحد بالسيرفر، بكل الحالة بتاعته.
- **thread**: "خيط تنفيذ"، يعني مسار شغل مستقل جوه البرنامج يقدر يشتغل في نفس الوقت مع مسارات تانية.
- **event loop / asyncio**: النظام اللي بيخلّي Python يتعامل مع كذا اتصال في نفس الوقت من غير ما يقف مستني واحد منهم.

الصورة الكبيرة في سطور

المتصفح (أبلكيشن React) بيفتح الكاميرا، ويضغط كل فريم لـ JPEG، ويبعته لسيرفر Python عن طريق WebSocket. السيرفر بيشغّل خط الذكاء الاصطناعي كله (`shinobi/engine.py`: MediaPipe للإيدين، والـ YOLOX seal detector، والـ landmark classifier، والـ sign fusion، والـ sequence controller) وكود التأثيرات على الفريم، وبعدين يضغط النتيجة تاني لـ JPEG ويرجّعها ومعاها وصف JSON لكل اللي حصل (ده اسمه `FrameState`).
المتصفح دوره بس إنه يعرض الصورة اللي رجعت ويشغّل الأصوات لما السيرفر يبلّغ عن events. **المتصفح عمره ما بيقرر لوحده إن حركة أو jutsu حصلت.** القرار كله عند السيرفر.

ليه اتعمل كده؟ عشان يبقى فيه "مصدر حقيقة" واحد بس (السيرفر)، وعشان الواجهة تفضل بسيطة، وعشان اللي بيتشاف على الشاشة هو بالظبط اللي الموديل شافه.

مسار البيانات الكامل بين الواجهة والسيرفر

ده أهم قسم عشان تفهمي السيستم كله. هنمشي خطوة خطوة من لحظة فتح الصفحة لحد ما الصوت يشتغل.

1. المستخدمة بتفتح الأبلكيشن في المتصفح. السيرفر (`python -m shinobi.server`) بيرجّع ملفات React المبنية من `frontend/dist`.
2. الواجهة بتطلب بيانات ثابتة عن طريق REST: `GET /api/info` (السيرفر يقدر يعمل إيه)، و`GET /api/jutsu` (كروت الحركات والـ jutsu وروابط الصور)، و`GET /api/credits` (حقوق الصور والأصوات)، و`GET /api/voices` (ملفات أصوات الشخصيات المتركّبة، وهي كلها `null` النهارده).
3. الواجهة بتفتح WebSocket على `/ws`. السيرفر بيقبل الاتصال ويحاول ياخد "مكان" (slot). لو الأماكن كلها مليانة بيبعت خطأ `busy` ويقفل بكود 1013.
4. لو فيه مكان، السيرفر بيعمل `Session` جديدة، وجوه thread خاص بيها بيبني `Engine` جديد ويعمل له warm-up (تسخين للموديلات).
5. السيرفر بيبعت أول رسالة: `hello`، فيها رقم البروتوكول ورقم الجلسة وكل الـ `ServerInfo` والـ mode الحالي.
6. الواجهة بتاخد فريم من الكاميرا، تصغّره لو لازم، تضغطه JPEG بجودة 0.8، وتبعته كرسالة binary: 4 بايت طول + header JSON فيه `{"t": ms}` + بايتات الـ JPEG.
7. السيرفر بيستقبل الرسالة في `_reader()`، ويفحصها بسرعة في `_on_frame()` (الحجم، الـ header، توقيع الـ JPEG)، ويحطها في الطابور (queue).
8. الـ `_worker()` بياخد الفريم من الطابور ويشغّل `_process()` على thread الجلسة: فك الـ JPEG، تصغير، عكس الصورة (mirror)، وبعدين `engine.process(...)`.
9. الـ engine بيشغّل التعرّف على الإيد والحركات، والـ sequence controller، ولو jutsu اكتملت بيبدأ الـ effect، وبيرسم الـ effect على الفريم.
10. السيرفر بيبني `FrameView` ويبعته لـ `EventTracker.update()` اللي بيطلّع قائمة الـ events بتاعة الفريم ده.
11. السيرفر بيضغط الفريم الناتج JPEG بجودة 90 ويبني الـ header (`FrameState`) ويرجّع الرسالة binary بنفس الشكل.
12. الواجهة بتعرض الصورة اللي رجعت، وتحدّث الكروت والعدادات من الـ `FrameState`، وتعدّي الـ `events` على `soundsForFrame()` في `eventCues.ts` (المؤثرات الصوتية) و`voiceForFrame()` في `voiceCues.ts` (أصوات

الشخصيات) و `musicForFrame()` في `jutsuMusic.ts` (موسيقى الـ jutsu).

13. محرك الصوت (`frontend/src/audio/engine.ts`) بينفّذ الأوامر اللي طلعت: يشغّل صوت، يعمل loop، يوقف صوت بـ fade.
14. أول ما يوصل رد، الواجهة تبعت الفريم اللي بعده (مسموح لها يكون عندها بالكتير 2 فريم من غير رد).

وده رسم نصي للمسار:

```
المتصفح (React)                                  السيرفر (Python, FastAPI)
---------------                                  -------------------------
GET /api/info, /api/jutsu, /api/credits, /api/voices  ---->  REST routes
                                                 <----  JSON

فتح WebSocket /ws  ------------------------------------>  Slots.acquire()
                                                        Session -> Engine + warm_up()
                   <------------------------------------  {"type":"hello", ...}

كاميرا -> JPEG (0.8)
[N][{"t":ms}][JPEG]  ---- binary ---------------------->  _reader -> _on_frame -> queue
                                                        _worker -> _process (thread الجلسة)
                                                           decode -> resize -> mirror
                                                           engine.process (AI + effects)
                                                           EventTracker.update -> events
                                                           encode JPEG (90, 4:4:4)
[N][FrameState JSON][JPEG]  <---- binary --------------  pack(header, jpeg)

عرض الصورة + تحديث الواجهة
events -> eventCues / voiceCues / jutsuMusic -> SoundEngine -> السماعات

{"type":"mode"...} / {"type":"reset"} / {"type":"options"...}  ---->  queue (بالترتيب مع الفريمات)
{"type":"ping","t":n}  ------------------------------------------>  رد فوري
                       <------------------------------------------  {"type":"pong","t":n}
```

مثال عملي: المستخدمة عملت آخر حركة في (Monkey) Chidori. في الفريم ده الـ `FrameState.events` هيبقى فيه `sign` ثم `jutsu` بـ `stages` قيمتها `{"voice": 0.4, "charge": 1.2, "name": 0.4}`. الواجهة هتشغّل صوت الختم (hand seal) مع لمسة Chidori، وبعد 0.4 ثانية هييجي `effect_phase` بقيمة `charge` فيشتغل صوت الشحن الكهربي، وبعد 1.2 ثانية كمان `effect_phase` بـ `name`، وبعد 0.4 ثانية `effect_phase` بـ `release` فيقف الشحن ويشتغل صوت الضربة والـ loop بتاع الطقطقة. كل ده والسيرفر هو اللي بيرسم البرق على الصورة.

الملف `shinobi/server.py`

مسؤول عن إيه؟

`server.py` هو نقطة الدخول (entry point) للويب أب كله. يعني ده الملف اللي بتشغّليه عشان السيستم يقوم. بيتشغّل كده:

```
python -m shinobi.server [--host 127.0.0.1] [--port 8000] [--max-sessions 2]
                         [--ssl-certfile cert.pem --ssl-keyfile key.pem]
```

الحاجات اللي بين الأقواس المربعة اختيارية، ولو ما كتبتيهاش بتاخد القيم الافتراضية.

الملف ده بيعمل أربع شغلانات:

1. بيقدّم REST API صغيرة (`/api/info`، `/api/jutsu`، `/api/credits`، `/api/voices`) بتقول للويب أب السيرفر يقدر يعمل إيه، وبتديها النصوص وروابط الصور لكروت الـ jutsu.
2. بيقبل اتصالات WebSocket على `/ws`. كل اتصال هو "session" مستقلة ليها AI engine خاص بيها.
3. بيقدّم ملفات ثابتة للقراية بس (static files): صور حركات الإيد تحت `/references/...` والأصوات تحت `/audio/...`.
4. بيقدّم أبلكيشن React المبني (`frontend/dist`) لأي عنوان تاني.

العقد (contract) بين الملف ده وبين الواجهة مكتوب في `docs/web_protocol.md` (protocol version 1). يعني الطرفين متفقين على شكل الرسايل بالظبط من الملف ده. والدليل العربي `docs/13_web_server.md` بيشرح إزاي تشغّليه، والـ HTTPS للموبايلات، والمشاكل المشهورة. كل اللي هيتشرح تحت هو اللي الكود بيعمله فعلاً.

ليه اتعمل؟ لأن الهدف النهائي للمشروع هو ويب أب (React + TypeScript في الواجهة و FastAPI + WebSocket في الباك إند)، والذكاء الاصطناعي لازم يفضل في Python. فاحتجنا "كوبري" بين المتصفح و Python، وده هو الكوبري.

الـ imports والإعداد على مستوى الملف

الملف بيستورد من FastAPI: `FastAPI` و`WebSocket` و`HTTPException` و`FileResponse` و`HTMLResponse` و`StaticFiles`. وبيستورد OpenCV (`cv2`) و NumPy عشان فك وضغط الصور. وبيستورد كمان موديولات من المشروع نفسه:

- `shinobi.config`: كل الثوابت (البورتات، حدود الفريمات، جودة الـ JPEG، مسارات الفولدرات، تعريفات الـ jutsu).
- `shinobi.catalog`: نصوص كروت الحركات والـ jutsu، والـ credits، والدالة المساعدة لملفات أصوات الشخصيات.
- `shinobi.classifier.GestureClassifier`: موديل الـ landmarks المتدرّب (اختياري، ممكن ما يبقاش موجود).
- `shinobi.engine.Engine`: خط الذكاء الاصطناعي الكامل لجلسة واحدة.
- `shinobi.events`: فيه `EventTracker` و`FrameView` و`EffectView`، ودول اللي بيحوّلوا حالة الـ engine لقائمة events تتبعت للمتصفح.

بعد الـ imports على طول فيه تصليح صغير بس مهم جداً خاص بـ Windows. الـ Windows بيقرا أنواع الملفات (MIME types)، يعني "نوع المحتوى" اللي السيرفر بيقوله للمتصفح) من الـ registry، وعلى بعض الأجهزة امتداد `.js` بيبقى متسجّل على إنه `text/plain` (نص عادي). المتصفحات بترفض تشغّل JavaScript modules جاية بالنوع ده، فالويب أب كان هيظهر صفحة فاضية. عشان كده السيرفر بيسجّل الأنواع الصح بنفسه:

```
for _type, _ext in (("text/javascript", ".js"), ("text/javascript", ".mjs"), ("text/css", ".css"),
                    ("image/svg+xml", ".svg"), ("application/wasm", ".wasm"), ("application/json",
".json"),
                    ("application/manifest+json", ".webmanifest"), ("audio/wav", ".wav"),
("audio/flac", ".flac")):
    mimetypes.add_type(_type, _ext)
```

الكود ده بيعمل إيه خطوة خطوة:

1. بيلف على قائمة أزواج: كل زوج فيه نوع (زي `text/javascript`) وامتداد (زي `.js`).

2. لكل زوج بينادي `mimetypes.add_type` اللي بتقول لـ Python: "أي ملف بالامتداد ده نوعه كذا".
3. النتيجة: لما السيرفر يرجّع ملف `.js` أو `.css` أو `.wav` أو `.flac`، بيقول للمتصفح النوع الصح، مهما كان الـ registry بتاع Windows مكتوب فيه إيه.

ثوابت تانية على مستوى الملف:

الاسم	القيمة / المعنى
`YOLOX_SIGNS`	الحركات اللي في `config.GESTURES` واللي الـ YOLOX seal detector يقدر يتعرّف عليها (اللي موجودة في `config.SEAL_CLASSES`)
`HEADER_LEN`	`struct.Struct(">I")`: رقم صحيح 32-bit من غير إشارة بترتيب big-endian، وده بادئة الطول في كل رسالة binary
`OUTPUT_JPEG`	إعدادات OpenCV لضغط الفريمات اللي بترجع: الجودة `config.OUTPUT_JPEG_QUALITY` (90)، ولو `config.OUTPUT_JPEG_444` بـ True بيضيف دقة ألوان كاملة (chroma sampling 4:4:4) عشان الخطوط الملونة الرفيعة زي برق Chidori تفضل حادة
`BUILD_HELP`	صفحة HTML صغيرة بتظهر على `/` لو `frontend/dist` لسه مش موجود؛ بتشرح `npm install` / `npm run build` أو `npm run dev`

معنى "4:4:4": الـ JPEG العادي بيوفّر مساحة بإنه يخزّن الألوان بدقة أقل من الإضاءة (4:2:0). ده بيخلّي الخطوط الرفيعة الملونة تبهت أو تتلطّخ. الـ 4:4:4 بيخزّن الألوان بدقة كاملة، فالبرق الأزرق الرفيع يفضل واضح.

التغليف الـ binary: الدالتين `pack()` و `unpack()`

الاتجاهين في الـ WebSocket بيستخدموا نفس الشكل الـ binary لرسايل الصور:

```
[ uint32 big-endian N ][ N bytes: UTF-8 JSON header ][ JPEG bytes ]
```

يعني أول 4 بايت بيقولوا طول الـ header الـ JSON كام بايت (N). بعدين ييجي الـ header نفسه (object JSON). وباقي الرسالة كله هو صورة الـ JPEG. الفكرة دي بتخلّي رسالة WebSocket واحدة تشيل بيانات منظمة وصورة مع بعض، من غير base64 (base64 طريقة لتحويل البايتات لنص، بس بتكبّر حجم الصورة حوالي 33 %).

```
def pack(header, payload=b""):
    head = json.dumps(header, separators=(",", ":")).encode("utf-8")
    return HEADER_LEN.pack(len(head)) + head + payload
```

بتعمل إيه خطوة خطوة:

1. `json.dumps(header, separators=(",", ":"))`: بتحوّل الـ dictionary لنص JSON مضغوط (من غير مسافات زيادة بعد الفاصلة والنقطتين) عشان الحجم يبقى أصغر.
2. `.encode("utf-8")`: بتحوّل النص لبايتات.
3. `HEADER_LEN.pack(len(head))`: بتكتب طول الـ header في 4 بايت.
4. بتلزق التلاتة ورا بعض: الطول + الـ header + الـ payload (بايتات الـ JPEG). لو مفيش payload بتبقى فاضية (`b""`).

مين بيستخدمها؟ `_process()` في آخر خطوة، عشان يبني الرد اللي راجع للمتصفح.

`unpack(data)` بتعمل العكس، وبترمي `ValueError` في كل حالة الرسالة فيها غلط: الرسالة أقصر من 4 بايت، أو الطول المكتوب أكبر من الرسالة نفسها، أو الـ header مش JSON صالح بـ UTF-8، أو الـ JSON مش object. وبترجّع `(header_dict, payload_bytes)`. مين بيستخدمها؟ `_on_frame()` عشان تفك فريم الكاميرا اللي جاي من المتصفح.

فيه دالتين صغيرين مساعدين:

- `_number(value)`: بترجّع رقم JSON على شكل `float`، وبترفض الـ booleans (لأن في Python الـ `True` هو كمان `int`!)، وإلا بترجّع `None`. بتستخدم عشان تقرا `t` من header الفريم بأمان.
- `_clean(text)`: بتحوّل أي مسافات متكررة لمسافة واحدة. نصوص الكروت في `catalog.py` بتستخدم مسافتين كفاصل بصري في الكروت المطبوعة، لكن الويب أب عايز مسافة واحدة.

الكلاس `Capabilities`

مسؤول عن إيه؟ بيقرر مرة واحدة بس، وقت ما السيرفر يقوم، السيرفر ده يقدر يتعرّف على إيه. ده بيعتمد على ملفات الموديلات الموجودة فعلاً على الجهاز.

- `self.yolox` بيبقى True لو `config.SEAL_MODEL_PATH` (ملف YOLOX-Nano بصيغة ONNX) موجود، ومكتبة `onnxruntime` ممكن تتستورد.
- `self.classifier`: مع القيمة الافتراضية `"auto"`، بيحمّل `GestureClassifier()` لو ملف موديل الـ landmarks المتدرّب موجود. لو التحميل فشل (مثلاً الموديل اتدرّب بنسخة features قديمة) بيطبع `landmark model off: ...` ويكمّل من غيره. الـ classifier اللي اتحمّل بيتشارك "للقراية بس" بين كل الجلسات، فبيتحمّل مرة واحدة بس.
- `self.landmark_model` بيبقى True لو اتحمّل classifier.
- `self.recognizable`: قائمة الحركات اللي ممكن التعرّف عليها، بترتيب `config.GESTURES`. القاعدة نفس قاعدة `SignFusion`: لو فيه YOLOX، يبقى حركات YOLOX زائد `config.LANDMARK_SIGNS` (بس لو موديل الـ landmarks موجود)؛ لو مفيش YOLOX، يبقى كل labels موديل الـ landmarks ما عدا `idle`؛ لو مفيش الاتنين، يبقى ولا حاجة.
- `self.jutsu_available[j]` بيبقى True لو كل حركات الـ jutsu رقم `j` ممكن التعرّف عليها.

الدالة `recognizer(sign)` بترجّع `"yolox"` لحركة من حركات YOLOX لو YOLOX شغّال (أو لو كمان مفيش موديل landmarks)، وإلا بترجّع `"landmarks"`. القيمة دي بتظهر على كروت الحركات في الأبلكيشن عشان المستخدمة تعرف مين اللي بيتعرّف على الحركة دي.

مثال: لو ملف YOLOX موجود ومفيش موديل landmarks، يبقى `recognizable` هي الحركات السبعة الرسمية بس، و`clone_seal` و`rasengan_palm` مش هيبقوا فيها، وبالتالي `shadow_clone` و`rasengan` هيبقوا `false` في `jutsu_available` (وده بالظبط المثال المكتوب في `docs/web_protocol.md`).

الدالتين `build_catalog()` و `_sign_art()`

`build_catalog(caps)` بتبني الـ JSON اللي بيرجع من `GET /api/jutsu`، مرة واحدة وقت التشغيل.

- لكل حركة في `catalog.SIGNS` بتعمل object فيه: `id`، `name`، `jp` (الاسم الياباني)، `hands` (كام إيد)، `official` (رسمية ولا لأ)، `cues` (نصايح الأداء بعد التنضيف بـ `_clean`)، `art` (رابط الصورة)، `recognizer`،

و `recognizable`.

- لكل jutsu في `config.JUTSU` (بترتيب الـ config: fire_style، chidori، shadow_clone، rasengan) بتعمل object فيه: العنوان، الاسم، الكانجي، علامة official، الـ tag، الـ provenance (المصدر)، الوصف، الـ effect id، قائمة `steps` (رقم الخطوة، id الحركة، ورابط الكارت `/references/<jutsu>/step_<n>.png`)، ورابط `sequence_image`، و `available`، و `missing_signs` (الحركات الناقصة لو فيه).

`_sign_art(sid)` بترجّع `/references/_sources/signs_bold/<sid>.png` لو النسخة التخينة (bold) من الرسمة موجودة (بيعملها `tools/build_bold_signs.py` عشان كروت الويب)، وإلا بترجّع الرسمة الأصلية الرفيعة من `signs/`.

الكلاس `Slots`

مسؤول عن إيه؟ بيعدّ الجلسات المفتوحة. بيستخدم thread lock (قفل بيمنع اتنين threads يعدّلوا العداد في نفس اللحظة)، لأن الـ slot بيتحرر من thread الجلسة نفسها، مش من الـ event loop.

- `acquire()` بترجّع `False` لو `active >= max`، وإلا بتزوّد العداد وترجّع `True`.
- `release()` بتنقّص العداد بس عمره ما ينزل تحت الصفر.

الحد الأقصى جاي من `--max-sessions` (الافتراضي `config.SERVER_MAX_SESSIONS` = 2). ليه فيه حد؟ لأن كل جلسة بتشغّل نسخها الخاصة من MediaPipe و YOLOX والـ segmenter، فالحد ده بيحمي الـ CPU من إنه يتخنق.

الكلاس `Session`: اتصال متصفح واحد

مسؤول عن إيه؟ فيه object واحد من `Session` لكل WebSocket مفتوح. هو "مدير" الاتصال ده من أوله لآخره: بيبني الـ engine، بيستقبل الرسايل، بيرتّب الشغل، بيعالج الفريمات، وبيبعت الردود.

الحقول (fields) بتاعته:

الحقل	معناه
`ws`	الـ `WebSocket` بتاع FastAPI
`server`	الـ object المشترك `Server` (الـ capabilities، الـ slots، الـ catalog)
`id`	6 حروف hex من `secrets.token_hex(3)`، بيظهر في الـ logs وبيتبعت في `hello`
`mode`	`{"mode": "free", "jutsu": None}` أو `{"mode": "practice", "jutsu": "<id>"}`
`engine`	الـ `Engine` الخاص بالجلسة (بيتعمل على thread الجلسة)
`tracker`	`EventTracker(config.JUTSU)` اللي بيحوّل حالة الـ engine لـ events
`executor`	`ThreadPoolExecutor` فيه worker thread واحد بالظبط اسمه `session-<id>`
`queue`	`deque` (طابور) فيه الشغل المستني، بترتيب وصوله
`wakeup`	`asyncio.Event` بيصحّي الـ worker لما يدخل الطابور حاجة
`send_lock`	`asyncio.Lock` عشان عمر ما اتنين coroutines يكتبوا على الـ socket في نفس اللحظة
`seq`، `fps`، `_last_done`	عدّاد الفريمات ومعدّل المعالجة بعد التنعيم

ليه thread مخصوص لكل جلسة؟ لسببين:

- objects بتاعة MediaPipe ممنوع يستخدمها اتنين threads في نفس الوقت.
- معالجة الذكاء الاصطناعي بطيئة (عشرات الملّي ثانية لكل فريم). لو اشتغلت على الـ event loop كانت هتوقف كل حاجة تانية.

فلما بنشغّلها في thread الجلسة الخاص، الـ asyncio event loop بيفضل فاضي (فالجلسات التانية وطلبات الـ REST ما بتتعطلش)، وبنضمن إن موديلات الجلسة الواحدة ما بيلمسهاش غير thread واحد بس.

الإرسال

- `_send(text=None, data=None)`: بتبعت نص أو بايتات وهي ماسكة `send_lock`. أي exception (يعني الـ socket اتقفل) بيتحوّل لـ exception خاص اسمه `_Gone`، ومعناه "العميل مشي، اقفل بهدوء".
- `send_json(message)`: بتحوّل الرسالة لـ JSON مضغوط مع `ensure_ascii=False` (عشان النصوص اليابانية تفضل مقروءة بدل ما تتحوّل لرموز هروب).
- `send_error(code, message, fatal=False)`: بتبعت `{"type": "error", "code", "message", "fatal"}`.

دورة الحياة: الدالة `run()`

`run()` هي حياة الجلسة كلها من أولها لآخرها:

1. تبني الـ engine على thread الجلسة (`_create_engine()`): `Engine(classifier=..., yolox=...)` وبعدها `engine.warm_up()`. لو ده فشل، تبعت خطأ `internal` بـ fatal، وتقفل بكود 1011 وترجع.
2. تبعت رسالة `hello`: رقم البروتوكول، رقم الجلسة، الـ `ServerInfo` كامل، والـ mode الحالي.
3. تبدأ اتنين asyncio tasks: `_reader()` (بيستقبل الرسايل) و`_worker()` (بيعالج الطابور). وتستنى لحد ما أي واحد فيهم يخلص.
4. لو task خلصت بـ exception مش متوقع (مش `_Gone`)، تسجّله في الـ log، وتبعت خطأ `internal` بـ fatal وتقفل بـ 1011.
5. في `finally` (اللي بيشتغل دايماً، حتى لو السيرفر لغى الـ task): تلغي الاتنين tasks، وتبعت `_close_engine` لـ thread الجلسة (فيشتغل بعد الفريم اللي بيتعالج دلوقتي)، وتحرر الـ slot لما شغلانة القفل دي تخلص، وتسجّل شغلانة القفل عند `server.track_closing`، وتقفل الـ executor من غير ما تستنى.

التعليق في الكود بيشرح ليه مفيش `await` جوه الـ `finally`: لو السيرفر لغى الـ task، أي `await` هناك ممكن يتقطع والتنضيف كله يضيع.

الـ Warm-up (التسخين). أول تشغيل لـ MediaPipe و YOLOX والـ person segmenter ممكن ياخد ثواني (الـ docstring بتاع الـ engine بيذكر 1.8 ثانية اتقاست على أول فريم في جلسة). `Engine.warm_up()` بيشغّل كل موديل مرة على صورة رمادي 640x480 قبل ما `hello` يتبعت: الإيدين مرتين، استنتاج YOLOX واحد (بيستنى نتيجته لحد 15 ثانية)، و segmentation واحدة (عشان Shadow Clone ما يقفش أول ما يظهر). وبعدين بيعمل reset لحالة التعرّف عشان ما يفضلش أي حاجة من الصورة الفاضية. وحتى بعد الـ warm-up، الـ engine بيقول `ready: false` لحد ما `config.READY_FRAMES` (5) فريمات ورا بعض يتعالجوا في أقل من `config.READY_MAX_MS` (500 ms)؛ قبل كده مفيش أي حركة بتتأكد.

الرسايل الداخلة: `_reader()` و `_on_frame()` و `_on_text()`

`_reader()` بتلف على `ws.receive()`. لو جه disconnect بتخلص. الرسايل الـ binary بتروح لـ `_on_frame`، والرسايل النصية لـ `_on_text`.

`_on_frame(data)` بتفحص فريم الكاميرا بسرعة، على الـ event loop، من غير ما تفك الصورة:

- لو أطول من `config.FRAME_MAX_BYTES` (4 MiB): وحش.
- لو `unpack` فشلت: وحش.
- لو الـ payload ما بيبدأش بتوقيع الـ JPEG `FF D8 FF`: وحش.

الفريم الوحش بيدخل الطابور كـ `("bad", t, reason)`؛ والكويس كـ `("frame", t, jpeg)`، و`t` هو الـ timestamp بتاع العميل من الـ header (أو `None`).

`_enqueue(item)` بتضيف العنصر وبعدين تطبّق حد "الفريمات اللي في الطريق": لو أكتر من `config.FRAME_MAX_IN_FLIGHT` (2) فريمات مستنيين، أقدم المستنيين بيتبدّلوا بـ `("skip", t)`. دول هيترد عليهم بـ `skipped` بدل ما يتعالجوا، فالسيرفر عمره ما بيتراكم عنده شغل متأخر، ودايماً بيشتغل على فريمات حديثة.

```
def _enqueue(self, item):
    self.queue.append(item)
    frames = [i for i, it in enumerate(self.queue) if it[0] == "frame"]
    for i in frames[:max(0, len(frames) - config.FRAME_MAX_IN_FLIGHT)]:
        self.queue[i] = ("skip", self.queue[i][1])
    self.wakeup.set()
```

خطوة خطوة:

1. `self.queue.append(item)`: يحط العنصر الجديد في آخر الطابور.
2. `frames = [...]`: يجيب أماكن كل العناصر اللي نوعها `"frame"` (فريمات لسه مستنية معالجة).
3. `frames[:max(0, len(frames) - config.FRAME_MAX_IN_FLIGHT)]`: ياخد الأقدم بس، بحيث يفضل آخر 2 فريم بس. لو فيه 3 فريمات مستنيين، أول واحد بس هيتبدّل.
4. يبدّل كل واحد من دول بـ `("skip", t)` محتفظ بالـ `t` بتاعه عشان الرد يحمل نفس الـ `client_t`.
5. `self.wakeup.set()`: يصحّي الـ worker لو كان نايم.

مثال: الشبكة كانت سريعة والسيرفر بطيء لحظة، فوصلوا 3 فريمات (t=100، t=133، t=166). الطابور هيبقى فيه `skip` لـ t=100، و`frame` لـ t=133 و t=166. المتصفح هياخد `{"type": "skipped", "client_t": 100}` وبعدين ردين فيهم صور.

`_on_text(text)` بتفك رسالة تحكّم JSON:

- `ping`: بترد على طول بـ `pong` (مش بتدخل الطابور ورا الفريمات)، عشان العميل يقدر يقيس تأخير الشبكة.
- `reset`: تحط في الطابور نداء لـ `_reset` (يعني `reset_sequence()` بتاعة الـ engine و `reset()` بتاعة الـ tracker). مفيش رد.
- `mode` بقيمة `free`: تحط في الطابور `_set_mode(None)`.
- `mode` بقيمة `practice` ومعاه jutsu id معروف: تحط في الطابور `_set_mode(jutsu)`.

- `options` ومعاه `landmarks` boolean: تحط في الطابور `_set_landmarks(show)`، اللي بتظبط `engine.draw_landmarks`. مفيش رد.
- أي حاجة تانية (مش JSON، نوع مش معروف، practice بـ jutsu مش معروف، `landmarks` مش boolean): خطأ `bad_message` مش fatal.

ليه `mode` و`reset` و`options` بيعدّوا على نفس طابور الفريمات؟ عشان يتطبّقوا بالترتيب: الفريمات اللي اتبعتت قبل تغيير الـ mode لسه هيترد عليها بالـ mode القديم. ده بيمنع لخبطة.

`_set_mode(jutsu)` بتختار يا كل الـ jutsu (free mode) يا واحدة بس (practice)، وبتنادي `engine.set_jutsu(chosen, practice=...)` و`tracker.set_mode(...)`، وبترجّع `{"type": "mode", ...}` اللي الـ worker بيبعته كتأكيد.

المعالجة: `_worker()` و `_process()`

`_worker()` بيستنى على `wakeup` طول ما الطابور فاضي، وبعدين بياخد عنصر واحد:

- `skip`: يبعت `{"type": "skipped", "client_t": t}`.
- `bad`: يبعت خطأ `bad_frame` وبعده `skipped` (عشان عدّاد الفريمات اللي من غير رد عند العميل يفضل مظبوط).
- `frame`: يشغّل `_process` على thread الجلسة؛ لو رجّعت مشكلة يتعامل معاها زي الفريم الوحش، وإلا يبعت الرد الـ binary المتغلّف.
- `call`: يشغّل الدالة على thread الجلسة ويبعت ردها لو رجّعت رد.

`_process(client_t, jpeg)` بتشتغل على thread الجلسة وبتعمل الشغل الحقيقي لفريم واحد:

1. تفك الـ JPEG بـ `cv2.imdecode`. لو فشلت يبقى "the JPEG image could not be decoded".
2. تصغّر (عمرها ما تكبّر) عشان الفريم يدخل في `FRAME_MAX_WIDTH` (1280 x 1280) `FRAME_MAX_HEIGHT` x، باستخدام `INTER_AREA` (أحسن فلتر للتصغير).
3. ترفض الفريمات اللي ضلعها الصغير أقل من `FRAME_MIN_SIZE` (32 بكسل).
4. لو `config.MIRROR` بـ True، تعكس الصورة أفقياً (منظر السيلفي). العملاء بيبعتوا صورة الكاميرا من غير عكس؛ والسيرفر هو اللي بيعكسها عشان التأثيرات تترسم على اللي المستخدمة هتشوفه فعلاً.
5. تنادي `engine.process(frame, time.perf_counter())`. ده بيشغّل التعرّف، والـ sequence controller، والتأثيرات، وبيرجّع object فيه النتيجة.
6. تبني `FrameView` (و`EffectView` لو فيه effect شغّال) من النتيجة وتديه لـ `tracker.update(view)`، اللي بيرجّع قائمة events الفريم ده.
7. تضغط الفريم الناتج JPEG بـ `OUTPUT_JPEG`.
8. تجمع التوقيتات بالملّي ثانية: `decode`، وتوقيتات الـ engine نفسه (`hands`، `model`، `yolox`، `seg`، `fx`)، و`encode`، و`total`.
9. تحدّث عدّاد الفريمات والـ FPS: المعدّل اللحظي = `1 / (الوقت من آخر فريم خلص)`، وبيتنعّم بمتوسط متحرك أُسّي (`0.9 * old + 0.1 * new`؛ الفريم التاني بياخد المعدّل اللحظي على طول).
10. تبني الـ header بـ `_frame_state` وترجّع `pack(header, jpeg_bytes)`.

حقول `EffectView` محتاجة ملاحظة:

- `key` هو رقم التفعيل (activation id) من الـ engine (أو رقم سالب من id الـ object كاحتياطي)، عشان الـ tracker يفرّق بين تفعيل جديد وبين نفس التفعيل.
- `clones` هو `int(effect.drawn)` لـ Shadow Clone بس.
- `staged` بيقول هل الـ effect ليه مرحلة تحضير (build-up) ولا لأ.

الدالة `_frame_state()`: الـ header اللي اسمه FrameState

`_frame_state` بتقرا الـ stabilizer (المكوّن اللي بيستنى الحركة تتمسك شوية قبل ما يأكدها):

- `candidate`: الحركة اللي مستني عليها (بس لو مختلفة عن الحركة المتأكدة حالياً).
- `latched`: آخر حركة اتأكدت ولسه ممسوكة.
- `hold`: تقدّم من 0 لـ 1 ناحية التأكيد.

الحركة المدمجة (fused) جاية من `r.fused` (`name`، `source`، و`score` متقرّب لـ 3 أرقام عشرية)، أو `idle` / `none` / 0 لو مفيش. والـ object بتاع الـ effect بيتبني لو فيه effect شغّال.

القائمة الكاملة لحقول FrameState:

الحقل	النوع	المعنى
`type`	string	دايماً `"frame"`
`seq`	int	عدد الفريمات اللي اتعالجت في الجلسة دي لحد دلوقتي (1، 2، ...)
`client_t`	number أو null	الـ `t` بتاع فريم العميل اللي ده رده (عشان قياس زمن الرحلة ذهاب وعودة)
`width`، `height`	int	مقاس الصورة اللي راجعة
`fps`	number	معدّل المعالجة بعد التنعيم للجلسة دي
`timings`	object	ملّي ثانية لكل مرحلة: `decode`، `hands`، `model`، `yolox`، `seg`، `fx`، `encode`، `total` (المفتاح بيختفي لو المرحلة دي ما اشتغلتش)
`hands`	int	عدد الإيدين اللي MediaPipe لقاها في الفريم ده
`sign`	object	`{name, source, score}`: قرار الدمج للفريم ده؛ `source` بيبقى `yolox` أو `landmarks` أو `landmarks-backup` أو `none`
`candidate`	string أو null	حركة ممسوكة بس لسه ما اتأكدتش
`hold`	من 0 لـ 1	`candidate` وصلت لفين في طريقها للتأكيد
`latched`	string أو null	آخر حركة متأكدة طول ما هي لسه ممسوكة
`history`	list	آخر الحركات المتأكدة (لحد 4، الأقدم الأول)
`progress`	object	لكل jutsu في الـ mode الحالي: عدد الخطوات اللي خلصت
`time_left`	number أو null	في free mode: الثواني الفاضلة للـ jutsu اللي في المقدمة عشان حركتها الجاية؛ دايماً null في practice mode

الحقل	النوع	المعنى
`time_left_per_jutsu`	object	في free mode: الثواني الفاضلة لكل jutsu شغّالة؛ `{}` في practice
`ready`	bool	false طول ما الـ AI لسه بيجهّز (شوفي الـ warm-up)
`cooldown`	number	الثواني الفاضلة قبل ما jutsu تانية يسمح لها تشتغل
`mode`	object	الـ mode الحالي
`effect`	object أو null	الـ effect الشغّال (تحت)
`events`	list	اللي حصل في الفريم ده، بالترتيب

الـ object بتاع `effect`:

الحقل	المعنى
`name`	id الـ effect: `fire`، `chidori`، `shadow_clone`، `rasengan`
`jutsu`	id الـ jutsu اللي شغّلته، أو null (لو اتشغّل باليد على نسخة الديسكتوب)
`age`	الثواني من ساعة ما الـ jutsu اشتغلت (`now - activated_at`)، شاملة الـ build-up
`stopping`	true طول ما هو بيختفي تدريجياً (fade out)
`clones`	لـ Shadow Clone بس: عدد النسخ الظاهرة في الفريم اللي راجع
`crowded`	لـ Shadow Clone بس: الفريم ضيّق أوي على 10 نسخ من غير تداخل
`phase`	`voice` أو `charge` أو `name` (الـ build-up) أو `release` (الـ effect الأساسي)
`phase_left`	الثواني الفاضلة في مرحلة الـ build-up الحالية، و null في `release`

مثال لـ header فريم أثناء شحن Rasengan (مختصر، والقيم للتوضيح):

```
{"type":"frame","seq":812,"client_t":53120.4,"width":1280,"height":720,"fps":17.9,
 "hands":1,"sign":{"name":"rasengan_palm","source":"landmarks","score":0.913},
 "progress":{"fire_style":0,"chidori":0,"shadow_clone":0,"rasengan":0},
 "ready":true,"cooldown":0.9,"mode":{"mode":"free","jutsu":null},
 "effect":{"name":"rasengan","jutsu":"rasengan","age":0.95,"stopping":false,
           "clones":null,"crowded":null,"phase":"charge","phase_left":0.65},
 "events":[]}
```

لاحظي: `age` = 0.95 يعني عدّى 0.4 ثانية في `voice` و 0.55 في `charge`، فالفاضل من الـ (1.2) charge هو 0.65، وده بالظبط `phase_left`. والـ `progress` رجع أصفار لأن الـ sequence اتعمل لها reset لما الـ jutsu اشتغلت.

الكلاس `Server` والدالة `create_app()`

`Server` شايل الحاجات اللي كل الجلسات بتتشاركها:

- `caps`: الـ Capabilities.
- `slots`: الـ Slots.
- `catalog`: بيتبني مرة واحدة.
- مجموعة futures اسمها "closing" (شغلانات القفل اللي لسه شغّالة).

الدوال:

- `track_closing(future)`: بتفتكر شغلانة تنضيف جلسة لحد ما تخلص.
- `wait_closed(timeout=10)`: بتستنى كل شغلانات التنضيف المعلّقة. بتتنادى لما السيرفر يقفل، لأن أول ما Python يبدأ يخرج، MediaPipe ما يقدرش يقفل موديلاته صح.
- `info()`: بترجّع dictionary الـ `ServerInfo`: name، version، protocol، yolox، landmark_model، recognizable، jutsu_available، الجلسات الحالية (`sessions`)، حدود الفريم (`frame`)، mirror، `sequence_max_gap`، وتوقيت التفعيل (`activation`: الـ `charge` لكل jutsu، و`voice_default`، و`voice_gap`). لاحظي إن `frame.jpeg_quality` بيتبعت من 0 لـ 1 (`FRAME_JPEG_QUALITY / 100` = 0.8)، لأن ده الشكل اللي الـ canvas encoder بتاع المتصفح بيستناه.

شكل `ServerInfo` بالظبط زي ما في `docs/web_protocol.md`:

```
{
  "name": "SHINOBI AI",
  "version": "0.1.0",
  "protocol": 1,
  "yolox": true,
  "landmark_model": false,
  "recognizable": ["ram", "snake", "tiger", "dog", "monkey", "boar", "horse"],
  "jutsu_available": {"fire_style": true, "chidori": true, "shadow_clone": false, "rasengan": false},
  "sessions": {"active": 0, "max": 2},
  "frame": {"max_width": 1280, "max_height": 1280, "jpeg_quality": 0.8, "max_in_flight": 2},
  "mirror": true,
  "sequence_max_gap": 7.0,
  "activation": {"charge": {"fire_style": 1.2, "chidori": 1.2, "shadow_clone": 1.2, "rasengan": 1.2},
                 "voice_default": 0.4, "voice_gap": 0.05}
}
```

(القيم بتاعة `yolox` و`landmark_model` و`recognizable` في المثال ده بتوضّح سيرفر من غير موديل landmarks؛ على جهازك هتطلع حسب الموديلات الموجودة فعلاً.)

`create_app(max_sessions, classifier)` بتعمل الـ `Server`، وبتعرّف handler اسمه `lifespan` الجزء بتاع القفل فيه بيشغّل `server.wait_closed` في thread، وبتعمل app الـ `FastAPI`، وبتسجّل الـ routes:

الـ route	الطريقة	بيرجّع إيه
`/api/info`	GET	`server.info()` (يعني `ServerInfo`)
`/api/jutsu`	GET	الـ catalog: `{signs, jutsu}`
`/api/credits`	GET	`{"sections": catalog.CREDITS}` (حقوق الصور والأصوات والموديلات، وإخلاء المسؤولية)
`/api/voices`	GET	`{"voices": catalog.voice_clips()}`: ملفات أصوات الشخصيات المتركّبة لكل jutsu (كلها null النهارده)
`/ws`	WebSocket	جلسة واحدة
`/references/...`	GET	ملفات ثابتة من `gesture_references/`
`/audio/...`	GET	ملفات ثابتة من `assets/audio/`
`/{path}`	GET	أبلكيشن React (SPA fallback)

الـ WebSocket handler بيقبل الاتصال الأول، وبعدين يحاول ياخد slot. لو مفيش مكان فاضي بيبعت `{"type": "error", "code": "busy", "fatal": true, "message": "All N sessions are in use. Try again later."}` ويقفل بكود 1013 (ومعناه "حاول تاني بعدين"). وإلا بيشغّل `Session(ws, server).run()`، اللي بتحرر الـ slot لما تخلص.

```
@app.websocket("/ws")
async def ws_session(ws: WebSocket):
    await ws.accept()
    if not server.slots.acquire():
        await ws.send_text(json.dumps({"type": "error", "code": "busy", "fatal": True,
                                       "message": f"All {server.slots.max} sessions are in use. Try
again later."}))
        await ws.close(code=1013)
        return
    await Session(ws, server).run()
```

خطوة خطوة:

1. `@app.websocket("/ws")`: بتقول لـ FastAPI إن الدالة دي بتتعامل مع أي WebSocket بيتفتح على `/ws`.
2. `await ws.accept()`: بتقبل الاتصال. ليه بنقبل الأول حتى لو مليانين؟ عشان نقدر نبعت رسالة `busy` مفهومة للواجهة بدل ما الاتصال يتقفل من غير سبب.
3. `server.slots.acquire()`: بتحاول تحجز مكان. لو رجّعت False يبقى فيه بالفعل جلستين شغّالين (مع الإعداد الافتراضي).
4. لو مفيش مكان: تبعت الخطأ وتقفل بـ 1013 وترجع.
5. لو فيه مكان: تشغّل الجلسة كلها بـ `Session(ws, server).run()` وتفضل مستنياها لحد ما الاتصال يخلص.

مين بيستخدمه؟ الواجهة (hook الجلسة في React) بتفتح الاتصال ده، ولو جالها `busy` بتعرض حالة "مشغول" للمستخدمة.

`/api/voices` بتنادي `catalog.voice_clips()`، اللي بتدوّر في `assets/audio/voice/<jutsu>/` على `activation.wav` و`name.wav`، وتقرا طولهم بموديول `wave` بتاع Python، وترجّع لكل واحد رابط و`seconds`، أو `null` لو الملف مش موجود أو مش PCM WAV ينفع يتقري. وكمان بترجّع `speaker` (Naruto أو Sasuke). الفولدر بيتقري مع كل طلب، فإضافة ملف صوت مش محتاجة إعادة تشغيل السيرفر. شكل الرد النهارده (كله null لأن مفيش أي صوت متركّب):

```
{"voices": {
  "fire_style":   {"speaker": "Naruto", "activation": null, "name": null, "seconds": {"activation":
null, "name": null}},
  "chidori":      {"speaker": "Sasuke", "activation": null, "name": null, "seconds": {"activation":
null, "name": null}},
  "shadow_clone": {"speaker": "Naruto", "activation": null, "name": null, "seconds": {"activation":
null, "name": null}},
  "rasengan":     {"speaker": "Naruto", "activation": null, "name": null, "seconds": {"activation":
null, "name": null}}
}}
```

(ملاحظة: المثال المكتوب في `docs/web_protocol.md` فيه `chidori` بـ `name` متركّب و`seconds` بـ 1.42، وده مثال توضيحي لشكل الرد لو ملف اتضاف، مش الحالة الحقيقية. الحالة الحقيقية إن الفولدرات الأربعة فاضية.)

الـ route الأخير `frontend(path)` (اللي بيمسك أي عنوان) بيتسجّل في الآخر، عشان الـ API والـ static mounts تكسب الأول. بيرجّع 404 لأي عنوان `/api/...` مش معروف (عشان غلطة إملائية في نداء API ما ترجعش صفحة HTML بهدوء ونفضل محتارين). لو `frontend/dist/index.html` مش موجود بيرجّع `BUILD_HELP`. لو العنوان ملف موجود جوه `dist` بيرجّعه؛ والفحص `file.is_relative_to(dist)` بعد `resolve()` بيمنع حيل الـ path traversal زي `../../secret` (يعني محاولة الوصول لملفات برّه الفولدر المسموح). أي عنوان تاني بيرجّع `index.html`، عشان الـ routes بتاعة React من ناحية العميل (زي `/practice`) تشتغل لما الصفحة تتعمل لها reload.

الدالة `main()`

`main()` بتقرا اختيارات سطر الأوامر (`--host`، `--port`، `--max-sessions`، `--ssl-certfile`، `--ssl-keyfile`)، وبتتأكد إن اختيارين الـ SSL متكتبين مع بعض، وإن `--max-sessions` على الأقل 1، وبتبني الـ app، وتطبع العنوان، وتشغّله بـ `uvicorn` (السيرفر اللي فعلياً بيشغّل تطبيقات FastAPI). الـ HTTPS مهم لأن المتصفحات ما بتسمحش بالكاميرا غير على `localhost` أو HTTPS؛ فالموبايلات على الشبكة المحلية محتاجة `--host 0.0.0.0` ومعاه شهادة (الدليل العربي بيوضّح إزاي تعملي شهادة self-signed بـ `openssl`).

ثوابت الـ config المهمة هنا:

الثابت	القيمة	بيستخدم في إيه
`SERVER_HOST` / `SERVER_PORT`	`127.0.0.1` / 8000	العنوان الافتراضي
`SERVER_MAX_SESSIONS`	2	عدد الجلسات في نفس الوقت
`FRAME_MAX_WIDTH` / `FRAME_MAX_HEIGHT`	1280 / 1280	الفريمات الأكبر بتتصغّر
`FRAME_MIN_SIZE`	32	الفريمات الأصغر بترفض
`FRAME_JPEG_QUALITY`	80	الجودة اللي المتصفح المفروض يستخدمها (بتتبعت 0.8)
`OUTPUT_JPEG_QUALITY` / `OUTPUT_JPEG_444`	90 / True	جودة الفريمات اللي راجعة
`FRAME_MAX_IN_FLIGHT`	2	الفريمات اللي من غير رد المسموحة للعميل
`FRAME_MAX_BYTES`	4 MiB	أكبر رسالة binary مقبولة
`MIRROR`	True	السيرفر بيعكس الفريمات
`PROTOCOL_VERSION` / `APP_VERSION`	1 / "0.1.0"	بيتبعتوا في `hello` و `/api/info`
`FRONTEND_DIST`، `AUDIO_DIR`، `REFERENCES_DIR`	مسارات	الأبلكيشن المبني، الأصوات، صور المراجع

كل رسايل الـ WebSocket في نظرة واحدة

من العميل للسيرفر:

الرسالة	الشكل	تأثيرها على السيرفر	الرد
فريم كاميرا	binary: header `{"t": ms}` + JPEG	بيدخل الطابور ويتعالج بالترتيب	فريم binary (FrameState + JPEG) أو `skipped`

الرسالة	الشكل	تأثيرها على السيرفر	الرد
`{"type": "mode", "mode": "free"}`	نص	الأربع jutsu ممكن يشتغلوا، حد 7 ثواني بين الخطوات	`{"type": "mode", "mode": "free", "jutsu": null}`
`{"type": "mode", "mode": "practice", "jutsu": "<id>"}`	نص	الـ jutsu دي بس، من غير حد وقت	`{"type": "mode", "mode": "practice", "jutsu": "<id>"}`
`{"type": "reset"}`	نص	بينسى كل sequence شغّالة (الـ effect الشغّال بيكمّل)	مفيش
`{"type": "options", "landmarks": true}`	نص	يرسم أو يخفي نقط الإيد (landmarks) على الفريمات الراجعة	مفيش
`{"type": "ping", "t": n}`	نص	بيترد عليه على طول	`{"type": "pong", "t": n}`

من السيرفر للعميل:

الرسالة	إمتى
`hello`	أول رسالة بعد ما الـ engine يجهز: البروتوكول، رقم الجلسة، ServerInfo، الـ mode
فريم binary	الرد على كل فريم كاميرا اتعالج
`mode`	تأكيد تغيير الـ mode
`pong`	الرد على `ping`
`skipped`	فريم اتشال (بسبب تراكم) أو كان وحش؛ بيحمل `client_t`
`error` بكود `busy`	كل الجلسات مشغولة (fatal، كود القفل 1013)
`error` بكود `bad_frame`	الفريم كبير أوي، أو الـ header بايظ، أو مش JPEG، أو مش بيتفك، أو صغير أوي (مش fatal، وبعده `skipped`)
`error` بكود `bad_message`	رسالة نصية مش صالحة (مش fatal)
`error` بكود `internal`	الـ engine فشل يقوم أو الجلسة وقعت (fatal، كود القفل 1011)

شكل رسالة الخطأ دايماً: `{"type": "error", "code": "...", "message": "...", "fatal": false}` (و`fatal` بـ true في `busy` و`internal`).

الـ events اللي جوه FrameState

قائمة `events` بيطلّعها `EventTracker` في `shinobi/events.py` (متشرح في جزء تاني). المتصفح بيشغّل كل صوت وكل أنيميشن في الواجهة من الـ events دي.

الـ event	الحقول (payload)	إمتى
`sign`	`sign`	الـ stabilizer أكّد حركة (مرة واحدة لكل حركة ممسوكة)
`step`	`jutsu`، `step`، `total`، `sign`	حركة متأكدة قدّمت jutsu خطوة (مش بيتبعت للخطوة الأخيرة)
`wrong_sign`	`jutsu`، `step`، `expected`، `got`	الحركة المتأكدة مش هي اللي عليها الدور؛ مفيش أي reset

الـ event	الحقول (payload)	إمتى
`sequence_failed`	`reason` ("timeout")، `jutsu` ، `step` ، `expected` ، `got`	free mode بس: عدّى أكتر من 7 ثواني من غير الحركة الجاية، فالـ jutsu دي بدأت من الأول
`jutsu`	`jutsu` ، `effect` ، `stages`	الـ sequence كلها اتأكدت؛ الـ build-up بيبدأ؛ فيها `stages` ثواني `voice` و `charge` و `name`
`effect_phase`	`effect` ، `jutsu` ، `phase`	التفعيل دخل `charge` أو `name` أو `release` (كل واحدة مرة، بالترتيب)
`effect_stopping`	`effect`	الـ effect بدأ يختفي تدريجياً
`effect_end`	`effect`	الـ effect خلص واختفى (وكمان لـ effect اتبدّل بـ jutsu جديدة)
`clones`	`count`	Shadow Clone: نسخ أكتر بقت ظاهرة (بيستخدم لتوقيت أصوات الدخان)

أمثلة للـ payload بالظبط زي ما في البروتوكول:

```
{"type": "sign", "sign": "tiger"}
{"type": "step", "jutsu": "chidori", "step": 2, "total": 4, "sign": "dog"}
{"type": "wrong_sign", "jutsu": "chidori", "step": 2, "expected": "tiger", "got": "boar"}
{"type": "sequence_failed", "reason": "timeout", "jutsu": "chidori", "step": 2, "expected": "tiger",
"got": null}
{"type": "jutsu", "jutsu": "chidori", "effect": "chidori", "stages": {"voice": 0.4, "charge": 1.2,
"name": 0.4}}
{"type": "effect_phase", "effect": "chidori", "jutsu": "chidori", "phase": "charge"}
{"type": "effect_stopping", "effect": "chidori"}
{"type": "effect_end", "effect": "chidori"}
{"type": "clones", "count": 3}
```

الترتيب جوه الفريم الواحد: `sequence_failed` ، `sign` ، `wrong_sign` ، `step` ، `effect_end` ، `jutsu` ، `effect_phase` ، `effect_stopping` ، `clones` .

ملاحظات مهمة من البروتوكول:

- `step` معناه "عدد الخطوات اللي خلصت دلوقتي". لو حركة واحدة قدّمت أكتر من jutsu، بييجي `step` لكل واحدة.
- `wrong_sign` عمره ما بيعمل reset: الخطوات اللي خلصت بتفضل، والمستخدمة تحاول تاني الحركة المطلوبة كام مرة ما هي عايزة.
- لو مرحلة اتنطّت بسبب فريم بطيء، بتتبلّغ برضه: كل المراحل بالترتيب في الفريم اللي شاف المرحلة اللي بعدها.
- `effect_stopping` دايماً قبل `effect_end` لنفس التفعيل. والـ effect اللي اتبدّل بياخد `effect_end` بس.
- `clones` بيتبعت لما عدد النسخ الظاهرة يبقى أكبر من أي وقت فات في التفعيل ده.

إزاي `server.py` متوصّل بباقي الملفات

- `engine.py` بيعمل كل شغل الـ AI والتأثيرات؛ السيرفر بس بيفك الصورة، وينادي `engine.process` ، ويضغط النتيجة.
- `events.py` بيحوّل حالة كل فريم لـ events؛ والسيرفر بيبني الـ `FrameView` اللي هو محتاجه.

- `catalog.py` بيدّي نصوص الكروت، والـ credits، واكتشاف ملفات أصوات الشخصيات.
- `config.py` بيدّي كل الثوابت.
- الواجهة (`frontend/src/...`) بتنفّذ النص التاني من البروتوكول؛ و`docs/web_protocol.md` هو العقد اللي الاتنين ماشيين عليه.

إزاي التفعيل النهائي للـ jutsu بيحصل (من آخر حركة لحد الـ effect)

قبل ما ندخل في ملفات التأثيرات، خلّينا نجمّع الصورة كاملة لأهم لحظة في السيستم: لحظة ما الـ jutsu "تنطلق". الحتة دي موزّعة على كذا ملف، فهنا هنربطها ببعض.

1. المستخدمة بتعمل الحركات الأربعة بالترتيب. كل حركة الـ stabilizer بيأكدها بتطلّع event اسمه `sign`، ولو قدّمت jutsu بيطلع `step` (الخطوات 1 و 2 و 3).
2. لما الحركة الرابعة تتأكد، الـ `ActionController` في الـ engine بيقول "الـ sequence اكتملت". لو مفيش cooldown شغّال (`COOLDOWN_SECONDS` = 1.5 ثانية بعد آخر jutsu)، الـ engine بينادي `start_effect(state.effect, now, jutsu=state.jutsu, staged=True)`.
3. `start_effect` بتنادي `activation_stages(jutsu)` من `shinobi/effects/__init__.py` عشان تعرف طول كل مرحلة: `voice` و`charge` و`name`. النهارده مفيش أي صوت شخصية متركّب، فالنتيجة دايماً `{"voice": 0.4, "charge": 1.2, "name": 0.4}`.
4. `EffectManager.start(name, now, stages)` بيبني object جديد للـ effect من `EFFECTS` وينادي `begin(now, stages)`. أي effect كان شغّال بيتبدّل.
5. في نفس الفريم، `EventTracker` بيطلّع `effect_end` للـ effect القديم (لو كان فيه)، وبعدين `jutsu` ومعاه `stages`.
6. طول الـ 2.0 ثانية الجاية، `EffectManager.render` بينادي `render_charge(...)` بتاعة الـ effect: صورة "تجميع الشاكرا" اللي بتكبر مع `charge_level`.
7. لما وقت كل مرحلة يخلص، الـ tracker بيطلّع `effect_phase` بـ `charge` (بعد 0.4 ثانية)، وبعدين `name` (بعد 1.6 ثانية)، وبعدين `release` (بعد 2.0 ثانية).
8. عند `release_at` بالظبط، `effect.start(effect.release_at)` بيتنادى، ومن الفريم ده `render(...)` هو اللي بيرسم الـ effect الأساسي لمدة `duration` بتاعته.
9. لما `age >= duration`، `opacity()` بتبدأ الـ fade out تلقائياً، فالـ tracker بيطلّع `effect_stopping`. ولما الـ fade يخلص (وكل الجزيئات تختفي في Fire و Shadow Clone)، الـ effect بيتشال ويطلع `effect_end`.
10. الواجهة بتسمع كل الـ events دي وتشغّل الأصوات المناسبة لكل مرحلة (التفاصيل في قسم الصوت).

الخط الزمني بالأرقام الافتراضية:

الوقت من لحظة الانطلاق	المرحلة	اللي بيحصل على الشاشة	اللي بيحصل في الصوت
0.0 ثانية	`voice` تبدأ (event `jutsu`)	بداية تجميع الشاكرا (charge level من 0 لـ 0.2)	صوت الختم للحركة الرابعة + لمسة الـ jutsu؛ وصوت الشخصية `activation` لو كان متركّب (مش متركّب)

الوقت من لحظة الانطلاق	المرحلة	اللي بيحصل على الشاشة	اللي بيحصل في الصوت
0.4 ثانية	event) `charge` (`effect_phase`	الشحن بيزيد (من 0.2 لـ 1.0)	صوت الشحن الخاص بالـ jutsu
1.6 ثانية	event) `name` (`effect_phase`	الشحن كامل (1.0)	الشحن مكمّل؛ وصوت اسم الـ jutsu `name` لو كان متركّب (مش متركّب)
2.0 ثانية	event) `release` (`effect_phase`	الـ effect الأساسي بيبدأ	الشحن بيقف بـ fade، وأصوات الانطلاق والضربة والـ loop بتشتغل
2.0 + المدة	`effect_stopping`	الـ fade out (0.5 ثانية، أو 0.6 لـ Shadow Clone)	صوت النهاية وإيقاف الـ loop
بعدها	`effect_end`	الـ effect اختفى	الـ loop بيقف نهائياً

مدد الـ effect الأساسي (`config.EFFECT_DURATION`): Shadow Clone 10 ثواني، Fire 6، Chidori 8، Rasengan 8.
يعني مثلاً Fire Style بتخلص "رسمياً" عند 2.0 + 6.0 = 8.0 ثواني من الانطلاق، وبعدين الـ fade.

حزمة التأثيرات (effects package): نظرة عامة

الفولدر `shinobi/effects/` فيه:

الملف	دوره
`__init__.py`	قاموس المصنع `EFFECTS` والدالة `activation_stages()`
`base.py`	الكلاس الأساسي `Effect` (الخط الزمني، الظهور والاختفاء التدريجي، مراحل الـ build-up) و `EffectManager`
`fx.py`	أدوات رسم مشتركة: `add_glow` ، `color_ramp` ، `Particles` ، `random_directions`
`fire.py`	Fire Style (`FireStyle`)
`chidori.py`	Chidori (`Chidori` ، `lightning_path`)
`rasengan.py`	Rasengan (`Rasengan`)
`shadow_clone.py`	Shadow Clone (`ShadowClone` ، `plan_row` ، `clearance`)

شوية مفردات بتتكرر في كل الملفات دي:

- **frame**: مصفوفة NumPy شكلها `(height, width, 3)` ، ونوعها `uint8` (أرقام من 0 لـ 255)، بترتيب ألوان **BGR** بتاع OpenCV (أزرق، أخضر، أحمر). يعني لون زي `(255, 150, 40)` ده أزرق، مش أحمر! دي نقطة بتلخبط ناس كتير.
- **layer (طبقة)**: صورة سودا بنفس المقاس، الـ effect بيترسم عليها الأول، وبعدين بتتضاف على الفريم.
- **Additive blending (الدمج بالجمع)**: قيم البكسلات بتتجمع على بعض (نور فوق نور)، ولو عدّت 255 بتقف عند 255. ده اللي بيخلّي التأثيرات المضيئة تبان كأنها نور حقيقي.
- **alpha**: درجة الشفافية/الوضوح من 0 (مخفي خالص) لـ 1 (ظاهر خالص).
- **mask (قناع)**: صورة بقيم من 0 لـ 1 بتقول كل بكسل تبع إيه (مثلاً 1 = شخص، 0 = خلفية).

- **particle (جُزيء)**: نقطة صغيرة بتتحرك ولها عمر (زي شرارة أو شعلة أو دخان). مئات منها مع بعض بيعملوا نار أو دخان.
- `tracking` : object من نوع `Tracking` من `shinobi/tracking.py` فيه: `now` (الوقت)، `visible` (الإيد ظاهرة، أو ضاعت من أقل من `HAND_LOST_GRACE` = 0.6 ثانية)، `anchor` (مركز كف الإيد بعد التنعيم بالبكسل)، `size` (حجم الإيد بعد التنعيم بالبكسل)، و`direction` (متجه طوله 1 من الرسغ لمفصل الصباع الوسطاني). القيم بتتنعّم بفلاتر One Euro عشان التأثيرات ما ترتعشش.
- `dt` : الوقت بالثواني من الفريم اللي فات. كل حركة بتتضرب في `dt` ، فالتأثيرات بتتحرك بنفس السرعة الحقيقية سواء السيرفر شغّال 5 أو 30 فريم في الثانية.
- `segmentation` : `(small_frame, mask)` من `shinobi/segmentation.py` ، وبيتحسب بس لما الـ effect الشغّال محتاجه (Shadow Clone).

الملف `shinobi/effects/__init__.py`

مسؤول عن إيه؟

الملف ده هو "المنيو" بتاع التأثيرات. باقي الكود عمره ما بيعمل objects تأثيرات بنفسه مباشرة؛ بيطلبها من الملف ده.

القاموس `EFFECTS`

```
EFFECTS = {
    "shadow_clone": lambda: ShadowClone(config.EFFECT_DURATION["shadow_clone"]),
    "rasengan": lambda: Rasengan(config.EFFECT_DURATION["rasengan"]),
    "chidori": lambda: Chidori(config.EFFECT_DURATION["chidori"]),
    "fire": lambda: FireStyle(config.EFFECT_DURATION["fire"]),
}
```

بيعمل إيه؟ كل مفتاح هو id الـ effect، وكل قيمة هي دالة (`lambda`) بتبني object تأثير جديد خالص.

ليه دالة ومش object جاهز؟ ده مهم: كل تفعيل محتاج حالة جديدة (جزيئات جديدة، تاريخ نسخ جديد، مؤقتات جديدة). لو خزّنا objects جاهزة، التفعيل التاني كان هيبدأ ببواقي التفعيل الأول (مثلاً دخان قديم أو نسخ قديمة).
المدد جاية من `config.EFFECT_DURATION` : Shadow Clone 10 ثواني، Fire 6، Chidori 8، Rasengan 8 (الـ effect الأساسي بس، من غير الـ build-up ومن غير الـ fade out).

مين بيستخدمه؟ الـ engine بيعمل `EffectManager(EFFECTS)` ، والـ manager بينادي `self.factories[name]()` كل مرة jutsu تنطلق.

الدالة `activation_stages(jutsu, audio_dir=None)`

بترجّع خطة الـ build-up لـ jutsu انطلقت: `{"voice": s, "charge": s, "name": s}` .

- `voice` و`name` : لو ملف صوت الشخصية `assets/audio/voice/<jutsu>/activation.wav` (أو `name.wav`) متركّب، المدة = طوله + `config.VOICE_GAP` (0.05 ثانية)؛ وإلا `config.VOICE_DEFAULT` (0.4 ثانية). الطول بيتقري بـ `catalog.voice_seconds` .

- `charge`: `config.ACTIVATION_CHARGE[jutsu]` (1.2 ثانية للأربعة)، أو `ACTIVATION_CHARGE_DEFAULT` (1.2).

```
def spoken(part):
    clip = seconds.get(part)
    return config.VOICE_DEFAULT if clip is None else clip + config.VOICE_GAP
```

الدالة الداخلية دي بتقول: لو مفيش ملف (`None`) خد 0.4، ولو فيه خد طوله + 0.05. مثال افتراضي: لو بعدين اتضاف `rasengan/name.wav` طوله 0.9 ثانية بترخيص سليم، مرحلة `name` للـ Rasengan هتبقى 0.95 ثانية تلقائياً، من غير أي تعديل في الكود.

بما إن مفيش أي ملف صوت متركّب النهارده، كل jutsu حالياً بتعمل build-up لمدة 0.4 + 1.2 + 0.4 = 2.0 ثانية قبل الـ effect الأساسي. الـ engine بينادي الدالة دي في `start_effect(..., staged=True)`، والخطة بتتبعت للمتصفح في `stages` بتاعة event الـ `jutsu`، فالأصوات والرسم بيفضلوا ماشيين مع بعض.

الملف `shinobi/effects/base.py`

مسؤول عن إيه؟

بيعرّف الحاجات المشتركة بين كل التأثيرات: الخط الزمني (مراحل الـ build-up، الـ effect الأساسي، الظهور التدريجي، الاختفاء التدريجي، النهاية)، و`EffectManager` اللي بيشغّل الـ effect الوحيد الشغّال.

الخط الزمني: voice و charge و name و release

الـ jutsu اللي انطلقت بتسلسل حركاتها ما بتعرضش الـ effect الأساسي على طول. بتعدّي على أربع مراحل (الثابت `PHASES = ("voice", "charge", "name")` بيحدد ترتيب التلاتة الأولى):

1. `voice`: المفروض الشخصية تزعّق بنداء التفعيل (لو فيه ملف صوت متركّب). الشاكرا بتبدأ تتجمّع (charge level من 0 لـ 0.2).
2. `charge`: الشاكرا بتتشحن (charge level من 0.2 لـ 1.0).
3. `name`: المفروض الشخصية تقول اسم الـ jutsu. الـ charge level بيفضل 1.0.
4. `release`: الـ effect الأساسي بيشتغل لمدة `duration`، وبعدين يختفي تدريجياً.

في التلات مراحل الأولى، `render_charge()` بتاعة الـ effect بترسم صورة أصغر لـ "تجميع الشاكرا". الـ effect اللي اتشغّل باليد (أزرار 1-4 في نسخة الديسكتوب، أو الاختبارات) ما لوش build-up وبيبقى في `release` من أول فريم.

```
jutsu fires                        release_at                          duration   fade_out
    |-- voice --|---- charge ----|-- name --|======== main effect ========|~~ fade ~~| gone
      0.4 s          1.2 s          0.4 s
```

الكلاس `Effect`

خصائص على مستوى الكلاس (كل effect بيغيّرها لنفسه):

الخاصية	الافتراضي	المعنى
`name`	`"base"`	id الـ effect
`needs_segmentation`	False	لو True الـ engine بيشغّل الـ person segmenter عشان الـ effect ده
`follows_hand`	False	لو True الـ effect بيترسم عند الإيد المتتبعة
`fade_in`	0.3 ثانية	الوضوح بيطلع من 0 لـ 1 في المدة دي
`fade_out`	0.5 ثانية	الوضوح بينزل لـ 0 في المدة دي بعد الإيقاف

حقول كل object: `duration`، `t0` (بداية الـ effect الأساسي، و`None` أثناء الـ build-up)، `stop_at` (إمتى الـ fade out بدأ)، `age` (ثواني من الـ release)، `activated_at` (إمتى الـ jutsu انطلقت)، `stages` (قائمة `(phase, seconds)`).

الدوال:

- `begin(now, stages=None)`: بتبدأ تفعيل. لو جالها dictionary مراحل بتخزّن التلات مراحل (القيم السالبة بتبقى 0)؛ ولو مفيش بتنادي `start(now)` على طول.
- `stage_plan` (property): `{phase: seconds}` متقرّبة؛ بتستخدم في event الـ `jutsu`.
- `release_at` (property): `مجموع أطوال المراحل + activated_at`.
- `building` (property): True طول ما `t0` بـ `None` وفيه مراحل.
- `phase(now)`: بترجّع `(phase, seconds_left)` بإنها تمشي على المراحل واحدة واحدة؛ و`("release", None)` أول ما الـ build-up يخلص.
- `charge_level(now)`: من 0 لـ 1 زي ما اتشرح فوق. في `voice` بتطلع بخط مستقيم من 0 لـ 0.2؛ وبعدين من 0.2 لـ 1.0 على طول مرحلة `charge`؛ وبعدين تفضل 1.0.
- `render_charge(frame, tracking, dt, segmentation, level, alpha)`: بترسم الـ build-up. النسخة الأساسية بترجّع الفريم زي ما هو؛ وكل effect بيكتب نسخته.
- `start(now)`: بتحط `t0`.
- `stop(now)`: بتحط `stop_at` مرة واحدة (لو اتنادت تاني ما بتعملش حاجة).
- `min_on_screen` (property): ثواني الـ effect الأساسي قبل ما ضياع الإيد يسمح بإنهائه. مع `config.HAND_EFFECT_MIN_SECONDS = None` بتساوي `duration` كلها، يعني تأثيرات الإيد عمرها ما بتخلص بدري لأن MediaPipe ضيّع الإيد. الأختام اللي بإيدين ساعات كتير بتخبّي الإيد، فده بيخلّي الـ effect يفضل على الشاشة في آخر مكان معروف للإيد.
- `opacity(now)`: غلاف الظهور والاختفاء. وكمان بتحدّث `age` وبتبدأ الـ fade out تلقائياً لما `age >= duration`.
- `finished(now)`: True أول ما الـ fade out يكمل.
- `render(...)`: لازم كل effect يكتبها بنفسه.

```
def opacity(self, now):
    self.age = now - self.t0
    if self.stop_at is None and self.age >= self.duration:
        self.stop_at = now
    a = min(1.0, self.age / self.fade_in)
    if self.stop_at is not None:
        a = min(a, max(0.0, 1.0 - (now - self.stop_at) / self.fade_out))
    return a
```

بتعمل إيه خطوة خطوة:

1. `self.age = now - self.t0`: تحسب عمر الـ effect الأساسي من ساعة الـ release.
2. لو لسه ما اتوقفش و`age` وصل لـ `duration`: تحط `stop_at = now`، يعني "ابدأ الاختفاء دلوقتي". ده اللي بيخلّي الـ effect يخلص لوحده.
3. `a = min(1.0, age / fade_in)`: الظهور التدريجي. مع `fade_in` = 0.3: عند 0.15 ثانية `a` = 0.5، وعند 0.3 أو أكتر `a` = 1.
4. لو فيه `stop_at`: تحسب الاختفاء `(الوقت من الإيقاف - 1) / fade_out`. مع `fade_out` = 0.5: بعد 0.25 ثانية من الإيقاف القيمة 0.5، وبعد 0.5 ثانية القيمة 0. وبتاخد الأصغر بين الظهور والاختفاء.
5. ترجّع `a` من 0 لـ 1، والـ effect بيستخدمها كـ alpha للرسم.

مثال: Chidori مدتها 8 ثواني. عند `age` = 8.0 الـ `stop_at` بيتحط، وعند 8.25 الـ alpha = 0.5، وعند 8.5 الـ alpha = 0 و`finished()` بترجّع True.

الكلاس `EffectManager`

شايل قاموس `factories` (يعني `EFFECTS`) و`active`، الـ effect الوحيد الشغّال (أو `None`). effect واحد بس بيشتغل في المرة.

- `start(name, now, stages=None)`: بيبني effect جديد من المصنع وينادي `begin`. أي effect شغّال بيتبدّل ببساطة.
- `stop(now)`: بيوقف الـ effect الشغّال (فيبدأ يختفي تدريجياً).
- `needs_segmentation` (property): True لو الـ effect الشغّال محتاج الـ segmenter. الـ engine بيبص عليها قبل كل فريم، فالـ segmentation الغالية بتشتغل بس طول ما Shadow Clone شغّال.
- `render(frame, tracking, dt, segmentation)`: المنطق بتاع كل فريم:

1. مفيش effect شغّال: رجّع الفريم زي ما هو.
2. لسه في الـ build-up واتوقف بدري: ارسم الـ build-up بـ alpha بيقل؛ وشيل الـ effect لما الـ fade يوصل 0.
3. لسه في الـ build-up و`now < release_at`: ارسم الـ build-up بـ alpha كامل.
4. الـ build-up لسه خالص: نادي `effect.start(effect.release_at)`. لاحظي إنه بيبدأ في الوقت المخطط بالظبط، مش "دلوقتي"، فالفريم البطيء ما بيزحزحش الخط الزمني.
5. لتأثيرات الإيد: لو الإيد مش ظاهرة، والـ effect مش بيتوقف أصلاً، والـ effect الأساسي اشتغل على الأقل `min_on_screen`، وقّفه.
6. لو الـ effect خلص، حط `active = None` ورجّع الفريم.
7. وإلا رجّع `effect.render(...)`.

الارتباطات

الـ engine (`engine.py`) بيعمل `EffectManager(EFFECTS)`، وبينادي `start` لما الـ controller يطلق jutsu، وبينادي `render` كل فريم، وبيقرا `effect.phase(now)` و`stage_plan` و`stop_at` و`activated_at` و(لـ Shadow Clone) `drawn` و`crowded` عشان يملا الـ FrameState.

الملف `shinobi/effects/fx.py`

مسؤول عن إيه؟

أدوات رسم مشتركة ومتظبطة على السرعة، بتستخدمها تأثيرات الإيد التلاتة (والجزيئات كمان بيستخدمها دخان Shadow Clone).

الدالة `add_glow(frame, layer, center, extent, strength=1.0, blur=25, opacity=1.0, soften=0.0, backdrop=0.0)`

بتضيف `layer` على `frame` بالجمع، وبتضيف هالة ناعمة مغبّشة حواليها (اسمها "bloom"). ده اللي بيخلّي شكل مرسوم يبان كأنه نور منوّر. وبترجّع فريم جديد.

المدخلات: `center` مركز الـ effect، `extent` نص حجمه تقريباً، `strength` قوة الهالة، `blur` قد إيه الهالة واسعة، `opacity` وضوح الكل، `soften` تنعيم القلب نفسه، `backdrop` قد إيه نضلّم تحت النور.

خطوة خطوة:

1. تحسب منطقة مربعة حوالين `center` (اسمها ROI، يعني "منطقة الاهتمام") نص ضلعها `extent + 2 * blur`، ومقصوصة على حدود الفريم. لو أصغر من 8 بكسل (يعني الـ effect برّه الشاشة) ترجّع نسخة من الفريم.
2. لو `soften > 0`، تغبّش الطبقة الحادة شوية بنص الدقة (أرخص 4 مرات). ده بيدوّب الجزيئات المنفصلة في شكل واحد، وبيستخدمه Fire.
3. تعمل الهالة: تصغّر الـ ROI بتاع الطبقة 4 مرات (أو 8 لو `blur > 40`)، وتغبّشها هناك بـ sigma قيمتها `blur / shrink`، وترجّع تكبّرها. الهالة ناعمة أصلاً، فتغبيش صورة صغيرة بيدّي نفس الشكل وأسرع بكتير.
4. لو `backdrop > 0`، تضلّم الصورة تحت النور الأول، وأقوى تضليم مكان ما الهالة أنور. السبب المكتوب في الكود: النور المضاف ما يبانش على حيطة بيضا (هي أصلاً 255)، فلما الأوضة منوّرة، شوية ضل تحت الـ effect بيخلّيه زاهي.
5. تضيف القلب الحاد (`addWeighted` بـ `opacity`) والهالة (`strength * opacity`). الـ `addWeighted` بتاعة OpenCV على `uint8` بتقف عند 255 بدل ما ترجع تلف من الصفر.

خدعتين للسرعة باختصار: بنعالج بس الـ ROI حوالين الـ effect، عمرنا ما بنعالج الفريم كله؛ والتغبيش بيتعمل على نسخ متصغّرة.

مين بيستخدمها؟ `FireStyle` و`Chidori` و`Rasengan` في آخر `render` و`render_charge`.

الدالة `color_ramp(t, stops)`

بتعمل تدرّج ألوان. `t` مصفوفة قيم من 0 لـ 1 (زي أعمار الجزيئات) و`stops` قائمة `(position, (b, g, r))`. بتستخدم `np.interp` لكل قناة لون وبترجّع مصفوفة `(N, 3)`. Fire بيستخدمها عشان الجزيء يبدأ أبيض من السخونية، وبعدين أصفر، برتقاني، أحمر، وبعدين دخان غامق وهو بيكبر في السن.

الكلاس `Particles`

نظام جزيئات متخزّن على شكل "structure of arrays": مصفوفة NumPy لكل خاصية، وصف لكل جزيء، من غير object Python لكل جزيء. ده بيخلّي تحديث مئات الجزيئات مجرد كام عملية على متجهات.

المصفوفة	الشكل	المعنى
`pos`	(N, 2)	المكان بالبكسل (أو بالكسور في دخان Shadow Clone)
`vel`	(N, 2)	السرعة في الثانية
`life`	(N,)	الثواني اللي عاشها لحد دلوقتي
`max_life`	(N,)	عمره الكلي
`size`	(N,)	نص القطر

- `emit(pos, vel, max_life, size)`: بتضيف `n = len(vel)` جزيء جديد، كلهم بيبدأوا من `pos`.
- `update(dt, gravity=(0, 0), drag=0)`: تكامل Euler بسيط: السرعة += الجاذبية في dt، والسرعة بتقل بالـ drag (المقاومة)، والمكان += السرعة في dt، والعمر += dt. وبعدين كل المصفوفات بتتفلتر بـ mask منطقي `life < max_life`، فالجزيئات الميتة بتتشال في خطوة واحدة.
- `t` (property): العمر من 0 (اتولد) لـ 1 (بيموت).
- `draw(layer, colors, sizes=None, positions=None)`: بترسم كل جزيء دايرة مليانة بحواف ناعمة (anti-aliased). المصفوفات بتتحوّل لقوائم Python مرة واحدة قبل اللفة، والتعليق بيقول إن ده أسرع بكتير من تحويل كل قيمة جوه اللفة. الجزيئات اللي برّه الطبقة بتتنط.

الدالة `random_directions(n, rng, center_angle=None, spread=pi)`

بترجّع `n` متجه طول كل واحد 1. لو فيه `center_angle`، الزوايا بتبقى في حدود زائد أو ناقص `spread` منه (مخروط، بيستخدم لنفاثة النار)؛ ولو مفيش، أي اتجاه.

الملف `shinobi/effects/fire.py`

مسؤول عن إيه؟

Fire Style (Great Fireball): تيار من جزيئات اللهب طالع من الإيد. id الـ effect هو `fire`، وبيطلقه الـ jutsu اللي اسمها `fire_style`. و`follows_hand = True` (بيترسم عند الإيد).

الثوابت

- `FIRE_RAMP`: خمس محطات ألوان حسب عمر الجزيء: أبيض من السخونية `(190, 250, 255)` عند 0، أصفر عند 0.15، برتقاني عند 0.40، أحمر عند 0.70، دخان غامق `(10, 10, 30)` عند 1.0 (كلها BGR).
- `EMBER_RAMP`: للجمرات بتاعة الـ build-up: برتقاني وهي بعيدة، وأبيض من السخونية لما توصل لنقطة التجمّع.

الكلاس `FireStyle`

الـ constructor `FireStyle(duration, rate=650, seed=None)`: مولّد أرقام عشوائية، ومعدّل الإطلاق `rate` (650 جزيء لهب في الثانية)، ونظامين جزيئات: `flames` (الـ effect الأساسي) و`embers` (الـ build-up).

`_origin(tracking, size)` (static): النقطة اللي اللهب بيطلع منها، `anchor + direction * size * 0.9`، يعني بعد الصوابع بشوية في الاتجاه اللي الإيد شاورة عليه. لو الاتجاه مش معروف، بيشاور لفوق `(0, -1)`.

`render_charge(...)`: الـ build-up. الجمرات بتتولد على حلقة حوالين نقطة الخروج (على بعد من 1.0 لـ 1.8 حجم إيد، والحلقة بتضيق كل ما الشحن يزيد) بسرعة متوجهة ناحية النقطة زائد لفّة جانبية، فبتتشفط وتلف حلزوني. معدّل الإطلاق بيزيد مع `level` (من 80 لـ 400 في الثانية). دايرتين مليانين بيعملوا توهّج حراري (برتقاني غامق بزّه، وأصفر منوّر جوه) نص قطره بيكبر من 0.15 لـ 0.5 حجم إيد. وكل ده بيعدّي على `add_glow` بقوة ووضوح و backdrop بيزيدوا مع `level`.

`render(...)`: الـ effect الأساسي.

1. `self.opacity(now)` بتحدّث `age` وبتبدأ الـ fade out في آخر المدة.
2. لو مكان الإيد مش معروف، رجّع الفريم.
3. طول ما مش بيتوقف، أطلق `rate * dt` جزيء (بيزيد تدريجياً في أول 0.3 ثانية) جوه مخروط زاويته 0.35 radian حوالين اتجاه الإيد، بسرعة من 3 لـ 7 أحجام إيد في الثانية، من نقط عشوائية شوية جنب نقطة الخروج. العمر من 0.45 لـ 0.9 ثانية، والحجم من 0.09 لـ 0.2 حجم إيد.
4. حدّث اللهب بـ "جاذبية" لفوق قيمتها `-3 * size` (الهوا السخن بيطلع لفوق؛ وفي إحداثيات الصورة y السالب معناه فوق) و drag قوي 2.2 (النفاثة بتبطّأ وتنتفخ). والجمرات اللي فاضلة بتكمّل تتحرق.
5. لوّن حسب العمر بـ `FIRE_RAMP`، مضروب في `1 - t^2` عشان تبهت وهي بتبرد؛ والحجم بيكبر مع العمر (`1 + 2t`)، فاللهب بيتمدد ويندمج.
6. ارسم اللهب بنص الإضاءة ("dim: overlaps add up to bright"): مكان ما جزيئات كتير بتتداخل، الطبقة المجمّعة بتبقى أصفر-أبيض منوّر، زي قلب اللهب الحقيقي.
7. احسب المربع اللي حاوي كل اللهب وانادي `add_glow` حواليه بس، بـ `soften` (عشان الجزيئات تدوب في بعض) و`backdrop=0.35`.

`finished(now)` متكتوبة من جديد: الـ effect ما بيخلصش غير لما الـ fade out يخلص **و** كل شعلة تتحرق للآخر، فالنار عمرها ما بتختفي فجأة.

ولما الـ effect بيتوقف، مفيش جزيئات جديدة بتتطلق، فالنار بتموت طبيعي بدل ما تختفي كبلوك واحد.

مثال: إيد حجمها 120 بكسل. نقطة الخروج على بعد 108 بكسل (0.9 × 120) قدام الكف. الجزيئات بتطلع بسرعة من 360 لـ 840 بكسل في الثانية، والجاذبية لفوق 360 بكسل/ثانية^2. على 20 فريم في الثانية، كل فريم بيطلّع حوالي 33 جزيء جديد.

الملف `shinobi/effects/chidori.py`

مسؤول عن إيه؟

Chidori: صواعق برق بتطقطق حوالين الإيد. id الـ effect هو `chidori`. و`follows_hand = True`.

الثوابت

`GLOW = (255, 185, 100)` (أزرق كهربي فاتح، BGR) و `CORE = (255, 245, 225)` (أبيض من السخونية، مايل للأزرق شوية).

الدالة `lightning_path(start, end, rng, detail=5, roughness=0.28)`

بتولّد خط برق متعرّج (polyline) بخوارزمية "midpoint displacement" (إزاحة نقطة النص):

1. ابدأ بخط مستقيم من `start` لـ `end`.
2. لكل جزء، خد نقطة نصه وزقّها لجنب (على العمودي بتاع الجزء) بمقدار عشوائي لحد `offset`.
3. حط نقط النص بين النقط القديمة (القائمة بتتضاعف).
4. قسّم `offset` على 2 وكرّر `detail` مرة.

التعرّجات الكبيرة بتيجي من أول اللفات، والرعشة الصغيرة من اللفات الأخيرة، وده بيبان زي البرق الحقيقي. مع `detail=5` الصاعقة فيها 33 نقطة. الـ offset الأولاني = `length * roughness`.

```
for _ in range(detail):
    mids = (points[:-1] + points[1:]) / 2
    seg = points[1:] - points[:-1]
    normal = np.stack([-seg[:, 1], seg[:, 0]], axis=1)
    normal /= np.linalg.norm(normal, axis=1, keepdims=True) + 1e-6
    mids += normal * rng.uniform(-offset, offset, (len(mids), 1))
    out = np.empty((len(points) + len(mids), 2), np.float32)
    out[0::2], out[1::2] = points, mids
    points, offset = out, offset / 2
```

خطوة خطوة:

1. `mids`: نقطة نص كل جزء (متوسط كل نقطتين ورا بعض).
2. `seg`: متجه كل جزء (النقطة اللي بعدها ناقص اللي قبلها).
3. `normal`: العمودي على كل جزء. لو الجزء `(x, y)`، العمودي `(-y, x)`.
4. القسمة على الطول بتخلّي العمودي طوله 1 (والـ `1e-6` عشان ما نقسمش على صفر).
5. `mids += normal * rng.uniform(-offset, offset, ...)`: زق كل نقطة نص لجنب بمقدار عشوائي.
6. `out[0::2], out[1::2] = points, mids`: القديم في الأماكن الزوجية والجديد في الفردية، فالنقط بتفضل بالترتيب.
7. `offset / 2`: كل لفة الزقّة بتبقى نص اللي قبلها.

العدد: 2 نقط ← 3 ← 5 ← 9 ← 17 ← 33 بعد 5 لفات.

الكلاس `Chidori`

الـ constructor `Chidori(duration, n_bolts=7, seed=None)`: مولّد عشوائي، عدد الصواعق، قائمة `bolts` فيها `(polyline, thickness)` منسوبة لـ (0, 0)، الوقت `next_regen` بتاع التجديد الجاي، ونظام جزيئات `sparks` (شرارات).

`_regenerate(radius, n_bolts=None, reach=(0.7, 1.8), branches=0.6)`: بتعمل صواعق جديدة في اتجاهات عشوائية بطول من 0.7 لـ 1.8 مرة `radius`. وباحتمال `branches` الصاعقة بياخد فرع جانبي أرفع بيتفرّع من نقطة عشوائية في التلتين الأخيرين منها، بزاوية 0.8 radian وبطول 0.4 من طولها. الصواعق بتتخزّن منسوبة لـ (0, 0) وبتتزق لمكان الإيد وقت الرسم، فالإيد المتحركة مش محتاجة صواعق جديدة.

`_draw_bolts(layer, offset, scale)`: بترسم كل صاعقة مرتين: الأول خط عريض بلون التوهّج متخفّف (70 % عشان التداخلات تفضل تحت الأبيض)، وبعدين خط قلب رفيع منوّر. عرض الخطوط بيتناسب مع حجم الإيد (`scale`، وأقصاه 1.8)، فالصواعق تفضل تخينة لما الإيد تبقى قريبة من الكاميرا.

`render_charge(...)`: الـ build-up. نص القطر بيكبر من 0.3 لـ 0.75 من النص قطر الكامل مع `level`. كل 0.06 ثانية بتتولّد طقطقات صغيرة جديدة: من 2 لـ 7 صواعق، بمدى أقصر (0.35 لـ 1.0) وفروع أقل. نقطة منوّرة صغيرة عند الكف بتكبر. شرارات بتتطلق على حلقة حوالين الإيد وبتجري لجوه (سرعتها متظبطة بحيث توصل للمركز في آخر عمرها). و`add_glow` بقوة ووضوح و backdrop بيزيدوا.

`render(...)`: الـ effect الأساسي.

1. `alpha = self.opacity(now)`؛ ارجع لو مفيش مكان للإيد أو الـ alpha صفر.
2. `radius = max(30, hand size * 1.5)`.
3. كل 0.05 ثانية (حوالي 20 مرة في الثانية) جدّد كل الصواعق. التغيير السريع ده هو ترعيش الكهربا.
4. ارسم قرص توهّج عند الكف (نص قطر 0.28)، وبعدين الصواعق.
5. أطلق حوالي 250 شرارة في الثانية في كل الاتجاهات، بسرعة، بجاذبية لتحت (400 px/s^2) و drag، ملوّنة من `CORE`، وبتبهت مع العمر.
6. `add_glow` بقوة 1.5 و`backdrop=0.45`.
7. باحتمال `0.12 * alpha` كل فريم، ضيف `(60, 25, 0)` على الفريم كله: فلاش أزرق سريع على المشهد، زي ومضة برق بتنوّر الأوضة.

الملف `shinobi/effects/rasengan.py`

مسؤول عن إيه؟

Rasengan: كرة طاقة زرقا بتلف قاعدة على كف الإيد. الـ id الـ effect هو `rasengan`. و`follows_hand = True`. آخر حركة في Rasengan هي كف مفتوح واحد والصوابع مفرودة ومتباعدة (`rasengan_palm`)، والكرة بتترسم في مركز الكف ده. (مهم: إيد واحدة بس، مش إيدين.)

الثوابت

`BLUE = [255, 150, 40]` و`LIGHT = [255, 225, 170]` (مصفوفات NumPy بـ BGR، عشان نقدر نضربها في رقم ونطلع درجات أغمق). `CHARGED = 0.45`: الكرة بتوصل لـ 45 % من حجمها الكامل في آخر الـ build-up.
`bgr(color)` بتحوّل لون NumPy لـ tuple أرقام Python عادية، لأن دوال الرسم في OpenCV عايزة كده.

الكلاس `Rasengan`

الـ constructor `Rasengan(duration, n_orbit=70, seed=None)` بيجهّز 70 شرارة بتدور في مدارات. كل واحدة ليها نص قطر مدار خاص (من 0.45 لـ 1.15 نص قطر الكرة)، وسرعة (من 5 لـ 11 radian في الثانية، واتجاه عشوائي)، وطور بداية، وميل (1 = دايرة، صغير = بيضاوي مفلطح)، ودوران للبيضاوي بتاعها. ده بيدّي إحساس بكرة طاقة 3D بتدوّم. وكمان بيعمل نظام جزيئات `wind` (هوا) ومصفوفة `streaks` (خطوط) للـ build-up (صف لكل خط: زاوية البداية، نص قطر البداية، العمر، العمر الكلي، اللفّة).

`_sphere(layer, center, radius)` (static): تلات دوايـر مليانة: قشرة برّانية ناعمة (r 1.05، أزرق غامق)، ونص (r 0.55، أزرق)، وقلب منوّر (r 0.28، فاتح).

`render_charge(...)`: الـ build-up.

- كرة صغيرة بتكبر من 0.12 لـ `CHARGED` (0.45) من الحجم الكامل مع `level`.
- قوسين بيلفوا (أجزاء بيضاوي 220 درجة) بسرعة 420 درجة في الثانية في اتجاهين عكس بعض.
- خطوط بتتولد حوالين الكف (من 30 لـ 140 في الثانية) وبتلف حلزوني لجوه: نص قطرها بيصغر بـ `(1 - t)^1.3` وزاويتها بتزيد بـ `spin * t`. المحور الرأسي متضغوط 0.75 عشان شكل منظور. كل خط بيترسم كخط قصير بين مكانه دلوقتي ومكانه من 0.015 ثانية (ذيل) زائد نقطة منوّرة عند الراس، وبيظهر بسرعة وبيبقى أنور في نص الطريق.
- `add_glow` بقوة ووضوح و backdrop بيزيدوا.

`render(...)`: الـ effect الأساسي، في أربع طبقات رسم:

1. **الكرة**: `grow = min(1, age / 0.6) ** 0.5` (تسهيل بالجذر التربيعي: سريع في الأول وبعدين بيبطّأ). بعد الـ build-up، الكبر بيبدأ من `CHARGED`، فالكرة بتكمّل من حجم الشحن بدل ما ترجع للصفر فجأة. نص القطر `max(25, hand size * 0.95) * grow`.
2. **الأقواس اللي بتلف**: 8 أقواس بيضاوية بمحاور مختلفة؛ زاويتها بتتغير كل فريم (`now * 420 * direction`)، نصهم مع عقارب الساعة ونصهم عكسها بـ 1.4 مرة السرعة. الزوايا المتغيرة بتعمل خدعة الدوران.
3. **الشرارات اللي بتدور**: أماكن الـ 70 شرارة بتتحسب في خطوة NumPy واحدة: نقطة على بيضاوي مايل، وبعدين بتلف بزاوية دوران المدار.
4. **الهوا**: حوالي 120 جزيء في الثانية بيتطلقوا على حافة الكرة، بيتحركوا غالباً مماسياً (المماس لاتجاه `(x, y)` هو `(-y, x)`) زائد شوية لبرّه، مع drag: هوا مرمي من كرة بتلف.

وفي الآخر `add_glow` بقوة 1.4، و`blur = radius * 0.6`، و`backdrop=0.4`.

مثال على الـ grow: بعد build-up، عند `age` = 0.15 ثانية: `min(1, 0.25) ** 0.5` = 0.5، فـ `grow` = 0.45 + 0.55 × 0.5 = 0.725، يعني الكرة 72.5 % من حجمها. وعند 0.6 ثانية أو أكتر `grow` = 1.

الملف `shinobi/effects/shadow_clone.py`

مسؤول عن إيه؟

Shadow Clone (Kage Bunshin): نسخ متأخرة ومقصوصة من المستخدمة بتظهر حواليها في نفخات دخان. بيظهر **10 نسخ بالظبط** (يعني 11 شخص على الشاشة مع المستخدمة)، والعدد ده نهائي ومش بيتقلل أبداً عشان

نكسب سرعة. id الـ effect هو `shadow_clone` . و `needs_segmentation = True` (الـ engine بيشغّل الـ person segmenter طول ما الـ effect ده شغّال). و `fade_out = 0.6` .

الفكرة: بدل فيديوهات متسجلة قبل كده، كل نسخة هي المستخدمة نفسها، مقصوصة بالـ segmentation mask من فريم اتسجّل من 0.25 لـ 1.6 ثانية فاتت. فالنسخ بتقلّد حركات المستخدمة بتأخير، كل واحدة متأخرة شوية عن اللي قبلها، فعمرهم ما بيتحركوا مع بعض بالظبط.

الـ mask جاي منين؟ (الملف `shinobi/segmentation.py` باختصار)

`PersonSegmenter.segment(frame)` في `shinobi/segmentation.py` بيصغّر الفريم لعرض `config.SEGMENT_WIDTH` (640 بكسل)، ويشغّل MediaPipe selfie segmenter (موديل بيفصل الشخص عن الخلفية)، ويرجّع `(small_frame, mask)` . الـ mask نوعه float32 من 0 لـ 1 (1 = شخص). وبيتحسّن بتلات طرق:

1. متوسط متحرك أُسّي مع الـ mask اللي فات (`MASK_SMOOTHING` = 0.5) بيشيل الترعيش بين الفريمات.
2. guided filter بيلزق حافة الـ mask على الحواف الحقيقية في الصورة (الشعر، الكتاف). الصورة المرشدة رمادي ومتقسومة على 255 (من 0 لـ 1)؛ التعليق في الكود بيشرح إن بقيم من 0 لـ 255 الحيطان السادة كانت بتطلّع NaN (اتقاس: 82 من 180 فريم متسجل؛ وصفر مع النسخة من 0 لـ 1).
3. منحنى تباين `(mask - 0.3) / 0.4` (ومقصوص بين 0 و 1) بيخلّي الحافة حادة بس فيها نعومة خفيفة.

الـ engine بيشغّل الـ segmentation على worker thread بالتوازي مع تتبّع الإيد.

الثوابت

الثابت	القيمة	المعنى
`SMOKE_COLOR`	235	درجة الرمادي بتاعة الدخان
`AURA_COLOR` / `AURA_EDGE`	برتقاني `(60, 170, 255)` / أصفر باهت `(200, 240, 255)` (BGR)	هالة الشاكرا في الـ build-up
`POP_START`	0.3 ثانية	أول نسخة بتظهر بعد الـ release بالمدة دي
`POP_STEP`	0.12 ثانية	وبعدين نسخة كمان كل 0.12 ثانية (العاشرة عند 1.38 ثانية)
`BOTTOM_FADE`	0.3	النسخ المرفوعة بتبهت على آخر 30 % من تحت بدل قطع حاد
`TOP_FADE`	0.06	حافة علوية ناعمة لما راس المستخدمة تلمس أعلى الصورة
`EDGE`	مصفوفة 3x3 كلها واحد	kernel للـ erosion (تآكل) لحواف النسخ
`FIT_STEPS`	1.0، 0.95، 0.9، 0.85، 0.8	الصف ممكن يصغر لحد 80 % لو مفيش مساحة كفاية
`SPACING`	0.55	أقل مسافة بين نسخ نفس الصف، بوحدة عرض جسم النسخة
`SIDE_HYSTERESIS`	0.3	النسخة ما بتغيّرش جنبها غير لما نصيب صفها يتحرك بوضوح
`ROOM_SLACK`	0.25	سماحية قبل ما جنب يتحسب إنه صغير أوي

الثابت	القيمة	المعنى
`GLIDE`	8.0 في الثانية	النسخة بتتزحلق لمكانها الجديد بسرعة قد إيه

ملاحظة صغيرة: التعليق جنب `POP_START` في الكود بيقول "after the jutsu fires"، لكن الكود فعلياً بيحسب الظهور من `t0` (`now >= self.t0 + clone["appear"]`)، و `t0` هو لحظة الـ release مش لحظة الانطلاق. فالصح هو "بعد الـ release"، وده اللي مكتوب في الجدول.

من `config.py` (قيمة لكل نسخة، بالـ index، الصف الأمامي الأول):

القائمة	القيم
`CLONE_COUNT`	10
`CLONE_OFFSETS`	-0.36, 0.36, -0.46, -0.27, 0.27, 0.46, -0.19, 0.19, -0.37, 0.37
`CLONE_SCALES`	0.68, 0.68, 0.56, 0.56, 0.56, 0.56, 0.45, 0.45, 0.45, 0.45
`CLONE_LIFTS`	0.00, 0.00, 0.22, 0.22, 0.22, 0.22, 0.40, 0.40, 0.40, 0.40
`CLONE_DELAYS`	0.25, 0.40, 0.55, 0.70, 0.85, 1.00, 1.15, 1.30, 1.45, 1.60
`CLONE_PARALLEL`	True

يعني فيه تلات صفوف:

الصف	عدد النسخ	الحجم (`scale`)	الرفع (`lift`)	التأخير (`delay`)
الأمامي	2	68 %	0 (واقفين على الحافة اللي تحت)	0.25 و 0.40 ثانية
الأوسط	4	56 %	مرفوعين 22 % من ارتفاع الفريم	من 0.55 لـ 1.00 ثانية
الخلفي	4	45 %	مرفوعين 40 %	من 1.15 لـ 1.60 ثانية
المجموع	10			

الصفوف الأصغر والأعلى بتبان أبعد (ده إحساس العمق). الـ `offset` بيدّي بس الترتيب من الشمال لليمين جوه الصف (وأول مكان قبل ما المستخدمة تتلاقى)؛ الأماكن الحقيقية بتتخطط من جديد كل فريم.

ورسم نصي تقريبي للترتيب لما المستخدمة في النص:

```
الصف الخلفي (45%, مرفوع 40%):     C9   C7        C8   C10
الصف الأوسط (56%, مرفوع 22%):   C3   C4             C5   C6
الصف الأمامي (68%, على الأرض):     C1      [ المستخدمة ]      C2
```

(الأرقام هنا للتوضيح بس؛ التوزيع الفعلي بيتحسب كل فريم حسب المساحة على كل جنب.)

الدالتين `clearance(body, scale, lift)` و `plan_row(...)`

`clearance` هي المسافة اللي مركز جسم النسخة بيسيبها من مركز جسم المستخدمة: نص عرض جسم المستخدمة زائد نص عرض جسم النسخة، وبتقل للنسخ المرفوعة (راسهم فوق كتاف المستخدمة، فممكن يقفوا أقرب). المعادلة: `body * (0.5 + 0.5 * scale) * (1.0 - lift)`.

`plan_row(n, center, body, w, scale, lift, phase, left_before=None)` بتقرر الـ `n` نسخ بتوع صف واحد هيقفوا فين، بمعلومية مركز جسم المستخدمة وعرضه وعرض الفريم `w`. بترجّع `(xs, clones_on_left, size_factor, crowded)`.

1. لكل معامل حجم في `FIT_STEPS` (1.0، 0.95، 0.9، 0.85، 0.8): احسب المساحة الفاضية شمال ويمين المستخدمة (من الـ clearance لحد حافة الفريم، بحيث جسم النسخة كله يفضل جوه)، وكام نسخة تدخل في كل جنب بمسافة `SPACING`. وقّف عند أول حجم الـ `n` كلهم يدخلوا فيه.
2. لو مفيش حجم نفع حتى عند 80 %: احشرهم في اللي فاضل جنب المستخدمة، متداخلين، ورجّع `crowded = True`. النسخ عمرها ما بتتحط ورا المستخدمة (كانت هتستخبى خالص) و**عمر ما نسخة بتتشال**.
3. وإلا قسّم النسخ على الجنبين بنسبة المساحة في كل جنب (المستخدمة في النص: 5 و 5 في المجموع؛ المستخدمة على الحافة اليمين: كلهم على الشمال). عشان نمنع الترعيش، التقسيمة اللي فاتت (`left_before`) بتفضل زي ما هي إلا لو النصيب العادل اتحرك أكتر من 0.5 + `SIDE_HYSTERESIS`، أو جنب بقى صغير أوي حتى مع `ROOM_SLACK`.
4. وزّع النسخ بالتساوي على كل جنب؛ `phase` بـ 0.5 بيحطهم في نص خاناتهم، و0 بيبدأ من عند المستخدمة. الصف الخلفي بيستخدم phase بـ 0 عشان الصفوف تتداخل في بعض وروس الصف الخلفي ما تستخباش ورا روس الصف الأمامي.

مثال: فريم عرضه 640 (دقة الـ segmentation)، المستخدمة في النص (center = 320) وعرض جسمها 120. للصف الأمامي (scale 0.68، lift 0): الـ clearance = 120 × 0.84 × 1 = 100.8، ونص عرض النسخة = 40.8، والمسافة بين النسخ = 0.55 × 120 × 0.68 = 44.9. المساحة على الشمال = (320 - 100.8) - 40.8 = 178.4، وعلى اليمين نفس الحاجة. فكل جنب ياخد 1 من النسختين.

الكلاس `ShadowClone`: التجهيز

الـ constructor بيتأكد إن قوايم الـ CLONE الأربعة بنفس الطول وإن `count` داخل فيهم، وبعدين بيبني:

- `clones`: قائمة dictionaries فيها `offset` و`scale` و`lift` و`delay` و`appear` (`POP_START + POP_STEP * i`) و`shown` و`gone` و`x` و`fit` و`side`.
- `order`: النسخ مترتبة بالحجم، الأصغر (الأبعد) الأول، عشان النسخ الأقرب تترسم بعدها وتغطيها.
- `rows`: النسخ متجمّعة بـ `(scale, lift)`، الصف الأمامي الأول، وكل صف مترتب بالـ offset.
- `buffer`: `deque` فيه تاريخ `(time, small_frame, mask, person)`، الأقدم الأول؛ بيحتفظ بـ `max(delay) + 0.3` = 1.9 ثانية.
- `smoke`: نظام `Particles` الأماكن فيه متخزنة ككسور من الفريم (0 لـ 1)، فمش مرتبط بالدقة.
- `anchor`: `(مركز الجسم، عرض الجسم)` بتاع الشخص الحقيقي بعد التنعيم.
- `drawn`: عدد النسخ اللي اترسمت في آخر فريم (بيتبعت كـ `clones` في FrameState)؛ و`crowded`.
- `_worker`: لما `parallel` شغّال، executor خاص فيه thread واحد. تعليق الكود بيشرح ليه لكل effect: مع كذا مستخدم على الويب، نسخ كل واحد بتتبني بالتوازي، و worker واحد بيحافظ على ترتيب شغل الـ effect ده (شغلانتين لنفس الـ effect ممنوع يشتغلوا مع بعض، لأنهم بيتشاركوا الـ buffer).

الدالتين `measure(mask)` و `_remember(now, small, mask)`

`measure` بتلاقي الشخص في mask: `left` و`right` و`top` (المربع اللي هنقص منه النسخة، بالدراعات) و`center` / `body` محسوبين من الـ 80 % اللي في النص من "الكتلة" الأفقية للـ mask (من 10 % لـ 90 % من المجموع

التراكمي للأعمدة). استخدام نسب الكتلة معناه إن دراع مرفوع بالكاد بيحرّك مركز الجسم. بترجّع `None` لو مفيش حد.

`_remember` بتضيف الفريم للـ `buffer`، وتشيل اللي أقدم من `keep`، وتحدّث `anchor` بنعومة: المركز بيتبع بـ 15 % كل فريم، والعرض بـ 6 % بس ("a moving arm must not reshuffle the crowd"، يعني دراع بيتحرك ما يلخبطش الزحمة).

الدالتين `layout(w, dt)` و `place(...)`

`layout` بتشغّل `plan_row` لكل صف وبتحرّك كل نسخة ناحية الـ x المخطط ليها:

- أول تخطيط: النسخة بتبدأ هناك على طول.
- نفس الجنب زي قبل كده: بتتزحلق بـ `ease = 1 - exp(-GLIDE * dt)` (اقتراب ناعم مش مرتبط بعدد الفريمات في الثانية).
- الجنب التاني من المستخدمة: النسخة ما بتتزحلقش من خلال المستخدمة؛ بتختفي وتظهر تاني بنفخة دخان في المكان القديم والجديد.

`place(clone, person, w, h, x=None, fit=None)` بتحسب المستطيل اللي النسخة هتترسم فيه على الطبقة: الحجم = مربع الشخص × scale النسخة × fit؛ والقاعدة عند خط الأرض المرفوع بتاع النسخة (`h * (1 - lift)`)؛ ومركز الجسم بيروح عند x، بس بيتزق لجوه الفريم لو كان هيطلع برّه.

الدالتين `_edge_fade` و `_poof`

`_edge_fade(th, bottom, top)` بترجّع عمود شفافية متخزّن (cached): تلاشي خطي على آخر 30 % من تحت للنسخ المرفوعة (عشان جسم طاير ما يخلصش بخط حاد) و/أو على أول 6 % من فوق لما راس المستخدمة مقطوعة من أعلى الصورة.

`_poof(clone, w)` بتطلق دفعة جزيئات دخان في نص جسم النسخة (حوالي `90 * scale` جزيء، بتنتشر وتطلع لفوق شوية). الدخان بيظهر لما نسخة تظهر، أو تختفي، أو تغيّر جنبها؛ وكمان بيخبّي أول فريمات النسخة اللي ممكن تبقى خشنة.

الدالة `render_charge()`: الـ build-up

بتشتغل على الـ main thread (الـ worker ما بيبدأش غير مع الـ effect الأساسي). وفي نفس الوقت buffer التاريخ بيتملا (عشان النسخ تقدر تظهر عند الـ release وفريماتها المتأخرة جاهزة):

- بتبني هالة شاكرا حوالين حدود جسم المستخدمة بنص دقة الـ segmentation: الـ mask بيتمدّد (dilate) بنص قطر بيكبر مع `level`، والحلقة بين الـ mask الممدود والأصلي بتتغبّش، وبتنبض (`0.8 + 0.2 * sin(9 * now)`). الهالة بتضلّم شوية (`keep`) وبتضيف نور برتقاني (`add`)، لأن النور المضاف لوحده كان هيبقى أبيض على حيطة منوّرة.
- بتطلق خيوط دخان على الجنبين بتاع المستخدمة، بتتحرك لجوه ولفوق. دي نفس جزيئات الدخان بتاعة نفخات النسخ، فآخر خيوط بتفضل عايشة لما النسخ تظهر.
- بتدمج كل حاجة بعملية ضرب واحدة وجمع واحدة بمقاس الفريم الكامل.

الدالة `render()`: الـ effect الأساسي والـ worker المتوازي

```
job = (now, dt, small, mask, self.stop_at)
if self.parallel:
    previous, self._pending = self._pending, self._worker.submit(self._clone_layer, *job)
    if previous is None:
        self.drawn = 0
        return frame
    color, alpha, density, self.drawn = previous.result()
else:
    color, alpha, density, self.drawn = self._clone_layer(*job)
alpha = cv2.multiply(alpha, 1.0 - mask)
```

بيعمل إيه خطوة خطوة:

1. `job = (...)`: بيجهّز كل اللي الـ worker محتاجه للفريم ده، ومعاه `stop_at` كقيمة (مش بيقراها من الـ object بعدين، لأن الـ main thread ممكن يغيّرها).
2. لو `parallel`: بيبعت الشغلانة الجديدة للـ worker (`submit`) وبياخد الشغلانة اللي اتبعتت الفريم اللي فات (`previous`).
3. لو `previous` بـ `None` (أول فريم): مفيش طبقة لسه، فـ `drawn = 0` ويرجّع الفريم زي ما هو.
4. وإلا: `previous.result()` بيجيب طبقة الفريم اللي فات (لون، شفافية، كثافة الدخان، عدد النسخ المرسومة).
5. لو مش `parallel`: بيبني الطبقة على طول في نفس الـ thread.
6. `alpha = cv2.multiply(alpha, 1.0 - mask)`: بيشيل مكان المستخدمة الحقيقية من طبقة النسخ بالـ mask **الحالي**.

ليه كده؟ مع `CLONE_PARALLEL`، طبقة النسخ الغالية للفريم ده بتتبني على الـ worker thread والـ main thread مكمّل (الـ engine بيكمّل على تتبّع الإيد للفريم الجاي)، والطبقة اللي اتبنت للفريم اللي فات هي اللي بتتعرض. النسخ أصلاً متأخرة 0.25 لـ 1.6 ثانية بقصد، ففريم تأخير زيادة مش هيبان. وفي أول فريم خالص مفيش طبقة لسه.

وبعدين:

1. **الشخص الحقيقي يفضل قدام**: شفافية طبقة النسخ بتتضرب في `1 - mask` بالـ mask بتاع الفريم **الحالي**، فحدود المستخدمة نفسها عمرها ما بتتأخر حتى لو النسخ جاية من الـ worker.
2. **الدخان فوق**: صورة رمادي موحدة بتتحط "over" الطبقة وكثافة الدخان هي شفافيتها.
3. لو مفيش حاجة ظاهرة، رجّع الفريم.
4. كبّر طبقة اللون والشفافية لمقاس الفريم الكامل وادمج مرة واحدة بـ `cv2.blendLinear(layer, frame, a, 1 - a)`.

الدالة `_clone_layer()`: بناء النسخ

بتشتغل بدقة الـ segmentation (عرض 640 بكسل)، وممكن على الـ worker thread؛ وبتقرا `stop_at` بس زي ما اتبعتلها، لأن الـ main thread ممكن يوقّف الـ effect في النص.

1. `_remember` للفريم؛ و`layout` للصفوف لو المستخدمة اتلاقت.

2. لكل نسخة جه وقت `appear` بتاعها، علّمها `shown` وأطلق نفخة. لو الـ effect بيتوقف، علّم النسخ الظاهرة `gone` بنفخة. وأثناء التوقف، ظهور النسخ بيقل على 0.25 ثانية.
3. لكل نسخة بالترتيب من البعيد للقريب: هات الفريم المتخزن اللي عمره على الأقل `delay` ثانية (`_delayed`)؛ اقطع مربع الشخص من الفريم ده ومن الـ mask بتاعه؛ صغّره للمقاس المطلوب؛ خلّي حافة الـ mask أقوى (`a * a`) واعمل erosion ببكسل واحد عشان أقل جزء من الحيطة اللي ورا المستخدمة ينوّر حوالين النسخة؛ طبّق تلاشي الحواف والظهور؛ اقطع على حدود الطبقة؛ وادمجها "over" الطبقة.
4. رجّع `(color, alpha, smoke_density, drawn)`.

الدالة `_over()`: الدمج بطريقة Porter-Duff

`_over(color, alpha, region, img, a)` بتحط `img` بشفافية `a` قدام اللي موجود في الطبقة، في نفس المكان (in place). اللون الناتج `(img * a + old * old_a * (1 - a)) / (a + old_a * (1 - a))` والشفافية الجديدة `a + old_a * (1 - a)`. الكود بيستخدم `cv2.blendLinear(img, old, a, old_a * (1 - a))`، اللي بتقسم على مجموع الوزنين، فبتحسب المعادلة دي بالظبط في نداء OpenCV واحد سريع.

مثال بالأرقام لبكسل واحد: الطبقة فيها نسخة بعيدة لونها 100 وشفافيتها 1.0، ونسخة قريبة لونها 200 وشفافيتها `a` = 0.6. الوزن القديم = 1.0 × (1 - 0.6) = 0.4. اللون الجديد = (200 × 0.6 + 100 × 0.4) / (0.6 + 0.4) = 160. الشفافية الجديدة = 0.6 + 0.4 = 1.0.

الدالتين `_smoke_density()` و `finished()`

`_smoke_density(w, h, dt, scale_up=True)` بتحدّث الدخان (طلوع خفيف لفوق، drag بـ 1.5)، وترسم كل جزيء كقرص على canvas بنص الدقة بقوة بتطلع بسرعة وتنزل ببطء (`sin(pi/2 * min(4t, 1)) * (1 - t) * 140`) وحجم بيكبر مع العمر، وتغبّشه (sigma 4.5) عشان النفخات تندمج في سحابة، وترجّع خريطة كثافة من 0 لـ 1.

`finished(now)` بتستنى الـ fade out **و** إن كل الدخان يختفي.

خدع السرعة (ملخص)

- كل حاجة (النسخ، الدخان، قص الشخص) بتتجمّع في طبقة واحدة بدقة الـ segmentation، وبعدين بتتدمج على الفريم الكامل مرة واحدة، بدل دمج بالمقاس الكامل لكل نسخة.
- كل نسخة بتلمس المربع بتاعها بس، مش الطبقة كلها.
- الدخان والهالة بيترسموا بنص الدقة.
- طبقة النسخ بتتبني على worker thread متوازي، قبلها بفريم.
- أعمدة تلاشي الحواف متخزنة حسب المقاس.
- لو مقاس الفريم اتغير أثناء Shadow Clone (الكاميرا اتغيرت، الموبايل اتلف)، الـ engine بينهي الـ effect لأن الطبقة المتأخرة بفريم ما ينفعش تتدمج على المقاس الجديد.

الخط الزمني بالإعدادات الافتراضية

الوقت بعد انطلاق الـ jutsu	اللي على الشاشة
من 0 لـ 2.0 ثانية	الـ build-up: هالة بتنبض وخيوط دخان (voice 0.4، charge 1.2، name 0.4)

الوقت بعد انطلاق الـ jutsu	اللي على الشاشة
2.0 ثانية	release: الـ effect الأساسي بيبدأ
2.3 ثانية	أول نسخة بتظهر بالدخان
من 2.3 لـ 3.38 ثانية	نسخة كمان كل 0.12 ثانية لحد ما العشرة يظهروا
12.0 ثانية	المدة (10 ثواني) خلصت: النسخ بتختفي في الدخان، fade out 0.6 ثانية
بعد كده	الـ effect بيخلص أول ما آخر دخان يتشال

وتوقيت ظهور كل نسخة (من لحظة الـ release، ومن لحظة الانطلاق):

النسخة	الصف	`appear` بعد الـ release	من الانطلاق	التأخير عن المستخدمة
1	أمامي	0.30	2.30	0.25
2	أمامي	0.42	2.42	0.40
3	أوسط	0.54	2.54	0.55
4	أوسط	0.66	2.66	0.70
5	أوسط	0.78	2.78	0.85
6	أوسط	0.90	2.90	1.00
7	خلفي	1.02	3.02	1.15
8	خلفي	1.14	3.14	1.30
9	خلفي	1.26	3.26	1.45
10	خلفي	1.38	3.38	1.60

ومع كل نسخة جديدة بتظهر، الـ tracker بيطلّع event `clones` بالعدد الجديد، والواجهة بتشغّل صوت `smoke_poof` (بحد أقصى 3 لكل event وبمسافة 110 ms على الأقل بينهم).

الصوت: الفولدر `assets/audio/` وملفات `frontend/src/audio/`

مسؤول عن إيه؟

الفولدر `assets/audio/` فيه كل أصوات الويب أب. السيرفر بيعرضه للقراية بس على `/audio/...`؛ وكود Python نفسه **ما بيشغّلش أي صوت** (الحاجة الوحيدة اللي بيقراها هي أطوال ملفات أصوات الشخصيات، لو موجودة). الواجهة هي اللي بتعرّف الأصوات وبتقرر إمتى تشتغل، ودايماً كرد فعل على events السيرفر:

ملف الواجهة	دوره
`frontend/src/audio/cues.ts`	قائمة `CUES`: كل صوت ليه id (زي `'chidori.crackle_loop'`) ورابط (`/audio/...`) وهل هو loop ولا مرة واحدة (one-shot)
`frontend/src/audio/gains.ts`	`CUE_GAIN`: معامل علو ثابت لكل صوت عشان يتظبط على دوره في المكس؛ ودوال حساب الأحجام من الإعدادات؛ وثوابت الـ ducking
`frontend/src/audio/eventCues.ts`	الدالة `soundsForFrame()`: بتحوّل events الفريم لأوامر صوت (شغّل، loop، وقّف)

ملف الواجهة	دوره
`frontend/src/audio/voiceCues.ts`	الدالة `voiceForFrame()` : أوامر أصوات الشخصيات (لو متركّبة)
`frontend/src/audio/voices.ts`	تعريف ملفات أصوات الشخصيات (`activation` و `name`) والمتكلم، وقراية `GET /api/voices`
`frontend/src/audio/jutsuMusic.ts` و `jutsuMusicPlayer.ts`	موسيقى الـ jutsu وانتي بتعملي حركاتها
`frontend/src/audio/engine.ts`	محرك Web Audio (الـ `SoundEngine`): بيفك الملفات ويشغّلها بالـ fades والـ gains والـ limiter
`frontend/src/audio/useSessionSounds.ts`	الـ hook اللي بيربط كل فريم جاي من الجلسة بالتلات دوال وبينفّذ الأوامر
`frontend/src/konoha/music/player.ts`	مشغّل موسيقى الخلفية `leaf_trail_loop.flac` على graph صوت صغير لوحده

مهم: الملفات `eventCues.ts` و `voiceCues.ts` و `jutsuMusic.ts` هي "دوال نقية" (pure functions): بتاخد الحالة والفريم وترجّع حالة جديدة وأوامر، من غير ما تلمس Web Audio ولا React. ده بيخلّيها سهلة الاختبار (فيه `eventCues.test.ts` و `voiceCues.test.ts` و `jutsuMusic.test.ts`). والتعليق في أول `eventCues.ts` بيأكد القاعدة الذهبية: "Sounds follow the backend's FrameState.events ONLY: nothing is played on a guess of the frontend".

ترتيب الفولدر

```
assets/audio/
  ASSET_LICENSES.md    كل ملف بمصدره وصانعه وترخيصه والـ attribution والاستخدام التجاري
  SOURCES.md           المصادر والتعديلات والمستويات وتفاصيل الطبقات وبلوك الـ credits
  fire_style/          7 ملفات WAV
  chidori/             6 ملفات WAV
  rasengan/            8 ملفات WAV
  shadow_clone/        7 ملفات WAV
  ui/                  4 ملفات WAV
  jutsu_music/         4 WAV loops (واحد لكل jutsu)
  music/               leaf_trail_loop.flac + compose_leaf_trail.py
  voice/               README.md + 4 فولدرات فاضية (fire_style, chidori, shadow_clone, rasengan)
```

ده 37 ملف صوت في المجموع (32 مؤثر صوتي + 4 موسيقى jutsu + 1 موسيقى خلفية). كل المؤثرات الصوتية WAV-16 bit PCM بـ 44.1 kHz.

الخط الزمني للصوت مع التفعيل

الخط الزمني في المتصفح ماشي ورا events السيرفر بالظبط:

1. `step` : صوت الختم (`ui/hand_seal.wav`)، وتحته "لمسة" الـ jutsu الخاصة (`<jutsu>/hand_sign.wav`). صوت واحد بس لكل فريم حتى لو أكتر من jutsu اتقدّمت (الأكتر تقدّماً هي اللي بتدّي اللمسة)، وعمره ما بيتكرر لنفس الخطوة من نفس السلسلة.
2. `jutsu` (ومعاه `stages`): صوت الختم بتاع الحركة الرابعة (الأخيرة) مع لمسة الـ jutsu؛ وكان المفروض صوت الشخصية `activation` يشتغل هنا (مش متركّب). وموسيقى الـ jutsu بتوطى (duck) لـ 0.25.

3. `effect_phase` بـ `charge`: صوت الشحن الخاص بالـ jutsu (`CHARGE`).
4. `effect_phase` بـ `name`: كان المفروض صوت الشخصية `name` يشتغل هنا (مش متركّب)؛ والشحن مكمّل تحته.
5. `effect_phase` بـ `release`: الشحن بيقف بـ fade مدته 250 ms (`CHARGE_RELEASE_FADE_MS`)، وأصوات الانطلاق والضربة بتشتغل، والـ loop بيبدأ (`RELEASE`)؛ وموسيقى الـ jutsu بتختفي على 1.5 ثانية.
6. `clones` (Shadow Clone بس): `smoke_poof` للنسخ الجديدة.
7. `effect_stopping`: صوت النهاية (`ENDING`).
8. `effect_end`: الـ loop بتاع الـ effect بيقف (fade 150 ms).

`wrong_sign` **ما بيعملش أي صوت خالص**، وده مقصود. و`sequence_failed` (timeout) بيشغّل `ui/sequence_failed.wav` الهادي.

ولو السيرفر قديم وبيبعت `jutsu` من غير `stages` (الـ effect بيبدأ على طول)، الواجهة بتشغّل `RELEASE` على طول.

الكود اللي بيحسب صوت الختم

```
export const SEAL_RISE = [0, 1, 2, 3, 4]
export const ACCENT_VOLUME = 0.55

export function stepSound(ev: Pick<StepEvent, 'jutsu' | 'step'>): PlayCommand {
  const rise = SEAL_RISE[Math.min(SEAL_RISE.length - 1, Math.max(0, ev.step - 1))]
  return { kind: 'play', cue: 'ui.hand_seal', volume: SEAL_VOLUME, rate: 2 ** (rise / 12) }
}
```

بيعمل إيه خطوة خطوة:

1. ياخد رقم الخطوة `ev.step` (1، 2، 3، أو 4 للحركة الأخيرة).
2. `ev.step - 1` يحوّله لـ index من 0، ومقصوص بين 0 وآخر عنصر.
3. `SEAL_RISE[...]` يدّي عدد الـ semitones (نص النغمة الموسيقية) اللي هيعلّي بيها الصوت: 0 للخطوة 1، 1 للخطوة 2، 2 للخطوة 3، 3 للخطوة 4.
4. `rate: 2 ** (rise / 12)`: في الموسيقى، رفع الصوت semitone واحد = ضرب سرعة التشغيل في الجذر الـ 12 لـ 2 (حوالي 1.0595). يعني الخطوة 2 بتشتغل بسرعة 1.0595، والخطوة 4 بسرعة 2^(3/12) حوالي 1.189.
5. النتيجة: كل ختم أعلى شوية من اللي قبله، فالسلسلة بتحسسك إنها "بتبني" لحد الانطلاق.

و`handSignSounds()` بتحط تحت الختم لمسة الـ jutsu بعلو `ACCENT_VOLUME` (0.55). مين بيستخدمها؟ `soundsForFrame()` لما يلاقي `step` أو `jutsu` في الـ events.

أوامر كل jutsu بالتفصيل (من `eventCues.ts`)

الشحن (`CHARGE`)، بيشتغل مع `effect_phase` بـ `charge`:

الـ jutsu	الأصوات	التفاصيل
`fire_style`	`fire_style.prepare_inhale` ثم `fire_style.charge_rise`	التاني بيبدأ بعد 150 ms (`CHARGE_DELAY_MS`) بـ fade-in 400 ms (`CHARGE_FADE_IN_MS`)
`chidori`	`chidori.lightning_charge`	fade-in طويل 900 ms عشان الطنين يطلع تدريجياً

الـ jutsu	الأصوات	التفاصيل
`shadow_clone`	`shadow_clone.charge_chakra`	fade-in 150 ms
`rasengan`	`rasengan.charge_magic` ثم `rasengan.chakra_charge`	الأول fade-in 250 ms؛ التاني بعد 150 ms بـ fade-in 600 ms وعلو 0.6

الانطلاق (`RELEASE`)، بيشتغل مع `effect_phase` بـ `release`:

الـ effect	الأصوات بالترتيب والتأخير
`fire`	`fire_style.ignition` (0 ms)، `fire_style.flame_burst` (90 ms)، `fire_style.impact` (140 ms)، loop `fire_style.burning_loop` (650 ms، fade-in 700 ms)
`chidori`	`chidori.activation_zap` (0)، `chidori.thunder_impact` (40 ms)، loop `chidori.crackle_loop` (90 ms، fade-in 250 ms)
`rasengan`	`rasengan.release_whoosh` (0)، `rasengan.wind_impact` (60 ms)، loop `rasengan.energy_loop` (250 ms، fade-in 500 ms)
`shadow_clone`	`shadow_clone.duplicate` (0)، `shadow_clone.clone_appear` (120 ms)؛ مفيش loop

النهاية (`ENDING`)، بتشتغل مع `effect_stopping`:

الـ effect	الأصوات
`fire`	وقف `fire_style.burning_loop` بـ fade 700 ms (مفيش صوت نهاية، النار بتطفى لوحدها)
`chidori`	`chidori.impact`، ووقف `chidori.crackle_loop` بـ fade 450 ms
`rasengan`	`rasengan.impact` بيتقطع بعد 900 ms بـ fade 1200 ms (لأن ليه ذيل طوله 6.9 ثانية)، ووقف `rasengan.energy_loop` بـ fade 450 ms
`shadow_clone`	`shadow_clone.clone_disappear`، و`shadow_clone.smoke_poof` بعد 60 ms بعلو 0.9

نفخات الدخان مع `POOF_GAP_MS` = 110 : `clones` event (أقل مسافة بين نفختين)، `MAX_POOFS_PER_EVENT` = 3، ونفخة متأخرة أكتر من 700 ms بتتشال لأنها مش هتبقى ماشية مع الصورة. العلو بتاع أول تلات نفخات: 1، 0.8، 0.65.

كل ملف صوت وإمتى بيشتغل

الملف	إمتى بيشتغل
`ui/hand_seal.wav`	كل خطوة متأكدة (`step` event) وكمان الحركة الأخيرة مع `jutsu`، وبيعلى semitone مع كل خطوة. وهو كمان صوت الاختبار في صفحة الإعدادات الحالية
`ui/sequence_failed.wav`	`sequence_failed` (timeout)
`ui/step_success.wav`	متعرّف كـ cue بس مش بيشتغل في الأبلكيشن الحالي (شوفي الملاحظة تحت)
`ui/mode_switch.wav`	بيشغّله الـ sound context لما الصوت يتفك قفله (unlock) بأول تفاعل من المستخدمة
`fire_style/hand_sign.wav`	لمسة Fire Style تحت الختم (رفّة لهب قصيرة، بتتقطع بعد 380 ms بـ fade 220 ms)

الملف	إمتى بيشتغل
`fire_style/prepare_inhale.wav`، `charge_rise.wav`	مرحلة الشحن في Fire Style
`fire_style/ignition.wav`، `impact.wav`، `flame_burst.wav`	انطلاق Fire Style
`fire_style/burning_loop.wav`	loop طول النار، وبيختفي تدريجياً لما تقف
`chidori/hand_sign.wav`	لمسة Chidori (زقزقة كهربا)
`chidori/lightning_charge.wav`	شحن Chidori
`chidori/activation_zap.wav`، `thunder_impact.wav`	انطلاق Chidori
`chidori/crackle_loop.wav`	loop طول Chidori
`chidori/impact.wav`	نهاية Chidori (`effect_stopping`)
`rasengan/hand_sign.wav`	لمسة Rasengan (هبّة هوا)
`rasengan/charge_magic.wav`، `chakra_charge.wav`	شحن Rasengan (طبقتين)
`rasengan/release_whoosh.wav`، `wind_impact.wav`	انطلاق Rasengan
`rasengan/energy_loop.wav`	loop طول Rasengan (بدل `spin_loop.wav`)
`rasengan/impact.wav`	نهاية Rasengan (ذيله 6.9 ثانية بيختفي بعد حوالي ثانية)
`rasengan/spin_loop.wav`	متعرّف كـ cue بس اتبدّل بـ `energy_loop.wav`، مش بيشتغل
`shadow_clone/hand_sign.wav`	لمسة Shadow Clone (حركة إيد سريعة)
`shadow_clone/charge_chakra.wav`	شحن Shadow Clone
`shadow_clone/duplicate.wav`، `clone_appear.wav`	انطلاق Shadow Clone
`shadow_clone/smoke_poof.wav`	كل event `clones` (بحد أقصى 3 نفخات لكل event، وبينهم 110 ms على الأقل) وفي النهاية
`shadow_clone/clone_disappear.wav`	نهاية Shadow Clone
`shadow_clone/seal_charge.wav`	متعرّف كـ cue بس مش مستخدم في الـ mapping (اتبدّل بـ `charge_chakra.wav`)
`jutsu_music/<jutsu>.wav`	موسيقى وانتي بتعملي حركات الـ jutsu دي؛ بتوطى تحت الـ build-up، وبتختفي عند الـ release أو لما التقدّم يرجع صفر
`music/leaf_trail_loop.flac`	موسيقى خلفية هادية للأبلكيشن (مقفولة افتراضياً)

ملاحظة تصحيح عن `ui/step_success.wav`: النسخة الإنجليزية من الدليل بتقول إنه "بيشتغل كاختبار صوت في صفحة الإعدادات". الكود الحالي بيقول غير كده: الـ route بتاع `settings` في `App.tsx` بيعرض `KonohaSettingsPage` (في `frontend/src/konoha/pages/SettingsPage.tsx`)، وده بيشغّل `ui.hand_seal` كاختبار. الصفحة القديمة `frontend/src/pages/SettingsPage.tsx` هي اللي كانت بتشغّل `ui.step_success`، وهي لسه

في الريبو بس مش متوصّلة بأي route. والتعليق في `gains.ts` نفسه بيقول: "the former hand-sign tone (kept: the Settings test plays the seal now)". يعني `step_success.wav` النهارده متعرّف بس ما بيتسمعش في الأبلكيشن.

قنوات الصوت (buses) والتحكم في العلو

محرك الصوت (`engine.ts`) بيوصّل الأصوات في كذا "قناة" (bus)، يعني مسار صوتي ليه علو خاص بيه. الرسم من التعليق في أول الملف:

```
  مؤثر صوتي (buffer source -> gain: CUE_GAIN x علو التشغيل x الـ fade)
    -> output gain (علو المستخدمة: master x sfx) -> duck (بيوطى تحت صوت شخصية) -> limiter ->
السماعات
  صوت شخصية -> clip gain (fades) -> voice gain (master x voice) -> limiter
  موسيقى الـ jutsu -> music gain (master x music) -> music duck (تحت صوت شخصية) -> limiter
  موسيقى الخلفية (Leaf Trail) -> MusicPlayer لوحده: gain (master x music) + duck خاص بيه
```

القناة	المعادلة	الافتراضي	الدالة
المؤثرات الصوتية (SFX)	`masterVolume * sfxVolume` (أو 0 لو `muted`)	0.8 × 0.9 = 0.72	`outputVolume()` في `gains.ts`
أصوات الشخصيات (Voice)	`masterVolume * voiceVolume`	0.8 × 1 = 0.8	`voiceOutputVolume()`
الموسيقى (Music)	`masterVolume * musicVolume`	0.8 × 0.3 = 0.24	`musicOutputVolume()` (موسيقى الـ jutsu)، ونفس المعادلة في `MusicContext.tsx` لموسيقى الخلفية

القيم الافتراضية جاية من `DEFAULT_SETTINGS` في `frontend/src/contexts/SettingsContext.tsx`:

`masterVolume: 0.8`، `sfxVolume: 0.9`، `voiceVolume: 1`، `muted: false`، `audioEnabled: true`، `musicOn:` `false` (موسيقى الخلفية مقفولة لحد ما المستخدمة تفتحها)، `musicVolume: 0.3`، `jutsuMusic: true`. كل القيم من 0 لـ 1، والنتيجة بتتقص بين 0 و 1.

```
export function outputVolume(settings: { masterVolume: number; sfxVolume: number; muted: boolean }):
number {
  if (settings.muted) return 0
  const v = settings.masterVolume * settings.sfxVolume
  return Number.isFinite(v) ? Math.min(1, Math.max(0, v)) : 0
}
```

خطوة خطوة: لو كتم الصوت شغّال يرجّع 0؛ وإلا يضرب العلو الرئيسي في علو المؤثرات؛ ولو الناتج رقم صالح يقصّه بين 0 و 1، ولو مش صالح (NaN مثلاً من إعدادات بايظة) يرجّع 0 للأمان.

الـ Ducking (توطية القنوات): لما صوت شخصية بيتكلم، قناة المؤثرات بتنزل لـ `VOICE_DUCK_SFX` = 0.4 (حوالي -8 dB)، والموسيقى لـ `VOICE_DUCK_MUSIC` = 0.3 (حوالي -10 dB)، بسرعة نزول `DUCK_ATTACK_S` = 0.02 ثانية ورجوع `DUCK_RELEASE_S` = 0.18 ثانية. صوت الشخصية بيبدأ بعد 0.03 ثانية (`SPEECH_START_S`) عشان التوطية تلحق تحصل الأول. (النهارده مفيش أصوات شخصيات، فالـ ducking ده عمره ما بيتفعّل عملياً.)

الـ Limiter: في آخر السلسلة فيه `DynamicsCompressor` بإعدادات: threshold -6 dB، knee 6، ratio 12، attack 0.003، release 0.2. وظيفته يمسك أي ذروات لما أصوات كتير تتداخل عشان ما يحصلش تشويش (clipping).

والمحرك بيشغّل لحد 32 صوت في نفس الوقت (`MAX_VOICES`).

تسوية العلو لكل صوت (`CUE_GAIN`): الملفات مختلفة جداً في علوها (من -38 لـ -9 dBFS)، فكل cue ليه معامل ثابت بيوصّله لعلو دوره في المكس:

الدور	الهدف (أعلى نافذة 400 ms بعد الـ gain)	أمثلة
seal	حوالي -14 dBFS (أعلى 100 ms)	`ui.hand_seal` (gain 1)
accent	حوالي -20 dBFS	`fire_style.hand_sign` 0.87، `chidori.hand_sign` 1.19، `shadow_clone.hand_sign` 0.33، `rasengan.hand_sign` 0.89
ui	حوالي -24 dBFS	`ui.sequence_failed` 0.2، `ui.mode_switch` 1.6
charge	حوالي -21 dBFS	`fire_style.charge_rise` 0.48، `chidori.lightning_charge` 0.62، `rasengan.chakra_charge` 0.22
activation	حوالي -13 dBFS (أعلى لحظة)	`fire_style.ignition` 1.35، `chidori.activation_zap` 1.4، `shadow_clone.clone_appear` 0.45
loop	حوالي -23 dBFS	`fire_style.burning_loop` 0.3، `chidori.crackle_loop` 0.95، `rasengan.energy_loop` 1.23
ending	حوالي -17 dBFS	`chidori.impact` 0.8، `rasengan.impact` 0.55، `shadow_clone.clone_disappear` 0.85

أي gain أكبر من 1 متقصوص بحيث ذروة الملف تفضل عند حوالي -1 dBFS أو أقل.

موسيقى الـ jutsu (`jutsuMusic.ts`): كل jutsu ليها track وgain بيسوّي الأربعة لحوالي -16 dBFS: rasengan 1.35، shadow_clone 0.93، chidori 1.2، fire_style 0.87. الثوابت: `FADE_IN_MS` = 1500 (أول track بيظهر)، `CROSSFADE_MS` = 800 (من jutsu لـ jutsu تانية متقدّمة أكتر)، `DUCK_ACTIVATION` = 0.25 (تحت الـ build-up) و`DUCK_RAMP_MS` = 300، `RELEASE_FADE_MS` = 1500 (عند الانطلاق)، `RESET_FADE_MS` = 1000 (timeout أو Start over أو mode تاني)، `STOP_FADE_MS` = 600 (الكاميرا وقفت أو الاتصال قطع). ومع كل ختم الموسيقى بتعمل "غطسة" قصيرة (`pulse`) عشان صوت الختم يفضل واضح. وطول ما فيه تفعيل شغّال مفيش track جديد بيبدأ. ولو مفيش فريم وصل لمدة 2 ثانية (`FRAME_GAP_STOP_MS` في `useSessionSounds.ts`)، موسيقى الـ jutsu بتختفي.

موسيقى الخلفية: `frontend/src/konoha/music/player.ts` بيشغّل `/audio/music/leaf_trail_loop.flac` بـ loop مظبوط بالعينة (من غير فجوة عند نقطة اللف)، بـ fade-in 2.2 ثانية و fade-out 0.8 ثانية، وبيوطى تحت صوت شخصية (نزول 0.05 ثانية ورجوع 0.35). علوه `musicVolume * masterVolume`، وما بيبدأش غير بعد أول click أو ضغطة زرار (قاعدة المتصفحات).

ليه لازم تفاعل الأول؟ المتصفحات بتسيب الـ AudioContext واقف لحد ما المستخدمة تتفاعل مع الصفحة، فالمحرك بيتعمل ويتفك قفله في `unlock()`، اللي لازم تتنادى من click أو لمسة أو زرار. ولما ده يحصل، `SoundContext.tsx` بيشغّل `ui.mode_switch` كتأكيد.

التراخيص

حسب `ASSET_LICENSES.md` (37 ملف):

الترخيص	عدد الملفات	الـ attribution (ذكر صاحب العمل)
CC0 1.0 (ملكية عامة)	16	مش مطلوب (اختياري)
CC BY 4.0، أو خليط من طبقات CC0 و CC BY 4.0	16	مطلوب
CC BY 3.0 (`smoke_poof.wav` فيه كمان طبقة CC BY 4.0)	4	مطلوب
شغل المشروع نفسه (موسيقى الخلفية)	1	مفيش

مفيش أي ملف صوت بترخيص NC (ممنوع تجاري)، ولا ND (ممنوع تعديل)، ولا SA (لازم تشارك بنفس الترخيص). كل الأصوات من أطراف تالتة اتنزلت مباشرة من OpenGameArt.org (اتنين منهم إعادة رفع لأصوات من Freesound، وصفحاتهم الأصلية اتراجعت كمان). كلها مؤثرات صوتية عامة: مفيش صوت من Naruto أو أي أنمي، مفيش حوار، مفيش soundtrack. الصنّاع منهم: Julien، BMacZero، pyranostudios، qubodup، faxcorp، Matthey، wobbleboxx، dklon، fins، Fupi، Kenney، rubberduck، artisticdude، jaggedstone، leohpaz، ogrebane، zeroisnotnull، Hansjörg Malthaner، Jordan Irwin، iochi-glaucus، و JC Sounds.

تحديث 2026-11-01: الـ 15 مؤثر صوتي الجداد بطبقات

في 2026-11-01 اتعمل تحديث كبير: 15 مؤثر صوتي اتعادت بنايتهم بإنهم اتعملوا "طبقات" (layering) فوق الأصوات القديمة بأصوات من حزمة "JC Sounds "Fantasy SFX Pack Vol 1 (ترخيص CC BY 4.0، والـ credit المطلوب "JC Sounds")، وحزمة Kenney "Sci-Fi Sounds" (CC0)، وطبقات مولّدة بالكود (ضوضاء متفلترة وموجات sine). الهدف إن كل خطوة وشحن وانطلاق يبقى صوته أتقل وأقوى بستايل الألعاب. أسماء الملفات والصيغة (16-bit PCM WAV، 44.1 kHz) والعلو اتظبطوا بالظبط زي الملفات القديمة (نفس علو أعلى نافذة 400 ms، وأعلى 100 ms للختم)، عشان كده معاملات العلو في `frontend/src/audio/gains.ts` لسه صح ومفيش أي كود اتغير. والنسخ القديمة اتحفظت برّه الريبو في `C:\Users\user\Desktop\SHINOBI-AI_backup_20261101_sfx_originals\`.

الـ 15 ملف:

الملف	الطبقات الجديدة	الترخيص بعد التحديث
`fire_style/hand_sign.wav`	رفّة اللهب + fwoosh من Kenney `thrusterFire_001`	CC0 (الطبقتين)
`fire_style/charge_rise.wav`	رمبل + JC Fireball Buildup + Fireball Hold + Kenney thruster منخفض مولّد	CC BY 4.0 + CC0
`fire_style/impact.wav`	انفجار Kenney crunch + JC Fireball Impact + leohpaz و sub boom	CC BY 4.0 + CC0
`fire_style/flame_burst.wav`	JC Fireball Launch + Kenney اتنين + fireball Julien Matthey thruster و sub boom	CC BY 4.0 + CC0
`chidori/hand_sign.wav`	زقزقة Kenney + طقطقة من JC Electric Spell Hit	CC BY 4.0 + CC0
`chidori/thunder_impact.wav`	رعد JC Electric Launch + leohpaz و Kenney low boom + Hit	CC BY 4.0 + CC0
`chidori/lightning_charge.wav`	طنين JC Electric Hold + faxcorp (بيعلى في الطبقة) + Buildup + Launch	CC BY 4.0 + CC0
`shadow_clone/charge_chakra.wav`	لمعة JC Mana Drain + jaggedstone + انتفاخ هوائي مولّد	CC BY 4.0 + CC0

الملف	الطبقات الجديدة	الترخيص بعد التحديث
`shadow_clone/duplicate.wav`	انفجار دخان مولّد + teleport ogrebane + JC Teleport In + Kenney low boom	CC BY 4.0 + CC0
`shadow_clone/smoke_poof.wav`	poof dklon + JC Teleport Out + نفخة وضربة منخفضة مولّدين	CC BY 3.0 + CC BY 4.0
`rasengan/hand_sign.wav`	هبّة rubberduck + JC Sword Swing	CC BY 4.0 + CC0
`rasengan/charge_magic.wav`	شحن leohpaz + Kenney engineCircular (بيسرّع) + هوا من thruster	CC BY 4.0 + CC0
`rasengan/wind_impact.wav`	هوا leohpaz + Kenney sub boom و crunch + JC Magic Shield	CC BY 4.0 + CC0
`rasengan/release_whoosh.wav`	swoosh qubodup + JC Heavy Sword Swing	CC BY 4.0 + CC0
`ui/hand_seal.wav`	الختم الأصلي (قماش + صفعة، CC0) + JC Dagger Swing + ضربة sine مولّدة	CC BY 4.0 + CC0

(ملاحظة: النسخة الإنجليزية من الجزء ده كانت فعلاً بتذكر التحديث في سطر واحد؛ هنا فصّلناه من `ASSET_LICENSES.md` و`SOURCES.md` الحاليين.)

موسيقى الـ jutsu والخلفية

موسيقى الـ jutsu الأربعة هي tracks مرخّصة اتقصّت لـ loops من غير فواصل (seamless): Rasengan "Blackmoor Ninjas" لـ Matthew Pablo (CC BY 3.0)، النسخة من غير غنا عشان ما تتخانقش مع صوت شخصية)، "Fire Style" لـ Chidori "Revenge"، "congusbongus / jobro (CC BY 3.0) لـ TAD (CC BY 4.0)، "Shadow Clone" لـ TAD (CC BY 4.0). **ولا واحدة فيهم من الـ soundtrack الأصلي بتاع Naruto.** (الـ soundtrack الأصلي تسجيلات تجارية ومحتاج تراخيص master-use و synchronisation مش موجودة للمشروع.)

موسيقى الخلفية "Leaf Trail" اتألّفت واتولّدت للمشروع ده بالسكربت `music/compose_leaf_trail.py` (Python، numpy/scipy): مقطوعة pentatonic صغير (minor) بسرعة 90 BPM، طولها 64 ثانية، بـ koto و ناي bamboo و taiko و bass كلهم مولّدين بالكود، ومعمولة عشان تلف من غير فاصل، ومتخزنة FLAC.

نصوص الـ attribution المطلوبة متجمّعة في أقسام "Credits block" في `SOURCES.md` و`ASSET_LICENSES.md`، والسيرفر بيرجّع الـ credits للأبلكيشن عن طريق `GET /api/credits`.

ملاحظة على `SOURCES.md`: الملخص اللي في أوله لسه بيقول "3 files are CC BY 16 ... files are CC0" (ده ملخص أول دفعة 20 ملف يوم 2026-10-30). الأرقام الحالية الصح للـ 37 ملف هي اللي في `ASSET_LICENSES.md` (16 / 16 / 4 / 1)، والملف نفسه بيوضح في السطر 6 إن 15 مؤثر اتحدّثوا بعدين في نفس اليوم.

الفولدر `voice/`: أماكن أصوات الشخصيات (فاضية)

الفولدر `assets/audio/voice/` فيه فولدر فاضي لكل jutsu و`README.md`. **مفيش أي ملف صوت شخصية متركّب، لأن مفيش صوت مرخّص متاح.** وده قرار صاحبة المشروع: الأماكن دي تفضل جاهزة بس لتسجيلات أصلية مرخّصة بشكل سليم، و**مفيش أصوات مولّدة صناعياً (synthetic) ولا مقلّدة خالص.**

الـ README بيحدد الـ 8 ملفات المتوقعة بأساميهم بالظبط:

الملف	المتكلم	بيقول إيه
`fire_style/activation.wav`	Naruto	صرخة التفعيل
`fire_style/name.wav`	Naruto	"!Katon: Gōkakyū no Jutsu"
`chidori/activation.wav`	Sasuke	نداء التفعيل / الشحن
`chidori/name.wav`	Sasuke	"!Chidori"
`shadow_clone/activation.wav`	Naruto	صرخة الحركة
`shadow_clone/name.wav`	Naruto	"!Kage Bunshin no Jutsu"
`rasengan/activation.wav`	Naruto	صرخة الشحن
`rasengan/name.wav`	Naruto	"!Rasengan"

الصيغة المطلوبة: 16-bit PCM WAV، mono، 44.1 أو 48 kHz، صوت "ناشف" (من غير موسيقى أو مؤثرات، والصدى أقل من 0.3 ثانية)، الصوت يبدأ خلال 30 ms، `activation` من 0.25 لـ 2 ثانية، `name` من 0.25 لـ 2.5 ثانية، حوالي -16 LUFS، والذروات بالكتير -1 dBFS، والتمانية بنفس العلو. وفيه أداة فحص:

```
python tools/check_voices.py
```

الـ README كمان بيوثّق البحث: أصوات الأنمي (Junko Takeuchi في دور Naruto و Noriaki Sugiyama في دور Sasuke) ملك أصحاب الحقوق (Masashi Kishimoto / Shueisha و TV Tokyo و Studio Pierrot)، ومفيش مصدر قانوني بيقدّمها للاستخدام ده، وملفات "Naruto voice" اللي على النت يا إما نسخ AI مقلّدة يا إما متقطّعة من الحلقات، والاتنين ممنوع استخدامهم. الطرق القانونية الوحيدة: ترخيص مكتوب من أصحاب الحقوق، أو تسجيلات جديدة من الممثلين الأصليين بموافقة أصحاب حقوق الشخصيات.

الكود بيعمل إيه مع ده:

- `catalog.voice_clips()` (ورا `GET /api/voices`) بيبلّغ عن كل ملف إنه `null`.
- `activation_stages()` بتستخدم `VOICE_DEFAULT` (0.4 ثانية) لمرحلتي `voice` و`name`.
- الويب أب بيتخطى أي ملف `null` (`voiceCues.ts`) بيحطه في قائمة `missing` وبس).
- مفيش أي صوت عام أو مولّد أو مقلّد بيشتغل مكانه.
- لو اتضافت ملفات مرخّصة بعدين، الـ build-up هيبقى طوله تلقائياً قد طول الملفات بالظبط، من غير أي تعديل في الكود.

وإزاي كانت هتشتغل لو موجودة (من `voiceCues.ts`): مع event `jutsu` اللي فيه `stages` بيشتغل ملف `activation` على طول، وملف `name` بيستنى event `effect_phase` بـ `name`. ولو الـ effect خلص قبل مرحلة الاسم، الاسم ما بيشتغلش. ولو الـ mode اتغير الصوت بيقف (fade 250 ms). وتفعيل جديد بيقطع صوت القديم (fade 120 ms). وصفحة الإعدادات فيها "Character voices" لعلوهم لوحدهم.

الفولدر `gesture_references/`

مسؤول عن إيه؟

رسومات بتعلّم المستخدمة حركات إيد كل jutsu. السيرفر بيعرض الفولدر للقراية بس على `/references/...`، والـ catalog (`GET /api/jutsu`) بيشاور على الصور دي. **دي مش بيانات تدريب**: الموديلات بتتعلم من تسجيلات الكاميرا، مش من الصور دي.

الترتيب

```
gesture_references/
  README.md                          نظرة عامة (بالعامية المصرية)
  fire_style/   chidori/   shadow_clone/   rasengan/
      step_1.png ... step_4.png      كارت لكل خطوة
      sequence_reference.png         الأربع خطوات في صورة واحدة
      instructions.md                شرح خطوة بخطوة
  _sources/
      signs/                         الرسومات الخطية الأصلية (PNG) + CREDITS.txt
      signs_bold/                    نسخ متخنة لكروت الويب + CREDITS.txt
      rasengan_palm_landmarks.json   نقط الإيد اللي اترسم منها كف Rasengan
```

الكروت وصور التسلسل بيعملهم `tools/build_gesture_references.py`؛ والرسومات المتخنة بيعملها `tools/build_bold_signs.py`. السيرفر بيفضّل `signs_bold/<sign>.png` لرابط `art` لو موجود. وفي الفولدرين `signs/` و`signs_bold/` فيه 9 صور: `ram`، `snake`، `tiger`، `dog`، `monkey`، `boar`، `horse`، `clone_seal`، `rasengan_palm`.

التسلسلات الأربعة

الـ jutsu	الخطوات	الحالة
Fire Style: Great Fireball	Monkey، Boar، Horse، Tiger	رسمي (آخر 4 من الـ 6 أختام الرسمية)
Chidori	Snake، Dog، Tiger، Monkey	رسمي (آخر 4 من نسخة الـ 9 أختام)
Shadow Clone	Ram، Snake، Tiger، Clone seal	تركيبة مخصوصة من أختام رسمية
Rasengan	Clone seal، Dog، Horse، Rasengan palm	مخصوص (الـ Rasengan رسمياً ما لوش أختام إيد)

ودي نفس القيم اللي في `config.JUTSU` بالظبط (`["monkey", "boar", "horse", "tiger"]` إلخ).

كل jutsu بتبدأ بحركة مختلفة عشان ما يتلخبطوش في أول خطوة. وكف الـ Rasengan مش ختم: هو **كف واحد مفتوح والصوابع مفرودة ومتباعدة**، وشه للكاميرا، والإيد التانية برّه الصورة. (عمره ما بيبقى إيدين.)

التراخيص

الرسومات التمانية للأختام (Ram، Snake، Tiger، Dog، Monkey، Boar، Horse، Clone seal) رسومات خطية لـ Schlum على Wikimedia Commons، متحوّلة لـ vector من صور فوتوغرافية، بترخيص CC BY-SA 4.0. وبما إن المشروع قصّها وحطها على كروت عليها أسامي، وتخّنها للنسخ الـ bold، فالصور المشتقة كمان CC BY-SA 4.0 (لازم ذكر صاحب العمل ونفس الترخيص لما تتشارك). رسمة كف الـ Rasengan من شغل المشروع نفسه، اتعملت من فريم واحد متسجل لنقط إيد MediaPipe (`_sources/rasengan_palm_landmarks.json`) بيسجّل ملف المصدر

`data/landmarks/rasengan/me_day_20261030_174647.npz` ، الفريم رقم 247، ونسبة الأبعاد، وإحداثيات الـ 21 نقطة). مفيش أي لقطة من الأنمي أو المانجا مستخدمة.

ملاحظة: القسم 6 من `gesture_references/README.md` (بعنوان "لسه ماخلصش") اتكتب يوم 2026-10-30، قبل ما التعرّف على الأختام يتبني، ولسه بيقول إن التعرّف لسه ما اتدرّبش. الجزء ده بيوصف الحالة في التاريخ ده، مش الكود الحالي.

متوصّل بإيه؟

- `server.py` بيعمل mount للفولدر على `/references` وبيستخدم `_sign_art()` عشان يختار الرسمة المتخنة أو الرفيعة.
- `catalog.py` شايل نفس نصوص الكروت (الأسامي، الأسامي اليابانية، النصايح، الـ tags، المصدر) اللي اترسمت بيها الكروت، فالأبلكيشن والصور بيعرضوا نفس النص بالظبط.
- أبلكيشن React بياخد روابط `step_<n>.png` و`sequence_reference.png` ورسمة كل حركة من `GET /api/jutsu`، وبيحمّلهم من `/references/...` .

مثال: كارت الخطوة التالتة في Chidori رابطه `/references/chidori/step_3.png` (Tiger)، ورسمة Tiger نفسها `/references/_sources/signs_bold/tiger.png` .

فروقات بين النسخة الإنجليزية والكود الحالي

اتراجع كل جزء في الشرح ده على الكود الفعلي. معظم النسخة الإنجليزية مطابق للكود. الفروقات اللي اتلاقت:

النقطة	النسخة الإنجليزية / التعليق القديم	الكود الحالي (والمعتمد هنا)
`ui/step_success.wav`	"بيشتغل كاختبار صوت في صفحة الإعدادات"	صفحة الإعدادات المتوصّلة (`KonohaSettingsPage`) بتشغّل `ui.hand_seal` ؛ `step_success` بيشتغل بس في الصفحة القديمة `frontend/src/pages/SettingsPage.tsx` اللي مش متوصّلة بأي route، فعملياً ما بيتسمعش
وصف `step_success.wav` في `ASSET_LICENSES.md`	"the hand-sign tone of every jutsu"	صوت كل خطوة النهارده هو `ui/hand_seal.wav`
ملخص التراخيص في أول `SOURCES.md`	"CC0 … 3 16 CC BY"	الأرقام الصح للـ 37 ملف في `ASSET_LICENSES.md` : CC0، 16 CC 16 BY 4.0 (أو خليط)، 4 1، CC BY 3.0 شغل المشروع
تعليق أول `frontend/src/audio/engine.ts`	"decodes the 19 cues of cues.ts"	`cues.ts` فيه 32 cue
تعليق أول `frontend/src/audio/gains.ts`	"The 31 files"	فيه 32 مؤثر صوتي في `CUES` و`CUE_GAIN`

النقطة	النسخة الإنجليزية / التعليق القديم	الكود الحالي (والمعتمد هنا)
تعليق `POP_START` في `shadow_clone.py`	"first clone" appears this long after the "jutsu fires	الكود بيحسبها من `t0` يعني من الـ release (أول نسخة عند 2.3 ثانية من الانطلاق)
مثال `GET /api/voices` في `docs/web_protocol.md`	فيه `chidori` بـ `name` متركّب (1.42 ثانية)	ده مثال توضيحي؛ الفولدرات الأربعة فاضية والرد الحقيقي كله `null`
القسم 6 في `gesture_references/README.md`	التعرّف لسه ما اتدرّبش	التعرّف متبني (YOLOX + landmarks)؛ القسم بيوصف حالة 2026-10-30
تحديث الأصوات 2026-11-01	مذكور في سطر واحد	متفصّل هنا ملف ملف (15 ملف) من `ASSET_LICENSES.md` و `SOURCES.md`

خلاصة: لو حد سألك تشرحي الجزء ده في دقيقتين

- السيرفر (`shinobi/server.py`) مبني بـ FastAPI. فيه 4 REST endpoints (`/api/info`، `/api/jutsu`، `/api/voices`، `/api/credits`) و WebSocket واحد `/ws`، وبيقدّم الصور والأصوات والأبلكيشن المبني.
- كل WebSocket = جلسة ليها engine خاص وthread خاص، وبحد أقصى جلستين في نفس الوقت.
- الفريمات بتروح وترجع بشكل `[4 بايت طول][JSON header][JPEG]`. الرايح فيه `{"t": ms}`، والراجع فيه `FrameState` كامل.
- العميل مسموح له بـ 2 فريم من غير رد؛ الزيادة بتترد بـ `skipped`.
- كل القرارات (حركة، خطوة، jutsu) من السيرفر. الواجهة بتعرض وبتشغّل أصوات من الـ `events` بس.
- الـ jutsu لما تنطلق بتعدّي على `voice` (0.4) و`charge` (1.2) و`name` (0.4) وبعدين `release`، يعني 2.0 ثانية build-up قبل الـ effect الأساسي، لأن مفيش أصوات شخصيات متركّبة.
- التأثيرات الأربعة: Fire (جزيئات لهب، 6 ثواني)، Chidori (برق بيتجدد 20 مرة في الثانية، 8 ثواني)، Rasengan (كرة على كف واحد مفتوح، 8 ثواني)، Shadow Clone (10 نسخ بالظبط في 3 صفوف 2 و 4 و 4، متأخرة من 0.25 لـ 1.6 ثانية، 10 ثواني).
- الأصوات: 37 ملف، كلها مرخّصة (CC0 و CC BY)، و 15 منهم اتحدّثوا يوم 2026-11-01 بطبقات من JC Sounds و Kenney. التحكم في العلو بـ music / voice / sfx × master.
- أماكن أصوات الشخصيات فاضية ومستنية تسجيلات أصلية مرخّصة بس، ومفيش أي صوت مولّد أو مقلّد.

الجزء التالت: الواجهة (React Frontend)

الجزء ده بيشرح فولدر `frontend/` كله: يعني تطبيق الويب اللي بتفتحيه في المتصفح وتشوفي فيه الكاميرا والكروت والجوتسو. التطبيق مكتوب بـ **React 19** و **TypeScript**، وبيتبني بأداة اسمها **Vite**، وبيكلم الباك إند المكتوب بـ Python (FastAPI) اللي شغال على العنوان `http://127.0.0.1:8000`. في وضع الـ production (يعني النسخة النهائية المبنية) الباك إند نفسه هو اللي بيقدّم التطبيق المبني من فولدر `frontend/dist`، فالمتصفح طول الوقت بيكلم عنوان واحد بس.

الهدف من الجزء ده إنك بعد ما تخلصيه تقدري تشرحي لأي حد: الصفحات إيه، كل ملف بيعمل إيه، الإشارة اللي إيدك بتعملها بتمشي إزاي من الكاميرا للباك إند وترجع تبقى كارت بيتملى وصوت بيطلع. كل ملف من الـ 118 ملف `.ts` / `.tsx` اللي في `frontend/src` مذكور هنا باسمه، ومعاه ملفات الـ CSS modules والصور وسكريبتات الـ e2e.

ملحوظة مهمة عن الدقة: الشرح ده اتعمل من النسخة الإنجليزية للجزء التالت، **واتراجع على الكود الحقيقي**. في الأماكن اللي النص الإنجليزي كان فيها قديم أو ناقص، اتبعنا الكود وكتبنا ده صراحة تحت عنوان "ملحوظة: فرق بين الكود والوصف القديم". وفي آخر الجزء فيه قسم كامل بكل الفروق دي.

قبل ما نبدأ: أساسيات React و TypeScript اللي هتحتاجيها

لو أول مرة تتعاملي مع React، القسم ده هيوفّر عليكي كتير. كل مصطلح هنشرحه مرة واحدة هنا وبعد كده هنستخدمه على طول.

يعني إيه React؟

React مكتبة JavaScript بتبني واجهات المستخدم. الفكرة الأساسية: بدل ما تقولي للمتصفح "غيّر الزرار ده، امسح العنصر ده" خطوة بخطوة، إنتي بتوصفي **الشكل المفروض يبقى عامل إزاي** حسب البيانات الحالية، و React هو اللي بيحدّث الصفحة لما البيانات تتغير.

TypeScript

TypeScript هي JavaScript بس مضاف لها "أنواع" (types). يعني تقدري تقولي إن المتغير ده رقم، والتاني نص، والتالت object فيه خانات معينة. الفايدة إن الأخطاء بتتكشف وإنتي بتكتبي الكود، قبل ما التطبيق يشتغل. مثلًا لو الباك إند بيبعت `seq` كرقم وإنتي حاولتي تتعاملي معاه كنص، TypeScript هيطلّع خطأ. الملفات اللي امتدادها `.ts` فيها كود TypeScript عادي، والملفات اللي امتدادها `.tsx` فيها كمان JSX (هنشرحه حالًا).

JSX

JSX طريقة كتابة شبه HTML جوه الكود. لما تشوفي:

```
return <DojoView heading="SHINOBI AI: form the hand signs, unleash the jutsu" documentTitle={null} />
```

ده معناه "ارسم الكومبوننت `DojoView` وابعتله القيم دي". اللي بين `{ }` ده كود JavaScript عادي جوه الـ JSX.

Component (كومبوننت)

Component يعني "قطعة" من الواجهة: زرار، كارت، صفحة كاملة. في React الكومبوننت عبارة عن function بترجع JSX. اسمها لازم يبدأ بحرف كبير، زي `SequenceCard` أو `CameraCard`. الصفحة كلها بتتبني من كومبوننتس جوه كومبوننتس، زي مكعبات الليجو.

Props

Props هي "المدخلات" اللي الكومبوننت الأب بيبعتها للكومبوننت الابن، زي بارامترات الـ function. مثلًا `SequenceCard` بياخد prop اسمها `done` (عدد الخطوات اللي اتعملت) و prop اسمها `timed` (هل نعرض العدّاد ولا لأ). الابن **ما بيغيرش** الـ props بتاعته؛ هو بس بيقراها.

State

State هي "ذاكرة" الكومبوننت: بيانات بتتغير مع الوقت، ولما تتغير React بيعيد رسم الكومبوننت. بتتعمل بـ `useState`:

```
const [warning, setWarning] = useState<CardNote | null>(null)
```

يعني: `warning` هي القيمة الحالية (أولها `null`)، و `setWarning` هي الـ function اللي بتغيرها. أول ما تنادي `setWarning(...)`، React بيرسم الكومبوننت تاني بالقيمة الجديدة.

Hook

Hook function خاصة اسمها بيبدأ بـ `use`، بتدّي الكومبوننت قدرات زي الـ state أو التأثيرات الجانبية. React فيه hooks جاهزة (`useState`, `useEffect`, `useRef`, `useMemo`, `useCallback`, `useContext`, `useLayoutEffect`, `useSyncExternalStore`)، وتقدري تعملي hooks بتاعتك بتجمّع منطق بتستخدميه في أكتر من مكان. في المشروع ده فيه hooks مهمة جدًا زي `useCamera` و `useShinobiSession` و `useDojo` و `useSessionSounds`.

Effect (useEffect)

Effect يعني "حاجة بتحصل بعد الرسم وليها علاقة بالعالم بره React": فتح WebSocket، تشغيل timer، الاشتراك في event. بتتكتب كده:

```
useEffect(() => {
  const timer = setTimeout(...)
  return () => clearTimeout(timer)
}, [success])
```

الجزء اللي جوه بيشتغل لما القيم اللي في القوسين المربعين (`[success]`) تتغير. والـ function اللي بترجع (`return () => ...`) اسمها **cleanup**: بتنضف (توقف الـ timer، تقفل الاتصال) قبل ما الـ effect يشتغل تاني أو لما الكومبوننت يختفي.

Ref (useRef)

Ref صندوق بيحتفظ بقيمة من غير ما يعيد الرسم لما تتغير. بيُستخدم لحاجتين: إمساك عنصر حقيقي في الصفحة (زي الـ `<canvas>`)، أو تخزين قيمة لازم تفضل موجودة بين الرسمات بس مش لازم تظهر (زي حالة مخطط الأصوات `CueState`).

Context

Context طريقة لمشاركة بيانات مع التطبيق كله من غير ما تعدّيها prop prop في كل مستوى. بتعملي "Provider" بيحط القيمة، وأي كومبوننت تحته يقدر يقراها بـ hook. مثلًا `SettingsProvider` بيحط إعدادات المستخدم، وأي صفحة تقرا الإعدادات بـ `useSettings()`.

CSS Module

CSS Module ملف CSS اسمه بينتهي بـ `.module.css`. لما الكومبوننت يعمل `import styles from './X.module.css'`، Vite بيغير أسماء الـ classes لأسماء فريدة، فـ class اسمها `card` في ملف ما تتخانقش مع class بنفس الاسم في ملف تاني. وفي الكود بتستخدميها كده: `className={styles.card}`.

Route و Router

Route يعني "مسار" في عنوان الصفحة (URL) زي `/live` أو `/practice`. والـ **Router** (هنا مكتبة `react-router-dom`) هو اللي بيقرر أنهي صفحة تترسم حسب العنوان، من غير ما المتصفح يعيد تحميل الصفحة.

WebSocket

WebSocket اتصال مفتوح على طول بين المتصفح والسيرفر، الاتنين يقدروا يبعتوا فيه رسايل في أي وقت (عكس HTTP العادي اللي هو سؤال وإجابة). المشروع بيستخدمه عشان يبعت صور الكاميرا ويستقبل النتيجة عشرات المرات في الثانية.

Vite و npm

npm هو مدير الحزم بتاع JavaScript (بينزّل المكتبات ويشغّل السكريبتات). و **Vite** أداة بتشغّل سيرفر تطوير سريع (بيحدّث الصفحة أول ما تحفظي ملف) وبتبني النسخة النهائية.

نظرة عامة: الصفحات وطريقة مرور البيانات

الصفحات

الـ router (في `src/App.tsx`) بيعرّف الصفحات دي. كلها بتترسم جوه إطار "Konoha" الفاتح (خلفية القرية، هيدر NARUTO، قايمة تنقل مرسومة بالفرشاة).

الـ URL	بتعرض إيه	الملف الأساسي
`/` (الرئيسية Home)	"الدوجو": الكاميرا المباشرة، قايمة الجوتسو الأربعة، كارت تسلسل الإشارات	`konoha/dojo/DojoView.tsx`

الـ URL	بتعرض إيه	الملف الأساسي
`/live`	نفس حالة الدوجو، بس الكاميرا كبيرة ومعاها عمود جانبي ضيق	`konoha/live/LiveView.tsx`
`/practice` و `/practice/:jutsuId`	Carousel تختاري منه جوتسو، وبعدين جلسة تدريب (من غير حد زمني)	`konoha/practice/PracticeView.tsx`
`/library` و `/library/:jutsuId`	"Jutsu Scrolls": carousel للجوتسو الأربعة وإشارات الجوتسو المختار	`konoha/jutsu/JutsuView.tsx`
`/settings`	الكاميرا، الصوت، الموسيقى، الأنيميشن، إعادة الضبط	`konoha/pages/SettingsPage.tsx`
`/about`	وصف المشروع والـ credits (بالتصميم الغامق القديم)	`pages/AboutPage.tsx`
أي عنوان تاني	صفحة 404 (بالتصميم الغامق القديم)	`pages/NotFoundPage.tsx`

الـ `:jutsuId` في المسار معناها "جزء متغير": مثلًا `/practice/fire_style` بيفتح التدريب على Fire Style مباشرة.

طريقة مرور البيانات خطوة بخطوة

أهم فكرة في الفرونت إند كله: **المتصفح عمره ما بيقرر لوحده إن إشارة اتعملت**. هو بس بيبعت صور الكاميرا ويعرض اللي الباك إند بيرد بيه. ده مقصود: الذكاء الاصطناعي كله في الباك إند، والواجهة "شاشة عرض" أمينة.

1. `useCamera` (في `src/hooks/useCamera.ts`) بيطلب من المتصفح الويب كام (`getUserMedia`) وبياخد `MediaStream` (يعني "تيار" الفيديو الحي).
2. `useShinobiSession` (في `src/hooks/useShinobiSession.ts`) بيشغّل التيار ده في `<video>` مخفي، بيرسم كل صورة على canvas، بيضغطها لـ **JPEG** وبيبعتها على **WebSocket** (`/ws`) كرسالة binary شكلها: `[4-byte length][JSON header {"t": time}][JPEG bytes]`.
3. الباك إند بيتعرف على إشارة الإيد، بيحدّث التسلسل، بيرسم المؤثرات (نار، برق، نسخ...) على الصورة، وبيرجّع رسالة binary: `[4-byte length][FrameState JSON][processed JPEG]`.
4. الفرونت إند بيفك الرد (`services/socket/framing.ts`)، بيرسم الـ JPEG المعالَج على الـ `<canvas>` الظاهر، وبيدّي الـ `FrameState` لكل "مستمع" (listener).
5. المستمعين بيحوّلوا الـ `FrameState` والـ `events` اللي فيه (`step`, `wrong_sign`, `sequence_failed`, `jutsu`, `effect_phase`, `effect_end`...) لواجهة (كروت الدوجو، الرسايل الصغيرة toasts، طبقة التفعيل overlay — عن طريق `useDojo` و `useLiveEvents`) ولصوت (`useSessionSounds` ← محرك Web Audio).

```
camera -> <video> -> canvas -> JPEG -> WebSocket /ws -> backend (AI + effects)
backend -> FrameState + processed JPEG -> canvas on screen
                                       -> events -> cards, toasts, overlay
                                       -> events -> sounds, voices, jutsu music
```

الرسم ده معناه: الكاميرا بتدخل في فيديو مخفي، الفيديو بيترسم على canvas، الـ canvas بيتحول JPEG، الـ JPEG بيروح على WebSocket للباك إند. الباك إند بيرجّع حاجتين: صورة معالجة بتترسم على الشاشة، و `FrameState` فيه events بتتحول لكروت ورسايل وطبقة تفعيل، ولأصوات وأصوات شخصيات وموسيقى جوتسو.

غير الـ WebSocket، الفرونت إند بيستخدم طلبات REST (يعني طلبات HTTP عادية بترجع JSON): `GET /api/jutsu` (الكتالوج: الإشارات والجوتسو)، `GET /api/info` (السيرفر يقدر يتعرف على إيه)، `GET /api/credits` (لصفحة About)، و `GET /api/voices` (أنهي مقاطع أصوات الشخصيات متركبة). والملفات الثابتة بتيجي من `/audio/...` (الأصوات) و `/references/...` (رسومات الإشارات).

ملحوظة: فرق بين الكود والوصف القديم. النص الإنجليزي بيقول "three REST calls" (تلات طلبات) وبعدين بيعدّ أربعة. الصح من الكود إنهم **أربعة**: التلاتة الأولانيين موجودين في `services/api.ts` (`api.catalog()`, `api.info()`, `api.credits()`)، والرابع `/api/voices` بيتطلب من ملف تاني هو `audio/voices.ts` (الدالة `fetchVoiceManifest`).

شجرة الـ Providers

الملف `src/main.tsx` بيلف التطبيق في "providers" (يعني React contexts) بالترتيب ده؛ اللي جوه يقدر يستخدم اللي بره:

```
<StrictMode>
  <BrowserRouter>
    <ThemeProvider>          Naruto / Sasuke colours
      <SettingsProvider>     user preferences (localStorage)
        <SoundProvider>      Web Audio engine (effects, voices, jutsu music)
          <MusicProvider>    background music loop
            <CatalogProvider>  /api/jutsu + /api/info
              <App />
```

ليه الترتيب ده بالذات؟ لأن `SoundProvider` محتاج يقرا الإعدادات (مستوى الصوت) والثيم (عشان يشغّل صوت لما تبدّلي Naruto/Sasuke)، فلازم يكون جوه `SettingsProvider` و `ThemeProvider`. و `MusicProvider` محتاج محرك الصوت فلازم يكون جوه `SoundProvider`. و `CatalogProvider` مستقل تقريبًا فهو الأقرب للتطبيق.

ملفات المشروع في جذر `frontend/`

الملف `frontend/package.json`

ده "بطاقة تعريف" مشروع npm اسمه `shinobi-ai-web` (النسخة 0.1.0، و `"type": "module"` معناها إن الملفات بتستخدم `import` / `export` الحديثة).

السكريبت	الأمر	بيعمل إيه
`npm run dev`	`vite`	بيشغّل سيرفر التطوير على البورت 5173 مع hot reload (الصفحة بتتحدث أول ما تحفظي)
`npm run build`	`tsc -b && vite build`	بيعمل فحص أنواع لكل حاجة، وبعدين بيبني التطبيق المحسّن في `dist/`
`npm run typecheck`	`tsc -b`	فحص TypeScript بس
`npm run lint`	`oxlint`	linter سريع (بيدور على الأخطاء الشائعة)
`npm test`	`vitest run`	بيشغّل اختبارات الوحدة (`*.test.ts`) مرة واحدة

السكريبت	الأمر	بيعمل إيه
`npm run preview`	`vite preview`	بيقدّم `dist/` المبني عشان تبصي عليه بسرعة

الـ dependencies الأساسية (المكتبات اللي التطبيق محتاجها وهو شغال): `react` , `react-dom` , `react-router-dom` (الصفحات والعناوين)، `framer-motion` (الأنيميشن)، `lucide-react` (الأيقونات)، وكذا حزمة `@fontsource` (الخطوط متضمنة جوه التطبيق، فمفيش طلب لـ Google Fonts: Inter, Nunito, Bebas Neue, Caveat Brush, Permanent Marker, Rajdhani, Yuji Syuku).

الـ dev dependencies (مكتبات التطوير بس): `vite` , `@vitejs/plugin-react` , `typescript` , `vitest` (اختبارات الوحدة)، `oxlint` , `playwright-core` (بيتحكم في Microsoft Edge في اختبارات e2e)، وحزم الأنواع (`@types/node` , `@types/react` , `@types/react-dom`).

الملف `frontend/vite.config.ts`

ده إعداد Vite. الكود بالظبط:

```
const BACKEND = process.env.SHINOBI_BACKEND ?? 'http://127.0.0.1:8000'

export default defineConfig({
  plugins: [react()],
  cacheDir: process.env.VITE_CACHE_DIR ?? 'node_modules/.vite',
  server: {
    port: 5173,
    strictPort: true,
    proxy: {
      '/api': BACKEND,
      '/references': BACKEND,
      '/audio': BACKEND,
      '/ws': { target: BACKEND, ws: true },
    },
  },
  test: {
    include: ['src/**/*.test.ts', 'src/**/*.test.tsx'],
    environment: 'node',
  },
})
```

بيعمل إيه، سطر سطر:

- `BACKEND` : عنوان الباك إند. لو فيه متغير بيئة اسمه `SHINOBI_BACKEND` بيستخدمه، وإلا الافتراضي `http://127.0.0.1:8000`. علامة `??` معناها "لو اللي على الشمال مش موجود، خد اللي على اليمين".
- `plugins: [react()]` : بيفعّل JSX والـ fast refresh (تحديث الكومبوننت من غير ما الـ state تضيع).
- `cacheDir` : فولدر الكاش ممكن يتغير بمتغير البيئة `VITE_CACHE_DIR` ، عشان تقدري تشغّلي سيرفرين تطوير جنب بعض من غير ما يتخانقوا على نفس الكاش.
- `server.port: 5173` مع `strictPort: true` : لو البورت مشغول، يفشل بدل ما يختار بورت تاني بالسكوت (عشان الاختبارات بتفترض 5173).
- `server.proxy` : في التطوير، أي طلب لـ `/api` أو `/references` أو `/audio` أو الـ WebSocket `/ws` (`ws:true`) بيتحوّل للباك إند. بفضل ده التطبيق دايمًا بيكلم "نفسه" (نفس الـ origin)، فنفس العناوين النسبية

بتشتغل في التطوير وفي النسخة المبنية.

- `test`: Vitest بيشغّل `src/**/*.test.ts(x)` في بيئة `node` (من غير DOM متصفح)، لأن الكود اللي بيتختبر منطق صافي.

مين بيستخدمه؟ أوامر `npm run dev` و `npm run build` و `npm test` كلها بتقراه.

الملف `frontend/index.html`

صفحة الـ HTML الوحيدة في التطبيق كله (تطبيق "صفحة واحدة"). بيحط `lang="en"` و `data-theme="naruto"` على عنصر `<html>`، أيقونات التاب (`/konoha-leaf.svg?v=3`) ومقاسات PNG؛ الـ `?v=3` بتجبر المتصفح يرمي أيقونة قديمة متخزنة عنده)، الـ viewport، لون الثيم، الوصف، وتاجات Open Graph (اللي بتظهر لما حد يشارك اللينك)، والعنوان `NARUTO`.

وفيه سكريبت صغير جوه الصفحة:

```
<script>
  // apply the saved theme before the first paint (no flash of the wrong colours)
  try {
    var t = JSON.parse(localStorage.getItem('shinobi.theme'))
    if (t === 'sasuke' || t === 'naruto') document.documentElement.dataset.theme = t
  } catch (e) {}
</script>
```

بيعمل إيه: بيقرا `shinobi.theme` من `localStorage` (تخزين صغير في المتصفح بيفضل بعد قفل الصفحة) **قبل أول رسم للصفحة**. ليه؟ عشان المستخدم اللي اختار Sasuke ما يشوفش ألوان Naruto لحظة وبعدين تتبدل ("وميض"). الـ `try/catch` عشان لو التخزين مقفول أو فيه قيمة بايظة، الصفحة ما تقعش.

الـ body فيه بس `<div id="root">` وبيحمّل `/src/main.tsx`. React بيرسم التطبيق كله جوه الـ div ده.

ملحوظة صغيرة من الكود: لسه فيه `<meta name="theme-color" content="#05060b" />` و `<meta name="color-scheme" content="dark" />`، ودول من أيام التصميم الغامق القديم، مع إن التصميم الحالي فاتح. ده مش بيكسر حاجة، بس كويس تعرفيه.

ملفات تانية في الجذر

- `tsconfig.json` و `tsconfig.app.json` و `tsconfig.node.json`: إعدادات TypeScript (الهدف ES2023، و `jsx: react-jsx`، و `noEmit` لأن Vite هو اللي بيبني، وقواعد صارمة "ممنوع متغيرات أو بارامترات مش مستخدمة").
- `public/`: ملفات بتتنسخ زي ما هي لجذر الموقع: `favicon.svg`، `favicon.ico`، `favicon-32/48.png`، `apple-touch-icon.png`، `konoha-leaf.svg` ومقاساتها PNG.
- `README.md`: ملاحظات للمطورين. `dist/` هو ناتج البناء، و `node_modules/` الحزم المتنزلة. (دول مش جزء من الكود اللي بنشرحه.)

نقطة البداية والـ Routing

الملف `frontend/src/main.tsx`

مسؤول عن إيه: ده أول ملف TypeScript بيشتغل. بيعمل import لكل الخطوط ولملفين الـ CSS العامين (`konoha/konoha.css` و `styles/global.css`)، وبعدين بيرسم شجرة الـ providers اللي فوق حوالين `<App />` جوه `#root`.

```
createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <BrowserRouter>
      <ThemeProvider>
        <SettingsProvider>
          <SoundProvider>
            <MusicProvider>
              <CatalogProvider>
                <App />
              </CatalogProvider>
            </MusicProvider>
          </SoundProvider>
        </SettingsProvider>
      </ThemeProvider>
    </BrowserRouter>
  </StrictMode>,
)
```

خطوة بخطوة:

1. `document.getElementById('root')!` بيجيب الـ div اللي في `index.html`. علامة `!` معناها في TypeScript "أنا متأكد إنه موجود".
2. `createRoot(...).render(...)` بيقول لـ React "ابدأ ارسم هنا".
3. `StrictMode`: وضع بيخلي React في التطوير يشغّل الـ effects مرتين عشان يكشف الأخطاء. عشان كده كل hook في المشروع ده بينضف ورا نفسه (cleanup) بعناية؛ لو hook فتح WebSocket ومقفلوش، StrictMode هيكشف ده.
4. `BrowserRouter`: بيفعّل الـ routing بالعناوين الحقيقية.
5. باقي الـ providers زي ما شرحنا في الشجرة.

الخطوط اللي بتتعمل لها import بالظبط: `@fontsource-variable/inter`، `@fontsource/bebas-neue`، أوزان Rajdhani (500 و 600 و 700)، `@fontsource/yuji-syuku`، `@fontsource/caveat-brush`، `@fontsource/permanent-marker`، `@fontsource-variable/nunito`.

الملف `frontend/src/App.tsx`

مسؤول عن إيه: بيعرّف الـ routes بـ `<Routes>` بتاعة React Router.

```
<Routes>
  <Route element={<KonohaShell />}>
    <Route index element={<HomePage />} />
    <Route path="live" element={<LiveDemoPage />} />
    <Route path="practice" element={<KonohaPracticePage />} />
    <Route path="practice/:jutsuId" element={<KonohaPracticePage />} />
    <Route path="library" element={<KonohaJutsuPage />} />
    <Route path="library/:jutsuId" element={<KonohaJutsuPage />} />
    <Route path="settings" element={<KonohaSettingsPage />} />
    <Route element={<LegacyFrame />}>
      <Route path="about" element={<AboutPage />} />
      <Route path="*" element={<NotFoundPage />} />
    </Route>
  </Route>
</Routes>
```

بيعمل إيه:

- فيه route أب ملهوش `path`، عنصره `<KonohaShell />`. ده اسمه **layout route**: بيرسم الهيدر والخلفية والفوتر، وجواه مكان اسمه `<Outlet />` بتترسم فيه الصفحة الابن حسب العنوان.
- `index` معناها "الصفحة الافتراضية" للمسار `/`.
- `/about` والـ catch-all `*` (يعني أي عنوان مش معروف ← 404) جوه layout زيادة هو `<LegacyFrame />`، اللي بيحط الصفحات الغامقة القديمة في لوحة غامقة عشان تفضل مقروءة على الخلفية الفاتحة.
- فيه تعليق في الكود بيقول إن الصفحات الغامقة القديمة `pages/PracticePage.tsx` و `pages/LibraryPage.tsx` و `pages/SettingsPage.tsx` لسه موجودة في الريبو بس **مش متوصلة بأي route**: الـ routes بتستخدم نسخ Konoha.

مثال عملي: لما تكتبي `http://localhost:5173/practice/chidori`، React Router بيلاقي `practice/:jutsuId`، فبيرسم `KonohaShell` وجواه `KonohaPracticePage`، والـ `jutsuId` بيبقى `chidori`.

الأنواع (Types): العقد بين الفرونت إند والباك إند

قبل ما نشرح أي كود بيشتغل، لازم نفهم "شكل البيانات". الفولدر `src/types/` ما فيهوش أي منطق؛ فيه بس **تعريفات أنواع** TypeScript بتوصف كل رسالة جاية أو رايحة. فكري فيه كأنه "عقد" مكتوب: الباك إند وعد إنه يبعت كذا، والفرونت إند وعد إنه يفهم كذا.

الملف `frontend/src/types/protocol.ts`

مسؤول عن إيه: أنواع TypeScript لكل حاجة بتعدّي على الـ WebSocket (نسخة البروتوكول `PROTOCOL_VERSION = 1`). أسماء الخانات مكتوبة بـ snake_case (يعني `time_left` مش `timeLeft`) بالظبط زي ما Python بيبعتها، عشان ما يحصلش أي تحويل يغلّط.

ليه اتعمل: عشان أي ملف يستخدم `FrameState` مثلًا، TypeScript يعرف إن فيه `progress` و `events` ويمنعك تكتبي اسم غلط.

الأنواع الأساسية:

- `SignId`: أسماء الإشارات: `idle`, `ram`, `snake`, `tiger`, `dog`, `monkey`, `boar`, `horse`, `clone_seal`, `rasengan_palm`. (`idle` معناها "مفيش إشارة"، و `rasengan_palm` هي كف إيد واحدة مفتوحة والصوابع مفرودة.)
- `JutsuId` / `EffectId`: الجوتسو ومؤثره: `fire_style` / `fire`، `chidori`، `shadow_clone`، `rasengan`.
- `ServerInfo` (الرد على `/api/info` وجوه رسالة `hello`): الاسم، النسخة، هل موديل YOLOX وموديل الـ landmarks متحملين، الإشارات اللي يقدر يتعرف عليها `recognizable`، الجوتسو المتاحة `jutsu_available`، أعداد الجلسات، حدود الفريم (`frame`, `max_width`, `max_height`, `jpeg_quality`, `max_in_flight`)، و `mirror`، و `sequence_max_gap` الاختياري (عدد الثواني المسموح بيها بين إشارتين في Live، الافتراضي 7).
- `Mode`: الوضع: يا إما الوضع الحر (Home و Live، كل الجوتسو) يا إما وضع التدريب على جوتسو واحد (شوفي الملحوظة تحت القايمة).
- `EffectState` و `EffectPhase` (`voice` ← `charge` ← `name` ← `release`): الخط الزمني لتفعيل جوتسو اتعمل: الأول صوت الشخصية، بعدين شحن التشاكرا، بعدين اسم الجوتسو، بعدين الإطلاق. ولـ Shadow Clone كمان `clones` (عدد النسخ الظاهرة) و `crowded` (المكان مش كفاية للـ 10 نسخ كلهم من غير تداخل؛ بس كلهم بيترسموا برضه). وفيه كمان `phase_left` (الثواني الباقية في مرحلة التحضير).
- `FrameState`: الـ JSON header اللي مع كل فريم معالَج. فيه: `seq` (رقم الفريم)، `client_t` (صدى الوقت اللي الفرونت إند بعته)، المقاس، `fps`، `timings` (وقت كل مرحلة)، `hands` (عدد الإيدين)، `sign` (الإشارة المدموجة ومعاها `source` و `score`)، `candidate` / `hold` / `latched` (حالة المثبّت stabiliser اللي بيتأكد إن الإشارة ثابتة)، `history`، `progress` (عدد الخطوات اللي اتعملت لكل جوتسو)، `time_left` و `time_left_per_jutsu`، `ready` (بتبقى false وهو الـ AI بيسخّن)، `cooldown`، `mode`، `effect`، و `events`.
- الـ Events (الأحداث): `SignEvent`، `StepEvent` (jutsu, step, total, sign)، `SequenceFailedEvent` (`reason:'timeout' | 'wrong_sign'`)، `WrongSignEvent` (expected و got؛ والتقدم **بيفضل محفوظ**)، `JutsuEvent` (فيه `stages` اختياري بأطوال مراحل voice/charge/name)، `EffectPhaseEvent`، `EffectStoppingEvent`، `EffectEndEvent`، `ClonesEvent`.
- رسايل من الفرونت للسيرفر (`ClientMessage`): `mode`، `reset`، `options` (`landmarks` شغال/مقفول)، `ping`.
- رسايل نصية من السيرفر: `hello`، `mode`، `pong`، `skipped`، `error` (`busy`, `bad_frame`, `bad_message`, `internal`، ومعاها `fatal`).

ملحوظة: فرق بين الكود والوصف القديم (نوع `Mode`). النص الإنجليزي كتب الوضع الحر `{mode:'free'}`، لكن النوع في الكود بالظبط:

```
export type Mode = { mode: 'free'; jutsu: null } | { mode: 'practice'; jutsu: JutsuId }
```

يعني الوضع الحر جوه الفرونت إند فيه كمان `jutsu: null` صريحة، ووضع التدريب فيه اسم الجوتسو. العلامة `|` معناها "يا ده يا ده" (union type). أما الرسالة اللي بتتبعت فعلًا على السلك فبتتبني في `useShinobiSession` بالدالة `modeMessage`: للوضع الحر بتبعت `{ type: 'mode', mode: 'free' }` من غير `jutsu`، وللتدريب `{ type: 'mode', mode: 'practice', jutsu }`.

مثال يوضح الفرق بين `wrong_sign` و `sequence_failed` من تعليقات الكود: السيرفرات القديمة كانت بتبعت `sequence_failed` بسبب `wrong_sign` ومعاه reset، لكن السيرفر الحالي بيبعت `WrongSignEvent` ومش بيمسح

التقدم؛ المستخدم بيحاول نفس الخطوة تاني. الـ reset الوحيد في Live هو `timeout` (عدّى أكتر من `sequence_max_gap` من غير إشارة صح).

مين بيستخدمه: تقريبًا كل ملف في الفرونت إند: `framing.ts` بيتحقق من الشكل، `useShinobiSession` بيورّع الـ `FrameState`، ومخططات الصوت والكروت بيقروا الـ events.

الملف `frontend/src/types/catalog.ts`

مسؤول عن إيه: أنواع ردود الـ REST.

- `SignInfo`: الاسم، الاسم الياباني، عدد الإيدين، `official` (إشارة أصلية ولا لأ)، `cues` (تعليمات إزاي تعمليها)، `art` (عنوان الرسمة الخطية)، `recognizer` (`'yolox' | 'landmarks'`: أنهي موديل بيتعرف عليها)، `recognizable` (السيرفر يقدر يتعرف عليها دلوقتي ولا لأ).
- `JutsuStep`: رقم الخطوة، الإشارة، و `card` (عنوان صورة الكارت المرجعي).
- `JutsuInfo`: id، title، name، kanji، `official` (الأختام الأصلية ولا تسلسل مخصص)، effect، description، `steps`، `available`، `missing_signs`.
- `Catalog = { signs, jutsu }`.
- `Credits`: أقسام فيها عناصر (نص، url، رخصة) لصفحة About.

الخدمات (Services): الكلام مع السيرفر

الملف `frontend/src/services/api.ts`

مسؤول عن إيه: طلبات REST للباك إند.

```
async function getJSON<T>(path: string, signal?: AbortSignal): Promise<T> {
  let response: Response
  try {
    response = await fetch(path, { signal, headers: { Accept: 'application/json' } })
  } catch (error) {
    if (error instanceof DOMException && error.name === 'AbortError') throw error
    throw new ApiError('The SHINOBI AI server cannot be reached. Is it running?', null)
  }
  if (!response.ok) throw new ApiError(`${path} answered ${response.status}`, response.status)
  return (await response.json()) as T
}
```

خطوة بخطوة:

1. `fetch(path, ...)` بيبعت الطلب ومعاه `Accept: application/json` (يعني "عايز JSON").
2. لو `fetch` نفسه فشل (السيرفر مقفول، مفيش نت)، بيحصل exception. لو السبب إن الطلب اتلغى بقصد (`AbortError`، مثلًا الكومبونننت اختفى) بيرميه زي ما هو. غير كده بيحوله لرسالة لطيفة: "The SHINOBI AI server cannot be reached. Is it running?" ومعاها `status = null`.
3. لو السيرفر رد بس بكود خطأ (زي 502)، بيعمل `ApiError` برسالة زي `"/api/jutsu answered 502"`.
4. لو تمام، بيرجّع الـ JSON.

باقي الملف:

- `ApiError`: نوع Error فيه `status` (رقم HTTP، أو `null` لو السيرفر ما اتوصلناش ليه أصلًا).
- `api.info()`، `api.catalog()`، `api.credits()`: تلات طلبات REST (`/api/info`، `/api/jutsu`، `/api/credits`).
- `sessionUrl()`: بيبني `ws://host/ws` (أو `wss://` لو الصفحة https) من عنوان الصفحة الحالي.

مين بيستخدمه: `CatalogContext` (catalog و info)، صفحة About (credits)، و `useShinobiSession` (sessionUrl).

الملف `frontend/src/services/socket/framing.ts` (ومعاه `framing.test.ts`)

مسؤول عن إيه: functions صافية (من غير WebSocket ومن غير DOM) للتعامل مع الشكل الـ binary: `[uint32 big-endian N][N bytes UTF-8 JSON][JPEG]`. يعني: أول 4 bytes رقم بيقول طول الـ JSON، بعده الـ JSON نفسه، وبعده صورة الـ JPEG.

ليه binary ومش JSON بس؟ لأن الصورة نفسها bytes، ولو حولناها نص (base64) هتكبر حوالي الثلث وتبطّأ. فبنحط JSON صغير وصورة خام في رسالة واحدة.

```
export function encodeBinaryMessage(header: unknown, payload: ArrayBuffer | Uint8Array): ArrayBuffer {
  const json = encoder.encode(JSON.stringify(header))
  const body = payload instanceof Uint8Array ? payload : new Uint8Array(payload)
  const out = new Uint8Array(4 + json.byteLength + body.byteLength)
  new DataView(out.buffer).setUint32(0, json.byteLength, false) // big-endian length
  out.set(json, 4)
  out.set(body, 4 + json.byteLength)
  return out.buffer
}
```

خطوة بخطوة:

1. `JSON.stringify(header)` بيحول الـ header (مثلًا `{t: 1234.5}`) لنص، و `encoder.encode` بيحوله bytes بترميز UTF-8.
2. `body` هي bytes الصورة.
3. بيحجز مصفوفة bytes جديدة طولها 4 + طول الـ JSON + طول الصورة.
4. `setUint32(0, json.byteLength, false)` بيكتب طول الـ JSON في أول 4 bytes. الـ `false` معناها big-endian (الـ byte الأكبر الأول)، وده اللي Python متفق عليه.
5. بيحط الـ JSON بعد أول 4 bytes، والصورة بعده.
6. بيرجّع الـ buffer جاهز يتبعت.

مثال: لو الـ JSON طوله 12 byte والصورة 30000 byte، الرسالة هتبقى 30016 byte، أول 4 منهم قيمتهم `00 00 00 0C`.

باقي الـ functions:

- `decodeBinaryMessage`: العكس. بيرمي `FramingError` لو الرسالة أقصر من اللازم، أو الطول بيعدّي آخر الرسالة، أو الـ header مش JSON صالح بـ UTF-8. الصورة بترجع كـ "view" (من غير نسخ، عشان السرعة).

- `parseFrameState(header)`: بيتأكد إن الـ header شكله `FrameState` فعلًا (النوع `frame`، و `seq` / `width` / `height` أرقام، فيه `sign`، فيه `mode`)، بيفلتر الـ events (أي event مش معروف أو بايظ بيترمي، والمعروفين بيتفحصوا خانة خانة في `isEvent`)، وبيملا قيم افتراضية آمنة (`fps` = 0، `ready` = true للسيرفرات القديمة، `time_left` = null...). ده بيحمي الصفحة من إنها تقع لو البروتوكول اتغير شوية.
- `parseServerText(text)`: بيفك ويتحقق من الرسايل النصية؛ `skipped` ممكن تيجي بـ `client_t: null`؛ و `error` من غير `fatal` بيتحسب مش قاتل.
- `fitWithin(w, h, maxW, maxH)`: بيصغّر مقاس عشان يدخل في حدود معينة مع الحفاظ على النسبة، ومن غير تكبير أبدًا. مثال: 1920×1080 جوه 1280×1280 يبقى 1280×720.
- `InFlightFrames`: دفتر حسابات للفريمات اللي اتبعتت ولسه ماجلهاش رد، متسجلة بالـ `t` بتاعها. لأن السيرفر بيرد بالترتيب، `answer(t)` بتقفل كمان أي فريم أقدم؛ `skip(t)` بتقفل واحد؛ `dropOldest()` للـ `skipped` اللي ملهاش وقت؛ و `expire(now, maxAge)` بترمي فريمات عمرها ما جالها رد، عشان رد ضايع عمره ما يوقف البث.

`framing.test.ts` بيختبر: ترتيب الـ bytes، الذهاب والعودة (encode ثم decode يرجّع نفس الحاجة، حتى مع JSON فيه حروف مش ASCII وheaders أطول من 255 و 65535 byte)، كل حالات الرفض، التحقق من الفريمات والـ events والرسايل النصية، و `fitWithin`، و `InFlightFrames`.

مين بيستخدمه: `client.ts` (encode/decode والـ parse)، و `useShinobiSession` (و `fitWithin` و `InFlightFrames`).

الملف `frontend/src/services/socket/client.ts` (ومعاه `client.test.ts`)

مسؤول عن إيه: `ShinobiSocket` هو عميل WebSocket بأنواع واضحة. نسخة واحدة = جلسة واحدة مع الباك إند. ما يعرفش أي حاجة عن الكاميرا أو React؛ ده مقصود عشان يتختبر لوحده.

- **الحالات** (`ConnectionState`): `idle` (مش شغال)، `connecting` (بيحاول)، `connected` (بعد ما السيرفر يبعت `hello`)، `reconnecting` (الاتصال وقع وبيحاول تاني)، `offline` (فشل كذا مرة)، `busy` (مفيش مكان جلسة فاضي في السيرفر)، `error` (اختلاف في نسخة البروتوكول أو خطأ قاتل). و `SocketStatus` بيضيف رسالة للمستخدم، وعدد مرات الفشل المتتالية `failures`، و `retryAt` (إمتى المحاولة الجاية).
- **الـ Handlers** (الدوال اللي بتتنادى لما حاجة تحصل): `onStatus`، `onHello`، `onFrame(state, jpeg)`، `onServerError`، `onPong(t, rtt)`، `onMode`، `onSkipped`.
- الدوال: `connect()`، `disconnect()`، `retryNow()` (تخطي وقت الانتظار: زرار Retry أو رجوع المتصفح أونلاين)، `send(message)` و `sendFrame(header, jpeg)` (الاتنين بيرفضوا لو مش متوصل).

إعادة الاتصال بـ exponential backoff (يعني كل مرة نستنى ضعف المرة اللي قبلها):

```
const DEFAULT_BACKOFF: BackoffOptions = { baseMs: 600, maxMs: 8000, factor: 2, jitter: 0.2 }

export function backoffDelay(failures: number, opts: BackoffOptions, random: () => number =
Math.random): number {
  const raw = Math.min(opts.maxMs, opts.baseMs * opts.factor ** Math.max(0, failures - 1))
  const jitter = 1 + opts.jitter * (2 * random() - 1)
  return Math.round(Math.min(opts.maxMs * (1 + opts.jitter), raw * jitter))
}
```

بيعمل إيه: المدة = `base × 2^(failures-1)` وأقصاها `maxMs`، مضروبة في jitter عشوائي (±20%). القيم: 600 ms أول مرة، 1200 تاني مرة، 2400، 4800، وبعدين 8000 كحد أقصى. الـ jitter (العشوائية الصغيرة) ليه؟ عشان لو كذا متصفح وقعوا مع بعض، ما يرجعوش يضربوا السيرفر في نفس اللحظة بالظبط.

باقي القواعد (كلها قيم افتراضية في الـ constructor):

- بعد 3 مرات فشل (`offlineAfter: 3`) الحالة بتبقى `offline` (وبيفضل يحاول في الخلفية).
- رد `busy` (خطأ `busy` أو كود إغلاق 1013 = `CLOSE_TRY_AGAIN_LATER`) بيستنى 4 ثواني على الأقل (`busyRetryMs: 4000`).
- **مهلة الـ hello**: محاولة ماجالهاش `hello` خلال 8 ثواني (`helloTimeoutMs: 8000`) بتتقفل.
- **فحص البروتوكول**: `hello` بنسخة بروتوكول تانية بيقفل الـ socket بكود 4001 (`CLOSE_PROTOCOL`) وبيوقف المحاولات برسالة واضحة.
- **نبض القلب واكتشاف التعليق**: كل 4 ثواني (`pingIntervalMs: 4000`) بيبعت `ping`؛ ولو اتبعت حاجة ومجاش أي رد لمدة 12 ثانية (`staleMs: 12000`)، بيقفل بكود 4000 (`CLOSE_STALE`) ويعيد الاتصال.
- عدّاد `generation` بيزيد مع كل WebSocket جديد، فأي callback متأخر من socket قديم بيتجاهل.

`client.test.ts` بيستخدم WebSocket مزيف و timers مزيفة عشان يختبر: نمو الـ backoff والـ jitter، `connecting` ثم `connected` بعد الـ hello، الإرسال بس وهو متوصل، توصيل الفريمات والرسايل النصية، `offline` والمحاولات، `reconnecting` ثم `offline` ثم الرجوع، التعامل مع busy (حتى إغلاق 1013 لوحده)، التوقف عند اختلاف البروتوكول، الخطأ الداخلي القاتل، مهلة الـ hello، `disconnect()` النضيف، `retryNow()`، وإعادة الاتصال بسبب ping/التعليق.

مين بيستخدمه: `useShinobiSession` بس.

الملف `frontend/src/services/socket/index.ts`

ملف "تجميع": بيعيد تصدير الأجزاء العامة من `client.ts` (`ShinobiSocket`, `backoffDelay`, `CLOSE_NORMAL`, `CLOSE_TRY_AGAIN_LATER` والأنواع) و `framing.ts` (`FramingError`, `InFlightFrames`, `decodeBinaryMessage`, `encodeBinaryMessage`, `fitWithin`, `parseFrameState`, `parseServerText`)، عشان الملفات التانية تكتب سطر واحد زي `import { ShinobiSocket, InFlightFrames } from '../services/socket'`.

الـ Hooks: الكاميرا والجلسة

الملف `frontend/src/hooks/useCamera.ts`

مسؤول عن إيه: hook حوالين `navigator.mediaDevices` (واجهة المتصفح للكاميرا والمايك).

بيرجّع إيه: `status` (`idle`، `requesting` بيطلب الإذن، `active` شغالة، `denied` الإذن اترفض، `unavailable` مفيش كاميرا، `insecure` الصفحة مش آمنة، `error`)، نص `stream` للمستخدم، قايمة `devices` (الكاميرات المتاحة)، `activeDeviceId`، والدوال `start(deviceId?)`، `stop()`، `refreshDevices()`.

تفاصيل مهمة:

- `start` بيطلب 1280×720 بـ 30 fps (قيم "مثالية" يعني لو الكاميرا ما تقدرش، المتصفح بيدّي أقرب حاجة):

```
width: { ideal: 1280 },
height: { ideal: 720 },
frameRate: { ideal: 30 },
```

- لو id كاميرا محفوظ بقى مش موجود (`OverconstrainedError` / `NotFoundError`) بيرجع للكاميرا الافتراضية.
- الأخطاء بتتحول لجمل واضحة؛ مثلًا الكاميرا مشغولة: "The camera is busy or not readable. Close other apps that use it (Zoom, Teams, the Python camera window) and try again.".
- عدّاد `requestId` بيتجاهل نتيجة `start()` قديم لو المستخدم داس تاني في النص (حماية من السباق race)، وبيوقف الـ stream الخسران.
- لو التراك خلص (الكاميرا اتشالت)، الحالة بتبقى `error` برسالة "The camera stopped".
- أسماء الأجهزة بتظهر بس بعد الإذن، فالقايمة بتتحدث بعد start ناجح ومع event `devicechange`.

مين بيستخدمه: `useDojo` (Home و Live)، `PracticeSession`، وقسم الكاميرا في Settings (كل واحد ليه نسخة خاصة).

الملف `frontend/src/hooks/useShinobiSession.ts`

مسؤول عن إيه: قلب الاتصال الحي. ده أهم hook في الفرونت إند. الخيارات: `stream` (من `useCamera`)، `mode`، `landmarks`، `enabled`. الجلسة بتشتغل بس وفيه stream: `active = enabled && stream !== null && !hiddenLong`.

بيرجّع إيه (`ShinobiSession`): `status`، `connected`، `info`، `sessionId`، آخر `frame` (FrameState)، `live` (فيه فريم معالَج على الـ canvas)، `latencyMs` (التأخير بعد التنعيم)، `serverFps`، `serverMode`، `paused` (التاب مخفي)، والدوال `attachCanvas`، `subscribe(listener)`، `reset()`، `retry()`.

الثوابت في أول الملف:

```
const HIDDEN_DISCONNECT_MS = 60_000 // a hidden tab gives its session slot back after this long
const MAX_FRAME_AGE_MS = 5000 // an unanswered frame older than this no longer blocks sending
const MAX_BUFFERED_BYTES = 1_500_000 // socket backpressure: wait instead of queueing more
const LATENCY_SMOOTHING = 0.2
const SAFETY_TICK_MS = 400
```

معنى القيم: تاب مخفي 60 ثانية بيرجّع مكانه في السيرفر؛ فريم ماجلوش رد في 5 ثواني ما يوقفش الإرسال؛ لو الـ socket عنده أكتر من 1.5 MB مستنية تتبعت، نستنى؛ تنعيم التأخير بمعامل 0.2؛ ومؤقت أمان كل 400 ms.

الإرسال (`pump`)

```
const pump = useCallback(function pump() {
  const p = pipe.current
  const socket = socketRef.current
  if (!socket || !socket.isConnected || p.encoding || p.hidden) return
  const video = p.video
  if (!video || video.readyState < 2 || !video.videoWidth || !video.videoHeight) return
  const now = performance.now()
  p.inflight.expire(now, MAX_FRAME_AGE_MS)
  if (p.inflight.size >= p.maxInFlight) return
  const wait = Math.max(p.lastSentAt + p.frameIntervalMs - now, socket.bufferedAmount >
MAX_BUFFERED_BYTES ? 50 : 0)
  ...
  ctx.drawImage(video, 0, 0, width, height) // as the camera delivers it: the server mirrors
  canvasToJpeg(canvas, p.quality)
    ...
      if (socket.sendFrame({ t }, bytes)) {
        p.inflight.add(t, performance.now())
        p.lastSentAt = t
      }
    ...
    .finally(() => {
      p.encoding = false
      pump()
    })
}, [])
```

خطوة بخطوة:

1. لو مفيش socket متوصل، أو فيه فريم بيتضغط دلوقتي، أو التاب مخفي: ما يعملش حاجة.
2. لو الفيديو لسه مجهزش: ما يعملش حاجة.
3. يرمي الفريمات القديمة اللي ماجلهاش رد (`expire`).
4. لو عدد الفريمات اللي مستنية رد وصل `max_in_flight` بتاع السيرفر: يستنى (الرد الجاي هينادي `pump` تاني).
5. يحسب لازم يستنى قد إيه: عشان ما يبعتش أسرع من معدل الكاميرا (أقصى 30 fps)، أو 50 ms لو الـ buffer مليان (backpressure يعني "ضغط راجع": ما نكدسش رسايل).
6. يصغّر الفريم بـ `fitWithin` لحدود السيرفر `max_width/max_height` ، ويرسمه **من غير مراية** (السيرفر هو اللي بيعمل المراية)، ويضغطه JPEG بجودة السيرفر `jpeg_quality` (باستخدام `OffscreenCanvas.convertToBlob` لو موجود).
7. يبعته بـ header فيه `{t: performance.now()}` ويسجله في `inflight` .
8. في الآخر (`finally`) ينادي `pump()` تاني.

و `pump` بتتنادى كمان بعد كل رد، وكل `skipped` ، وبمؤقت أمان كل 400 ms.

الاستقبال (`onFrame`)

```
onFrame: (frame, jpeg) => {
  const now = performance.now()
  if (frame.client_t !== null) {
    p.inflight.answer(frame.client_t)
    const rtt = now - frame.client_t
    if (rtt >= 0 && rtt < 60_000) {
      p.latency = p.latency === null ? rtt : p.latency + LATENCY_SMOOTHING * (rtt - p.latency)
    }
  }
  listeners.current.forEach((listener) => {
    try {
      listener(frame)
    } catch (e) {
      console.error('[shinobi] frame listener failed:', e)
    }
  })
  setView({ frame, latencyMs: p.latency, serverFps: frame.fps })
  draw(jpeg, frame.seq, socket)
  pump()
},
```

خطوة بخطوة:

1. يقفل الفريم ده في دفتر `inflight` (و `answer` بتقفل اللي أقدم منه كمان).
2. يحسب الـ RTT (الوقت من الإرسال للرد) وينعّم التأخير: القيمة الجديدة = القديمة + 0.2 × (الفرق). يعني التأخير المعروض ما يتنطط مع كل فريم.
3. **ينادي كل listener بشكل متزامن، قبل ما React يعيد الرسم.** ده نقطة مهمة جدًا: لو اعتمدنا على الـ state بس، React ممكن يدمج كذا تحديث ورا بعض وساعتها event زي `step` ممكن يضيع. بالطريقة دي كل event بيوصل لكل مستمع. والـ `try/catch` بيضمن إن مستمع بايظ ما يوقفش الباقيين.
4. بعدين يخزن الفريم في الـ state (عشان الواجهة تتحدث).
5. يرسم الـ JPEG.
6. ينادي `pump()` يبعت الفريم الجاي.

الرسم (`draw`)

بيعمل `createImageBitmap` للـ JPEG، وبعدين `drawImage` على الـ canvas المتوصل. أي فريم `seq` بتاعه أقدم من آخر واحد اترسم بيترمي (فالصورة عمرها ما ترجع لورا). وأول فريم يترسم بيخلي `live = true`.

باقي السلوك

- **لما يجي `hello`**: بيقرا حدود الفريم (`max_in_flight`, `max_width`, `max_height`, `jpeg_quality`) بقيم احتياطية 1 و 1280 و 1280 و 0.8)، وبعدين بيبعت الـ `mode` الحالي والـ `options` (landmarks).
- **تغيير الوضع أو الـ landmarks** بيتبعت أول ما يتغير.
- **ظهور التاب**: مخفي ← الإرسال بيقف؛ مخفي 60 ثانية ← الجلسة بتتقفل عشان ترجّع مكانها في السيرفر.
- **حدث `online`** في المتصفح بينادي `socket.retryNow()`.

- **قاعدة الأمان**: أي وقت الحالة مش `connected`، كل حاجة (frame، info، latency، live) بتتمسح (`clearSession`)، فالواجهة عمرها ما تعرض تعرّف من اتصال ميت.

مين بيستخدمه: `useDojo` (Home و Live) و `PracticeSession`، والنسخة القديمة `pages/PracticePage.tsx`.

مثال عملي: فتحتي `/live` ودوستي Start Camera. `useCamera` رجّع stream ← `active` بقت true ← الـ effect عمل `ShinobiSocket` و `connect()` ← السيرفر بعت `hello` ← الهوك بعت `{ type: 'mode', mode: 'free' }` و `options` ← `pump` بدأ يبعت فريمات ← أول رد اترسم على الـ canvas و `live` بقت true ← الكارت بيبدأ يعرض اللي الـ AI شايفه.

الـ Contexts (الحالة المشتركة على مستوى التطبيق كله)

افتكري: الـ context زي "لوحة إعلانات" معلقة فوق التطبيق، أي كومبوننت تحتها يقدر يقرا منها من غير ما حد يعدّيله props. الفولدر `src/contexts/` فيه أربعة.

الملف `frontend/src/contexts/ThemeContext.tsx`

مسؤول عن إيه: بيمسك `theme` (`naruto` أو `sasuke`)، و `setTheme`، و `toggleTheme`. القايمة الثابتة `THEMES = ['naruto', 'sasuke']`.

بيعمل إيه: بيكتب الثيم على `<html data-theme>` (متغيرات الـ CSS بتغير الألوان حسب القيمة دي)، وبيحفظه في `localStorage` تحت المفتاح `shinobi.theme` (نفس المفتاح اللي سكريبت `index.html` بيقراه قبل أول رسم). **عمره ما بيغيّر التعرّف**؛ هو ألوان بس.

مين بيستخدمه: `SoundProvider` (بيشغّل `ui.mode_switch` لما الثيم يتغير)، قسم Reset في Settings (بيرجّع الثيم Naruto)، و `ThemeToggle` القديم.

الملف `frontend/src/contexts/SettingsContext.tsx`

مسؤول عن إيه: تفضيلات المستخدم، محفوظة في `localStorage` تحت المفتاح `shinobi.settings`. مفيش قاعدة بيانات.

نوع `Settings`: `masterVolume`، `sfxVolume`، `voiceVolume`، `muted`، `audioEnabled`، `cameraId`، `animations` (`full` / `reduced` / `off`)، `showLandmarks`، `musicOn`، `musicVolume`، `jutsuMusic`.

القيم الافتراضية بالظبط من الكود:

```
export const DEFAULT_SETTINGS: Settings = {
  masterVolume: 0.8,
  sfxVolume: 0.9,
  voiceVolume: 1,
  muted: false,
  audioEnabled: true,
  cameraId: null,
  animations: 'full',
  showLandmarks: false,
  musicOn: false,
  musicVolume: 0.3,
  jutsuMusic: true,
}
```

معنى القيم: الصوت الرئيسي 80%، المؤثرات 90%، أصوات الشخصيات 100%، مش مكتوم، الصوت مفعّل، الكاميرا الافتراضية بتاعة المتصفح (`null`)، أنيميشن كامل، نقط الإيد (landmarks) مش ظاهرة، موسيقى الخلفية **مقفولة**، مستواها 0.3، وموسيقى الجوتسو شغالة.

الدوال المهمة:

- `sanitize()`: بيفحص كل قيمة اتحملت من التخزين: الـ volumes بتتحصر بين 0 و 1 (`clamp01`)، أي نوع غلط بيتبدل بالافتراضي. فتخزين بايظ عمره ما يكسر التطبيق. مثال: لو حد كتب يدوي `masterVolume: 5` هتبقى 1، ولو `animations: 'crazy'` هتبقى `'full'`.
- `update(patch)`: بيدمج التغيير ويعمل sanitize تاني. مثال: `update({ muted: true })`.
- `reset()`: يرجّع الافتراضي.
- `motion`: مستوى الأنيميشن الفعلي. لو نظام التشغيل طالب "reduced motion" (حركة أقل، لراحة بعض الناس)، `full` بتبقى `reduced`. القيمة بتتكتب على `<html data-motion>` عشان الـ CSS يستخدمها.

مين بيستخدمه: تقريبًا كل الصفحات: `useSettings()` في `useDojo` (showLandmarks)، `SequenceCard` (motion)، `SoundProvider` (الأصوات)، `MusicProvider`، `KonohaShell` (motion)، وصفحة Settings.

الملف `frontend/src/contexts/SoundContext.tsx`

مسؤول عن إيه: بيعمل `SoundEngine` واحد (شوفي قسم الصوت) وبيخليه متزامن مع الإعدادات.

```
const [engine] = useState(() => new SoundEngine())
const ready = useSyncExternalStore(engine.subscribe, engine.isReady, () => false)

const volume = outputVolume(settings)
useEffect(() => engine.setVolume(volume), [engine, volume])
const voiceVolume = voiceOutputVolume(settings)
useEffect(() => engine.setVoiceVolume(voiceVolume), [engine, voiceVolume])
const musicVolume = musicOutputVolume(settings)
useEffect(() => engine.setMusicVolume(musicVolume), [engine, musicVolume])
useEffect(() => engine.music.setEnabled(settings.jutsuMusic), [engine, settings.jutsuMusic])
useEffect(() => engine.setEnabled(settings.audioEnabled), [engine, settings.audioEnabled])
```

بيعمل إيه خطوة بخطوة:

1. `useState(() => new SoundEngine())`: بيعمل المحرك **مرة واحدة بس** طول عمر التطبيق (الـ function اللي جوه `useState` بتشتغل أول مرة بس).

2. `useSyncExternalStore` : hook بيخلي React "يتابع" قيمة عايشة بره React. هنا بيتابع `engine.isReady()` (هل المتصفح سمح بالصوت). كل ما المحرك يبلّغ إن حالته اتغيرت، الكومبوننت بيتحدث.
3. كل `useEffect` بيبعت قيمة من الإعدادات للمحرك أول ما تتغير: مستوى المؤثرات (`outputVolume`)، أصوات الشخصيات، موسيقى الجوتسو، تشغيل/قفل موسيقى الجوتسو (`jutsuMusic`)، و `audioEnabled` .

وكمان:

- أول ضغطة ماوس/لمس أو زرار كيبورد في أي مكان (`pointerdown` و `keydown`) بتنادي `engine.unlock()` . ليه؟ المتصفحات بتمنع الصوت لحد ما المستخدم يتفاعل مع الصفحة.
- لما الثيم يتبدل Naruto/Sasuke بيشغّل `ui.mode_switch` (مش في أول رسم).
- بيدّي hook-ين، الأول `useSound()` : واجهة بسيطة فيها `ready` ، `unlock` ، `play` ، `startLoop` ، `stopLoop` ، `stopAll` . ولو اتنادى بره الـ provider بيرجّع نسخة "صامتة" ما بتعملش حاجة (`silent`).
- والتاني `useSoundEngine()` : المحرك نفسه، بيستخدمه `useSessionSounds` وكونتكست الموسيقى.

مين بيستخدمه: `useSound()` في CameraCard (زرار Start Camera بيعمل unlock) وصفحة Settings (Test sound)؛ `useSoundEngine()` في `useSessionSounds` و `MusicContext` .

الملف `frontend/src/contexts/CatalogContext.tsx`

مسؤول عن إيه: بيحمّل `/api/jutsu` و `/api/info` مرة واحدة بالتوازي وبيشاركهم: `catalog` ، `info` ، `loading` ، `error` ، `reload()` ، `jutsu(id)` .

```
useEffect(() => {
  const abort = new AbortController()
  Promise.all([api.catalog(abort.signal), api.info(abort.signal)])
    .then(([c, i]) => {
      setCatalog(c)
      setInfo(i)
    })
    .catch((e: unknown) => {
      if (abort.signal.aborted) return
      setError(e instanceof Error ? e.message : String(e))
    })
    .finally(() => {
      if (!abort.signal.aborted) setLoading(false)
    })
  return () => abort.abort()
}, [attempt])
```

خطوة بخطوة:

1. `AbortController` : أداة بتلغي الطلب. الـ cleanup (`return () => abort.abort()`) بيلغيه لو الـ provider اختفى.
2. `Promise.all([...])` : بيبعت الطلبين مع بعض ويستنى الاتنين.
3. لو نجحوا: يحفظ الكتالوج والـ info.
4. لو فشل (ومش بسبب الإلغاء): يحفظ رسالة الخطأ (زي "The SHINOBI AI server cannot be reached. Is it running?").

5. في الآخر: `loading = false`.
6. الـ effect معتمد على `attempt`. فـ `reload()` بيزوّد `attempt` بواحد (ويرجع `loading` لـ true ويمسح الخطأ) ← الـ effect بيشتغل تاني ← طلب جديد.

و `jutsu(id)` بيدور على جوتسو بالـ id في الكتالوج.

مين بيستخدمه: `useDojo`، `JutsuView`، `PracticeView`، `PracticeSession`، والصفحات القديمة.

الملف `frontend/src/lib/storage.ts`

مسؤول عن إيه: تلات دوال مساعدة لـ `localStorage`: `loadJSON`، `saveJSON`، `removeKey`، و**عمرهم ما بيرموا exception**.

```
export function loadJSON<T>(key: string): T | null {
  try {
    const raw = window.localStorage.getItem(key)
    return raw === null ? null : (JSON.parse(raw) as T)
  } catch {
    return null
  }
}
```

بيعمل إيه: يحاول يقرا ويفك الـ JSON؛ لو أي حاجة فشلت (وضع التصفح الخاص، التخزين مليان، JSON بايظ) يرجّع `null` بهدوء. و `saveJSON` لو فشل ما يعملش حاجة، لأن التفضيلات "راحة" والتطبيق يشتغل من غيرها.

مين بيستخدمه: `SettingsContext` و `ThemeContext`.

واجهة Konoha: الإطار والقطع المشتركة

الفولدر `src/konoha/` فيه التصميم الفاتح الحالي: كروت شكلها "ورق بردي" (parchment) فوق لوحة قرية Konoha.

الملف `frontend/src/konoha/konoha.css`

مسؤول عن إيه: الـ design tokens العامة (يعني متغيرات CSS ثابتة زي الألوان والخطوط) لشكل Konoha، وكلها بتبدأ بـ `--k-` عشان ما تتخانقش مع متغيرات التصميم الغامق القديم: الخطوط (`--k-logo` = Permanent Marker، `--k-brush` = Caveat Brush، `--k-body` = Nunito)، ألوان الورق والحبر، لمسات Naruto البرتقالي و Sasuke الأزرق، ألوان حالة "Ready" و "Training"، الحواف المدورة، الظلال، ومنحنيات الحركة (easing).

مين بيستخدمه: بيتحمّل في `main.tsx`، وكل CSS modules بتاعة Konoha بتستخدم المتغيرات دي.

الملف `frontend/src/konoha/KonohaShell.tsx` (ومعاه `KonohaShell.module.css`)

مسؤول عن إيه: الـ layout route لكل الصفحات (الإطار اللي حوالين أي صفحة).

```
<MotionConfig reducedMotion={motion === 'full' ? 'never' : 'always'}>
  <div className={styles.shell}>
    <a href="#main" className={styles.skip}>
      Skip to content
    </a>
    <KonohaBackdrop />
    <KonohaHeader />

    <main id="main" ref={mainRef} className={styles.main} tabIndex={-1}>
      <div key={section} className={styles.page}>
        <Outlet />
      </div>
    </main>

    <footer className={styles.footer}>
      ...
      <NavLink to="/about" className={styles.footerLink}>
        About
      </NavLink>
    </footer>
  </div>
</MotionConfig>
```

بيعمل إيه:

- لينك "Skip to content" (بيظهر بالكيبورد بس، عشان مستخدم الكيبورد يقفز للمحتوى على طول).
- `<KonohaBackdrop />` الخلفية، و `<KonohaHeader />` الهيدر.
- `<main>` وجواه `<Outlet />` للصفحة.
- فوتر فيه تنويه إن ده مشروع معجبين غير رسمي ("Unofficial fan project for personal and educational use...") ولينك About.
- مع كل تغيير في المسار: يطلع لفوق الصفحة (`window.scrollTo`) وينقل تركيز الكيبورد لـ `<main>` (مفيد لقارئات الشاشة).
- `MotionConfig` من framer-motion: بيقفل أنيميشن framer لو مستوى الحركة مش `full`.
- الـ `div` اللي جوه main عليه `key={section}` (أول جزء من المسار)، فأنيميشن الدخول بيشتغل مرة واحدة لكل قسم (`/library` و `/library/fire_style` نفس القسم فمفيش إعادة).

الـ CSS module بيرتب الإطار: ارتفاع الشاشة، مكان الهيدر، عرض المحتوى، شكل لينك التخطي والفوتر.

مين بيستخدمه: `App.tsx` كعنصر الـ route الأب.

الملف `frontend/src/konoha/shell/KonohaBackdrop.tsx` (ومعاه `KonohaBackdrop.module.css`)

الخلفية الزخرفية الثابتة: اللوحة `assets/konoha/background.jpg` (Naruto على الشمال، Sasuke على اليمين)، طبقة تفتيح (wash)، ضوء برتقالي ناحية Naruto وضوء أزرق ناحية Sasuke، وكمان `<FallingLeaves />` **بس** لما الأنيميشن `full`. الـ CSS module بيثبّت الطبقات دي ورا كل حاجة (`position: fixed`) ويحدد التدرجات.

الملف `frontend/src/konoha/shell/FallingLeaves.tsx`

مسؤول عن إيه: canvas فيه من 4 لـ 12 ورقة شجر برتقالي (تقريبًا ورقة لكل 150000 بكسل مربع) بتنزل.

```
return Math.max(4, Math.min(12, Math.round((width * height) / 150_000)))
```

يعني: مساحة الشاشة على 150000، مقربة، ومحصورة بين 4 و 12. شاشة 1440×900 = 1296000 ÷ 150000 ≈ 9 ورقات.

- كل ورقة بترسم مرة واحدة على canvas صغير 64 بكسل (`SPRITE = 64`): شكل لوزة، عِرق في النص، وعروق جانبية.
- كل فريم أنيميشن بيحرك كل ورقة (نزول، انجراف، تمايل يمين وشمال، دوران، وضغط أفقي كأنها بتتقلب) ولما تخرج من الشاشة بترجع من فوق.
- بيقف لما التاب يتخفى، وبيتأقلم مع تغيير حجم الشباك وكثافة البكسل.

مين بيستخدمه: `KonohaBackdrop`.

الملفات `frontend/src/konoha/shell/KonohaHeader.tsx` (ومعاه `KonohaHeader.module.css`) و `frontend/src/konoha/shell/BrushSwash.tsx` و `frontend/src/konoha/shell/nav.ts`

- `KonohaHeader`: بيعرض شعار ورقة الشجر، فاصل، وكلمة "NARUTO" على ضربة فرشاة برتقالي (كلهم لينك واحد لـ `/`)، وقايمة التنقل من `KONOHA_NAV` (Home, Live, Practice, Jutsu, Settings؛ و About موجود في الفوتر)، وكمان `<MusicControl />`. اللينك النشط بيبقى تحته خط حبر بيتزحلق بين اللينكات بفضل `layoutId` المشترك في framer-motion (لما عنصرين ليهم نفس الـ layoutId، framer بيحرك واحد لمكان التاني بنعومة).
- `BrushSwash.tsx`: الـ SVG بتاع ضربة الفرشاة البرتقالي (تدرج لوني، فلتر turbulence/displacement بيخلي الأطراف خشنة، وخطوط شعيرات الفرشاة).
- `nav.ts`: عناصر القايمة (`KONOHA_NAV`).
- `KonohaHeader.module.css`: ترتيب الهيدر في النص، الشعار، القايمة، الخط اللي تحت اللينك النشط، وشكل الموبايل.

الملف `frontend/src/konoha/shell/MusicControl.tsx` (ومعاه `MusicControl.module.css`)

زرار موسيقى الخلفية الصغير في الهيدر: تشغيل/إيقاف، و popover (نافذة صغيرة طالعة) للصوت بتتقفل بضغطة بره أو بزرار Escape. على الموبايل بيظهر زرار التشغيل بس. بيستخدم `useMusic()` وبيعلّم نفسه بـ `MUSIC_CONTROL_ATTR` (قيمته `data-music-control`) عشان منطق "شغّل الموسيقى مع أول ضغطة" ما يتخانقش معاه (لو ضغطتي على زرار الموسيقى نفسه، هو اللي يقرر). الـ CSS module بيرسم الزرار والـ popover والـ slider.

الملف `frontend/src/konoha/shell/LegacyFrame.tsx` (ومعاه `LegacyFrame.module.css`)

layout route بيلف الصفحات الغامقة القديمة (About و 404) في لوحة غامقة نص شفافة، عشان تفضل مقروءة على الخلفية الفاتحة. مستخدم في `App.tsx`.

الملف `frontend/src/konoha/LeafEmblem.tsx`

رمز قرية ورقة الشجر (Hidden Leaf) مرسوم من أول وجديد كـ SVG paths (`LEAF_SPIRAL`، `LEAF_OUTER`، `LEAF_STEM`، `LEAF_TIP`) في مربع 100×100، بيُستخدم كشعار (والـ favicon بيستخدم نفس الـ paths). الـ Props: `size`، `weight` (سُمك الخط)، `title` (لو اتبعت بيبقى صورة ليها وصف لقارئ الشاشة؛ غير كده زخرفة بس).

الملف `frontend/src/konoha/BrushButton.tsx` ومعاه (`BrushButton.module.css`)

زرار الشكل الجديد. `variant="primary"` بيرسم ضربة فرشاة برتقالي SVG ورا الكلام، و `secondary` زرار ورقي هادي. فيه `icon` اختياري. ولو اتبعت `to` بيرسم `<Link>` بتاع الراوتر بدل `<button>` (يعني ينقلك لصفحة). مثال: زرار "Practice" في `JutsuDetail` هو `BrushButton` بـ `to="/practice/<id>"`.

الملف `frontend/src/konoha/BrushTag.tsx` ومعاه (`BrushTag.module.css`)

لافتة سودا بفرشاة ناشفة عليها عنوان أبيض بخط اليد (عناوين زي "LIVE CAMERA" و "JUTSU"). الـ prop `as` بتختار `h1` / `h2` / `h3` (عشان ترتيب العناوين يبقى صح لقارئ الشاشة).

الملف `frontend/src/konoha/ParchmentCard.tsx` ومعاه (`ParchmentCard.module.css`)

`<section>` متصمم كورق بردي، اختياري يكون فيه عنوان `BrushTag` (`title`، `icon`) و `actions` على يمين الهيدر. بيُستخدم في كل الكروت تقريبًا (SequenceCard، JutsuListCard، CameraCard، كارت الكوتش في Practice، أقسام Settings).

الملف `frontend/src/konoha/jutsuLook.ts`

`JUTSU_LOOK`: أيقونة lucide ولون ودرجة لكل جوتسو. **الحالة: مش مستخدم.** مفيش أي ملف بيعمله import دلوقتي (اتأكدنا بالبحث في الكود)؛ الكروت بقت بتستخدم `JutsuIcon`.

الدوجو (الصفحة الرئيسية Home وصفحة Live)

"الدوجو" هو المكان اللي بتتدربي فيه: الكاميرا + قايمة الجوتسو + كارت التسلسل. الصفحة الرئيسية وصفحة Live الاتنين بيستخدموا **نفس الحالة** من hook واحد اسمه `useDojo`، والفرق بينهم في الترتيب على الشاشة بس.

الملف `frontend/src/konoha/dojo/useDojo.ts`

مسؤول عن إيه: الـ state hook المشترك بين Home و Live. بيجمّع كل حاجة في مكان واحد.

ليه اتعمل: عشان Home و Live يبقوا متطابقين في المنطق، ولو اتصلح bug يتصلح في الاتنين.

بيجمّع إيه

```
const FREE: Mode = { mode: 'free', jutsu: null }
const SUCCESS_MS = 4000
const NOTICE_MS = 6000

export function useDojo() {
  const { catalog, info: restInfo, loading, error, reload } = useCatalog()
  const { settings } = useSettings()
  const camera = useCamera()
  const session = useShinobiSession({ stream: camera.stream, mode: FREE, landmarks:
settings.showLandmarks })
  useSessionSounds(session)
  const events = useLiveEvents(session, catalog)
  const info = session.info ?? restInfo
  ...
```

خطوة بخطوة:

1. `useCatalog()`: الكتالوج والـ info من REST.
2. `useSettings()`: عشان `showLandmarks`.
3. `useCamera()`: الكاميرا.
4. `useShinobiSession({stream, mode: FREE, landmarks})`: الجلسة في الوضع الحر (كل الجوتسو).
5. `useSessionSounds(session)`: يربط الجلسة بالصوت (أي event ← صوت).
6. `useLiveEvents(session, catalog)`: يحوّل الـ events لطبقة التفعيل والـ toasts.
7. `info`: لو الجلسة جابت info من الـ `hello` بيستخدمها (أحدث)، وإلا اللي جه من REST.

باقي المنطق

- لو الكتالوج فشل لأن السيرفر كان مقفول، بيعيد تحميله أول ما جلسة تتوصل:

```
useEffect(() => {
  if (connected && !catalog && !loading) reload()
}, [connected, catalog, loading, reload])
```

- `live` = الكاميرا شغالة **و** فيه فريمات معالجة على الشاشة **و** `frame.ready !== false` (مش وهو الـ AI بيسخّن). و `frame` بيبقى `null` لو مش live. يعني الكروت عمرها ما تعرض تقدم من فريم قديم أو من AI لسه مجهزش:

```
const cameraOn = camera.status === 'active' && camera.stream !== null
const live = cameraOn && session.live && session.frame?.ready !== false
const frame = live ? session.frame : null
```

- **أنهي جوتسو كارت التسلسل بيعرضه (`focusId`)**: الجوتسو اللي المستخدم داس عليه (`selected`)، وبيتحدث بـ `followFocus` كل ما التقدم يتغير. `followFocus` **بينقل لجوتسو تاني بس لو عنده خطوات أكتر بشكل صريح (strictly more)**، فإشارة غلط بالصدفة بتبدأ جوتسو تاني عمرها ما "تسرق" الكارت. الكود من `labels.ts`:

```
const lead = leadingJutsu(progress, order)
if (lead === null) return current
const mine = current ? (progress?.[current] ?? 0) : 0
return mine > 0 && (progress?.[lead] ?? 0) <= mine ? current : lead
```

مثال: إنتي في Rasengan خطوة 1، وعملتي `monkey` غلط فبدأ Fire Style خطوة 1. الاتنين متعادلين (1 و 1)، فـ `<= mine` صح، فالكارت **بيفضل** على Rasengan. ده بالظبط اللي اختبار `live_tie` بيتأكد منه.

- التفعيل (event `jutsu`) بينقل الكارت للجوتسو اللي اتعمل.
- وفيه رسالة timeout ظاهرة، الكارت بيبقى "متمسك" (held) بجوتسوه.
- الاحتياطي: أول جوتسو متاح، وإلا أول واحد.
- `complete`: بيبقى true لمدة 4 ثواني (`SUCCESS_MS`) بعد ما الباك إند يفعّل الجوتسو المعروض (والتقدم بتاعه بيرجع 0 في اللحظة دي):

```
const complete = live && focus !== null && success?.jutsu === focus.id && done === 0
```

- `timeLeft`: عدّاد الجوتسو المعروض من `time_left_per_jutsu` (كل جوتسو ليه ساعته الخاصة)؛ السيرفرات القديمة بتبعت بس `time_left` بتاع الجوتسو اللي في المقدمة، فبيستخدمه بس لو الجوتسو المعروض هو المتقدم.
- بيشترك في الفريمات عشان يبني ملاحظات الكارت:

```
subscribe((f) => {
  const on = focusRef.current
  for (const ev of f.events) {
    if (ev.type === 'wrong_sign' && ev.jutsu === on) {
      setWarning({ title: wrongSignTitle(ev, catalog), detail: wrongSignDetail(ev, catalog) })
      setMissKey((k) => k + 1)
    } else if (ev.type === 'step' && ev.jutsu === on) {
      setWarning(null)
      setMissKey(0)
      setNotice(null)
    } else if (ev.type === 'step') {
      setNotice(null)
    } else if (ev.type === 'sequence_failed' && ev.reason === 'timeout' && ev.jutsu === on) {
      setWarning(null)
      setMissKey(0)
      setNotice({ key: Date.now(), note: timeoutNotice(ev, catalog, maxGap) })
    } else if (ev.type === 'jutsu') {
      setWarning(null)
      setMissKey(0)
      setNotice(null)
    }
  }
}),
```

معناه: `wrong_sign` للجوتسو المعروض ← `warning` (تحذير) ويزوّد `missKey` (الحلقة الكهرماني على الخطوة)؛ `step` للجوتسو ده ← يمسحهم؛ `step` لجوتسو تاني ← يمسح رسالة الـ timeout بس (تسلسل جديد بدأ)؛ `sequence_failed` بسبب `timeout` ← `notice` مبني بـ `timeoutNotice` (بيفضل 6 ثواني، `NOTICE_MS`)؛ `jutsu` ← يمسح كل حاجة. ولما الجوتسو المعروض يتغير، ملاحظاته بتتمسح.

- `startOver()`: بيبعت `{type:'reset'}` (عن طريق `session.reset()`) وبيمسح الملاحظات. ده الـ reset الوحيد غير حد الوقت.

بيرجّع إيه: كل ده للعروض: `catalog`، `info`، `loading`، `error`، `reload`، `camera`، `session`، `events`، `cameraOn`، `live`، `frame`، `focusId`، `focus`، `done`، `complete`، `timeLeft`، `warning`، `missKey`، `notice`، `startOver`، `select`. ولاحظي إن `warning` و `missKey` بيرجعوا بس لو `live && done > 0`، و `notice` بس لو `live`.

نمط مهم: "اشتقاق الـ state أثناء الرسم" (derive state during render):

```
const [seenProgressKey, setSeenProgressKey] = useState(progressKey)
if (progressKey !== seenProgressKey) {
  setSeenProgressKey(progressKey)
  const next = held ? selected : followFocus(selected, frame?.progress, order)
  if (next && next !== selected) setSelected(next)
}
```

بيعمل إيه: بدل ما يستخدم `useEffect` يستنى بعد الرسم، بيقارن مفتاح (هنا التقدم لكل الجوتسو) بآخر مفتاح شافه. لو اتغير، يحدّث الـ state فورًا أثناء الرسم. React بيسمح بده وبيوفر effect زيادة ورسمة زيادة. النمط ده متكرر كذا مرة في المشروع.

مين بيستخدمه: `DojoView` و `LiveView`.

الملف `frontend/src/konoha/dojo/DojoView.tsx` (ومعاه `DojoView.module.css`)

مسؤول عن إيه: ترتيب الصفحة الرئيسية: `CameraCard` و `JutsuListCard` جنب بعض فوق، و `SequenceCard` تحتهم، وكمان فقرة مخفية بصريًا `aria-live` بتعلن الخطوات والفشل والتفعيل لقارئات الشاشة (`aria-live` معناها "اقرا النص ده بصوت عالي لما يتغير").

الـ Props: `heading` (عنوان `h1` مخفي) و `documentTitle` (عنوان التاب، عن طريق `useDocumentTitle`).

الـ CSS module: شبكة (grid) عمودين على الشاشات الكبيرة وعمود واحد على الموبايل.

مين بيستخدمه: `pages/HomePage.tsx`.

الملف `frontend/src/konoha/dojo/CameraCard.tsx` (ومعاه `CameraCard.module.css`)

مسؤول عن إيه: كارت "LIVE CAMERA". **الـ Props:** `camera`، `session`، `catalog`، `events`، `className`، `size` (`default` أو `hero` لصفحة Live).

بيعرض إيه:

- وهو الـ AI شغال live: بيعرض الـ `<canvas>` (`data-testid="processed-canvas"`)، متوصل بـ `session.attachCanvas` بالفريمات المعالجة: معمولة مراية خلاص، والمؤثرات مرسومة من Python.
- غير كده: بيعرض معاينة الـ `<video>` المحلية (معمولة مراية بالـ CSS) باهتة، مع رسالة: "AI is getting ready..." (بيتوصل، أو متوصل بس لسه مفيش فريم، أو بيسخّن)، أو اسم حالة الاتصال (offline, busy...) وزرار "Retry now". والنص بيقول صراحة إن مفيش تعرّف هيظهر لحد ما الـ AI يبقى live.

- `ConnectionChip`: اسم حالة الـ AI (أو "Paused" لو التاب مخفي)، التأخير بالـ ms، و fps السيرفر (`data-testid="stat-latency"` و `stat-fps`).
- `SignChip`: "AI sees" + اسم الإشارة الحالية ودرجتها، أو "No sign" / "No hands"، وشريط صغير بيوضح تقدم الـ hold بتاع المثبّت (`data-testid="current-sign"`).
- `StageOverlay` (طبقة التفعيل والـ toasts) بترسم فوق الـ canvas وهو live.
- والكاميرا مقفولة: "Camera is off"، أو "Waiting for the camera..."، أو رسالة مشكلة (اتمنعت، الصفحة مش آمنة، مش موجودة، خطأ) بـ `data-testid="camera-error"`.
- أزرار التحكم: "Start Camera" (`start-camera`؛ وكمان بيعمل unlock للصوت؛ وبيبقى "Try Again" بعد خطأ)، "Stop Camera" (`stop-camera`)، واختيار كاميرا لو فيه أكتر من واحدة (`camera-picker`، والاختيار بيتحفظ في الإعدادات). زرار شاشة كاملة (`fullscreen`) بيستخدم Fullscreen API على صندوق الفيديو (`useFullscreen`)؛ والشاشة الكاملة بتتقفل لوحدها لما الكاميرا تقف.
- الصندوق بيحافظ على نسبة عرض لارتفاع بين 1 و 16/9 حسب مقاس الفريم أو المعاينة. و `useNow` بيعيد الرسم كل 500 ms بس عشان يحدّث عدّاد "Retrying in N s".
- عنصر الصندوق عليه `data-testid="camera-stage"` و `data-live="true"` لما يبقى live (اختبارات e2e بتستنى ده).

الـ CSS module (`CameraCard.module.css`، من أكبر ملفات الـ CSS) بيرسم الصندوق، المراية، الشرايح فوق الفيديو، الأزرار، وحالة الـ hero.

مين بيستخدمه: `DojoView`، `LiveView`، و `PracticeSession`.

الملف `frontend/src/konoha/dojo/JutsuListCard.tsx` (ومعاه `JutsuListCard.module.css`)

مسؤول عن إيه: كارت "JUTSU": زرار لكل جوتسو (بترتيب الـ API) فيه `JutsuIcon`، الاسم بحروف أولى كبيرة والاسم الفرعي، وعلى اليمين يا إما عدد الخطوات الحي `2/4` (من `progress`)، يا "Ready"، يا "Training Required" وسببها (من `jutsuAvailability`) بيظهر كـ tooltip وكوصف مخفي.

- الضغط بينادي `onSelect` (يعني `select` من `useDojo`، فالكارت التاني بيعرض الجوتسو ده).
- وهو بيحمّل: صفوف skeleton (مستطيلات رمادي مكان المحتوى).
- لو فيه خطأ: "The jutsu scrolls are sealed" مع تلميح تشغيل السيرفر وزرار Retry.
- `density="compact"` للعمود الجانبي في Live.
- `data-testid="jutsu-list"`.

مين بيستخدمه: `DojoView` و `LiveView`.

الملف `frontend/src/konoha/dojo/sequenceSteps.ts` (ومعاه `sequenceSteps.test.ts`)

مسؤول عن إيه: دوال مساعدة صافية لكارت التسلسل. "صافية" يعني: مدخلات ← مخرجات، من غير React ومن غير شبكة، فسهل نختبرها.

- `stepStates(total, done, live, complete)`: مصفوفة فيها `done` / `current` / `todo` لكل خطوة. الخطوة الجاية بتبقى `current` بس وهو live؛ ولما `complete` كل الخطوات `done`.

```
export function stepStates(total: number, done: number, live: boolean, complete: boolean): StepState[]
{
  const n = complete ? total : Math.max(0, Math.min(done, total))
  return Array.from({ length: total }, (_, i) => {
    if (i < n) return 'done'
    return live && !complete && i === n ? 'current' : 'todo'
  })
}
```

مثال: `stepStates(4, 2, true, false)` ← `['done', 'done', 'current', 'todo']`. ولو `live` = false ← `['done', 'done', 'todo', 'todo']`.

- `focusStep(...)`: أنهي خطوة "بتعمل zoom": الجاية، بس بعد ما الباك إند يأكد إشارة واحدة على الأقل، وهو live ومش complete.
- `shownDone(...)`: الرقم "N / 4".
- `nextCountdownAnchor(prev, timeLeft, now, step, maxExtrapolateS)`: المنطق اللي بيخلي العدّاد ناعم **وفي نفس الوقت** يقدر يرجع يطلع لفوق (مشروح بالتفصيل تحت). ومعاه الثابت `CLOCK_RESTART_S = 1`.
- `signGesture(sign)`: تلميح الحركة الصغير بالـ CSS لكل إشارة: `rise` (طالعة لفوق: tiger, ram, horse)، `press` (ضغط: monkey, boar, dog)، `twist` (تشابك: snake, clone_seal)، `push` (دفع: rasengan_palm). والافتراضي `press`.

`sequenceSteps.test.ts` بيختبر: خطوة جديدة بتبدأ من قيمة الباك إند، الـ jitter عمره ما يخليه يطلع جوه نفس الخطوة، **بيرجع يطلع لفوق لما الباك إند يعيد تشغيل الساعة بعد محاولة غلط**، حالات الخطوات وهو live / مش live / complete، الحصر (clamping)، وإمتى الـ zoom بيشتغل. مثال من الاختبار:

```
it('jumps back up when the backend restarts the clock after a wrong attempt', () => {
  const a = nextCountdownAnchor({ value: 2, at: at(0), step: 2 }, 7, at(0.1), 2, 0.4)
  expect(a).toEqual({ value: 7, at: 100, step: 2 })
})
```

معناه: كان العدّاد ظاهر 2 ثانية في الخطوة 2، وبعد 0.1 ثانية الباك إند بعت 7 (لنفس الخطوة). 7 أكبر من (2 − 0.1) + 1، فده "إعادة تشغيل ساعة" والمرساة الجديدة قيمتها 7.

الملف `frontend/src/konoha/dojo/SequenceCard.tsx` (ومعاه `SequenceCard.module.css`)

مسؤول عن إيه: الكارت الكبير "FIRE STYLE — HAND SIGN SEQUENCE" المستخدم في Home، وفي Live (`variant="side"`)، وفي Practice (`timed={false}`).

الـ Props: `jutsu`، `catalog`، `info`، `done`، `complete`، `live`، `cameraOn`، `status`، `timeLeft`، `loading`، `classNameٍ`، `variant`، `timed`، `missKey`، `onReset`، `warning`، `notice`.

الـ prop	معناها
`jutsu`	الجوتسو المعروض (أو `null` لو الكتالوج ما اتحملش)

الـ prop	معناها
`done`	الخطوات اللي اتعملت بالترتيب (`FrameState.progress` ؛ 0 لو مش live)
`complete`	الباك إند لسه مفعّل الجوتسو ده (`jutsu` event)
`live`	فيه فريمات معالجة من الـ AI على الشاشة
`cameraOn`	الكاميرا شغالة
`status`	حالة الـ socket (للرسايل وهو مش live)
`timeLeft`	الوقت الباقي للإشارة الجاية (من الباك إند)، أو `null`
`variant`	`'default'` أو `'side'` (ترتيب 2×2 مضغوط في العمود الجانبي)
`timed`	يعرض العدّاد ولا لأ (Live: أيوه، Practice: لأ)
`missKey`	بيتغير مع كل محاولة غلط على الخطوة الحالية (حلقة كهرماني)
`onReset`	زرار "Start over"
`warning`	محاولة غلط (والخطوات محفوظة)
`notice`	ليه التسلسل اتمسح بسبب حد الوقت

ملحوظة صغيرة من الكود: لو `jutsu` = `null` الكارت بيترسم بـ `data-testid="sequence-card"` (فيه skeleton أو رسالة "The jutsu could not be loaded from the server.")، ولو فيه جوتسو بيترسم بـ `data-testid="sequence-track"` ومعاه `data-jutsu` و `data-done` و `data-complete`.

بيرسم إيه

- **الهيدر:** `JutsuIcon`، العنوان، "Hand sign sequence —"، نقط تقدم بخط بيتملى، و `N / 4` (`data-testid="step-count"`).
- **لو `timed`:** صف الـ `Countdown` وزرار "Start over" (`sequence-reset`). المكان بتاعه موجود دايمًا بس `hidden` لو مفيش تسلسل شغال، فالترتيب عمره ما يتنطط.
- **أربع مربعات مترقمة (`<ol>`):** رسمة الإشارة (`SignPicture`، وأيقونة إيد لو الصورة فشلت)، سهم بين المربعات، علامة صح لما الخطوة تخلص، حلقة كهرماني (`data-testid="step-miss"`، عليها `key={missKey}` فالأنيميشن بتاعها بيتعاد كل مرة) على المربع الحالي بعد محاولة غلط، و "not trained yet" لو السيرفر مش قادر يتعرف على الإشارة دي. أنيميشن CSS (مع الحركة الكاملة بس) بيخلي الإشارة المنتظرة "تتنفس"، والخطوة المتأكدة "تفرقع" (pop)، ولمعة تعدي على المربعات لما الجوتسو يتفعل.
- **الفوتر، بترتيب الأولوية:** تلميح وهو مش live (شغّلي الكاميرا / الـ AI بيجهز / اسم حالة الاتصال) ← "X activated!" ← رسالة الـ timeout (`notice`) ← `warning` (إشارة غلط، التقدم محفوظ) ← "Training required" مع السبب ← "Next: HORSE — first cue".
- **`SignCredit`:** سطر رخصة الرسومات (Schlum / Wikimedia Commons, CC BY-SA 4.0؛ ورسمة الكف من عمل المشروع نفسه: "Palm pose drawn for this project").

كود أولوية الفوتر:

```
{!live ? (
  <p ... data-testid="recognition-offline">{offlineHint(status, cameraOn)}</p>
) : complete ? (
  <p ... data-testid="sequence-complete"><Sparkles aria-hidden="true" /> {name} activated!</p>
) : notice ? (
  <p ... data-testid="sequence-notice" role="status">...</p>
) : warning && next ? (
  <p ... data-testid="sequence-warning" role="status">...</p>
) : !avail.available ? (
  <p ... data-testid="training-note">...</p>
) : next ? (
  <p ... data-testid="next-hint">Next: <strong>{signName(catalog, next.sign)}</strong>...</p>
) : null}
```

ده سلسلة "لو ... وإلا لو ...": أول شرط صح هو اللي بيظهر. يعني لو الجوتسو اتفعل، رسالة "activated!" بتغطي على أي تحذير قديم.

العدّاد وليه بيقدر يرجع يطلع لفوق

دي من أهم النقط في الفرونت إند، وكتير بيتسأل عنها.

المشكلة: الباك إند بيبعت `time_left` للجوتسو المعروض في كل فريم (حوالي 10 لـ 30 مرة في الثانية). لو عرضنا الرقم زي ما هو، هيتنطط بسبب اختلاف توقيت وصول الفريمات (jitter). وكمان عايزين الرقم ينزل بنعومة بين الفريمات.

الحل: `useSmoothCountdown` بيخزن **مرساة** (anchor) فيها (القيمة، الوقت، رقم الخطوة)، وكل 100 ms بيعرض `anchor.value - elapsed` (القيمة ناقص الوقت اللي عدى)، بس عمره ما يكمّل أكتر من `MAX_EXTRAPOLATE_S = 0.4` ثانية بعد آخر فريم (لأن ساعة الباك إند نفسها بتقف لما الفريمات توقف). ومع كل فريم جديد المرساة بتتحسب تاني بـ `nextCountdownAnchor`:

```
if (!prev || prev.step !== step) return { value: timeLeft, at: now, step } // new step: start fresh
const shown = Math.max(0, prev.value - Math.min(maxExtrapolateS, (now - prev.at) / 1000))
if (timeLeft > shown + CLOCK_RESTART_S) return { value: timeLeft, at: now, step } // clock restarted
return { value: Math.min(timeLeft, shown), at: now, step } // otherwise only go down
```

خطوة بخطوة:

1. **السطر الأول:** لو مفيش مرساة قبل كده، أو رقم الخطوة اتغير (خطوة جديدة اتأكدت): ابدأ من قيمة الباك إند زي ما هي.
2. **السطر التاني:** احسب الرقم اللي ظاهر دلوقتي على الشاشة: القيمة القديمة ناقص الوقت اللي عدى (بحد أقصى 0.4 ثانية)، ومش أقل من صفر.
3. **السطر التالت:** لو قيمة الباك إند **أكبر من المعروض بأكتر من ثانية** (`CLOCK_RESTART_S = 1`)، ده معناه إن الساعة اتعاد تشغيلها: خد قيمة الباك إند.
4. **السطر الرابع:** غير كده، خد الأصغر بين قيمة الباك إند والمعروض. يعني الرقم **بس بينزل**.

يبقى جوه الخطوة الواحدة الرقم بس بينزل (اختلاف توقيت الشبكة ما يخليهوش يرتعش لفوق)، **إلا** لما قيمة الباك إند تبقى أعلى من المعروض بأكتر من ثانية. وده بيحصل في Live بعد محاولة غلط: الباك إند بيدّي نفس الخطوة وقتها كامل تاني (7 ثواني)، والشريط بيتملى من جديد. وخطوة جديدة متأكدة برضه بتبدأ من قيمة الباك إند.

مثال رقمي:

1. خلصتي خطوة 2، الباك إند بعت 7.0 ← المرساة 7.0 (خطوة جديدة).
2. بعد 3 ثواني الباك إند بيبعت 4.0 ← المعروض حوالي 4.0 ← المرساة 4.0.
3. فريم وصل متأخر شوية وفيه 4.2 ← 4.2 مش أكبر من 4.0 + 1، فالمرساة `min(4.2, 4.0)` = 4.0. الرقم ما طلعش.
4. عملتي إشارة غلط (dog بدل horse). الباك إند بيدّي الخطوة 7 ثواني تاني ويبعت 7.0 ← 7.0 أكبر من 4.0 + 1 ← المرساة 7.0. العدّاد **رجع طلع لفوق** والشريط اتملى.

الهوك نفسه في الآخر بيرجّع:

```
return display !== null && display.step === step && timeLeft <= display.value + CLOCK_RESTART_S ?
display.value : timeLeft
```

يعني: لو القيمة المعروضة تخص الخطوة دي ومفيش إعادة تشغيل، اعرض القيمة الناعمة؛ غير كده (قبل أول tick، أو بعد إعادة تشغيل) اعرض قيمة الباك إند مباشرة. ده بيمنع إن الرقم القديم يفضل ظاهر لحظة بعد ما الساعة اتعادت.

الشريط: عرضه `left / maxGap` (`sequence_max_gap`، والافتراضي `DEFAULT_MAX_GAP = 7`)، وبيبقى "urgent" (لونه بيتغير) عند ثانيتين أو أقل. النص بيعرض الرقم بكسر عشري واحد (`left.toFixed(1)`) و "for HORSE" (`data-testid="time-left"`).

إمتى العدّاد بيظهر؟ بس وهو live، ومش complete، وبعد خطوة واحدة على الأقل (`counting`):

```
const counting = live && !complete && n > 0 && timeLeft !== null
```

ملحوظة من الكود: تعليق الـ prop `timeLeft` في `SequenceCard` لسه بيقول "FrameState.time_left"، لكن القيمة الحقيقية اللي `useDojo` بيبعتها جاية من `time_left_per_jutsu` للجوتسو المعروض (ومن `time_left` بس مع السيرفرات القديمة).

الـ CSS module (`SequenceCard.module.css`، أكبر ملف CSS في المشروع، أكتر من 1000 سطر) فيه: شكل المربعات، أنيميشن التنفس والـ pop واللمعة، الحلقة الكهرماني، الـ zoom على الخطوة الحالية، الشريط والحالة urgent، وترتيب `side`.

مين بيستخدمه: `DojoView`، `LiveView`، و `PracticeSession`.

صفحة Live (الوضع الحي)

الملف `frontend/src/konoha/live/LiveView.tsx` (ومعاه `LiveView.module.css`)

مسؤول عن إيه: نفس حالة `useDojo()` اللي في Home، بس ترتيب مختلف: `CameraCard size="hero"` كبير، وجنبه على الكمبيوتر (وتحته على التابلت والموبايل) `JutsuListCard` مضغوط (`density="compact"`) و `SequenceCard` جانبي (`variant="side"`). وكمان فيه إعلان `aria-live` المخفي.

ليه صفحة منفصلة؟ عشان لما تكوني بتعملي الإشارات بجد، الكاميرا هي الأهم ومحتاجة أكبر مساحة، وفي نفس الوقت عايزة تشوفي الكارت والعدّاد من غير ما تنزلي.

إزاي Live بيشتغل (ملخص): الوضع الحر (`mode: 'free'`): كل الجوتسو الأربعة بتتحسب مع بعض في الباك إند، ولكل جوتسو ساعته. بين كل إشارتين فيه حد أقصى `sequence_max_gap` (7 ثواني افتراضي). إشارة غلط **ما بتمسحش** التقدم، بس بتدّي الخطوة الحالية وقتها كامل تاني (والعدّاد في الكارت بيطلع لفوق). لو عدّى الوقت من غير إشارة صح، الباك إند بيبعت `sequence_failed` بسبب `timeout` والتسلسل بيتمسح والكارت بيعرض "Time ran out — sequence reset.". وزرار "Start over" بيمسح فورًا.

الـ CSS module: بيعمل التقسيم: عمود عريض للكاميرا وعمود ضيق جانبي على الشاشات الكبيرة، وعمود واحد على الأصغر.

مين بيستخدمه: `pages/LiveDemoPage.tsx`.

الملفات `frontend/src/pages/LiveDemoPage.tsx` و `frontend/src/pages/HomePage.tsx`

كومبوننتس routes صغيرة جدًا:

```
export function HomePage() {
  return <DojoView heading="SHINOBI AI: form the hand signs, unleash the jutsu" documentTitle={null}
/>
}

export function LiveDemoPage() {
  return <LiveView heading="Live: form the hand signs of any jutsu" documentTitle="Live" />
}
```

`documentTitle={null}` معناها عنوان التاب يبقى `NARUTO` بس، و `"Live"` معناها "Live · NARUTO". الملفين دول موجودين في الفولدر القديم `pages/` بس هم **مستخدمين ومتوصلين** بالـ routes.

صفحة الجوتسو (المكتبة Library)

الملفات `frontend/src/konoha/pages/JutsuPage.tsx` و `frontend/src/konoha/pages/PracticePage.tsx`

غلافات routes من سطر واحد: `KonohaJutsuPage` بيرسم `<JutsuView />`، و `KonohaPracticePage` بيرسم `<PracticeView />`. ليه؟ عشان `App.tsx` يبقى نضيف وكل صفحة ليها "باب" واضح.

الملف `frontend/src/konoha/jutsu/JutsuView.tsx` (ومعاه `JutsuView.module.css`)

مسؤول عن إيه: صفحة "Jutsu Scrolls": عنوان `BrushTag`، و `Carousel` فيه `JutsuCard` لكل جوتسو، وتحته `JutsuDetail` للجوتسو المختار.

- `/library/<id>` بيختار الكارت ده من الأول؛ id مش معروف بيعرض رسالة قصيرة.
- اختيار كارت بيحدّث العنوان بـ `replaceUrl` (من غير تنقل حقيقي، فالـ shell ما يطلعش لفوق الصفحة).
- وهو بيحمّل: skeletons. لو خطأ: رسالة "sealed scrolls" وزرار Retry.
- الـ CSS module ده **بيتعاد استخدامه كمان في صفحة Practice** (`PracticeView` بيعمله import).

الملف `frontend/src/konoha/jutsu/pageState.ts`

دوال مساعدة مشتركة بين صفحتي Jutsu و Practice:

- `replaceUrl(path)`: بيستخدم `history.replaceState` (بيغير العنوان في شريط المتصفح من غير ما يضيف صفحة للـ history ومن غير ما الراوتر يعمل تنقل).
- `useSelectedJutsu(param)`: الاختيار بيبدأ من العنوان وبيتابع التنقلات الحقيقية (زي زرار Back).
- `selectedIndex(list, selected, fallback)`: ترتيب العنصر المختار في القايمة، أو الاحتياطي.

الملف `frontend/src/konoha/jutsu/sequenceKind.ts`

`sequenceKind(official)` بيرجّع الاسم والمعنى: "Official seals — The original hand seals, in their original order." أو "Custom sequence — Designed for SHINOBI AI: the source has fewer than four seals." يعني بعض الجوتسو تسلسلها أصلي من الأنمي، وبعضها المشروع صممه لأن المصدر فيه أقل من أربع أختام.

الملف `frontend/src/konoha/jutsu/JutsuCard.tsx` ومعاه (`JutsuCard.module.css`)

مسؤول عن إيه: كارت واحد في الـ carousel: حرف كانجي كبير زخرفي، الترتيب `01` … `04`، شارة Official/Custom، `JutsuIcon` متحرك، الاسم (العنوان جواه زرار ممدود على الكارت كله، و `aria-pressed` = مختار)، الاسم الفرعي والكانجي، الوصف (بيستخبى لو `compact`)، معنى الشارة، وفوتر فيه "Ready" / "Training Required" ومعاه يا (في `extra`) "Practice: "Training now" يا "4 hand signs".

ليه الزرار جوه العنوان وممدود؟ عشان الكارت كله يبقى قابل للضغط، وفي نفس الوقت قارئ الشاشة يقرا زرار واحد بالاسم بس.

مين بيستخدمه: `JutsuView` و `PracticeView`.

الملف `frontend/src/konoha/jutsu/JutsuDetail.tsx` ومعاه (`JutsuDetail.module.css`)

ورقة "Hand Signs" تحت الـ carousel (`data-testid="jutsu-detail"`): مربعات إشارات الجوتسو المختار (`SignSteps`)، مصدر التسلسل (جاي منين)، التوافر على السيرفر ده، وزرار "Practice" (`testid="practice-button"`) بيودّي لـ `/practice/<id>`. متحرك بـ framer-motion لما الاختيار يتغير. `BrushButton`، `data-`

الملف `frontend/src/konoha/jutsu/SignSteps.tsx` ومعاه (`SignSteps.module.css`)

الملف ده بيصدّر تلات كومبوننتس:

- `SignArt`: رسمة الإشارة الخطية (`<img>`)، أو أيقونة إيد لو فشلت تتحمل. مستخدم كمان في `PracticeSession`.
- `SignSteps`: مربعات بيضا مترقمة فيها الرسمة، الاسم، الاسم الياباني، والتعليمات (ممكن تتحدد بعدد معين بـ `cues`). وهج خفيف بيمشي على المربعات من 1 لـ 4 (`data-testid="sign-steps"`؛ بيقف مع reduced/off).

- `ArtCredit`: سطر رخصة الرسومات (`data-testid="art-credit"`).

الـ Carousel

الملف `frontend/src/konoha/carousel/Carousel.tsx` (ومعاه `Carousel.module.css`)

مسؤول عن إيه: carousel أفقي عام (`Carousel<T>`، يعني بيشتغل مع أي نوع عناصر)، مستخدم في صفحتي Jutsu و Practice.

الأب هو اللي ماسك الاختيار (`active`، `onActiveChange`)؛ ده اسمه "controlled component". باقي الـ props: `items`، `getKey`، `getLabel`، `renderItem(item, {index, active})`، `label`، `noun`، `testId`.

بيشتغل إزاي:

- `measure()` بيقرا `offsetLeft` لكل شريحة من الـ DOM (العروض جاية من CSS container queries) وبيحسب إزاحة الشريط اللي تجيب كل شريحة للمكان الأول (الشمال)، محصورة عشان آخر كارت يخلص عند الحافة اليمين. و `ResizeObserver` بيعيد القياس لما المقاس يتغير.
- الـ `x` بتاع الشريط قيمة framer-motion متحركة بـ spring (زمبلك): `stiffness 280, damping 34` (و `mass` في الكود). من غير حركة كاملة بيقفز فورًا.

```
const SPRING = { type: 'spring', stiffness: 280, damping: 34, mass: 0.9 } as const
```

- **المدخلات:** أسهم شمال/يمين، نقط، كيبورد (←، →، Home، End؛ التركيز بينتقل لزرار الكارت نفسه من غير scroll)، سحب بالماوس، وسحب باللمس.
- `onDragEnd` بيتوقع مكان الإفلات بالسرعة، بيختار أقرب نقطة توقف، وبيخلي "النفضة" القصيرة (أكتر من 36 px أو 380 px/s) تحرك كارت واحد على الأقل:

```
if (best === index && (Math.abs(info.offset.x) > 36 || Math.abs(info.velocity.x) > 380)) {
```

يعني: لو الحساب قال "خليكي على نفس الكارت"، بس إنتي سحبتي أكتر من 36 بكسل أو بسرعة أكتر من 380 بكسل في الثانية، يتحرك كارت.

- ضغطة بتنهي سحب ما بتختارش الكارت اللي تحت الماوس.
- لو Tab ركّز على كارت بره الشاشة، الإطار المقصوص بيفضل من غير scroll والشريط هو اللي بيتزحلق.

صفحة Practice (التدريب)

الملف `frontend/src/konoha/practice/PracticeView.tsx`

مسؤول عن إيه: صفحة `/practice` و `/practice/:jutsuId`. بتعيد استخدام `Carousel` و `JutsuCard` بتوع صفحة Jutsu (`compact`)، والفوتر "Training now" على الكارت النشط) والـ CSS module بتاعها، وبعدين بترسم `<PracticeSession>` للجوتسو المختار.

- من غير جوتسو في العنوان بتبدأ بأول جوتسو متاح.
- اختيار كارت تاني بيحدّث العنوان (`replaceUrl`) وبيبدّل جوتسو التدريب في الجلسة **والكاميرا لسه شغالة**. ليه الكاميرا ما بتقفش؟ لأن `<PracticeSession>` بيترسم من غير `key`، فـ React بيحتفظ بنفس النسخة (ونفس الكاميرا) وبس الـ props بتتغير، فالـ `mode` بيتغير والهوك بيبعت رسالة mode جديدة.
- لو الكتالوج فشل، بتعيد المحاولة لوحدها كل 5 ثواني (`setTimeout(reload, 5000)`).

الملف `frontend/src/konoha/practice/PracticeSession.tsx` (ومعاه `PracticeSession.module.css`)

مسؤول عن إيه: جلسة تدريب جوتسو واحد. بتعمل كاميرا وجلسة **خاصين بيها** في الوضع `{mode:'practice', jutsu}` (فالباك إند بيعدّ الجوتسو ده بس)، ومعاهم `useSessionSounds` و `useLiveEvents`.

```
const camera = useCamera()
const mode: Mode = { mode: 'practice', jutsu: jutsu.id }
const session = useShinobiSession({ stream: camera.stream, mode, landmarks: settings.showLandmarks })
const sounds = useSessionSounds(session)
const events = useLiveEvents(session, catalog)
```

الحالة اللي بتمسكها

بتشترك في الفريمات (بس الفريمات اللي الوضع بتاعها هو التدريب ده) وبتحتفظ بـ:

- `completedKey`: بيتحط لما ييجي event `jutsu` للجوتسو ده.
- `failure`: آخر `wrong_sign`، أو `sequence_failed` من سيرفرات قديمة.
- `missKey`: بيزيد مع كل محاولة غلط (الحلقة الكهرماني).

```
subscribe((frame) => {
  if (frame.mode.mode !== 'practice' || frame.mode.jutsu !== practiceId) return
  let failedNow = false
  for (const ev of frame.events) {
    if (ev.type === 'jutsu' && ev.jutsu === practiceId) {
      setCompletedKey(++counter.current)
      setFailure(null)
      setMissKey(0)
    } else if (ev.type === 'wrong_sign' && ev.jutsu === practiceId) {
      failedNow = true
      setCompletedKey(null)
      setFailure(ev)
      setMissKey((k) => k + 1)
    } else if (ev.type === 'sequence_failed' && ev.jutsu === practiceId) {
      failedNow = true
      setCompletedKey(null)
      setFailure(ev)
      setMissKey(0)
    } else if (ev.type === 'step' && ev.jutsu === practiceId) {
      setCompletedKey(null)
      setMissKey(0)
      if (!failedNow) setFailure(null)
    }
  }
}),
```

خطوة بخطوة: أول سطر بيتجاهل أي فريم من وضع تاني (مثلًا فريم متأخر قبل ما السيرفر يغير الوضع). بعدين لكل event: `jutsu` ← مكتمل؛ `wrong_sign` ← تحذير وحلقة؛ `sequence_failed` (سيرفرات قديمة بس، لأن الحالي عمره ما بيعمل timeout أو reset في التدريب) ← فشل؛ `step` ← يمسح الفشل، **إلا** لو الإشارة الغلط جت في نفس الفريم (`failedNow`) عشان رسالتها ما تضيعش.

- تغيير الجوتسو أو فقدان الاتصال بيمسح الحالة المحلية دي (نمط الـ derived state بمفتاح `scopeKey = practiceId:connected`).
- `done` جاي **بس** من `frame.progress[jutsu]` (وبيبقى كل الخطوات لما يكتمل).

الترتيب على الشاشة

- `CameraCard`.
- كارت "كوتش" (`ParchmentCard`، `data-testid="coach"`) عنوانه "Step N of 4" (`practice-step`) أو "Complete!" (`practice-complete`)، فيه الإشارة الحالية كبيرة (`SignArt`، الرقم، الاسم، الاسم الياباني، التعليمات، ولينك للكارت المرجعي للخطوة)، سطر "AI sees" (`practice-sees`): "Right sign: hold it" أو "Not this step's sign")، رسالة الفشل (`practice-failure`): كهرماني "not recognized" لـ `wrong_sign`، وتنبيه أحمر للـ resets القديمة)، ملاحظة "Training Required" لو السيرفر مش قادر يتعرف على كل الإشارات، وأزرار "Practise again" / "Start over" (`retry`): بتنادي `session.reset()` وبتوقف صوت الشخصية) و "Back to Jutsu" (`back-to-jutsu`).
- تحت كل ده `SequenceCard` بـ `timed={false}`: التدريب **ملهوش حد زمني** وإشارة غلط عمرها ما تعمل reset.
- framer-motion بيحرك تغيير الخطوة (slide)، الاكتمال (scale)، وهزة صغيرة مع الفشل.

الفرق بين Live و Practice باختصار:

الحاجة	Live / Home	Practice
الوضع المبعوت	`free` (كل الجوتسو)	`practice` + جوتسو واحد
حد الوقت	أيوه (`sequence_max_gap`، 7 ث)	لأ
إشارة غلط	تحذير، التقدم محفوظ، الساعة بتتعاد	تحذير، التقدم محفوظ
العدّاد في الكارت	ظاهر (`timed`)	مش ظاهر (`timed={false}`)
الكاميرا والجلسة	من `useDojo`	خاصين بـ `PracticeSession`

صفحة Settings (الإعدادات)

الملف `frontend/src/konoha/pages/SettingsPage.tsx`

مسؤول عن إيه: صفحة الإعدادات. بتستخدم الـ CSS بتاع `konoha/settings/SettingsPage.module.css`. الأقسام، كل واحد `ParchmentCard`:

- `CameraSection`: `useCamera()` خاص بيها ومعاينة محلية معمولة مراية (**من غير AI**)، اختيار كاميرا (الافتراضية بتاعة المتصفح، الكاميرات المكتشفة، و "Saved camera (not detected yet)" لو الـ id المحفوظ مش ظاهر)،

أزرار تشغيل/إيقاف، نصايح للتصوير (framing)، ورسالة لكل حالة كاميرا.

- `SoundSection`: مفاتيح "Enable sounds" و "Mute"، sliders لـ Master volume و Sound effects و Character voices (ومعاها تلميح من `useVoiceManifest`: كام من الـ 8 أماكن للأصوات متركب؛ مفيش ولا واحد مضمّن مع المشروع)، وزرار "Test sound" اللي بيعمل unlock وبيشغّل `ui.hand_seal`. الـ 8 أماكن = 4 جوتسو × 2 جزء (`activation` و `name`):

```
const voiceSlots = VOICE_JUTSU.length * VOICE_PARTS.length
```

- `MusicSection`: موسيقى الخلفية تشغيل/إيقاف (`useMusic().toggle`) وسطر حالة، slider "Music volume"، ومفتاح "Jutsu music" (موسيقى وإنتي بتعملي إشارات جوتسو).
- `ExperienceSection`: اختيار Animations (full / reduced / off)، مع ملحوظة لو نظام التشغيل فارض reduced motion) و "Show hand landmarks".
- `ResetSection`: تأكيد على خطوتين ("Reset settings"، وبعدين "Yes, reset" أو "Cancel") بينادي `reset()` وبيرجّع الثيم Naruto.

`KonohaSettingsPage` بيحط قسم الكاميرا فوق والباقي في عمودين.

مين بيستخدمه: route `/settings` في `App.tsx`.

الملفات `frontend/src/konoha/settings/controls.tsx` (ومعاه `controls.module.css`) و `frontend/src/konoha/settings/SettingsPage.module.css`

عناصر فورم بالشكل الفاتح:

- `SettingRow`: أيقونة، اسم، وصف، وعنصر التحكم.
- `SwitchRow`: زرار `role="switch"` ومعاه `aria-checked` (مفتاح on/off مفهوم لقارئ الشاشة).
- `SliderRow`: input من نوع range من 0 لـ 1 بيتعرض كنسبة مئوية.
- `Segmented<T>`: اختيار على شكل مجموعة radio.
- `SelectField`: قايمة منسدلة.

و `SettingsPage.module.css` فيه ترتيب الصفحة (الشبكة، العمودين، الملاحظات، صندوق التأكيد).

موسيقى الخلفية

في المشروع نوعين موسيقى مختلفين، ومهم ما تتلخبطيش بينهم:

- **موسيقى الخلفية** (ambient): لوب هادي بيشتغل وإنتي بتتصفحي الموقع. ملفاتها في `src/konoha/music/`، ودي **ليها رسم صوتي (Web Audio graph) صغير منفصل خاص بيها**.
- **موسيقى الجوتسو**: موسيقى خاصة بكل جوتسو بتشتغل وإنتي بتعملي إشاراته. ملفاتها في `src/audio/` (`jutsuMusic.ts` و `jutsuMusicPlayer.ts`) وبتعدّي من خلال محرك الصوت الرئيسي.

الملف `frontend/src/konoha/music/player.ts`

مسؤول عن إيه: `MusicPlayer`: لوب الخلفية الهادي (`MUSIC_SRC = /audio/music/leaf_trail_loop.flac`، لوب أصلي) على Web Audio graph صغير خاص بيه.

- الملف بيتفك (decode) مرة واحدة وبيشتغل بـ `AudioBufferSourceNode` في وضع loop (من غير أي فجوة، عكس `<audio loop>` اللي ساعات بيعمل سكتة صغيرة عند الرجوع للأول).
- بيعدّي على gain للـ fades (ظهور تدريجي 2.2 ثانية `FADE_IN_S`، اختفاء 0.8 ثانية `FADE_OUT_S`) و gain تاني للـ duck (التوطية).
- `preload()`: بينزّل الملف بدري.
- `play(level)`: بيبدأ أو بيتبع مستوى جديد (محتاج تفاعل من المستخدم الأول).
- `pause()`: بيختفي تدريجيًا وبيحتفظ بالمكان.
- `setDuck(level, seconds)`: بيوطّيه.
- `subscribe` / `isPlaying`: عشان `useSyncExternalStore`.
- `dispose()`: بيقفل كل حاجة.

ثوابت تانية في الملف: `FOLLOW_S = 0.25` (سرعة متابعة تغيير المستوى)، `DUCK_DOWN_S = 0.05` و `DUCK_UP_S = 0.35`.

الملف `frontend/src/konoha/music/MusicContext.tsx`

مسؤول عن إيه: `MusicProvider` و `useMusic()`.

إمتى الموسيقى بتشتغل؟ بس لما تكون كل الشروط دي صح:

```
const level = Math.min(1, Math.max(0, settings.musicVolume * settings.masterVolume))
const allowed = settings.musicOn && settings.audioEnabled && !settings.muted && level > 0 && !hidden
const want = allowed && interacted
```

يعني: مطلوبة (`musicOn`)، الأصوات مفعّلة، مش مكتوم، المستوى أكبر من صفر، التاب ظاهر، **و** المستخدم اتفاعل مع الصفحة (قاعدة المتصفح).

- لو الموسيقى كانت شغالة آخر مرة، بتبدأ مع أول ضغطة أو زرار في الزيارة (جوه الـ gesture نفسه، وده Safari بيطلبه)، ما عدا الضغطات على أزرار الموسيقى نفسها (اللي عليها `MUSIC_CONTROL_ATTR`). وزرار Escape و Tab والضغطات المتكررة ما بتتحسبش.
- **بتتابع محرك الصوت:**

```
const follow = () => {
  if (engine.music.isPlaying()) player.setDuck(0, JUTSU_MUSIC_DUCK_S)
  else player.setDuck(engine.isSpeaking() ? VOICE_DUCK_MUSIC : 1, engine.isSpeaking() ? undefined :
JUTSU_MUSIC_BACK_S)
}
```

معناه: وموسيقى الجوتسو شغالة، موسيقى الخلفية بتختفي للصمت (في 0.35 ثانية)؛ وصوت شخصية بيتكلم ← بتوطى لـ `VOICE_DUCK_MUSIC` (0.3)؛ وبعدين بترجع (في 0.8 ثانية).

- **الواجهة (API):** `on` ، `playing` ، `available` ، `waiting` ، `volume` ، `toggle` ، `setOn` ، `setVolume` .

مين بيستخدمه: `MusicControl` في الهيدر و `MusicSection` في Settings.

محرك الصوت (المؤثرات، أصوات الشخصيات، موسيقى الجوتسو)

القاعدة الذهبية: كل أصوات الجوتسو بتتبع **events الباك إند بس**. الفرونت إند عمره ما بيشغّل صوت على "تخمين".

التصميم: فصل بين حاجتين:

- **مخططات صافية (pure planners):** المدخل `FrameState` ، والمخرج قايمة أوامر. من غير Web Audio ومن غير React، فسهل جدًا نختبرها. دول `eventCues.ts` و `voiceCues.ts` و `jutsuMusic.ts` .
- **مشغلات (players):** بتنفّذ الأوامر على Web Audio. دول `engine.ts` و `jutsuMusicPlayer.ts` .

ليه الفصل ده؟ عشان نقدر نكتب اختبار يقول "لو جه event `step` للخطوة 3، لازم يطلع أمر play لـ `ui.hand_seal` بـ rate كذا" من غير ما نحتاج سماعات أو متصفح.

الملف `frontend/src/audio/cues.ts`

مسؤول عن إيه: `CUES` : قايمة ملفات المؤثرات الصوتية (الباك إند بيقدمها تحت `/audio/...`) ومعاها علامة `loop` ، والنوع `CueId` (اسم أي صوت من دول).

المجموعات: `ui.*` (`hand_seal` ، `step_success` ، `sequence_failed` ، `mode_switch`) ومجموعة لكل جوتسو: `fire_style.*` ، `chidori.*` ، `shadow_clone.*` ، `rasengan.*` (لمسات الإشارات، الشحن، أصوات التفعيل، الاصطدام، واللوبات زي `fire_style.burning_loop` و `chidori.crackle_loop` و `rasengan.energy_loop`).

مثال من الكود:

```
'ui.hand_seal': { url: '/audio/ui/hand_seal.wav', loop: false },
'chidori.crackle_loop': { url: '/audio/chidori/crackle_loop.wav', loop: true },
```

العدد الفعلي في الكود: **32 صوت** (4 للـ ui، و 8 لـ rasengan، و 6 لـ chidori، و 7 لـ fire_style، و 7 لـ shadow_clone)، منهم 4 لوبات (`rasengan.spin_loop` ، `rasengan.energy_loop` ، `chidori.crackle_loop` ، `fire_style.burning_loop`).

ملحوظة: فرق بين الكود والتعليقات. تعليق أول `engine.ts` بيقول "decodes the 19 cues of cues.ts"، وتعليق أول `gains.ts` بيقول "The 31 files". الاتنين قدام؛ القايمة الحالية فيها 32 cue، و `CUE_GAIN` فيه 32 قيمة. (فيه اتنين معلّمين في `gains.ts` إنهم "no longer used": `shadow_clone.seal_charge` و `rasengan.spin_loop` ، بس لسه موجودين في القايمة وبيتحمّلوا.)

الملف `frontend/src/audio/gains.ts`

مسؤول عن إيه: موازنة مستوى الأصوات والـ volumes والـ ducking.

- `CUE_GAIN`: gain ثابت لكل cue، لأن الملفات مختلفة جدًا في علوّ الصوت (من -38 لـ -9 dBFS). كل cue بيتجاب لمستوى دوره: ختم الإيد حوالي -14 dBFS (مسموع بوضوح)، اللمسات -20، الـ 24- UI، الشحن -21، التفعيل -13 (أعلى لحظة)، اللوبات -23، النهايات -17. والتعليقات فيها المستويات اللي اتقاست. (dBFS = وحدة قياس الصوت الرقمي؛ 0 هو الأعلى الممكن، وكل ما الرقم يبقى سالب أكتر يبقى أوطى.)
- `outputVolume`، `musicOutputVolume`، `voiceOutputVolume`: الرئيسي × (المؤثرات أو الموسيقى أو الصوت)، صفر لو مكتوم، ومحصور بين 0 و 1:

```
export function outputVolume(settings: { masterVolume: number; sfxVolume: number; muted: boolean }):
number {
  if (settings.muted) return 0
  const v = settings.masterVolume * settings.sfxVolume
  return Number.isFinite(v) ? Math.min(1, Math.max(0, v)) : 0
}
```

مثال بالقيم الافتراضية: 0.8 × 0.9 = 0.72.

- ثوابت الـ ducking (توطية الأصوات التانية وشخصية بتتكلم): المؤثرات بتنزل لـ `VOICE_DUCK_SFX = 0.4` (حوالي -8 dB) والموسيقى لـ `VOICE_DUCK_MUSIC = 0.3` (حوالي -10 dB)، بنزول في 0.02 ثانية (`DUCK_ATTACK_S`) ورجوع في 0.18 ثانية (`DUCK_RELEASE_S`).

الملف `frontend/src/audio/engine.ts`

مسؤول عن إيه: `SoundEngine`، واحد بس للتطبيق كله (بيتعمل في `SoundContext`).

الـ Buses (المسارات الصوتية)

"Bus" يعني مسار بيتجمع فيه كذا صوت عشان نتحكم فيهم مع بعض (زي مكسر الصوت في الاستوديو). الرسم:

```
effect one-shot/loop -> voice gain (cue gain x volume x fades)
   -> output gain (user sfx volume) -> duck (under a voice) -> limiter -> speakers
character voice clip -> clip gain -> voice gain (user voice volume) -> limiter
jutsu music (JutsuMusicPlayer) -> music gain (user music volume) -> music duck -> limiter
```

اقريه كده:

1. **مسار المؤثرات:** كل صوت (one-shot أو loop) ليه gain خاص بيه (= `CUE_GAIN` × الـ volume المطلوب × الـ fades)، بعدين بيدخل `output` (مستوى المؤثرات بتاع المستخدم)، بعدين `duck` (بيوطى وشخصية بتتكلم)، بعدين الـ limiter، بعدين السماعات.
2. **مسار أصوات الشخصيات:** المقطع ← gain المقطع (للـ fades) ← `voiceOut` (مستوى الأصوات بتاع المستخدم) ← الـ limiter. ملاحظة إنه **ما بيعديش** على `duck`، لأنه هو اللي بيعمل الـ duck للباقي.
3. **مسار موسيقى الجوتسو:** `JutsuMusicPlayer` ← `musicOut` (مستوى الموسيقى) ← `musicDuck` ← الـ limiter.

الـ Limiter: `DynamicsCompressor` مستخدم كـ "سقف": threshold -6 dB، knee 6، ratio 12، attack 0.003، release 0.2. لو كذا صوت اشتغلوا مع بعض وبقوا عاليين أوي، بيضغطهم عشان ما يحصلش تشويه (clipping).

الكود اللي بيوصّل المسارات في `unlock()`:

```
output.connect(duck)
duck.connect(limiter)
voiceOut.connect(limiter)
musicOut.connect(musicDuck)
musicDuck.connect(limiter)
limiter.connect(ctx.destination)
```

الدوال

- `unlock()`: بيعمل أو يكمّل الـ `AudioContext` (لازم ييجي بعد تفاعل من المستخدم)، بيبني الرسم اللي فوق، وبعدين بيفك كل الـ cues (`load`) ومقاطع الأصوات المتركبة (`loadVoices`، من `/api/voices`). آمن تناديه كتير.
- `play(cue, options)`: one-shot مع خيارات: `delayMs`، `fadeInMs`، `fadeOutAfterMs` / `fadeOutMs`، `rate` (الطبقة/السرعة) و `filter`. **صوت لسه ما اتفكش بيتخطى** (صوت متأخر مش هيطابق الصورة). أقصى 32 صوت في نفس الوقت (`MAX_VOICES = 32`).
- `startLoop(cue)` (loop واحد لكل cue)، `stop(cue, fadeMs)`، `stopAll(fadeMs)`.
- `run(commands)`: بينفذ أوامر `SoundCommand` اللي طالعة من `eventCues.ts`.
- `playVoice(plan)` / `stopVoice()`: بيشغّل مقاطع صوت الشخصية في أوقاتها المخططة (بيبدأ بعد 0.03 ثانية `SPEECH_START_S` عشان يلحق يوطّي المؤثرات الأول)، بيعمل duck للمؤثرات والموسيقى لحد آخر الخطة، وبيبلّغ مستمعين `subscribeSpeech` (`isSpeaking()`).
- `setVolume`، `setVoiceVolume`، `setMusicVolume`، `setEnabled`، `voiceDuration(id)`، `musicBus()`، `isReady()` / `subscribe` (لـ `useSyncExternalStore`)، `dispose()`.
- `music`: الـ `JutsuMusicPlayer` المتوصل بالـ music bus.

مين بيستخدمه: `SoundContext` بيعمله، و `useSessionSounds` و `MusicContext` و صفحة Settings بيستخدموه.

الملف `frontend/src/audio/eventCues.ts` (ومعاه `eventCues.test.ts`)

مسؤول عن إيه: المخطط الصافي للمؤثرات الصوتية: `soundsForFrame(state, frame, nowMs)` بيرجّع `CueState` جديد وقايمة `SoundCommand` (`play`، `loop`، `stop`).

event من الباك إند	الصوت
`step`	ختم الإيد `ui.hand_seal`، أعلى بنص تون (semitone) مع كل خطوة (`SEAL_RISE`)، ومعاه لمسة الجوتسو الخاصة تحته (`HAND_SIGN_ACCENT`، volume 0.55)؛ مرة واحدة في الفريم (الجوتسو الأكتر تقدمًا هو اللي بيدّي اللمسة)، وعمره ما يتكرر لنفس الخطوة
`wrong_sign`	ولا حاجة (الحركة الغلط صامتة)
`sequence_failed` (timeout)	`ui.sequence_failed` (هادي)
`jutsu` ومعاه `stages`	صوت ختم الإشارة الرابعة؛ والباقي بيتبع المراحل
`effect_phase` charge	`CHARGE[jutsu]` (مثلًا Fire: شهيق، وبعدين شحن طالع بـ fade-in)
`effect_phase` release	الشحن بيختفي (250 ms)؛ و `RELEASE[effect]` (الأصوات الأساسية، الاصطدام، اللوب)

event من الباك إند	الصوت
`jutsu` من غير stages (سيرفر قديم)	`RELEASE` فورًا
`clones`	`shadow_clone.smoke_poof` للنسخ الجديدة، أقصى 3 في الـ event، وبينهم 110 ms على الأقل
`effect_stopping`	`ENDING[effect]` (اصطدامات، اختفاء، النسخ بتختفي)
`effect_end`	لوب المؤثر ده بيقف

صوت الخطوة بالتفصيل:

```
export const SEAL_RISE = [0, 1, 2, 3, 4]
export const ACCENT_VOLUME = 0.55

export function stepSound(ev: Pick<StepEvent, 'jutsu' | 'step'>): PlayCommand {
  const rise = SEAL_RISE[Math.min(SEAL_RISE.length - 1, Math.max(0, ev.step - 1))]
  return { kind: 'play', cue: 'ui.hand_seal', volume: SEAL_VOLUME, rate: 2 ** (rise / 12) }
}
```

بيعمل إيه: الخطوة 1 ← rise 0 ← rate = 1 (زي ما هو). الخطوة 2 ← rise 1 ← rate = 2^(1/12) ≈ 1.059 (أعلى بنص تون). الخطوة 4 ← rate ≈ 1.189. ليه؟ عشان الإحساس إن التسلسل "بيبني" لحد التفعيل. والمعادلة `2 ** (rise / 12)` هي معادلة الموسيقى: كل 12 نص تون = ضعف التردد.

والأوقات المهمة: `CHARGE_DELAY_MS = 150` (الطبقة التانية للشحن بتبدأ بعد 150 ms)، `CHARGE_FADE_IN_MS = 400`، `CHARGE_RELEASE_FADE_MS = 250`، `POOF_GAP_MS = 110`، `MAX_POOFS_PER_EVENT = 3`، وأي poof متأخر أكتر من 700 ms بيترمي (`MAX_POOF_DELAY_MS`).

`CueState` بيفتكر: التفعيل الشغال `activation` (اتشحن / اتطلق)، `lastStep` (عشان ما يحصلش صوت مكرر)، عدد النسخ، وأول وقت مسموح فيه بالـ poof الجاي.

`eventCues.test.ts` بيختبر قواعد صوت الإشارات، الخط الزمني الكامل للتفعيل (تأكيد، شحن، اسم، إطلاق)، الإيقاف أثناء التحضير، وإن كل cue ليه gain.

الملف `frontend/src/audio/voiceCues.ts` (ومعاه `voiceCues.test.ts`)

مسؤول عن إيه: المخطط الصافي لأصوات الشخصيات (Naruto أو Sasuke، شوفي `voices.ts`).

- **بس** event `jutsu` من الباك إند هو اللي بيخلي الشخصية تتكلم: عمر الخطوة أو الإشارة الغلط أو الـ timeout ما بيعملوا كده.
- مع الخط الزمني للتفعيل: مقطع `activation` بيشتغل فورًا، ومقطع `name` (اسم الجوتسو) بيستنى `effect_phase` name.
- من غير خط زمني (سيرفر قديم): الاتنين ورا بعض بفاصل 80 ms (`NAME_GAP_MS`).
- لو المؤثر خلص قبل مرحلة الاسم، مفيش مقطع اسم.
- تغيير الوضع بيوقف الصوت (`VOICE_STOP_FADE_MS = 250`).
- نفس التفعيل مرتين (نفس `seq`، أو نفس الجوتسو في أقل من ثانية `MIN_REPEAT_MS = 1000`) بيتجاهل.
- تفعيل جديد بيقطع صوت القديم بسرعة (`VOICE_REPLACE_FADE_MS = 120`).

- المقاطع الناقصة بتتسجل في `missing` و**عمرها ما بتتبدل بصوت تاني**.
- `voicePlan()` بيرتب المقاطع على الزمن.

الاختبارات بتغطي كل القواعد دي وكمان فك الـ voice manifest.

الملف `frontend/src/audio/voices.ts`

ids مقاطع الأصوات (`<jutsu>.activation` ، `<jutsu>.name`)، المتكلم لكل جوتسو:

```
export const VOICE_SPEAKER: Record<JutsuId, 'Naruto' | 'Sasuke'> = {
  fire_style: 'Naruto',
  chidori: 'Sasuke',
  shadow_clone: 'Naruto',
  rasengan: 'Naruto',
}
```

وكمان `parseVoiceManifest` (بيقبل بس عناوين تحت `/audio/voice/<jutsu>/`، عشان الأمان) و `fetchVoiceManifest` (`GET /api/voices`؛ سيرفر قديم أو مفيش نت = مفيش أصوات).

الملف `frontend/src/audio/useVoiceManifest.ts`

hook بيحمّل الـ voice manifest مرة واحدة، لصفحة Settings (`null` وهو بيحمّل). الصفحة بتستخدمه عشان تقول "X of 8 voice clips installed" أو إن مفيش ولا واحد.

الملف `frontend/src/audio/jutsuMusic.ts` (ومعاه `jutsuMusic.test.ts`)

مسؤول عن إيه: المخطط الصافي للموسيقى اللي بتشتغل وإنتي بتعملي إشارات جوتسو. لوب مرخّص لكل جوتسو في `TRACKS`، كل واحد بـ gain يوازن مستواه، و**مش** موسيقى أنمي Naruto الأصلية:

```
export const TRACKS: Record<JutsuId, JutsuTrack> = {
  rasengan: { url: '/audio/jutsu_music/rasengan.wav', gain: 0.87 }, // "Blackmoor Ninjas" (Matthew
Pablo, CC BY 3.0)
  fire_style: { url: '/audio/jutsu_music/fire_style.wav', gain: 1.2 }, // "Taiko drums" (jobro /
congusbongus, CC BY 3.0)
  chidori: { url: '/audio/jutsu_music/chidori.wav', gain: 0.93 }, // "Revenge" (TAD, CC BY 4.0)
  shadow_clone: { url: '/audio/jutsu_music/shadow_clone.wav', gain: 1.35 }, // "Samurai" (TAD, CC BY
4.0)
}
```

`musicForFrame(state, frame)` بيرجّع أوامر `MusicCommand` (`start` ، `duck` ، `pulse` ، `fadeOut`):

- الجوتسو **المتقدم** (أكتر خطوات، 1 على الأقل، من `leader()`؛ ولو تعادل، اللي شغال بيفضل) بيبدأ التراك بتاعه بـ fade-in 1.5 ثانية (`FADE_IN_MS`)؛ وجوتسو تاني ياخد المقدمة بخطوات **أكتر** بيعمل crossfade (0.8 ثانية، `CROSSFADE_MS`).
- كل خطوة: `pulse` (وطية قصيرة) عشان صوت الختم يفضل واضح.
- duck: `jutsu` event لـ `DUCK_ACTIVATION = 0.25` (25%)، في `DUCK_RAMP_MS` (300 ms) تحت التحضير؛ و `effect_phase` release: fade out في 1.5 ثانية (`RELEASE_FADE_MS`).

- التقدم رجع 0 من غير `jutsu` (timeout، Start over، وضع تاني): fade out في ثانية (`RESET_FADE_MS`). **إشارة غلط بتخلي الموسيقى مكملة.**
- مفيش تراك جديد بيبدأ وفيه تفعيل شغال.
- والكاميرا وقفت / الاتصال وقع / سبتي الصفحة: `STOP_FADE_MS = 600`.

`jutsuMusic.test.ts` بيختبر المخطط، والمشغل (بـ AudioContext مزيف)، وإن التلات volumes بتوع المحرك مستقلين عن بعض.

الملف `frontend/src/audio/jutsuMusicPlayer.ts`

مسؤول عن إيه: `JutsuMusicPlayer` بينفّذ أوامر الموسيقى على context المحرك:

```
loop source -> track gain (fades, track level) -> activation duck -> step dip -> music bus
```

- تراك واحد في المرة: الجديد بيعمل crossfade من القديم أول ما يتفك؛ والرجوع لتراك لسه بيختفي بيرجّع نفس النسخة (عمره ما يبقى فيه نسختين).
- آخر تراكين بس بيفضلوا مفكوكين في الذاكرة (`KEEP_DECODED = 2`، لأنهم كبار).
- `prefetch()` بينزّل الملفات مع أول فريم live؛ وتراك اتطلب قبل ما الصوت يتسمح بيبدأ أول ما يتسمح.
- `setEnabled` بيتبع إعداد "Jutsu music"؛ وكمان `stop(fadeMs)` و `dispose()`.
- الوطية تحت الختم: لـ 0.5 (`DIP_LEVEL`) في 0.015 ثانية، وبترجع بعد 0.12 ثانية في 0.12 ثانية.

الملف `frontend/src/audio/useSessionSounds.ts`

مسؤول عن إيه: الـ hook اللي بيربط جلسة بالصوت. ده "الكوبري" بين الـ events والسماعات.

```
const unsubscribe = subscribe((frame) => {
  const now = performance.now()
  lastFrameAt.current = now
  if (!prefetched.current) {
    prefetched.current = true
    engine.music.prefetch()
  }
  // the voice first: it ducks the sound effects before this frame's activation cues start
  const spoken = voiceForFrame(voice.current, frame, engine.voiceDuration, now)
  voice.current = spoken.state
  for (const c of spoken.commands) {
    if (c.kind === 'play') engine.playVoice(c.plan)
    else engine.stopVoice(c.fadeMs)
  }
  ...
  const result = soundsForFrame(state.current, frame, now)
  state.current = result.state
  if (result.commands.length > 0) engine.run(result.commands)
  const planned = musicForFrame(music.current, frame)
  music.current = planned.state
  if (planned.commands.length > 0) engine.music.run(planned.commands)
})
```

خطوة بخطوة لكل فريم:

1. يسجّل وقت آخر فريم (للـ watchdog).
2. أول فريم: يبدأ تنزيل تراكات الموسيقى (`prefetch`).
3. **الصوت الأول** (`voiceForFrame`): عشان صوت الشخصية يوطّي المؤثرات قبل ما أصوات التفعيل بتاعة نفس الفريم تبدأ.
4. المؤثرات (`soundsForFrame` ← `engine.run`).
5. الموسيقى (`musicForFrame` ← `engine.music.run`).

وكمان:

- في التطوير بيكتب في الـ console مرة واحدة لكل مقطع صوت ناقص.
- **Watchdog** (حارس): كل 500 ms بيبص؛ لو مفيش فريم وصل من ثانيتين (`FRAME_GAP_STOP_MS = 2000`، يعني الكاميرا وقفت) بيوقف موسيقى الجوتسو.
- لما الاتصال يقع أو الكومبوننت يختفي، بيرجّع كل حالات المخططات للأول وبيوقف كل حاجة: **مفيش صوت من غير الباك إند**.
- بيرجّع `stopVoice()` (بيستخدمه زرار Start over في Practice).

الملف `frontend/src/audio/voicePlayback.test.ts`

بيختبر `SoundEngine` بـ Web Audio context مزيف: أصوات الشخصيات (الجدولة، الاستبدال، الإيقاف، توطية المؤثرات)، صوت ختم الإيد والشحن، وموسيقى الخلفية وهي بتوطى تحت صوت.

رحلة الإشارة: من الإيد للكارت والصوت (خطوة بخطوة)

القسم ده بيجمع كل اللي فات في قصة واحدة. المثال: إنتي في صفحة `/live`، وعملتي الإشارة التالتة `horse` من Fire Style (monkey ← boar ← horse ← tiger)، وكنتي عاملة أول اتنين صح.

```
[your hand: HORSE]
      |
      v
useCamera  --MediaStream-->  useShinobiSession.pump()
      (hidden <video> -> canvas, NOT mirrored -> JPEG, quality from hello)
      |
      v   binary: [4-byte len][{"t": 1234.5}][JPEG]
WebSocket /ws  ----------------------------------------->  backend
                                                (recognise HORSE, hold, latch,
                                                 Fire Style progress 2 -> 3,
                                                 restart the 7 s clock, draw frame)
      binary: [4-byte len][FrameState JSON][processed JPEG]
      |   FrameState.events = [{type:'sign',...}, {type:'step', jutsu:'fire_style', step:3, total:4,
sign:'horse'}]
      v
ShinobiSocket.onmessage -> decodeBinaryMessage -> parseFrameState
      |
      v
useShinobiSession.onFrame
      |-- inflight.answer(t), latency smoothing
      |-- listeners (synchronous, before React re-renders):
      |      |-- useSessionSounds: voiceForFrame (nothing) -> soundsForFrame -> engine.run
      |      |        -> ui.hand_seal at rate 2^(2/12) + fire_style.hand_sign x 0.55
      |      |        -> musicForFrame -> pulse (short dip under the seal)
      |      |-- useLiveEvents: (step: announcement only)
      |      |-- useDojo subscriber: step for focused jutsu -> clear warning, missKey 0, notice
      |-- setView({frame,...})  -> React re-render
      |      |-- useDojo: progress key changed -> followFocus -> focus stays fire_style
      |      |-- done = 3, timeLeft = time_left_per_jutsu.fire_style (about 7)
      |      `-- SequenceCard: tile 3 "pop" + check, tile 4 becomes current,
      |          step-count "3 / 4", countdown anchor reset (new step) -> bar full
      `-- draw(jpeg) -> createImageBitmap -> canvas on screen
```

ودلوقتي بالكلام، مرقّم:

1. **الكاميرا:** `useCamera` فاتح الويب كام ومدّي `MediaStream` لـ `useShinobiSession`.
2. **الإرسال:** `pump()` بياخد الصورة الحالية من الـ `<video>` المخفي، بيصغرها بـ `fitWithin`، بيرسمها من غير مراية، بيضغطها JPEG، وبيبعتها بـ `sendFrame({ t }, bytes)`، وبيسجلها في `inflight`.
3. **الباك إند:** بيتعرف على `horse`، المثبّت بيتأكد إنها ثابتة، تقدم Fire Style بيبقى 3، ساعة الخطوة بتتعاد (7 ثواني)، والصورة بتترسم وتتضغط.
4. **الرد:** رسالة binary فيها `FrameState` (ومعاه `events` فيها `step` بـ `jutsu: 'fire_style'` و `step: 3`) وصورة JPEG.
5. **الفك:** `ShinobiSocket` بيفك الرسالة (`decodeBinaryMessage`) وبيتحقق منها (`parseFrameState`)، اللي بيرمي أي event بايظ) وبينادي `onFrame`.
6. **`onFrame` في `useShinobiSession`:** بيقفل الفريم في `inflight`، بيحدّث التأخير، وبينادي كل المستمعين **بشكل متزامن**.
7. **الصوت (`useSessionSounds`):** `voiceForFrame` ما بيرجعش حاجة (مفيش `jutsu`). `soundsForFrame` بيشوف `step` رقم 3 فبيرجّع `play` لـ `ui.hand_seal` بـ rate = 2^(2/12) ≈ 1.12 و `play` لـ `fire_style.hand_sign` بـ volume 0.55. `engine.run` بينفذهم: كل صوت بيعدّي على gain بتاعه ← limiter ← duck ← output ← السماعات. و `musicForFrame` بيرجّع `pulse` (وطية قصيرة في موسيقى الجوتسو تحت الختم؛ والتراك كان بدأ من الخطوة الأولى بـ fade-in).

8. **ملاحظات الكارت (مستمع `useDojo`):** `step` للجوتسو المعروض ← يمسح `warning` و `missKey` و `notice`.
9. **إعلان قارئ الشاشة (`useLiveEvents`):** بيحدّث نص الـ `announcement`.
10. **تحديث React ← `setView` ← `followFocus` ← `useDojo` بيعيد الرسم. **React:** بيلاقي مفتاح التقدم اتغير الكارت فاضل على `fire_style`. `done` = 3، و `timeLeft` من `time_left_per_jutsu.fire_style`.
11. **`SequenceCard`:** المربع 3 بيعمل "pop" وعليه علامة صح، المربع 4 بيبقى `current` وبيتنفس ويعمل zoom، العدد "3 / 4"، والعدّاد: رقم الخطوة اتغير فـ `nextCountdownAnchor` بيبدأ من جديد من قيمة الباك إند ← الشريط مليان، والفوتر بيقول "Next: TIGER — ...".
12. **الصورة:** `draw(jpeg)` ← `createImageBitmap` ← `drawImage` على الـ canvas، فبتشوفي نفسك والمؤثرات اللي الباك إند رسمها.

ولو كملتي وعملتي `tiger` (الإشارة الرابعة)، الباك إند بيبعت event `jutsu` ومعاه `stages`، فبيحصل كمان:

1. `voiceForFrame` بيشغّل مقطع `fire_style.activation` (لو متركب) والمؤثرات والموسيقى بتوطى تحته.
2. `soundsForFrame` بيشغّل صوت ختم الإشارة الرابعة.
3. `musicForFrame` بيعمل duck لـ 25%.
4. `useLiveEvents` بيظهر طبقة التفعيل (`jutsu-overlay`) باسم الجوتسو والكانجي.
5. `useDojo` بيحط `success` لمدة 4 ثواني، فالكارت بيعرض "Fire Style activated!" واللمعة بتعدي.
6. بعد كده events `effect_phase` (charge ← صوت الشحن، name ← مقطع الاسم، release ← أصوات النار واللوب، والموسيقى بتختفي في 1.5 ثانية، والطبقة بتتحول لبانر صغير "active").
7. وفي الآخر `effect_stopping` (أصوات النهاية) و `effect_end` (اللوب بيقف والبانر بيختفي).

ولو بدل `horse` عملتي `dog` (غلط): الباك إند بيبعت `wrong_sign` (expected horse, got dog)، **مفيش صوت**، `useDojo` بيحط `warning` ويزوّد `missKey` (الحلقة الكهرماني على المربع 3)، `useLiveEvents` بيظهر toast لطيف، التقدم بيفضل 2، الموسيقى مكملة، والباك إند بيدّي الخطوة 7 ثواني تاني، فالعدّاد في الكارت **بيطلع لفوق** (لأن القيمة الجديدة أكبر من المعروض بأكتر من ثانية).

كومبوننتس الطبقة الحية (`src/components/live`)

الملف `frontend/src/components/live/useLiveEvents.ts`

مسؤول عن إيه: بيحوّل events الباك إند لحالة الطبقة (overlay) اللي `StageOverlay` بيرسمها.

بيرجّع: `activation` (`{key, jutsu, effect, phase}`) والمرحلة `reveal` الأول، بعدين `active`، بعدين `ending`)، `toasts`، `dismissToast`، و `announcement` (نص لقارئات الشاشة).

- الطبقة بتظهر **بس** مع event `jutsu`. الكشف الكبير (reveal) بيطوّل قد مدة التحضير المخططة (`voice + charge + name) × 1000 + 800 ms`، أو `REVEAL_MS = 2200` ms مع السيرفرات القديمة، وبيمشي بدري عند `effect_phase` release عشان عمره ما يغطي المؤثر.

```
const build = ev.stages ? (ev.stages.voice + ev.stages.charge + ev.stages.name) * 1000 +
REVEAL_SLACK_MS : REVEAL_MS
```

مثال: لو `stages` = voice 1.0 و charge 1.2 و name 0.8، يبقى الكشف (1.0 + 1.2 + 0.8) × 1000 + 800 = 3800 ms كحد أقصى، إلا لو الـ release جه قبلها.

- إيقاف أو نهاية بتوصل أثناء الكشف بتتفتكر (`pending`) وبتتطبق بعده. ليه؟ لأن Chidori و Fire بيخلصوا فورًا لما الإيد تخرج من الصورة، وده ساعات بيحصل في نفس الفريم بتاع `jutsu`، فمن غير كده الطبقة كانت هتختفي قبل ما تشوفيها.
- بعد الكشف بتبقى بانر صغير "active"، بعدين `ending`، وبتختفي مع `effect_end`.
- الـ Toasts (أقصى 3، كل واحد 3.4 ثانية): ملاحظة لطيفة مع `wrong_sign`، ورسالة reset مع `sequence_failed`.
- لما الاتصال يقع، التفعيل والـ toasts بيتمسحوا.

مين بيستخدمه: `useDojo` و `PracticeSession` (والنسخة القديمة من Practice).

الملف `frontend/src/components/live/labels.ts` (ومعاه `labels.test.ts`)

مسؤول عن إيه: نصوص واشتقاقات صغيرة بتنسّق بس اللي الباك إند بعته (مفيش أي قرار تعرّف هنا):

- `signName`، `signInfo`: اسم الإشارة وبياناتها من الكتالوج.
- `SOURCE_LABEL`: اسم مصدر الإشارة (أنهي موديل).
- `failureTitle` / `failureDetail`: نصوص `sequence_failed`.
- `wrongSignTitle` / `wrongSignDetail`: "HORSE not recognized — that looked like DOG"؛ و "Your 2 completed steps are kept. Try HORSE again (step 3 of 4)." (ولو خطوة واحدة: "Your completed step is kept").
- `titleCase`.
- `followFocus` و `leadingJutsu` (شرحناهم في `useDojo`).
- `timeoutNotice`: "Time ran out — sequence reset.".
- `jutsuAvailability`: الجوتسو متاح ولا لأ، وليه (موديل الـ landmarks مش متدرب، YOLOX مش متحمل...).
- `connectionText`: الاسم، النبرة (tone)، والتفاصيل مع "Retrying in N s".
- `formatMs`، `formatFps`.

`labels.test.ts` بيغطي نصوص الإشارة الغلط، قواعد التعادل في `followFocus`، ورسالة الـ timeout.

الملف `frontend/src/components/live/StageOverlay.tsx` (ومعاه `StageOverlay.module.css`)

مسؤول عن إيه: بيترسم فوق الفيديو الحي (`CameraCard`) بيستخدمه):

- `ActivationOverlay`: الكشف بشريط ضوء قريب من فوق، `JutsuIcon`، اسم الجوتسو والكانجي (`data-testid="jutsu-overlay"`)؛ وبعدين البانر الصغير (`jutsu-banner`) اللي بيقول "active" أو "fading".
- `FailureToasts` (`failure-toast`).
- لـ Shadow Clone: رسالة (`crowded-notice`) لما `frame.effect.crowded` = true:

```
const crowded = frame?.effect?.name === 'shadow_clone' && frame.effect.crowded === true
...
<span>Not enough room for all 10 clones — step back or turn your device to landscape.</span>
```

يعني الباك إند بيرسم دايمًا الـ 10 نسخ، بس لو المكان ضيق وهيتداخلوا بيقولك ترجعي لورا أو تلفي الجهاز بالعرض.

كومبوننتس Live القديمة (موجودة؛ مستخدمة بس في الصفحة غير المتوصلة (`pages/PracticePage.tsx`)

الحالة لكل الملفات دي: قديمة. الكود شغال وبيتبني، بس الصفحة الوحيدة اللي بتستخدمها (`pages/PracticePage.tsx`) مش متوصلة بأي route، فالمستخدم عمره ما بيشوفها.

- `frontend/src/components/live/CameraStage.tsx` + `CameraStage.module.css`: مسرح الكاميرا القديم (فريمات معالجة جوه "إطار تشاكرا"، معاينة محلية وهو مش live، تشغيل/إيقاف، اختيار كاميرا، landmarks).
- `frontend/src/components/live/RecognitionPanel.tsx` + `RecognitionPanel.module.css`: الـ AI شايف إيه (الإشارة المدموجة بمصدرها ودرجة الثقة، تقدم الـ hold، الإشارات المتأكدة، الإيدين، أرقام الاتصال).
- `frontend/src/components/live/SequenceTrack.tsx` + `SequenceTrack.module.css`: شريط الأربع خطوات القديم بعدّاد.
- `frontend/src/components/live/ConnectionPill.tsx` + `ConnectionPill.module.css`: كبسولة حالة (مستخدمة في CameraStage و RecognitionPanel، يعني قديمة بالتبعية).
- `frontend/src/components/live/AvailabilityNotice.tsx` + `AvailabilityNotice.module.css`: أنهي جوتسو السيرفر ده يقدر يفعّله وليه لأ.
- `frontend/src/components/live/JutsuProgressList.tsx` + `JutsuProgressList.module.css`: قايمة قديمة للجوتسو الأربعة بتقدمهم؛ **مش مستوردة في أي مكان دلوقتي** (ميتة تمامًا).

كومبوننتس الفن (`src/components/art`)

الملف `frontend/src/components/art/JutsuIcon.tsx` ومعاه (`JutsuIcon.module.css`)

مسؤول عن إيه: الأيقونة الوحيدة للجوتسو المستخدمة في كل مكان (قايمة الجوتسو، هيدر التسلسل، الـ carousels، طبقة التفعيل).

الـ Props: `jutsu`، `size` (افتراضيًا بتملا الأب `'100%'`)، `animated`، `className`.

```
export function JutsuIcon({ jutsu, size = '100%', animated = true, className }: JutsuIconProps) {
  const visual = jutsuVisual(jutsu)
  const style = { '--ring': visual.ring, '--ring-hot': visual.ringHot, width: size, height: size } as
CSSProperties
  return (
    <span className={...} style={style} data-jutsu-icon={jutsu} aria-hidden="true">
      <span className={styles.disc}>
        {jutsu === 'shadow_clone' ? (
          <ShadowCloneIcon animated={animated} />
        ) : (
          <JutsuMotif kind={visual.motif} color={visual.color} color2={visual.color2} size="84%"
animated={animated} />
        )}
      </span>
      <svg className={styles.ring} viewBox="0 0 100 100" focusable="false">
        ...
```

بيعمل إيه: شباك دايري جواه حلقة طاقة SVG (توهج، شريط، علامات بتلف في اتجاه وأقواس بتلف في الاتجاه التاني) لونها من متغيرات CSS `--ring` / `--ring-hot` اللي جاية من `jutsuVisual()`. جوه: `ShadowCloneIcon` لـ Shadow Clone، وغير كده `JutsuMotif` (لهب، برق، أو كرة). الأنيميشن كلها CSS وبتحترم إعداد الحركة.

الملف `frontend/src/components/art/ShadowCloneIcon.tsx` (ومعاه `ShadowCloneIcon.module.css`)

مسؤول عن إيه: أيقونة Shadow Clone المتحركة. الرسمة المعتمدة متقطعة طبقات:

- `assets/konoha/shadow-clone/bg.webp`: المشهد والنسخ ممسوحين منه.
- `clone-left.webp` و `clone-right.webp`: كل نسخة لوحدها.
- `front.webp`: Naruto والسحب اللي قدامهم.

وفوقهم هالة برتقالي (`aura`)، نفختين دخان (خمس دوايز كل واحدة)، وشرارات.

الأنيميشن: CSS بيحرك بس `transform` و `opacity` (رخاص على المتصفح لأنهم ما بيعيدوش حساب الترتيب). النسخ بتطلع من ورا Naruto في دخان، تثبت شوية، وبعدين تختفي. اللوب كله 6 ثواني (`clone-left 6s`، `clone-right 6s`، `aura 6s`، `puff-left 6s`، `puff-right 6s`)، والشرارات 3 ثواني بتأخيرات مختلفة. والحالة الأساسية من غير أنيميشن هي الرسمة الثابتة والنسختين ظاهرين، فلو قفلتي الأنيميشن من Settings بتشوفي صورة كاملة.

```
useLayoutEffect(() => {
  const el = ref.current
  if (!animated || !el || typeof el.getAnimations !== 'function') return
  for (const animation of el.getAnimations({ subtree: true })) animation.startTime = 0
}, [animated])
```

بيعمل إيه: `useLayoutEffect` زي `useEffect` بس بيشتغل قبل ما المتصفح يرسم. بيجيب كل أنيميشن CSS شغالة جوه الأيقونة وبيحط `startTime = 0`. النتيجة: كل نسخة من الأيقونة على الصفحة (في القايمة وفي الكارت وفي الـ carousel) بتعرض **نفس اللحظة** من اللوب، مهما اتركبت إمتى. من غير كده كانوا هيبقوا مش متزامنين وشكلهم عشوائي.

ملحوظة مهمة: الأيقونة رسمة فيها نسختين بس (شمال ويمين) لأنها أيقونة صغيرة. أما المؤثر الحقيقي في الكاميرا (اللي الباك إند بيرسمه) فبيعمل **10 نسخ بالظبط** (11 شخص مع المستخدم)، وده رقم نهائي، وعشان كده رسالة الزحمة بتقول "all 10 clones".

الملف `frontend/src/components/art/JutsuMotif.tsx` (ومعاه `JutsuMotif.module.css`)

رسومات SVG صغيرة أصلية (`MotifKind`: `fire`، `lightning`، `sphere`، `smoke`) بالـ props `kind`، `color`، `color2`، `size`، `animated`. دي اللي جوه أيقونات Fire Style و Chidori و Rasengan.

الملف `frontend/src/components/art/jutsuVisuals.ts`

`JUTSU_VISUALS`: لكل جوتسو اللون (متغير CSS)، لون تاني، الـ motif، كانجي زخرفي واحد (火، 雷، 影، 螺) وألوان الحلقة. و `jutsuVisual(id)` بيرجّعه، ومعاه احتياطي لأي id مش معروف. الألوان بتيجي من متغيرات `--jutsu-fire`، `--jutsu-chidori`... اللي في `styles/tokens.css`.

ملفات فن تانية (مستخدمة بس في AppShell القديم، أو مش مستخدمة خالص)

- `frontend/src/components/art/ChakraBackground.tsx` + `ChakraBackground.module.css`: الخلفية الغامقة المتحركة القديمة (تدرجات، ضباب، تظليل حواف vignette، حبيبات grain). **الحالة: قديمة**؛ مستخدمة بس في `AppShell` اللي هو نفسه مش متوصل.
- `frontend/src/components/art/ParticleField.tsx`: canvas فيه جمر (Naruto) أو ذرات كهربا مع برق خفيف (Sasuke)؛ بيقف لما يتخفى أو يخرج من الشاشة. **الحالة: قديمة** (مستخدم في `ChakraBackground` بس).
- `frontend/src/components/art/particles.ts` (+ `particles.test.ts`): محاكاة الجزيئات الصافية (`particleCount`، `spawnParticle`، `stepParticle`، `particleOpacity`، `lightningBolt`، `seededRandom`)؛ والاختبارات بتفحص الأعداد، الظهور، الجمر الطالع، العمر، الاختفاء، والبرق المحدد (deterministic: نفس البذرة = نفس البرق). **الحالة: قديمة** (مستخدمة في `ParticleField` و `HeroArt`)، بس اختبارها لسه بيشتغل مع `npm test`.
- `frontend/src/components/art/HeroArt.tsx` + `HeroArt.module.css`: رسمة قديمة للصفحة الرئيسية (دايرة ختم تشاكرا)؛ **مش مستوردة دلوقتي** (ميتة).
- `frontend/src/components/art/ShinobiLogo.tsx` + `ShinobiLogo.module.css`: علامة "SHINOBI AI" القديمة (أربع شفرات حوالين مركز). **قديمة** (AppShell و MobileMenu بس).
- `frontend/src/components/art/ThemeShift.tsx` + `ThemeShift.module.css`: موجة ضوء لما الثيم يتغير. **قديمة** (AppShell بس).

الـ Layout القديم و UI kit (`src/components/layout` و `src/components/ui`)

الملف `frontend/src/components/layout/useDocumentTitle.ts`

مستخدم في كل صفحة (ده مش قديم): بيحط عنوان التاب `"<title> · NARUTO"`، أو `NARUTO` بس. مستخدم في `DojoView`، `LiveView`، `JutsuView`، `PracticeView`، 404، About، Settings، والصفحات القديمة.

الملفات `frontend/src/components/layout/AppShell.tsx` و `MobileMenu.tsx` و `ServerStatus.tsx` و `ThemeToggle.tsx` و `nav.ts` (ومعاهم ملفات الـ `.module.css` بتاعتهم)

الإطار الغامق القديم للتطبيق:

- `AppShell.tsx` + `AppShell.module.css`: خلفية متحركة، شريط علوي، درج موبايل، فوتر.
- `MobileMenu.tsx` + `MobileMenu.module.css`: درج فيه كانجي لكل عنصر.
- `ServerStatus.tsx` + `ServerStatus.module.css`: مؤشر إن السيرفر متاح.
- `ThemeToggle.tsx` + `ThemeToggle.module.css`: مفتاح Naruto / Sasuke.
- `nav.ts`: `NAV_ITEMS` بالأسماء والأيقونات والكانجي.

الحالة: غير نشطة. `AppShell` مش مستخدم في الراوتر (اتبدل بـ `KonohaShell`)، فكل الملفات دي حاليًا مش شغالة. ملاحظة: بما إن `ThemeToggle` مش ظاهر، مفيش حاليًا زرار في الواجهة الجديدة يبدّل Naruto/Sasuke مباشرة؛ الثيم المحفوظ بيتطبق، و Reset بيرجّعه Naruto.

الفولدر `frontend/src/components/ui/` (الملف `index.ts` وكل كومبوننت مع الـ CSS module بتاعه)

UI kit غامق صغير، بيتعاد تصديره من `index.ts`. لسه مستخدم في صفحتي About و 404 والصفحات القديمة غير المتوصلة.

الملف	الغرض
`Button.tsx` (+ `Button.module.css`)	زرار أو لينك راوتر، أنواع ومقاسات، أيقونة اختيارية
`Panel.tsx` (+ `Panel.module.css`)	كارت غامق بعنوان وأيقونة وأزرار وتوهج اختياري
`Badge.tsx` (+ `Badge.module.css`)	لافتة ملونة صغيرة (tone، أيقونة، نقطة)
`Slider.tsx`، `Switch.tsx`، `SegmentedControl.tsx`، `Select.tsx` (+ ملفات CSS بتاعتهم)	عناصر فورم
`SectionHeading.tsx` (+ `SectionHeading.module.css`)	عنوان فيه سطر صغير فوق، عنوان، ونص
`Skeleton.tsx` (+ `Skeleton.module.css`)	مكان رمادي وقت التحميل
`StatusMessage.tsx` (+ `StatusMessage.module.css`)	صناديق `StatusMessage` و `ErrorState` و `EmptyState`
`Lightbox.tsx` (+ `Lightbox.module.css`)	عارض صور شاشة كاملة بسابق / تالي
`SignArt.tsx` (+ `SignArt.module.css`)	رسمة إشارة ملونة (الشكل القديم)
`SequenceBadge.tsx`	شارة "Official" / "Custom"
`Reveal.tsx`	أنيميشن ظهور وطلوع مع الـ scroll
`ArtCredit.tsx` (+ `ArtCredit.module.css`)	سطر رخصة الرسومات
`serverError.ts`	`describeServerError` (بيحوّل "answered 502" لجملة مفهومة) و `START_SERVER_HINT`

الملف	الغرض
`index.ts`	بيعيد تصدير كل اللي فوق عشان `import { Button } from '../components/ui'`

استخدامها الفعلي: `AboutPage.tsx` و `NotFoundPage.tsx` (المتوصلين) و `pages/LibraryPage.tsx` و `pages/SettingsPage.tsx` و `pages/PracticePage.tsx` (القديمة). خلي بالك إن `SignArt` هنا غير `SignArt` اللي في `konoha/jutsu/SignSteps.tsx` (نفس الاسم، شكلين مختلفين).

الصفحات القديمة (`src/pages`)

- `frontend/src/pages/AboutPage.tsx` + `AboutPage.module.css` (متوصلة على `/about`): SHINOBI AI إيه، التعرّف بيشتغل إزاي، التقنيات، والـ credits اللي بتتحمل من `GET /api/credits` (حالة تحميل أو خطأ أو تمام)، وقسم إخلاء المسؤولية بيظهر لوحده. **الحالة: نشطة** (بالتصميم القديم جوه `LegacyFrame`).
- `frontend/src/pages/NotFoundPage.tsx` + `NotFoundPage.module.css` (متوصلة كـ `*`): "This path leads nowhere"، العنوان المجهول، وأزرار للرئيسية وللمكتبة. **الحالة: نشطة.**
- `frontend/src/pages/PracticePage.tsx` + `PracticePage.module.css`، و `frontend/src/pages/LibraryPage.tsx` + `LibraryPage.module.css`، و `frontend/src/pages/SettingsPage.tsx` + `SettingsPage.module.css`: النسخ الغامقة السابقة من Practice و Library و Settings. بتتبع نفس القواعد (التقدم جاي من الباك إند) بس **مش متوصلة بأي route**. **الحالة: قديمة.**
- `HomePage.tsx` و `LiveDemoPage.tsx`: اتشرحوا فوق (بيرسموا عروض Konoha). **نشطين.**

الـ Styles والـ Assets

- `frontend/src/styles/tokens.css`: design tokens بتاعة الشكل الغامق القديم (خطوط، مقياس خط مرن، ألوان لكل ثيم عن طريق `[data-theme]`، وألوان الجوتسو `--jutsu-fire`، `--jutsu-chidori`... اللي `jutsuVisuals.ts` لسه بيستخدمها). يعني الملف ده **لسه مهم** رغم إنه من التصميم القديم.
- `frontend/src/styles/global.css`: بيعمل import لـ `tokens.css`، resets (`box-sizing`، الهوامش)، styles أساسية للصفحة، والـ class `.visually-hidden` (نص لقارئات الشاشة بس، مستخدم في كل التطبيق).
- `frontend/src/assets/konoha/background.jpg`: لوحة القرية. و `frontend/src/assets/konoha/shadow-clone/*.webp` (`bg.webp`، `clone-left.webp`، `clone-right.webp`، `front.webp`): طبقات أيقونة Shadow Clone. Vite بيضمّهم وبيدّيهم عناوين فيها hash (فالمتصفح يعرف إمتى الصورة اتغيرت).
- الـ CSS modules (`*.module.css`): كل كومبوننت بيعمل `import styles from './X.module.css'`؛ و Vite بيغير أسماء الـ classes عشان عمرها ما تتخانق بين الكومبوننتس.

اختبارات الوحدة (Unit tests): إزاي تشغليها

`npm test` (Vitest) بيشغّل كل ملفات `src/**/*.test.ts` في Node:

ملف الاختبار	بيختبر إيه
`services/socket/framing.test.ts`	الشكل الـ binary والتحقق من الرسايل
`services/socket/client.test.ts`	الاتصال، الـ busy، الـ backoff، الـ hello، الـ ping
`components/live/labels.test.ts`	النصوص و `followFocus` و timeout
`konoha/dojo/sequenceSteps.test.ts`	حالات الخطوات والعدّاد اللي بيطلع لفوق
`components/art/particles.test.ts`	محاكاة الجزيئات (قديمة بس متختبرة)
`audio/eventCues.test.ts`	قواعد المؤثرات والخط الزمني
`audio/voiceCues.test.ts`	قواعد أصوات الشخصيات والـ manifest
`audio/jutsuMusic.test.ts`	موسيقى الجوتسو والمشغل والـ volumes
`audio/voicePlayback.test.ts`	المحرك مع Web Audio مزيف

سريعين لأنهم بيختبروا منطق صافي أو بيستخدموا حاجات مزيفة (WebSocket مزيف، timers مزيفة، AudioContext مزيف). الأمر من جوه فولدر `frontend/`:

```
npm test
```

اختبارات End-to-End في المتصفح (`frontend/e2e`)

دي اختبارات **حقيقية**: Microsoft Edge حقيقي (متحكم فيه بـ `playwright-core`)، والباك إند الحقيقي، والـ AI الحقيقي. الكاميرا المزيفة بتاعة Edge بتشغّل فيديو متبني من تسجيلات الإشارات بتاعة المستخدمة نفسها، فمفيش أي حاجة متزيفة في التعرّف: الباك إند لازم يتعرف على كل إشارة لوحده. كل سكريبت بيطبع سطور `PASS` / `FAIL` وملخص JSON، وبيخرج بكود 1 لو أي حاجة فشلت.

ليه اختبارات زي دي مهمة؟ اختبارات الوحدة بتقول "المنطق صح"، لكن الـ e2e بتقول "السيستم كله بيشتغل مع بعض": الكاميرا ← events ← AI ← WebSocket ← الكارت والصوت.

الملف `frontend/e2e/make_fixtures.py`

مسؤول عن إيه: بيبني فيديوهات الكاميرا المزيفة. بيتشغّل بـ Python بتاع المشروع:

```
.venv/Scripts/python.exe frontend/e2e/make_fixtures.py --out <folder outside the repo>
.venv/Scripts/python.exe frontend/e2e/make_fixtures.py --out <folder> --only fire_style chidori
```

- الملفات فيها وش المستخدمة، فالسكريبت **بيرفض** يكتب جوه الريبو ("refusing to write fixtures (they show the user's face) inside the repository").
- `FIXTURES` بيعرّف كل fixture كـ أجزاء `(sign or "idle", seconds)`. أمثلة من الكود:

الـ fixture	الأجزاء	الغرض
`fire_style`	idle 1.0، monkey 2.4، boar 2.4، horse 2.4، tiger 2.4، idle 3.0	Fire Style كامل

الـ fixture	الأجزاء	الغرض
`chidori`	idle 1.0، snake، dog، tiger، monkey (كل 2.4 واحدة)، idle 3.0	Chidori كامل
`wrong_order`	horse، idle بدل idle، monkey، boar، **dog** 4.5	إشارة غلط
`idle`	idle 6.0	مفيش إيدين
`practice_retry`	monkey، boar، dog، snake، dog، horse، tiger	3 محاولات غلط في Practice
`practice_wait`	monkey، boar، idle **32.0**، horse، tiger	وقفة 32 ثانية
`live_timeout`	monkey، boar، idle **8.5**، horse، tiger	وقفة أطول من حد الـ 7 ثواني
`live_retry`	monkey، boar، dog 1.6، snake 1.6، horse، tiger	محاولتين غلط ثم الصح
`live_tie`	clone_seal، monkey، idle 7.5	تعادل (Rasengan خطوة 1 و Fire Style خطوة 1)
`live_repeat`	monkey، boar، idle 1.2، boar، idle 6.0	تكرار آخر إشارة
`live_wrong_keep`	monkey، boar، dog، horse، tiger	غلط ثم يكمل
`live_retry_slow`	monkey، boar، dog، idle 2.0، ram، idle 2.0، horse، tiger	محاولات غلط أطول من 7 ثواني إجمالًا بس عمرها ما وصلت 7 ثواني من غير إشارة

- لكل إشارة بيقرا أول تسجيل `me_night_*.mp4` في `data/videos/<sign>/`. لو كاشف الأختام YOLOX قدر يتحمّل، `best_window` بيختار الجزء من التسجيل اللي YOLOX بيتعرف فيه على الإشارة دي في أكبر عدد فريمات (بدرجة `SEAL_MIN_SCORE` على الأقل)، عشان الاختبار يفحص الـ pipeline مش كليب وحش. من غير YOLOX بياخد نص التسجيل.
- الفريمات **بتتشال منها المراية** (`cv2.flip`) لأن التسجيلات اتحفظت معمولة مراية والسيرفر بيعمل مراية تاني؛ بتتصغر لـ 1280×720، بتتحول لـ I420، وبتتكتب كملف `.y4m` (header `YUV4MPEG2`) بمعدل فريمات التسجيلات). Edge بيشغّلها في لوب ككاميرا.

الملف `frontend/e2e/lib.mjs`

مسؤول عن إيه: دوال مساعدة مشتركة لكل السكريبتات.

متغيرات البيئة:

- `SHINOBI_WEB`: عنوان التطبيق (الافتراضي `http://localhost:5173`).
- `SHINOBI_FIXTURES`: (إجباري) الفولدر اللي فيه ملفات `.y4m`، ولازم يكون **بره** الريبو.
- `SHINOBI_E2E_OUT`: الصور (screenshots)؛ الافتراضي فولدر الـ fixtures.

الدوال:

- `launchEdge(fixture, {headless, profile})`: بيشغّل Edge (`channel: 'msedge'`) ببروفايل دائم (في `OUT/profiles/<profile>`)، إذن الكاميرا، والـ flags `--use-fake-ui-for-media-stream` و `--use-fake-device-for-media-stream` و `--use-file-for-fake-video-capture=<file>`، والـ autoplay مسموح (`--

`autoplay-policy=no-user-gesture-required`)، وكمان flags بتسرّع جهاز Windows مشغول (`--no-proxy-server`، `--disable-background-networking`، `--disable-component-update`، `--disable-sync`، `--no-first-run`، `--no-default-browser-check`).

- `recordSocket(page)`: بيسمع للـ WebSocket بتاع `/ws` في الصفحة (وبيتجاهل socket الـ HMR بتاع Vite) وبيسجل كل رسالة نصية، وكل header فريم (التقدم، الوقت الباقي، fps، ready)، وكل event، وكمان اللي الصفحة بعتته. لاحظي إنه بيفك الـ header بنفس الطريقة: `payload.readUInt32BE(0)` لطول الـ JSON.
- `openPage(context, route, viewport)`: بيفتح الصفحة، بيجمّع أخطاء الـ console، وبيستنى بـ `waitForRendering` لحد ما الصفحة ترسم طبيعي (10 فريمات أنيميشن في أقل من 1.5 ثانية)، لأن Edge headless جديد ممكن يبقى متجمد شوية.
- `startAndWaitLive(page)`: بيدوس `start-camera` وبيستنى `camera-stage` بـ `data-live="true"`.
- `recordActivations(context)`: init script فيه `MutationObserver` (أداة بتراقب أي تغيير في الـ DOM) بيسجل بالظبط إمتى طبقة التفعيل أو البانر ظهروا واختفوا، وكل عدد خطوات بيعرضه كارت التسلسل (لوب بيعمل polling ممكن يفوّت كشف مدته ثانيتين). و `readMarks(page)` بيقرا التسجيل ده.
- `noHorizontalScroll`، `readStats`، `median`، `sleep`، `check` (بيسجل PASS أو FAIL)، `summary`، `snap` (صورة بمهلة طويلة؛ والفشل بيتسجل بس).
- وكمان `fixture(name)` و `shot(name)` لبناء مسارات الملفات، والثوابت `BASE` و `FIXTURES` و `OUT`.

سكريبتات الاختبار

السكريبت	إزاي تشغليه	بيفحص إيه
`live-jutsu.mjs`	`node e2e/live-jutsu.mjs fire_style` (أو `chidori`)	على `/live`: الفريمات المعالجة بتظهر؛ الكارت بيعرض الخطوات 1، 2، 3؛ الباك إند بيبعت events `step` و event `jutsu`؛ الطبقة بتتبع الـ event ده خلال 3 ثواني وبتتقفل لبانر "active"؛ مفيش scroll أفقي على 1440×900؛ مفيش أخطاء console
`live-wrong-order.mjs`	`node e2e/live-wrong-order.mjs`	مع `wrong_order`: الباك إند بيبعت `wrong_sign` (expected horse, got dog) **من غير** reset (التقدم فاضل 2)؛ الواجهة بتعرض "HORSE not recognized — that looked like DOG"؛ مفيش event `jutsu` ومفيش طبقة
`practice.mjs`	`node e2e/practice.mjs fire_style`	`/practice/<jutsu>` بيبدأ عند "Step 1 of 4"، بيبعت mode practice، بيعرض الخطوات من 1 لـ 4 بالترتيب، الاكتمال بيظهر بعد event `jutsu` من الباك إند، بس خطوات الجوتسو ده بتتبلّغ، و "Practise again" بيبعت `reset`
`sequence-rules.mjs`	`node e2e/sequence-rules.mjs [case ...]`	قواعد التسلسل حالة حالة (شوفي الجدول اللي تحت)
`reconnect.mjs`	`node e2e/reconnect.mjs` (متغير البيئة `SHINOBI_PORT`، الافتراضي 8000)	بيقتل عملية الباك إند على البورت ده وهو live؛ بيتأكد إن الواجهة بتقول "reconnecting" وبعدين "offline" من غير أي تعرّف ظاهر؛ بيعيد تشغيل السيرفر (`python -m shinobi.server --port ...`)؛ بيتأكد من الرجوع التلقائي بجلسة جديدة والـ mode اتبعت تاني

السكريبت	إزاي تشغليه	بيفحص إيه
`two-browsers.mjs`	`node e2e/two-browsers.mjs`	نسختين Edge (تسجيلات Fire Style و Chidori) بياخدوا جلستين مختلفتين، وكل واحدة بتاخد الجوتسو بتاعها بس
`responsive.mjs`	`node e2e/responsive.mjs live` (أو `idle`)	صور لـ `/live` وصفحات practice على 1440×900، 1280×800، 1180×820، 820×1180، 390×844، 360×740، ومفيش scroll أفقي في أي مقاس
`audio-timeline.mjs`	`node e2e/audio-timeline.mjs [fixture ...]`	بيعدّل جوه الصفحة `fetch` و `decodeAudioData` و `AudioBufferSourceNode.start` و `WebSocket` عشان يسجل كل صوت بدأ وكل event من الباك إند؛ بيتأكد إن كل صوت اتفك، تون + لمسة واحدة لكل إشارة متأكدة، الإشارات الغلط صامتة، مفيش صوت بيشتغل من غير الـ event بتاعه، أصوات الإطلاق اشتغلت، والترتيب: تأكيد، شحن، اسم، إطلاق
`music-flow.mjs`	`node e2e/music-flow.mjs [case ...]`	بيسجل رسم Web Audio وأتمتة الـ gain: التراك بيبدأ بس والجوتسو بتاعه متقدم، بـ fade-in 1.5 ثانية أو crossfade 0.8 ثانية؛ التراك الصح شغال عند الاكتمال؛ duck لـ 25% عند التأكيد؛ fade-out 1.5 ثانية عند الإطلاق؛ ختم واحد لكل إشارة؛ `wrong_keep` بيخلي الموسيقى مكملة؛ `camera_stop` بيخفيها

ملحوظات من الكود: `audio-timeline.mjs` من غير بارامترات بيشغّل `fire_style chidori shadow_clone` حالاته: `music-flow.mjs` و `rasengan`؛ و `fire_style chidori shadow_clone rasengan wrong_keep camera_stop` ؛ و `responsive.mjs live` بيصوّر `/live` و `/practice/fire_style` والكاميرا شغالة، و `idle` بيصوّر `/live` و `/practice` و `/practice/shadow_clone` من غير كاميرا.

حالات `sequence-rules.mjs` بالتفصيل

الحالة	الصفحة	بتفحص إيه
`practice_retry`	`/practice/fire_style`	المحاولات الغلط تحذيرات، الخطوات محفوظة، حلقة كهرماني، مفيش عدّاد
`practice_wait`	`/practice/fire_style`	وقفة 30 ثانية ما بتغيرش حاجة
`live_timeout`	`/live`	reset بعد حوالي 7 ثواني من آخر خطوة، ورسالة "Time ran out"
`live_wrong_keep`	`/live`	إشارة غلط بتحفظ التقدم والصح بعدها بتكمّل Fire Style
`live_retry`	`/live`	كل محاولة غلط بتدّي الـ 7 ثواني كاملين تاني و**عدّاد الكارت بيطلع لفوق**، وبس ساعتها
`live_retry_slow`	`/live`	محاولات غلط أكتر من 7 ثواني إجمالًا بس مفيش 7 ثواني من غير إشارة: مفيش reset
`live_tie`	`/live`	الكارت بيفضل على Rasengan
`live_repeat`	`/live`	تكرار آخر إشارة ما بيعيدش تشغيل الساعة
`live_start_over`	`/live`	"Start over" بيعمل reset فورًا ومفيش timeout بعده (بيستخدم fixture `live_timeout`)

تشغيل اختبارات e2e (ملخص)

1. ابني الـ fixtures مرة واحدة بـ `make_fixtures.py --out <scratch folder>`.
2. شغّلي الباك إند (`python -m shinobi.server`) وسيرفر التطوير (`npm run dev` جوه `frontend/`).
3. حطي `SHINOBI_FIXTURES` على فولدر الـ fixtures (واختياري `SHINOBI_WEB` و `SHINOBI_E2E_OUT`).
4. شغّلي سكريبت بـ Node من جوه `frontend/`، مثلًا `node e2e/live-jutsu.mjs fire_style`. لازم Microsoft Edge يكون متسطب.

ملحوظة: `frontend/e2e/__pycache__/` فيه بس الكاش المترجم بتاع Python لـ `make_fixtures.py` (`make_fixtures.cpython-312.pyc`) وممكن تتجاهليه.

حالة الملفات القديمة وغير المستخدمة (ملخص واحد)

الجدول ده بيجمع حالة كل ملف مش جزء من الواجهة الحالية. "قديم" = موجود وبيتبني بس الطريق الوحيد ليه مقفول. "ميت" = مفيش أي ملف بيعمله import.

الملف (أو المجموعة)	الحالة	السبب
`src/pages/PracticePage.tsx` + CSS	قديم	مش متوصل بـ route (اتبدل بـ `konoha/practice/PracticeView.tsx`)
`src/pages/LibraryPage.tsx` + CSS	قديم	مش متوصل (اتبدل بـ `konoha/jutsu/JutsuView.tsx`)
`src/pages/SettingsPage.tsx` + CSS	قديم	مش متوصل (اتبدل بـ `konoha/pages/SettingsPage.tsx`)
`src/components/live/CameraStage.tsx`، `RecognitionPanel.tsx`، `SequenceTrack.tsx`، `AvailabilityNotice.tsx` + CSS	قديم	مستخدمين بس في `pages/PracticePage.tsx`
`src/components/live/ConnectionPill.tsx` + CSS	قديم	مستخدم بس في CameraStage و RecognitionPanel
`src/components/live/JutsuProgressList.tsx` + CSS	ميت	مفيش import
`src/components/layout/AppShell.tsx` + CSS	ميت	مفيش import (اتبدل بـ `KonohaShell`)
`src/components/layout/MobileMenu.tsx`، `ServerStatus.tsx`، `ThemeToggle.tsx`، `nav.ts` + CSS	قديم	مستخدمين بس جوه `AppShell`
`src/components/art/ChakraBackground.tsx`، `ThemeShift.tsx`، `ShinobiLogo.tsx` + CSS	قديم	مستخدمين بس في AppShell / MobileMenu
`src/components/art/ParticleField.tsx`، `particles.ts`	قديم	جوه ChakraBackground / HeroArt (والاختبار لسه شغال)
`src/components/art/HeroArt.tsx` + CSS	ميت	مفيش import
`src/konoha/jutsuLook.ts`	ميت	مفيش import (الكروت بتستخدم `JutsuIcon`)
`src/components/ui/*`	جزئيًا نشط	About و 404 بيستخدموا `Button` و غيره؛ باقي الكيت للصفحات القديمة

الملف (أو المجموعة)	الحالة	السبب
`frontend/e2e/__pycache__/`	مش كود	كاش Python

ونشطين رغم إنهم في فولدرات "قديمة": `pages/HomePage.tsx`، `pages/LiveDemoPage.tsx`، `pages/AboutPage.tsx`، `pages/NotFoundPage.tsx`، `components/layout/useDocumentTitle.ts`، `styles/tokens.css` (ألوان الجوتسو)، `styles/global.css`، و `components/live/labels.ts` و `useLiveEvents.ts` و `StageOverlay.tsx`.

فروق بين الكود والوصف الإنجليزي القديم

اللي لقيناه وإحنا بنراجع على الكود (والشرح فوق ماشي على الكود):

1. **عدد طلبات REST:** النص الإنجليزي قال "three REST calls" وعدّ أربعة. الصح أربعة: `/api/jutsu`، `/api/info`، `/api/credits` (في `services/api.ts`) و `/api/voices` (في `audio/voices.ts`).
2. **نوع `Mode`:** الإنجليزي كتب `{mode:'free'}`؛ النوع في الكود `{ mode: 'free'; jutsu: null }`. والرسالة اللي بتتبعت فعلًا للسيرفر في الوضع الحر هي `{ type: 'mode', mode: 'free' }` من غير `jutsu`.
3. **عدد ملفات الأصوات:** تعليق `engine.ts` بيقول 19 cue وتعليق `gains.ts` بيقول 31 ملف؛ الكود فيه 32 cue في `CUES` و 32 قيمة في `CUE_GAIN`. (التعليقات هي اللي قديمة، مش الكود.)
4. **تعليق prop `timeLeft` في `SequenceCard`:** بيقول "FrameState.time_left"، لكن `useDojo` بيبعت القيمة من `time_left_per_jutsu` للجوتسو المعروض (و `time_left` بس مع السيرفرات القديمة).
5. **`data-testid` بتاع كارت التسلسل:** مش مذكور في الإنجليزي: `sequence-card` لما مفيش جوتسو، و `sequence-track` لما فيه.
6. **flags بتاعة Edge في `lib.mjs`:** فيه كمان `--use-fake-ui-for-media-stream` غير اللي مذكور.
7. **`index.html`:** لسه فيه `theme-color` `#05060b` و `color-scheme` `dark` من التصميم الغامق، رغم إن الشكل الحالي فاتح.
8. **تبديل الثيم:** مفيش في الواجهة الحالية زرار Naruto/Sasuke ظاهر (اللي كان في `ThemeToggle` القديم غير مستخدم)؛ الاستخدام الوحيد لـ `setTheme` في الصفحات النشطة هو Reset اللي بيرجّعه Naruto.
9. **Carousel spring:** الإنجليزي ذكر `stiffness 280, damping 34`؛ الكود فيه كمان `mass: 0.9`.

فهرس كل ملفات `frontend/src` (مرجع سريع)

الفهرس ده بيذكر كل ملف `.ts` / `.tsx` (الـ 118) وكل ملف CSS و asset في `frontend/src`، بسطر واحد عن دوره وحالته. "نشط" = جزء من الواجهة الحالية أو اختباراتها.

الملف	بيعمل إيه	الحالة
`src/main.tsx`	نقطة البداية، الخطوط، شجرة الـ providers	نشط
`src/App.tsx`	الـ routes	نشط

الملف	بيعمل إيه	الحالة
`src/types/protocol.ts`	أنواع بروتوكول الـ WebSocket	نشط
`src/types/catalog.ts`	أنواع ردود REST	نشط
`src/services/api.ts`	طلبات REST و `sessionUrl`	نشط
`src/services/socket/framing.ts`	الشكل الـ binary والتحقق و `InFlightFrames`	نشط
`src/services/socket/framing.test.ts`	اختبار framing	نشط (اختبار)
`src/services/socket/client.ts`	`ShinobiSocket` بإعادة الاتصال	نشط
`src/services/socket/client.test.ts`	اختبار العميل	نشط (اختبار)
`src/services/socket/index.ts`	إعادة تصدير	نشط
`src/hooks/useCamera.ts`	الكاميرا	نشط
`src/hooks/useShinobiSession.ts`	الجلسة الحية (إرسال، استقبال، رسم)	نشط
`src/contexts/ThemeContext.tsx`	ثيم Naruto / Sasuke	نشط
`src/contexts/SettingsContext.tsx`	الإعدادات في localStorage	نشط
`src/contexts/SoundContext.tsx`	محرك الصوت للتطبيق	نشط
`src/contexts/CatalogContext.tsx`	الكتالوج والـ info	نشط
`src/lib/storage.ts`	localStorage آمن	نشط
`src/konoha/konoha.css`	tokens بتاعة Konoha	نشط
`src/konoha/KonohaShell.tsx` + `KonohaShell.module.css`	إطار الصفحات	نشط
`src/konoha/LeafEmblem.tsx`	شعار ورقة الشجر	نشط
`src/konoha/BrushButton.tsx` + `BrushButton.module.css`	زرار الفرشاة	نشط
`src/konoha/BrushTag.tsx` + `BrushTag.module.css`	لافتة العناوين	نشط
`src/konoha/ParchmentCard.tsx` + `ParchmentCard.module.css`	كارت البردي	نشط
`src/konoha/jutsuLook.ts`	أيقونة ولون لكل جوتسو	ميت
`src/konoha/shell/KonohaBackdrop.tsx` + `KonohaBackdrop.module.css`	الخلفية	نشط
`src/konoha/shell/FallingLeaves.tsx`	ورق الشجر النازل	نشط
`src/konoha/shell/KonohaHeader.tsx` + `KonohaHeader.module.css`	الهيدر	نشط
`src/konoha/shell/BrushSwash.tsx`	ضربة الفرشاة SVG	نشط

الملف	بيعمل إيه	الحالة
`src/konoha/shell/nav.ts`	`KONOHA_NAV`	نشط
`src/konoha/shell/MusicControl.tsx` + `MusicControl.module.css`	زرار الموسيقى	نشط
`src/konoha/shell/LegacyFrame.tsx` + `LegacyFrame.module.css`	لوحة الصفحات القديمة	نشط
`src/konoha/dojo/useDojo.ts`	حالة Home و Live	نشط
`src/konoha/dojo/DojoView.tsx` + `DojoView.module.css`	ترتيب Home	نشط
`src/konoha/dojo/CameraCard.tsx` + `CameraCard.module.css`	كارت الكاميرا	نشط
`src/konoha/dojo/JutsuListCard.tsx` + `JutsuListCard.module.css`	قايمة الجوتسو	نشط
`src/konoha/dojo/sequenceSteps.ts`	دوال كارت التسلسل والعدّاد	نشط
`src/konoha/dojo/sequenceSteps.test.ts`	اختبارها	نشط (اختبار)
`src/konoha/dojo/SequenceCard.tsx` + `SequenceCard.module.css`	كارت التسلسل والعدّاد	نشط
`src/konoha/live/LiveView.tsx` + `LiveView.module.css`	ترتيب Live	نشط
`src/konoha/pages/JutsuPage.tsx`	غلاف route المكتبة	نشط
`src/konoha/pages/PracticePage.tsx`	غلاف route التدريب	نشط
`src/konoha/pages/SettingsPage.tsx`	صفحة الإعدادات	نشط
`src/konoha/jutsu/JutsuView.tsx` + `JutsuView.module.css`	صفحة Jutsu Scrolls	نشط
`src/konoha/jutsu/pageState.ts`	`replaceUrl` والاختيار	نشط
`src/konoha/jutsu/sequenceKind.ts`	Official / Custom	نشط
`src/konoha/jutsu/JutsuCard.tsx` + `JutsuCard.module.css`	كارت الـ carousel	نشط
`src/konoha/jutsu/JutsuDetail.tsx` + `JutsuDetail.module.css`	ورقة الإشارات	نشط
`src/konoha/jutsu/SignSteps.tsx` + `SignSteps.module.css`	`SignArt` و `SignSteps` و `ArtCredit`	نشط
`src/konoha/carousel/Carousel.tsx` + `Carousel.module.css`	الـ carousel	نشط
`src/konoha/practice/PracticeView.tsx`	صفحة Practice	نشط
`src/konoha/practice/PracticeSession.tsx` + `PracticeSession.module.css`	جلسة التدريب	نشط

الملف	بيعمل إيه	الحالة
`src/konoha/settings/controls.tsx` + `controls.module.css`	عناصر الفورم	نشط
`src/konoha/settings/SettingsPage.module.css`	ترتيب صفحة الإعدادات	نشط
`src/konoha/music/player.ts`	`MusicPlayer` للخلفية	نشط
`src/konoha/music/MusicContext.tsx`	`MusicProvider` و `useMusic`	نشط
`src/audio/cues.ts`	قايمة الأصوات	نشط
`src/audio/gains.ts`	المستويات والـ ducking	نشط
`src/audio/engine.ts`	`SoundEngine`	نشط
`src/audio/eventCues.ts`	مخطط المؤثرات	نشط
`src/audio/eventCues.test.ts`	اختباره	نشط (اختبار)
`src/audio/voiceCues.ts`	مخطط أصوات الشخصيات	نشط
`src/audio/voiceCues.test.ts`	اختباره	نشط (اختبار)
`src/audio/voices.ts`	ids والمتكلم والـ manifest	نشط
`src/audio/useVoiceManifest.ts`	hook الـ manifest لـ Settings	نشط
`src/audio/jutsuMusic.ts`	مخطط موسيقى الجوتسو	نشط
`src/audio/jutsuMusic.test.ts`	اختباره	نشط (اختبار)
`src/audio/jutsuMusicPlayer.ts`	مشغل موسيقى الجوتسو	نشط
`src/audio/useSessionSounds.ts`	ربط الجلسة بالصوت	نشط
`src/audio/voicePlayback.test.ts`	اختبار المحرك	نشط (اختبار)
`src/components/live/useLiveEvents.ts`	حالة الطبقة والـ toasts	نشط
`src/components/live/labels.ts`	النصوص والاشتقاقات	نشط
`src/components/live/labels.test.ts`	اختبارها	نشط (اختبار)
`src/components/live/StageOverlay.tsx` + `StageOverlay.module.css`	الطبقة فوق الفيديو	نشط
`src/components/live/CameraStage.tsx` + `CameraStage.module.css`	مسرح الكاميرا القديم	قديم
`src/components/live/RecognitionPanel.tsx` + `RecognitionPanel.module.css`	لوحة التعرّف القديمة	قديم
`src/components/live/SequenceTrack.tsx` + `SequenceTrack.module.css`	شريط التسلسل القديم	قديم
`src/components/live/ConnectionPill.tsx` + `ConnectionPill.module.css`	كبسولة الحالة	قديم

الملف	بيعمل إيه	الحالة
`src/components/live/AvailabilityNotice.tsx` + `AvailabilityNotice.module.css`	التوافر	قديم
`src/components/live/JutsuProgressList.tsx` + `JutsuProgressList.module.css`	قايمة التقدم القديمة	ميت
`src/components/art/JutsuIcon.tsx` + `JutsuIcon.module.css`	أيقونة الجوتسو	نشط
`src/components/art/ShadowCloneIcon.tsx` + `ShadowCloneIcon.module.css`	أيقونة Shadow Clone المتحركة	نشط
`src/components/art/JutsuMotif.tsx` + `JutsuMotif.module.css`	رسومات اللهب والبرق والكرة	نشط
`src/components/art/jutsuVisuals.ts`	ألوان وكانجي كل جوتسو	نشط
`src/components/art/ChakraBackground.tsx` + `ChakraBackground.module.css`	الخلفية الغامقة	قديم
`src/components/art/ParticleField.tsx`	canvas الجزيئات	قديم
`src/components/art/particles.ts`	محاكاة الجزيئات	قديم
`src/components/art/particles.test.ts`	اختبارها	نشط (اختبار لكود قديم)
`src/components/art/HeroArt.tsx` + `HeroArt.module.css`	رسمة الرئيسية القديمة	ميت
`src/components/art/ShinobiLogo.tsx` + `ShinobiLogo.module.css`	الشعار القديم	قديم
`src/components/art/ThemeShift.tsx` + `ThemeShift.module.css`	موجة تغيير الثيم	قديم
`src/components/layout/useDocumentTitle.ts`	عنوان التاب	نشط
`src/components/layout/AppShell.tsx` + `AppShell.module.css`	الإطار الغامق	ميت
`src/components/layout/MobileMenu.tsx` + `MobileMenu.module.css`	درج الموبايل	قديم
`src/components/layout/ServerStatus.tsx` + `ServerStatus.module.css`	مؤشر السيرفر	قديم
`src/components/layout/ThemeToggle.tsx` + `ThemeToggle.module.css`	مفتاح الثيم	قديم
`src/components/layout/nav.ts`	`NAV_ITEMS`	قديم
`src/components/ui/index.ts`	إعادة تصدير الكيت	نشط جزئيًا
`src/components/ui/Button.tsx` + `Button.module.css`	زرار غامق	نشط (About، 404)
`src/components/ui/Panel.tsx` + `Panel.module.css`	كارت غامق	قديم (الصفحات الغامقة غير المتوصلة)

الملف	بيعمل إيه	الحالة
`Badge.module.css` + `src/components/ui/Badge.tsx`	لافتة صغيرة	قديم (عن طريق `SequenceBadge` والصفحات القديمة)
`Slider.module.css` + `src/components/ui/Slider.tsx`	slider	قديم
`Switch.module.css` + `src/components/ui/Switch.tsx`	مفتاح	قديم
`src/components/ui/SegmentedControl.tsx` + `SegmentedControl.module.css`	اختيار مجزأ	قديم
`Select.module.css` + `src/components/ui/Select.tsx`	قايمة منسدلة	قديم
`src/components/ui/SectionHeading.tsx` + `SectionHeading.module.css`	عنوان قسم	نشط (About)
`Skeleton.module.css` + `src/components/ui/Skeleton.tsx`	مكان التحميل	نشط (About)
`src/components/ui/StatusMessage.tsx` + `StatusMessage.module.css`	صناديق الحالة	نشط (`ErrorState` في About)
`Lightbox.module.css` + `src/components/ui/Lightbox.tsx`	عارض صور	قديم
`SignArt.module.css` + `src/components/ui/SignArt.tsx`	رسمة إشارة قديمة	قديم
`src/components/ui/SequenceBadge.tsx`	شارة Official / Custom	قديم
`src/components/ui/Reveal.tsx`	أنيميشن الظهور	نشط (About)
`src/components/ui/ArtCredit.tsx` + `ArtCredit.module.css`	رخصة الرسومات (القديمة)	قديم
`src/components/ui/serverError.ts`	رسائل أخطاء السيرفر	نشط (About)
`src/pages/HomePage.tsx`	route الرئيسية	نشط
`src/pages/LiveDemoPage.tsx`	route Live	نشط
`AboutPage.module.css` + `src/pages/AboutPage.tsx`	صفحة About	نشط
`NotFoundPage.module.css` + `src/pages/NotFoundPage.tsx`	صفحة 404	نشط
`PracticePage.module.css` + `src/pages/PracticePage.tsx`	Practice الغامق	قديم
`LibraryPage.module.css` + `src/pages/LibraryPage.tsx`	Library الغامق	قديم
`SettingsPage.module.css` + `src/pages/SettingsPage.tsx`	Settings الغامق	قديم
`src/styles/tokens.css`	tokens القديمة وألوان الجوتسو	نشط
`src/styles/global.css`	resets و `.visually-hidden`	نشط
`src/assets/konoha/background.jpg`	لوحة القرية	نشط
`src/assets/konoha/shadow-clone/bg.webp`، `clone-left.webp`، `clone-right.webp`، `clone-front.webp`	طبقات أيقونة Shadow Clone	نشط

ملحوظة عن الجدول: حالة ملفات `components/ui` مبنية على الـ imports الفعلية: `AboutPage.tsx` بيستورد `Button` و `ErrorState` و `Reveal` و `SectionHeading` و `Skeleton` و `START_SERVER_HINT` و `describeServerError`، و `NotFoundPage.tsx` بيستورد `Button` بس. أي كومبوننت تاني في الكيت مستخدم بس في الصفحات الغامقة غير المتوصلة.

خلاصة الجزء التالت

لو هتشرحي الفرونت إند في دقيقة: هو تطبيق React بيفتح الكاميرا، بيبعت صورها JPEG على WebSocket للباك إند، وبيعرض اللي الباك إند بيرجّعه: صورة فيها المؤثرات، و `FrameState` فيه events. الـ events دي هي **المصدر الوحيد** لأي حاجة بتحصل: الكارت بيتملى من `progress`، التحذير من `wrong_sign`، الـ reset من `sequence_failed`، التفعيل من `jutsu`، وكل صوت من مخطط صافي بيقرا نفس الـ events. Live فيه حد وقت لكل إشارة (والعدّاد بيرجع يطلع لفوق بعد محاولة غلط)، و Practice من غير حد وقت. وكل حاجة "ذكية" (التعرّف، التسلسل، الوقت) في الباك إند؛ الفرونت إند واجهة أمينة بتعرض وبتسمّع وبس.

الجزء الرابع: البيانات والتدريب والتقييم والأدوات والاختبارات

الجزء ده بيشرح كل حاجة حوالين "مخ" SHINOBI AI: إزاي بنسجل بيانات حركات الإيد (hand signs)، بتتخزن فين، إزاي بنقسمها لبيانات تدريب وبيانات اختيار (validation) وبيانات اختبار (test)، إزاي بندرب الـ landmark classifier، إزاي بنقيس الـ pipeline كله من أوله لآخره، كل أداة مساعدة في فولدر `tools/` بتعمل إيه، كل ملف اختبار في `tests/` بيتأكد من إيه، والمشروع واقف فين النهارده بالظبط. وفي الآخر فيه قايمة بكل الأوامر بالحرف عشان تشغلي كل جزء في المشروع على Windows.

باقي أجزاء الشرح بتتكلم عن التطبيق نفسه (الكاميرا، MediaPipe hands، الـ features، الـ fusion، الـ controller، الـ effects، السيرفر وواجهة الويب). هنا الحاجات دي هتظهر بس كـ "قطع بناء" بتستدعيها سكريبتات البيانات والتقييم.

ملحوظة مهمة عن طريقة الكتابة: الشرح ده مكتوب بعد ما اتراجع على الكود الحقيقي والبيانات والتقارير الموجودة في المشروع، مش ترجمة حرفية للنسخة الإنجليزي. في أي حتة النسخة الإنجليزي كانت قديمة أو ناقصة، اتبعنا الكود وقلنا ده صراحة. وفي آخر الجزء فيه قسم كامل اسمه "الفروقات اللي لقيناها بين الكلام والكود" بيجمع كل الحاجات دي في مكان واحد.

قبل ما نبدأ: أساسيات Machine Learning اللي هتحتاجيها

الجزء ده كله مبني على شوية أفكار أساسية في Machine Learning (تعلم الآلة: يعني بنخلي الكمبيوتر يتعلم من أمثلة بدل ما نكتبله القواعد بإيدينا). هشرح كل فكرة ببساطة دلوقتي، وبعدين هنشوفها جوه الكود.

Dataset (مجموعة البيانات)

Dataset يعني كل الأمثلة اللي بنعلم منها الموديل ونختبره عليها. في مشروعنا الـ dataset هي التسجيلات اللي سجلناها بالكاميرا: كل تسجيل 20 ثانية لحركة واحدة، ومحفوظ منه نقط الإيد (landmarks) فريم بفريم، وفيديو صغير للمراجعة.

Label (التصنيف أو الإجابة الصح)

Label يعني الإجابة الصح لكل مثال. لو التسجيل فيه حركة `tiger`، يبقى الـ label بتاعه `tiger`. الموديل بيتعلم يربط بين شكل الإيد والـ label ده. في مشروعنا الـ label بيتكتب جوه ملف التسجيل نفسه (في `meta`) وكمان اسم الفولدر اللي الملف فيه.

Feature (الخاصية أو الصفة)

Feature يعني رقم بنحسبه من البيانات الخام عشان يوصف المثال بطريقة الموديل يفهمها. MediaPipe بيدينا 21 نقطة لكل إيد (x, y, z)، لكن إحنا مش بنديهم للموديل كده على طول. بنحسب منهم أرقام زي زوايا مفاصل

الصوابع والمسافات بين أطراف الصوابع ومكان الإيدين بالنسبة لبعض. المجموعة دي من الأرقام اسمها **feature vector**، وفي مشروعنا طولها 206 رقم لكل فريم.

Model Training (تدريب الموديل)

Model Training يعني بندرب الموديل على البيانات عشان يتعلم يميز الحركات. بنديله آلاف الأمثلة (feature vector + label) وهو بيظبط نفسه لحد ما يبقى قادر يخمن الـ label صح من الـ features.

Training / Validation / Test split (تقسيم البيانات لتلات مجموعات)

ده من أهم الأفكار في الجزء ده كله:

- **Train (بيانات التدريب):** التسجيلات اللي الموديل بيتعلم منها فعلاً.
- **Validation (بيانات الاختيار والضبط):** تسجيلات الموديل ما اتعلمش منها، بنستخدمها عشان نختار بين موديلين أو نظبط إعدادات (زي الـ thresholds). زي امتحان تجريبي.
- **Test (بيانات الاختبار النهائي):** تسجيلات بنلمسها مرة واحدة بس في الآخر عشان نطلع رقم صادق عن أداء السيستم. زي الامتحان النهائي اللي ما ينفعش تشوفي أسئلته قبلها.

القاعدة الذهبية: **ما ينفعش أبداً نظبط أي حاجة على الـ test set.** لو ظبطنا عليه، الرقم اللي هيطلع هيبقى أحسن من الحقيقة، لأننا عملياً "ذاكرنا الامتحان".

Leakage (تسريب البيانات)

Leakage يعني إن معلومات من بيانات الاختبار "تتسرب" لبيانات التدريب من غير ما ناخد بالنا. في الفيديو، الفريمات اللي جنب بعض في نفس التسجيل تقريباً متطابقة. لو خدنا شوية فريمات من تسجيل للتدريب وفريمات تانية من نفس التسجيل للاختبار، الموديل هيتختبر على نسخ شبه طبق الأصل من اللي حفظه، والنتيجة هتطلع أحلى بكتير من الواقع. عشان كده في المشروع ده **وحدة التقسيم دايماً تسجيل كامل**، عمرنا ما بنقسم بالفريم.

Data Augmentation (تكبير البيانات صناعياً)

Augmentation يعني بنعمل نسخ معدلة شوية من الأمثلة الموجودة عشان الموديل يتعلم من تنوع أكبر. في مشروعنا بنعمل نسختين إضافيتين لكل فريم: نسخة معكوسة (mirror: اليمين بقى شمال) ونسخة فيها رعشة صغيرة عشوائية (jitter). مهم جداً: الـ augmentation بيتعمل على بيانات التدريب بس، عمره ما بيتعمل على البيانات اللي بنقيس عليها.

Cross-Validation (التقييم المتقاطع)

لما البيانات تبقى قليلة، بدل ما نسيب جزء صغير للـ validation، بنعمل **Cross-Validation**: بنقسم البيانات لكذا جزء (اسمهم folds)، وكل مرة ندرب على كل الأجزاء ما عدا جزء واحد ونختبر عليه، ونلف لحد ما كل جزء ياخد دوره. في الآخر بنجمع النتايج. في مشروعنا الأجزاء دي دايماً تسجيلات كاملة (أو مجموعات تسجيلات من نفس الظروف)، عشان ما يحصلش leakage. ده اسمه **Grouped Cross-Validation**.

Accuracy, Precision, Recall, F1

- **Accuracy (الدقة الكلية):** نسبة الفريمات اللي الموديل جابها صح من كل الفريمات.
- **Precision (الدقة لما بيقول):** من كل المرات اللي الموديل قال فيها "ده tiger"، كام مرة كان فعلاً tiger؟ لو الـ precision واطي يبقى الموديل بيقول tiger كتير وهو غلطان (false alarms).
- **Recall (الاسترجاع أو التغطية):** من كل الفريمات اللي كانت فعلاً tiger، الموديل لقى كام منهم؟ لو الـ recall واطي يبقى الموديل بيفوّت الحركة كتير.
- **F1:** رقم واحد بيجمع الـ precision والـ recall (متوسط توافقي ليهم). لو واحد فيهم واطي، الـ F1 بيقع.

مثال بسيط: عندنا 100 فريم tiger. الموديل قال "tiger" على 80 فريم، منهم 70 صح و10 كانوا حاجة تانية. يبقى precision = 70/80 = 0.875، و recall = 70/100 = 0.70، و F1 = 2 × 0.875 × 0.70 / (0.875 + 0.70) ≈ 0.778.

Macro-F1 و Balanced Accuracy

- **Macro-F1:** بنحسب F1 لكل حركة لوحدها، وبعدين ناخد المتوسط العادي، كل حركة بنفس الوزن. ليه؟ لأن الـ accuracy العادية بتتحكم فيها الحركات اللي ليها فريمات كتير (زي `idle` و `rasengan_palm`). Macro-F1 بيخلي حركة صغيرة الموديل بيفشل فيها تشد الرقم لتحت حتى لو فريماتها قليلة.
- **Balanced Accuracy:** متوسط الـ recall بتاع كل الحركات، برضه كل حركة بنفس الوزن.

Confusion Matrix (مصفوفة الالتباس)

Confusion Matrix جدول الصفوف فيه هي الحركة الحقيقية والأعمدة هي اللي الموديل قاله. الخانات اللي على القطر (نفس الصف ونفس العمود) هي الإجابات الصح. أي رقم بره القطر بيقولك الموديل بيلخبط بين أنهي حركتين. مثلاً لو في صف `tiger` وعمود `ram` فيه 108، يبقى 108 فريم tiger الموديل افتكرهم ram.

Random Forest

Random Forest موديل عبارة عن مجموعة كبيرة من "شجر القرارات" (decision trees). كل شجرة بتسأل أسئلة بسيطة على الـ features (زي: "هل زاوية الصباع ده أكبر من كذا؟") لحد ما توصل لإجابة. كل شجرة اتدربت على عينة عشوائية شوية مختلفة، وفي الآخر الشجر كله "بيصوّت" والنتيجة هي احتمال كل حركة. مميزاته: سريع، مستقر، وما بيحتاجش الأرقام تكون متظبطة على نفس المقياس. في مشروعنا بنستخدم 300 شجرة.

MLP (شبكة عصبية بسيطة)

MLP (Multi-Layer Perceptron) شبكة عصبية صغيرة: طبقة دخل (الـ 206 feature)، وبعدين طبقتين مخفيتين (128 خلية ثم 64 خلية)، وطبقة خرج فيها احتمال لكل حركة. بتتعلم أوزان بتربط بين الطبقات. محتاجة إن الـ features تكون متظبطة على نفس المقياس (عشان كده بنحط قبلها `StandardScaler`)، وأبطأ شوية من الـ Random Forest في كل فريم.

الصورة الكبيرة في صفحة واحدة

التطبيق بيتعرف على حركة الإيد بطريقتين في نفس الوقت:

- **YOLOX** (`models/yolox_nano.onnx`): ده hand-seal detector جاهز ومتدرب قبل كده (pretrained). حد تاني دربه على صور ناس تانيين، والمشروع ده عمره ما بيدربه. هو "صاحب" الأختام السبعة الرسمية: ram, snake, tiger, dog, monkey, boar, horse.
- **الـ landmark classifier** (`models/gesture_model.joblib`): ده اللي المشروع بيدربه بنفسه بـ `train.py`. بيقرا نقط الإيد التلاتية الأبعاد (3D hand landmarks) اللي MediaPipe بيلاقيها، ويطلع احتمال لكل حركة من العشر حركات اللي في `config.GESTURES`. في التطبيق هو بيقرر بس الحركتين المخصوصين: `clone_seal` و `rasengan_palm` (ودول اللي في `config.LANDMARK_SIGNS`).

`shinobi/fusion.py` (الـ SignFusion) بيدمج الإجابتين في كل فريم، و `shinobi/controller.py` (الـ ActionController) بيحول سلسلة الحركات لتفعيل jutsu.

خطوات شغل البيانات والـ machine learning:

1. نسجل sessions بالكاميرا (بـ `collect_data.py`، أو بالأدوات الموجّهة `tools/record_training_session.py` و `tools/record_test_session.py`). كل session بيبقى ملف `.npz` فيه نقط الإيد الخام، وفيديو `.mp4` صغير للمراجعة.
2. نحط كل تسجيل في مجموعة واحدة بس (train أو validation أو test) جوه `data/splits.json`. ده بيتعمل بالتسجيل، عمره ما بيتعمل بالفريم.
3. ندرب الـ landmark classifier على تسجيلات TRAIN بس (`train.py`). الموديل المدرب بيفتكر هو اتعلم من أنهي تسجيلات بالظبط.
4. نقيس الـ pipeline كله (الـ landmarks، والـ YOLOX، وقرار التطبيق بعد الدمج) على تسجيلات عمره ما اتعلم منها (`evaluate.py run`)، ونكتب تقرير في `reports/<date_time>/`.
5. دراسات أعمق اختيارية: grouped cross-validation حسب ظروف التسجيل (`tools/grouped_cv.py`)، والقياسات الإضافية في `tools/evaluate_pipeline.py`.

تلات كلمات لازم تفهميهم الأول

- **Recording (تسجيل أو session):** تسجيل واحد مدته 20 ثانية لحركة واحدة. الـ id بتاعه هو اسم الملف من غير `.npz`. مثلاً `me_day_20261101_162853` معناه: الشخص `me`، الإضاءة `day`، التاريخ 01-11-2026، الساعة 16:28:53.
- **Leakage:** زي ما شرحنا فوق، الفريمات اللي جنب بعض في نفس التسجيل شبه متطابقة. لو جزء من فريمات تسجيل راح للتدريب وجزء تاني من نفس التسجيل راح للاختبار، الموديل "بيتختبر" على نسخ تقريباً من اللي حفظه، والنتيجة بتبان أحسن بكتير من الحقيقة. عشان كده وحدة كل تقسيم في المشروع ده هي **تسجيل كامل**.
- **Train / validation / test:** تسجيلات TRAIN بتعلم الموديل. تسجيلات VALIDATION بنستخدمها نختار بين الموديلات ونظبط الإعدادات. تسجيلات TEST بنلمسها مرة واحدة بس في الآخر عشان نطلع رقم صادق. ولا حاجة بتتظبط على الـ test set أبداً.

فولدر `data/`: شكل البيانات وتنظيمها

كل البيانات موجودة في `data/` (المسار ده هو `config.DATA_DIR`). السكريبتات عمرها ما بتنقل أو تغير اسم أو تمسح أي حاجة في `data/`. الملف الوحيد اللي الأدوات بتكتبه في `data/` هو `data/splits.json` (وطبعاً

التسجيلات الجديدة اللي أدوات التسجيل بتحفظها).

المسار	فيه إيه
`data/landmarks/<label>/<id>.npz`	نقط الإيد الخام لتسجيل حركة واحدة
`data/videos/<label>/<id>.mp4`	فيديو المراجعة لنفس التسجيل (480p، بحد أقصى 15 فريم في الثانية)
`data/sequences/<jutsu>/<id>.npz`	تسجيلات موجّهة لسلسلة من 4 حركات من `record_sequences.py` (الفولدر ده مش موجود لسه: ما اتسجلش ولا take)
`data/videos/sequences/<jutsu>/<id>.mp4`	فيديوهات الـ takes دي
`data/_discarded/`	تسجيلين قدام (إيدين مفتوحين اتسجلوا باسم غلط)، متسابين بس كأمثلة سلبية "عمرها ما اتدربت"
`data/splits.json`	كل تسجيل تبع أنهي مجموعة
`data/evaluation_exclusions.json`	تسجيلات test متشالة من حساب النتايج لأن محتواها مش مطابق للـ label بتاعها

اتأكدنا من الكلام ده على الديسك: `data/_discarded/` فيها فعلاً ملفين `.npz` وفيديوهاتهم (`me_day_20261030_174011` و `me_day_20261030_174108`)، وفولدر `data/sequences/` مش موجود.

الملفات `data/landmarks/<label>/*.npz`

النهارده فيه 13 فولدر labels: `boar`، `chidori`، `clone_seal`، `dog`، `fire`، `horse`، `idle`، `monkey`، `ram`، `rasengan`، `rasengan_palm`، `snake`، `tiger` (77 ملف في المجموع).

عدد الملفات في كل فولدر (اتعد من الديسك):

الفولدر	عدد الملفات
`boar`	5
`chidori`	4
`clone_seal`	6
`dog`	6
`fire`	4
`horse`	5
`idle`	13
`monkey`	6
`ram`	6
`rasengan`	4
`rasengan_palm`	5
`snake`	7
`tiger`	6

- `rasengan` هو **الاسم القديم** لـ `rasengan_palm`. الإعداد `config.LEGACY_LABELS = {"rasengan": "rasengan_palm"}` بيترجمه في كل حتة، فالأربع ملفات دول بيتدربوا على إنهم `rasengan_palm`.
- `chidori` (إيد على شكل مخلب) و `fire` (علامة OK) دول labels من تصميم قديم. ما بقوش حركات في التطبيق، فـ `splits.py` بيحطهم في قايمة "ignored" وعمرهم ما بيتدربوا أو يتحسب عليهم نتايج. `tools/evaluate_pipeline.py` بيستخدمهم تاني كأمثلة سلبية عمرها ما اتدربت.

الـ labels (الحركات) اللي الموديل بيعرفها

دي القايمة الرسمية من `shinobi/config.py`:

```
GESTURES = ["idle", "ram", "snake", "tiger", "dog", "monkey", "boar", "horse", "clone_seal",
"rasengan_palm"]
```

بيعمل إيه: بيعرّف العشر حركات اللي الـ landmark classifier بيطلع احتمال لكل واحدة فيهم. الترتيب مهم: رقم كل حركة في القايمة (index) هو اللي بيتحفظ مع الموديل، و`idle` لازم يكون رقم 0. مين بيستخدمه: `train.py` (عشان يحول الـ label لرقم)، `collect_data.py` (زراير 0 لـ 9)، `splits.py`، `evaluate.py`، والتطبيق كله.

معنى كل حركة باختصار (من `config.GESTURE_HINTS`):

الـ label	الشكل
`idle`	أي حاجة تانية: الإيدين تحت، تلويح، لمس الوش، حركات نص مكتملة. ده بيعلم الموديل "إيه اللي ما يتفاعلش معاه"
`ram`	الإيدين: زي Tiger بس فوق بعض، أطراف الصوابع المرفوعة مش على نفس المستوى
`snake`	الإيدين متشبكين، العشر صوابع داخلين في بعض
`tiger`	الكفين لازقين في بعض، السبابتين لفوق، باقي الصوابع متشابكة
`dog`	إيد مقفولة (قبضة)، والإيد التانية مفرودة فوقها
`monkey`	الإيدين مفرودين، واحدة فوق التانية
`boar`	الكفين لتحت، الصوابع منحنية لتحت، أطرافها بتلمس بعض (شكل V)
`horse`	أطراف السبابتين بيلمسوا بعض زي السقف، باقي الصوابع متشابكة
`clone_seal`	الإيدين: السبابة والوسطى لفوق، متقاطعين زي علامة الزائد (+)
`rasengan_palm`	إيد واحدة: كف مفتوح ناحية الكاميرا، الصوابع مفرودة ومتباعدة

ملحوظة مهمة عن `rasengan_palm`: الاتفاق النهائي في المشروع إن الحركة دي **كف واحد مفتوح والصوابع متباعدة، مش إيدين أبداً**. ده هيفرق جامد لما نتكلم عن التسجيلات المستبعدة.

جوه ملف `.npz`

ملف `.npz` هو أرشيف NumPy مضغوط (يعني كذا array محفوظين في ملف واحد). ملف فيه N فريم بيحتوي على:

المفتاح (key)	الشكل (shape)	النوع	المعنى
`image_lm`	(N, 2, 21, 3)	float32	لكل فريم، لكل خانة إيد (2)، لكل نقطة (21): x و y من 0 لـ 1 بالنسبة للصورة، و z نسبي من MediaPipe
`world_lm`	(N, 2, 21, 3)	float32	نفس الـ 21 نقطة بالمتر، في إحداثيات MediaPipe التلاتية "world"
`present`	(N, 2)	float32	1.0 لو فيه إيد اتلقت في الخانة دي، وإلا 0.0 (ساعتها نقط الخانة دي كلها أصفار)
`timestamps`	(N,)	float	الثواني من أول التسجيل
`meta`	()	نص JSON	`label` ، `session` (الـ id)، `person` ، `lighting` ، `aspect` (عرض الصورة على طولها)؛ أدوات التسجيل الموجّهة بتضيف حاجات تانية زي `camera` ، `camera_note` ، `planned_set` ، `recording_session` ، `block` ، `note`

إزاي تقري الـ N : `(N, 2, 21, 3)` shape فريم، في كل فريم خانتين لإيدين، في كل إيد 21 نقطة، ولكل نقطة 3 أرقام. يعني لو التسجيل 350 فريم، الـ array فيه 350 × 2 × 21 × 3 = 44,100 رقم.

مثال من ملف حقيقي (`ram/me_day_20261101_162853.npz`): `image_lm` شكله (350, 2, 21, 3)، و `present` شكله (350, 2)، والـ meta هي `{"camera": "A", "planned_set": "train", "recording_session": 1, "block": 1, "label": "ram", "person": "me", "lighting": "day", "aspect": 1.777...}`.

ليه بنحفظ النقط الخام مش الـ features؟ الـ features (زوايا المفاصل، أماكن أطراف الصوابع بعد التطبيع، هندسة الإيدين مع بعض؛ في `shinobi/features.py`) ممكن تتغير في المستقبل. لأن الملفات حافظة النقط الخام، `train.py` يقدر يعيد حساب الـ features من التسجيلات القديمة في أي وقت. التحويل من إيدين MediaPipe للـ arrays دي بيتعمل بالدالة `shinobi.features.hands_to_arrays` ، وهي نفس الدالة اللي التطبيق الحي بيستخدمها، فبيانات التدريب والبيانات الحية ليهم نفس الشكل بالظبط.

تسجيلات السلاسل الموجّهة (`data/sequences/...` من `record_sequences.py`) فيها نفس المفاتيح، وزيادة عليهم تلات arrays لـ label كل فريم: `frame_phase` (int8، رقم في `lead_in / hold / transition / tail`)، `frame_step` (int8، رقم الخطوة أثناء الـ hold، وإلا -1)، و `frame_sign` (int16، رقم في `config.GESTURES` ؛ `idle` لما الإيدين تحت، و -1 وإحنا بنغير الإيد).

فولدر `data/videos/`

فيه `.mp4` واحد لكل تسجيل، بنفس الـ id ونفس فولدر الـ label بتاع الـ `.npz` . الفيديو بيتصغر لارتفاع 480 بكسل وبيتكتب بحد أقصى 15 مرة في الثانية (عملياً حوالي 10 فريم في الثانية). الفيديوهات ليها وظيفتين:

- `evaluate.py` بيشغل YOLOX عليها، لأن YOLOX محتاج صور، والـ `.npz` فيه نقط بس.
- الناس تقدر تتفرج عليها عشان تتأكد إن التسجيل فعلاً فيه الحركة بتاعته (والـ diagnosis عمل كده بالظبط).

فريم الفيديو بيتكتب في نفس لفة الـ loop اللي بيتحفظ فيها فريم النقط، كل ما يعدي على الأقل 1/15 ثانية من آخر فريم فيديو. `evaluate.py` بيعيد تطبيق القاعدة دي على الـ timestamps المحفوظة عشان يعرف أنهي فريم نقط تبع أنهي فريم فيديو (شوفي `saved_video_frames` تحت).

الملف `data/splits.json`

الملف ده بيقرر كل تسجيل مسموح يتستخدم في إيه. الشكل (format) اسمه `shinobi-splits-v1`:

```
{
  "format": "shinobi-splits-v1",
  "about": "...",
  "quotas": {"single": {"train": 3, "validation": 1, "test": 1},
             "take":   {"train": 0, "validation": 1, "test": 1}},
  "updated": "2026-10-31T02:03:19",
  "sessions": {
    "me_night_20261030_221934": {"set": "train", "kind": "single", "group": "idle",
       "label": "idle", "path": "landmarks/idle/me_night_20261030_221934.npz",
       "person": "me", "lighting": "night", "date": "2026-10-30", "time": "22:19:34",
       "assigned_by": "auto", "assigned_on": "2026-10-31T02:03:19"}, ...
  },
  "ignored": {"me_day_20261030_175555": {"path": "...", "label": "chidori", "reason": "..."}, ...}
}
```

شرح الملف خطوة خطوة:

- `format`: علامة إن ده فعلاً ملف splits. لو مش موجودة أو غلط، `splits.py` بيرفض الملف.
- `quotas`: الحصص (العدد المخطط له). لكل حركة واحدة (`single`): 3 train، 1 validation، 1 test. ولكل jutsu + نوع take: 0 train، 1 validation، 1 test.
- `updated`: آخر مرة الأداة الأوتوماتيك عدلت الملف.
- `sessions`: هنا كل تسجيل ليه entry، والمفتاح هو الـ id بتاعه.
- `ignored`: تسجيلات labels بتاعتها ما بقتش حركات (زي `chidori` و `fire`).

باقي التفاصيل:

- مفتاح كل entry هو الـ id بتاع التسجيل، فالتسجيل بيفضل في مجموعته حتى لو الملف اتنقل.
- `set` قيمته `train` أو `validation` أو `test`. و `kind` قيمته `single` (حركة واحدة متمسكة) أو `take` (سلسلة موجّهة).
- `assigned_by` قيمته `auto` (اتختار بقواعد `splits.py`) أو `manual` (اتحط بالإيد، بـ `--assign` أو `tools/make_test_set.py` أو `tools/assign_recordings.py`).
- الـ entries **ثابتة (sticky)**: أول ما التسجيل ياخد مجموعة، ولا حاجة بتنقله أوتوماتيك.

النهارده الملف فيه 68 entry (اتعدوا من الملف نفسه):

المجموعة	العدد	التفاصيل
train	43	16 `auto` من 2026-10-30، و 27 `manual` من الـ session الموجّهة يوم 2026-11-01
validation	3	`auto`: اتنين `idle` وواحد palm باسمه القديم `rasengan`، كلهم من 2026-10-30
test	22	`manual`، الـ session المستقلة للاختبار يوم 2026-11-01 من الساعة 14:04

وفيه تمن تسجيلات (أربعة `chidori` وأربعة `fire`) في قايمة ignored. وفيه تسجيل واحد على الديسك ملوش مجموعة خالص: الـ snake المرفوض `me_day_20261101_140757` (الإيدين ظاهرين في 0% من الفريمات). لأنه ملوش مجموعة، `train.py` و `evaluate.py` عمرهم ما بيستخدموه.

توزيع التسجيلات على الحركات (من الملف):

الحركة	train	validation	test
`idle`	7	2	4
`ram`	4	0	2
`snake`	4	0	2
`tiger`	4	0	2
`dog`	4	0	2
`monkey`	4	0	2
`boar`	3	0	2
`horse`	3	0	2
`clone_seal`	4	0	2
`rasengan_palm`	6	1	2

ملحوظة من الكود (مش مكتوبة في النسخة الإنجليزي): الحقل `updated` لسه بيقول `2026-10-31T02:03:19` رغم إن 49 تسجيل اتضافوا بالإيد بعد كده. السبب إن الدالة `splits.assign` (اللي بتستخدمها أدوات الإضافة اليدوية) بتحدّث `assigned_on` جوه كل entry بس، وما بتلمسش `updated`. يعني `updated` معناه الحقيقي "آخر تحديث أوتوماتيك"، مش "آخر تعديل في الملف".

أوعي تشغلي `python evaluate.py splits` على بيانات المشروع الحقيقية. الأمر ده بيدي مجموعة أوتوماتيك لكل تسجيل لسه ملوش مجموعة، حسب الـ quotas. مثلاً هيدي الـ snake المرفوض مجموعة (ممكن train أو test)، وهيعيد كتابة الملف. كل التقسيمات الحالية اتعملت عن قصد، بالإيد وبالـ id، وبعد dry run الأول. أي تسجيلات جديدة لازم تتضاف بـ `tools/assign_recordings.py` (لـ train و validation) أو `tools/make_test_set.py` (لـ test)، مش بـ `evaluate.py splits` أبداً. الأمر نفسه مشروح تحت لأنه موجود في الكود.

تحذير مهم لقيناه في الكود: فيه رسايل جوه الكود نفسه بتنصح بتشغيل الأمر الممنوع ده. `evaluate.py run` لما يلاقي تسجيلات من غير مجموعة بيطبع تحذير بيقول "Run `python evaluate.py splits`"، وبما إن الـ snake المرفوض ملوش مجموعة، **كل تشغيل لـ `evaluate.py run` النهارده هيطبع النصيحة دي**. نفس الكلام في `train.py` (بيقول "`python evaluate.py splits` to give them a set, then train again" run"). تجاهلي النصيحة دي تماماً: هي من وقت قبل ما قاعدة "التقسيم بالإيد بس" تتقرر.

الملف `data/evaluation_exclusions.json`

الشكل اسمه `shinobi-exclusions-v1`. بيعدد تسجيلات بتفضل في مجموعتها (ولا حاجة بتتمسح أو تتنقل) لكنها **بتتشال من حساب النتايج** لأن محتواها مش مطابق للـ label بتاعها. كل entry فيه `set`، `label`، `reason`، `evidence` (صورة في فولدر الـ diagnosis)، `decided_on` و `decided_by` (صاحبة المشروع، يوم 2026-11-01، بعد الـ diagnosis). التسجيلات الخمسة المستبعدة كلهم في الـ test set:

التسجيل	الـ label	ليه اتشال
me_day_20261101_140631	ram	جزئي: Ram موجود بس في أول ~9 ثواني، بعدين مفيش إيدين (10–14 ث) وإيد على البق (14–20 ث)؛ اتسجل بنسخة قديمة من أداة التسجيل كانت بتسبق المستخدمة
me_day_20261101_140826	tiger	مفيش Tiger خالص في أي وقت (موبايل في الإيد، ولمس الوش)؛ نفس النسخة القديمة من الأداة
me_day_20261101_141753	idle	كان المفروض علامة OK، بس بيبان فيه كفوف مفتوحة والصوابع متباعدة، يعني حركة Rasengan palm
me_day_20261101_141718	rasengan_palm	كفين مفتوحين في معظم التسجيل (إيدين ظاهرين في 83% من الفريمات)؛ والـ palm المتفق عليه كف واحد مفتوح والصوابع متباعدة
me_day_20261101_142407	rasengan_palm	نفس مشكلة الكفين

`evaluate.py run` بيقرا الملف ده افتراضياً. لسه بيقيّم التسجيلات المستبعدة واحد واحد، بس كل رقم مجمّع بيطلع مرتين: "valid recordings only" (التسجيلات السليمة بس، وده هو النتيجة) و "all recordings" (كل التسجيلات، للمقارنة بس). و `tools/assign_recordings.py` بيرفض يحط أي id مستبعد في train أو validation.

ليه الاستبعاد بالطريقة دي بالذات (مش مسح ولا نقل)؟ لأن الـ test set الأصلي لازم يفضل زي ما هو عشان أي حد يقدر يراجع. والاستبعاد مكتوب بسبب ودليل وتاريخ ومين قرر، فمفيش حاجة مخفية. والأرقام بتطلع بالطريقتين عشان الفرق يبقى واضح.

الكود اللي بيقرا الملف ده في `evaluate.py` صارم جداً:

```
def load_exclusions(path):
    path = Path(path)
    if not path.exists():
        return {}
    try:
        record = json.loads(path.read_text(encoding="utf-8"), object_pairs_hook=_unique_keys)
    except json.JSONDecodeError as e:
        raise ExclusionsError(f"{path} is not valid JSON ({e}). Fix it by hand; nothing was
evaluated.") from e
    ...
    if not isinstance(record, dict) or record.get("format") != EXCLUSIONS_FORMAT:
        raise ExclusionsError(...)
    ...
    for rid, entry in recordings.items():
        missing = [k for k in EXCLUSION_FIELDS if not isinstance(entry.get(k), str) or not
entry[k].strip()]
        if missing:
            raise ExclusionsError(...)
```

بيعمل إيه خطوة خطوة:

1. لو الملف مش موجود، بيرجع `{}` (يعني مفيش استبعاد).
2. بيقرا الـ JSON، والـ `_unique_keys` بيرفض لو نفس الـ id متكرر مرتين.
3. بيتأكد إن `format` هو `shinobi-exclusions-v1`.
4. لكل تسجيل، لازم الحقول `set`, `label`, `reason`, `decided_on`, `decided_by` (دي `EXCLUSION_FIELDS`) يكونوا نصوص مش فاضية، والـ `set` يكون مجموعة صح، والـ `evidence` لو موجود يكون نص.

ليه بالصرامة دي؟ لأن "أنهي تسجيلات بتتحسب" ده قرار ما ينفعش يتخمن. لو الملف بايظ، التقييم كله بيقف برسالة واضحة ومفيش ولا رقم بيطلع. مين بيستخدمه: `cmd_run` من خلال `exclusions_for`.

فولدر `models/`: كل ملف فيه

الملف	هو إيه	مين عمله
`gesture_model.joblib`	الـ bundle بتاع الـ landmark classifier (شوفي تحت)	`train.py`
`training_report.txt`	التقرير النصي اللي `train.py` طبعه لما عمل الموديل المتركب حالياً	`train.py`
`confusion_validation.png`	الـ confusion matrix بتاعة الموديل النهائي على تسجيلات الـ validation	`train.py`
`hand_landmarker.task`	موديل نقط الإيد بتاع MediaPipe (متنزل جاهز)	Google MediaPipe
`selfie_segmenter.tflite`	موديل فصل الشخص عن الخلفية بتاع MediaPipe، بيستخدمه الـ effect Shadow Clone	Google MediaPipe
`yolox_nano.onnx`	الـ YOLOX-nano hand-seal detector المتدرب جاهز، بيشتغل بـ onnxruntime على الـ CPU	طرف تالت
`THIRD_PARTY_YOLOX.md`	موديل YOLOX جاي منين واتراجع إزاي	ملاحظات المشروع
`YOLOX_NARUTO_HandSignDetection_LICENSE.txt`	رخصة موديل YOLOX	طرف تالت

`gesture_model.joblib` عبارة عن Python dictionary محفوظ بـ `joblib` (مكتبة بتحفظ objects بايثون في ملف وترجعها تاني). لما بيتدرب بوجود `data/splits.json` بيحتوي على:

- `model`: موديل scikit-learn المتدرب (يا إما `RandomForestClassifier`، يا إما pipeline فيه `StandardScaler` + `MLPClassifier`)؛
- `model_name` (`random_forest` أو `mlp`)، و `labels` (أسماء الحركات بنفس ترتيب classes الموديل)، و `feature_version` (لازم يطابق `shinobi.features.FEATURE_VERSION`)؛
- `selection` (نوع الموديل اتختار إزاي: `validation` أو `cv` أو `default`)، و `selection_macro_f1`، و `cv_macro_f1`، و `validation_macro_f1`؛
- `splits_file`، و `train_sessions`، و `validation_sessions` (الـ ids بالظبط اللي اتعلم منها واتختار بيها)، و `held_out_person`، و `created`.

القوايم `train_sessions` و `validation_sessions` هي **حارس الـ leakage**: `evaluate.py` و `tools/make_test_set.py` بيقروهم ويرفضوا يقيّموا الموديل على تسجيل هو اتعلم منه.

اتأكدنا من محتوى الموديل المتركب النهارده بقراية الملف بس (من غير أي تدريب):

الحقل	القيمة
نوع الموديل	`RandomForestClassifier`
`model_name`	`random_forest`

الحقل	القيمة
`created`	`2026-10-31T14:36:03`
`selection`	`default`
`validation_macro_f1`	1.0
`cv_macro_f1` و `selection_macro_f1`	فاضيين `{}`
`feature_version`	1
عدد `train_sessions`	16
`validation_sessions`	`me_day_20261030_175251`، `me_night_20261030_222524`، `me_night_20261030_222754`
md5 الملف	`be8485245e0c6cc959b8f4c020fc7d60` (بيبدأ بـ `be848524`)

ده كان الموديل الأصلي: Random Forest، اتعمل 2026-10-31T14:36:03، واتدرب على 16 تسجيل، `selection: default`، والـ validation macro-F1 بتاعه 1.000. بس خلي بالك: الـ validation كان فيه بس فريمات `idle` و palm، فالـ 1.000 ده ما بيقولش أي حاجة عن الأختام السبعة. **تحديث:** من يوم 2026-11-01 الموديل الـ production بقى **MLP** (md5 `ed330ca7…`)، والـ Random Forest ده محفوظ في `models/previous_production_20261101/` عشان لو حبينا نرجعله.

ملحوظة: `GestureClassifier` في `shinobi/classifier.py` لما بيحمّل الموديل بيقارن `feature_version` بتاع الملف بـ `FEATURE_VERSION` في الكود، ولو مختلفين بيرفض ويقول "Re-run train.py". ده بيمنع إن موديل متدرب على features قديمة يشتغل مع كود features جديد.

الملف `requirements.txt`

ده ملف الـ dependencies (المكتبات اللي المشروع محتاجها)، بإصدارات مثبتة لـ Python 3.12. الفكرة إنك لما تعملي `pip install -r requirements.txt` تنزلي نفس الإصدارات بالظبط اللي المشروع اتجرب عليها.

الـ package	ليه
`mediapipe==1.2.0`	نقط الإيد وفصل الشخص عن الخلفية؛ وهو بينزّل `-opencv-contrib` `python` بنفسه
`opencv-contrib-python==5.0.0.93`	الكاميرا، الرسم، ملفات الفيديو (ما تنزليش `opencv-python` كمان: الاتنين بيوفروا `cv2` وبيبوظوا بعض)
`numpy==2.5.4`	الـ arrays في كل حتة
`scikit-learn==1.9.1`	Random Forest، MLP، المقاييس (metrics)، و `StratifiedGroupKFold`
`onnxruntime==1.31.1`	بيشغل الـ YOLOX detector على الـ CPU
`fastapi==0.162.0`، `uvicorn==0.54.0`، `websockets==17.4`	سيرفر الويب لتطبيق React
`httpx==0.28.1`	للاختبارات بس (الـ TestClient بتاع FastAPI)
`joblib`، `matplotlib`، `pytest`	ملفات الموديل، رسومات التقارير، الاختبارات

ملاحظة مهمة: `tools/check_voices.py` و `tools/build_bold_signs.py` بيعملوا import لـ `scipy`، و `tools/build_gesture_references.py` بيعمل import لـ `PIL` (يعني Pillow). **الاتنين مش مكتوبين في `requirements.txt`.** اتأكدنا إنهم موجودين فعلاً في الـ `.venv` بتاع المشروع (scipy 1.18.1 و Pillow 13.0.0)، والسبب إن `scikit-learn` بيعتمد على `scipy` و `matplotlib` بيعتمد على Pillow، فبينزلوا معاهم أوتوماتيك. بس ده اعتماد غير مباشر: لو حد في يوم نزّل المكتبات بطريقة تانية ومالقاهمش، لازم ينزلهم بنفسه (`pip install scipy pillow`). والاختبار `tests/test_catalog.py` بيعمل `pytest.importorskip("PIL")`، يعني لو Pillow مش موجود، اختبار الكروت بيتعمل له skip بدل ما يفشل.

الملف `README.md`

دي الصفحة الأولى القصيرة للمشروع بالإنجليزي. بتشرح الـ pipeline في رسمة واحدة:

```
camera → MediaPipe hands → features (206) → classifier (MLP / RF) → controller → effect
```

وبتشرح الأفكار الأساسية: classifier بيشتغل على كل فريم لوحده + controller مبني على قواعد بدل LSTM (يعني الـ jutsu الجديدة بتتعرف في `config.py` من غير إعادة تدريب)؛ الـ cross-validation متقسم بالتسجيل؛ والـ mirror والـ jitter augmentation جوه الـ training folds بس. وفيها quick start لـ Windows بـ Python 3.12 (اعمل `.venv`، نزّل الـ requirements، شغّل `shinobi.main`، و `collect_data.py`، و `train.py`، و `pytest`)، وبتشاور على الشرح العربي الكامل في `docs/`.

ملاحظات من مقارنة الـ README بالكود:

- رقم 206 في الرسمة صح: في `shinobi/features.py`، `PER_HAND_SIZE = 1 + 63 + 15 + 10` يعني 89، و `INTER_HAND_SIZE = 1 + 2 + 25` يعني 28، و `FEATURE_SIZE` = 2 × 89 + 28 = 206.
- الرسمة بتقول `classifier (MLP / RF)`، والموديل المتركب فعلياً دلوقتي هو **MLP** (من 2026-11-01). الـ Random Forest القديم محفوظ كـ backup.
- الـ quick start بيشغل `train.py` كخطوة عادية، بس النهارده تشغيل `train.py` من غير `--out` **بيستبدل الموديل المتركب**، وده محتاج موافقة صاحبة المشروع الأول (شوفي قسم "إزاي تشغلي كل حاجة").
- الـ quick start ما بيذكرش السيرفر (`python -m shinobi.server`)، رغم إن الهدف النهائي للمشروع تطبيق ويب. الأوامر الكاملة في آخر الجزء ده.

الإعدادات (Configuration) اللي ليها علاقة بالجزء ده

كل الإعدادات في `shinobi/config.py`. دي اللي بتأثر على البيانات والتدريب والتقييم (القيم منقولة من الكود بالحرف):

الإعداد	القيمة	معناه ومين بيستخدمه
`DATA_DIR`	`ROOT / "data"`	فولدر البيانات
`LANDMARKS_DIR`	`DATA_DIR / "landmarks"`	فولدر ملفات `.npz`، الافتراضي لـ `train.py --data`
`VIDEOS_DIR`	`DATA_DIR / "videos"`	فولدر الفيديوهات

الإعداد	القيمة	معناه ومين بيستخدمه
`SEQUENCES_DIR`	`DATA_DIR / "sequences"`	تسجيلات السلاسل الموجّهة
`SEQUENCE_VIDEOS_DIR`	`VIDEOS_DIR / "sequences"`	فيديوهات السلاسل
`GESTURE_MODEL_PATH`	`MODELS_DIR / "gesture_model.joblib"`	مكان الموديل، الافتراضي لـ `train.py --out` و `evaluate.py --model`
`GESTURES`	العشر حركات	الـ labels بالترتيب
`LEGACY_LABELS`	`{"rasengan": "rasengan_palm"}`	ترجمة الأسماء القديمة
`LANDMARK_SIGNS`	`["clone_seal", "rasengan_palm"]`	الحركات اللي الـ landmark model مسؤول عنها في التطبيق
`RECORD_SECONDS`	20	مدة كل تسجيل
`COUNTDOWN_SECONDS`	3	العد التنازلي في `collect_data.py`
`SEQ_HOLD_SECONDS`، `SEQ_TRANSITION_SECONDS`، `SEQ_LEAD_SECONDS`	1.5 لكل واحد	توقيتات `record_sequences.py`
`SEAL_MIN_SCORE`	0.70	أقل score لـ YOLOX لما الـ box بتاعه على إيد MediaPipe شايفها
`SEAL_CONFIRM`	2	نفس الختم لازم يطلع في نتيجتين YOLOX ورا بعض
`SEAL_RATE_HZ`	10	أقصى عدد مرات يشتغل فيها YOLOX في الثانية
`LANDMARK_MIN_PROB`	0.60	أقل احتمال من الـ landmark model عشان يقبل `clone_seal` أو `rasengan_palm`
`LANDMARK_MIN_MARGIN`	0.30	الفرق المطلوب بين أعلى احتمال واللي بعده
`PALM_MIN_THUMB_GAP`	0.60	شرط فتحة الإبهام في فحص الكف المفتوح
`SEQUENCE_MAX_GAP`	7.0	أقصى ثواني بين خطوتين في الـ Live mode

القيم دي **ما اتغيرتش** في أي خطوة من خطوات التقييم والتشخيص. أي تغيير فيها لازم يتظبط على الـ validation أو cross-validation على الـ train، مش على الـ test.

الملف `collect_data.py`

مسؤول عن إيه: ده أداة التسجيل الأصلية البسيطة. بيفتح الكاميرا، يخليكي تختاري الحركة بزرار رقم، وكل ما تدوسي SPACE يسجل session واحدة مدتها 20 ثانية.

ليه اتعمل: عشان نجمع بيانات خام (نقط الإيد + فيديو) لكل حركة بأبسط طريقة. هو كمان المرجع لشكل الملفات: كل أدوات التسجيل التانية بتحفظ "زي `collect_data.py`" بالظبط.

مكانه في السيستم: في أول الخط خالص. مخرجاته (ملفات `.npz` و `.mp4`) هي مدخلات `splits.py` و `train.py` و `evaluate.py`. والأدوات الموجّهة بتعيد استخدام دالتين منه.

التشغيل: `python collect_data.py --person sara --lighting day`. الاختيارات:

الاختيار	الافتراضي	معناه
`--person`	`me`	مين بيسجل؛ بيتستخدم بعدين عشان نفرق بين الناس
`--lighting`	`normal`	الإضاءة، مثلاً day أو night أو lamp
`--seconds`	`config.RECORD_SECONDS` = 20	مدة التسجيل
`--no-video`	مش مفعّل	ما يحفظش فيديو المراجعة

الزرار: من 0 لـ 9 تختاري الحركة (رقمها في `config.GESTURES`، يعني 0 = `idle`، 1 = `ram`، ... 9 = `rasengan_palm`)، و SPACE يبدأ تسجيل، و Q (الحرف الصغير `q` في الكود) أو Esc يقفل.

الدوال الأساسية

- `count_sessions()`: بيعد ملفات `.npz` لكل حركة، ومعاها الملفات اللي بأسماء قديمة (`LEGACY_LABELS`)، عشان القايمة اللي على الشاشة. يعني `rasengan_palm` بيتعد معاه ملفات فولدر `rasengan`.
- `load_sign_pictures(height=220)`: بيحمّل الرسمة المرجعية لكل حركة من `gesture_references/_sources/signs/<label>.png` ويصغرها لركن الشاشة (`idle` ملوش صورة). و `paste_picture(view, picture)` بيرسمها فوق على اليمين على كارت أبيض. الاتنين بيتستخدموا تاني في أدوات التسجيل الموجّهة.
- `save_session(label, args, frames, video_frames, aspect)`: بيبني الـ id بالشكل `<person>_<lighting>_<YYYYMMDD_HHMMSS>`، يكتب `data/landmarks/<label>/<id>.npz` بـ `np.savez_compressed`، يكتب الفيديو بـ `cv2.VideoWriter` (mp4v، 15 fps)، ويطبع كام فريم كان فيه إيدين.
- `main()`: عبارة عن state machine صغيرة (آلة حالات: البرنامج بيبقى في حالة واحدة في كل وقت وبينتقل بينها): `idle` ثم `countdown` (`config.COUNTDOWN_SECONDS` = 3) ثم `recording` وبعدين يرجع `idle`. أثناء التسجيل كل فريم جديد من الكاميرا بيروح لـ `HandTracker.detect`، ويتحول بـ `hands_to_arrays`، ويتضاف مع الـ timestamp بتاعه؛ وفريم فيديو 480p بيتحفظ بحد أقصى كل 1/15 ثانية.

دالة الحفظ بالتفصيل

```
def save_session(label, args, frames, video_frames, aspect):
    session_id = f"{args.person}_{args.lighting}_{datetime.now():%Y%m%d_%H%M%S}"
    out_dir = config.LANDMARKS_DIR / label
    out_dir.mkdir(parents=True, exist_ok=True)
    np.savez_compressed(
        out_dir / f"{session_id}.npz",
        image_lm=np.stack([f[0] for f in frames]),
        world_lm=np.stack([f[1] for f in frames]),
        present=np.stack([f[2] for f in frames]),
        timestamps=np.array([f[3] for f in frames]),
        meta=json.dumps({"label": label, "session": session_id, "person": args.person,
                         "lighting": args.lighting, "aspect": aspect}),
    )
```

بيعمل إيه خطوة خطوة:

1. يبني الـ id من اسم الشخص والإضاءة والتاريخ والساعة لحد الثانية. مثلاً `me_day_20261101_162853`.

2. يتأكد إن فولدر الـ label موجود (ولو مش موجود يعمله).
3. كل عنصر في `frames` عبارة عن tuple من 4 حاجات: `(image_lm, world_lm, present, timestamp)`.
   `np.stack` بيرص الفريمات فوق بعض فيطلع array شكله `(N, 2, 21, 3)`.
4. الـ `meta` بيتحفظ كنص JSON جوه نفس الملف.

بعد كده (في باقي الدالة) بيكتب الفيديو لو فيه فريمات فيديو، ويطبع سطر زي `saved ram/me_day_...: 350 frames, 340 with hands`.

ليه بالشكل ده؟ عشان كل تسجيل يبقى ملف واحد مستقل فيه كل حاجة (البيانات + معلوماتها)، والـ id فيه التاريخ والإضاءة، ودول بالظبط اللي `splits.py` بيستخدمهم بعدين عشان يعرف "ظروف" كل تسجيل. ملحوظة من الكود: `save_session` ما بتتأكدش إن الملف مش موجود قبل ما تكتب؛ الأدوات الأحدث (`save_new` في `tools/record_test_session.py`) اتعملت عشان تمنع الكتابة فوق أي ملف موجود.

تفاصيل مهمة

- الزراير بتتقرا في أول كل لفة من الـ loop، قبل ما يستنى فريم، فزرار اتداس والكاميرا بطيئة ما بيضيعش.
- العد التنازلي ما بيتحفظش. وقت التسجيل 0 هو أول فريم اتسجل. عشان كده `evaluate.py` بيتخطى أول 0.5 ثانية بس.
- الشاشة بتقولك "move, rotate, change distance" وإنتي بتسجلي، فالـ session الواحدة بتغطي شوية تنوع (زوايا ومسافات مختلفة).

مثال عملي

عايزة تسجلي `tiger` بالنهار: شغلي `python collect_data.py --person me --lighting day`، دوسي 3 (رقم `tiger` في `GESTURES`)، دوسي SPACE، استني 3 ثواني عد تنازلي، اعملي الحركة وحركي إيدك شوية لمدة 20 ثانية. هيتحفظ `data/landmarks/tiger/me_day_<التاريخ_الساعة>.npz` و `data/videos/tiger/me_day_<...>.mp4`. التسجيل الجديد ده **مش هيدخل في أي مجموعة** لحد ما تضيفيه بالإيد بالأداة المناسبة، فـ `train.py` و `evaluate.py` هيتجاهلوه لحد ساعتها.

الربط بين البيانات الخام والـ features (Preprocessing و Feature Extraction)

Preprocessing يعني تجهيز البيانات الخام قبل ما تدخل الموديل، و **Feature Extraction** يعني استخراج الأرقام المفيدة منها. في مشروعنا الاتنين بيتعملوا في `shinobi/features.py` (الشرح الكامل في الجزء الخاص بالـ features)، بس لازم تفهمي الرابط هنا.

السلسلة كاملة:

1. MediaPipe بيدي `Hand` objects لكل فريم.
2. `hands_to_arrays(hands)` بيحولهم لـ `image_lm (2,21,3)`، `world_lm (2,21,3)`، `present (2,)`، وده الشكل اللي بيتحفظ في `.npz`.
3. `extract(image_lm, world_lm, present, aspect)` بيحولهم لـ feature vector طوله 206.
4. `mirror(image_lm, world_lm, present)` بيعمل نسخة معكوسة (للـ augmentation).

```
PER_HAND_SIZE = 1 + 63 + 15 + 10      # present flag + coords + angles + tip distances
INTER_HAND_SIZE = 1 + 2 + 25          # both flag + wrist offset + 5x5 tip distances
FEATURE_SIZE = 2 * PER_HAND_SIZE + INTER_HAND_SIZE
```

معنى القيم: لكل إيد 89 رقم: علامة "الإيد موجودة" (1)، والـ 21 نقطة × 3 إحداثيات بعد ما نخلي الرسغ هو نقطة الصفر ونقسم على حجم الإيد (63)، و15 زاوية مفصل (3 لكل صباع)، و10 مسافات بين أطراف الصوابع الخمسة. وبين الإيدين 28 رقم: علامة "الإيدين موجودين" (1)، ومكان رسغ الإيد التانية بالنسبة للأولى (2)، و 5 × 5 مسافات بين أطراف صوابع الإيدين (25). المجموع 89 + 89 + 28 = 206.

ليه التطبيع ده مهم؟ لأن الرسغ بيبقى الأصل والإيد بتتقسم على حجمها، فالـ features ما بتتأثرش بمكان الإيد في الصورة ولا بقربها أو بعدها عن الكاميرا. والاختبار `test_features_ignore_position_and_scale_of_world_coords` في `tests/test_core.py` بيتأكد من ده.

نفس الدوال دي بالظبط بيستخدمها `train.py` (وقت التدريب) والتطبيق الحي (وقت التشغيل) و `evaluate.py` (وقت القياس)، فمفيش فرق بين اللي الموديل اتعلمه واللي بيشوفه في الحقيقة. و `FEATURE_VERSION = 1` بيتحفظ مع الموديل عشان لو حد غير الـ features في يوم، الموديل القديم يترفض.

الملف `record_sequences.py`

مسؤول عن إيه: تسجيل (أو مجرد تمرين على) **سلاسل jutsu** موجّهة من 4 حركات، ومعاها سلاسل غلط عن قصد، عشان الـ controller يتختبر بعدين على سلاسل حركات حقيقية.

ليه اتعمل: الـ controller بيتعرف على الـ jutsu من ترتيب الحركات. عشان نقيسه بجد، محتاجين تسجيلات فيها شخص بيعمل السلسلة كاملة بالترتيب، وكمان تسجيلات غلط (ترتيب غلط، ناقصة، فيها حركة غلط) عشان نتأكد إن مفيش jutsu بيشتغل من غير ما يتطلب.

التشغيل: `python record_sequences.py --person me --lighting day`. الاختيارات: `--hold`، `--transition`، `--lead` (بالثواني، والافتراضي `config.SEQ_HOLD_SECONDS` و `SEQ_TRANSITION_SECONDS` و `SEQ_LEAD_SECONDS`، كلهم 1.5)، و `--no-video`.

الزراير:

الزرار	بيعمل إيه
1–4	تختاري الـ jutsu (1 Fire Style، 2 Chidori، 3 Shadow Clone، 4 Rasengan)
TAB	يغير نوع الـ take: `correct`، `wrong_order`، `incomplete`، `wrong_sign`
R	يسحب حركات عشوائية جديدة لـ take سلبي
SPACE	يسجل take
P	تمرين (مفيش حاجة بتتحفظ)
`[` و `]`	أبطأ أو أسرع
C	يلغي take شغال
Q	يقفل

السلاسل نفسها جاية من `config.JUTSU`:

الـ jutsu	السلسلة
`fire_style`	monkey، boar، horse، tiger
`chidori`	snake، dog، tiger، monkey
`shadow_clone`	ram، snake، tiger، clone_seal
`rasengan`	clone_seal، dog، horse، rasengan_palm

القطع الأساسية

- `Segment` (dataclass، يعني class بسيطة لتخزين بيانات): `phase` (`sign`، `step`، `start`، `end`).
- `build_timeline(signs, hold, transition, lead)`: الإيدين تحت، ثم حركة 1، ثم تغيير، ثم حركة 2، ... ، آخر حركة، ثم الإيدين تحت. و `segment_at(timeline, t)` بيرجع الـ segment اللي شغال في الوقت t (و None في الآخر).
- `fires(signs)`: بيشغل الـ `SequenceMatcher` الحقيقي بتاع التطبيق على قايمة حركات ويرجع الـ jutsu اللي كان هيشتغل.
- `make_signs(kind, jutsu_key, rng)`: لـ `correct` بيرجع سلسلة الـ jutsu؛ وللأنواع السلبية بيلخبط الترتيب، أو يقطع السلسلة بعد 2 أو 3 خطوات، أو يبدل حركة واحدة، و**يعيد السحب لحد 500 مرة لحد ما `fires()` يرجع ولا حاجة**. فالـ take السلبي عمره ما يكمّل jutsu بالصدفة.
- `frame_labels(segments)`: بيبني arrays لكل فريم: `frame_phase`، `frame_step`، `frame_sign`؛ و `save_take(path, frames, meta)` بيكتبهم مع النقط.
- `take_quality(frames, signs)`: نسبة فريمات الـ hold اللي فيها أي إيد، ونسبة اللي فيها الإيدين للحركات اللي محتاجة إيدين (كل الحركات ما عدا `rasengan_palm`).
- دوال رسم: `draw_strip` (كروت الحركات تحت؛ اللي خلصت باهتة، والحالية بإطار أخضر)، `draw_timeline` (شريط ملون للمراحل وعليه مؤشر بيتحرك)، `banner`، `prompt_text`.

```
def make_signs(kind, jutsu_key, rng, jutsu=config.JUTSU):
    seq = list(jutsu[jutsu_key]["sequence"])
    if kind == "correct":
        return seq
    vocabulary = sorted({s for spec in jutsu.values() for s in spec["sequence"]})
    for _ in range(500):
        if kind == "wrong_order":
            signs = [str(s) for s in rng.permutation(seq)]
        elif kind == "incomplete":
            signs = seq[:int(rng.integers(2, len(seq)))]          # stops after 2 or 3 steps
        elif kind == "wrong_sign":
            signs = seq.copy()
            i = int(rng.integers(len(seq)))
            signs[i] = str(rng.choice([s for s in vocabulary if s != seq[i]]))
        else:
            raise ValueError(f"unknown kind {kind}")
        if signs != seq and not fires(signs, jutsu):
            return signs
    raise RuntimeError(f"could not build a '{kind}' take for {jutsu_key}")
```

بيعمل إيه خطوة خطوة: بياخد سلسلة الـ jutsu الصح. لو النوع `correct` يرجعها على طول. لو سلبي، بيعمل محاولة: `wrong_order` = نفس الحركات بترتيب عشوائي؛ `incomplete` = أول 2 أو 3 حركات بس؛ `wrong_sign` = يبدل حركة واحدة بحركة تانية من كل الحركات اللي بتستخدمها الـ jutsu. بعدين يتأكد من شرطين: إن السلسلة فعلاً مختلفة عن الصح، وإن الـ matcher الحقيقي ما بيشغلش أي jutsu بيها. لو الشرطين اتحققوا يرجعها، وإلا يحاول تاني (لحد 500 مرة)، ولو فشل خالص يرمي `RuntimeError`. ليه؟ لأن مثلاً ترتيب عشوائي ممكن بالصدفة يطلع نفس الترتيب الصح، أو حركة مبدلة ممكن تكمّل jutsu تانية. مين بيستخدمها: `main()` في نفس الملف، والاختبار `test_generated_takes_fire_only_when_they_should`.

الـ meta المحفوظة (`format: shinobi-sequence-v1`) فيها الـ timeline كله، والحركات، والنوع، و `expected_activation` (الـ jutsu لو الـ take صح، و `None` للسلبي). والـ id بتاع الـ take شكله `<jutsu>_<kind>_<person>_<lighting>_<YYYYMMDD_HHMMSS>`. الـ docstring بيحذر إن الناس بتتفاعل بعد 0.3–0.8 ثانية من ظهور الطلب، فالكود اللي بيستخدم الـ labels دي لازم يثق في النص التاني من كل فترة hold أكتر من الأول. الفيديو بيتكتب وإحنا بنسجل (مفيش تراكم في الذاكرة)، والـ take اللي اتلغى أو اتقطع بيتمسح الفيديو الناقص بتاعه.

الحالة: ما اتسجلش ولا take لسه (`data/sequences/` مش موجود)، فتقييم السلاسل حالياً بيستخدم تسجيلات حركة واحدة متقطعة ومتلزقة ورا بعض بدلاً منها (`tools/evaluate_pipeline.py sequences`). يعني الأداة **implemented ومتختبرة** (فيه اختبارات في `tests/test_sequences.py`)، بس **ما اتستخدمتش لجمع بيانات فعلية**.

الملف `splits.py`

مسؤول عن إيه: ده المكتبة اللي ورا `data/splits.json`. بيعمل scan للتسجيلات، يقرا ويكتب الملف بأمان، يدي التسجيلات الجديدة مجموعات بقواعد واخدة بالها من الـ leakage، ويطبع جدول حالة. عمره ما بيكتب أو ينقل أو يمسح تسجيل؛ بيكتب `data/splits.json` بس.

ليه اتعمل: عشان قرار "التسجيل ده تبع أنهي مجموعة" يبقى مكتوب في مكان واحد، وثابت، ومحمي من الأخطاء، ومش بيتعمل بالفريم.

مين بيعتمد عليه: `train.py` (بيقرا المجموعات)، `evaluate.py` (الأمرين `splits` و `run`)، `tools/assign_recordings.py`، `tools/make_test_set.py`، `tools/grouped_cv.py`، `tools/evaluate_pipeline.py`، والاختبارات في `tests/test_splits.py`.

الثوابت

الثابت	القيمة	معناه
`SETS`	`("train", "validation", "test")`	المجموعات المسموحة
`FILL_ORDER`	`("train", "test", "validation")`	ترتيب ملء الحصص الدنيا
`FORMAT`	`"shinobi-splits-v1"`	علامة شكل الملف
`QUOTAS`	single: 3 train / 1 validation / 1 test لكل حركة؛ take: 0 / 1 / 1 لكل jutsu + نوع	الحصص الافتراضية. الـ takes ليها 0 train لأن `train.py` ما بيدربش على takes

الـ dataclass `Recording`

فيه: `id`، `kind` (`single` أو `take`)، `group` (الحركة بعد `LEGACY_LABELS`، أو `<jutsu>/<kind>` للـ take)، `label`، `path`، `person`، `lighting`، `date`، `time`، `signs`. وفيه property اسمها `condition` قيمتها `(date, lighting)`: تسجيلين ليهم نفس الـ condition يبقوا اتعملوا في نفس اليوم وبنفس الإضاءة.

```
@property
def condition(self):
    """(date, lighting): two recordings with the same condition were made the same day in the same light."""
    return (self.date, self.lighting) if self.date and self.lighting else None
```

بيعمل إيه: بيرجع (التاريخ، الإضاءة) كمفتاح واحد، أو `None` لو واحد منهم مش معروف. ليه؟ لأن قاعدة الـ test الأساسية بتقول إن تسجيل الـ test لازم يكون من "ظروف" مختلفة عن تسجيلات الـ train لنفس الحركة. والـ condition ده هو تعريف "الظروف" في الكود.

القراية

- `read_meta(path)`: بيفتح entry الـ `meta` الصغير بس (الفريمات ما بتتفكش من الضغط)، فالـ scan سريع حتى مع ملفات كتير.
- `_when(stem, meta)`: بياخد التاريخ والوقت من آخر الـ id بالشكل `_YYYYMMDD_HHMMSS` (أو من `created` للـ takes).
- `scan(data_dir)`: بيعدد كل تسجيل في `landmarks/*/*.npz` و `sequences/*/*.npz`. الملف البايظ بيبقى note (ملاحظة) مش crash. الـ label اللي ما بقاش حركة بيروح لـ `ignored`. لو الـ label المتخزن في الملف مختلف عن اسم الفولدر، الـ label اللي في الملف هو اللي بيكسب (زي `train.py`) وبتتطبع note. ولو فيه ملفين بنفس الـ id بيرمي `SplitsError`، لأن الـ id هو المفتاح.

الملف نفسه

- `load(path)`: بيرجع None لو الملف مش موجود؛ بيرفض JSON غلط ويرفض مفتاح متكرر مرتين (`_no_duplicate_keys`)، و `check()` بيتأكد من الـ format وإن كل `set` قيمته صح.
- `save(splits, path)`: بيعمل check، يكتب في `splits.json.tmp`، وبعدين `os.replace` يبدّله مكان الأصلي: يعني لو الجهاز وقع في النص، عمره ما هيسيب نص ملف.
- `set_of(splits)`: بيرجع `{id: set}`.

```
def save(splits, path=DEFAULT_PATH):
    """Write the file in one go (a crash never leaves half a file)."""
    path = Path(path)
    check(splits, str(path))
    tmp = path.with_name(path.name + ".tmp")
    tmp.write_text(json.dumps(splits, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")
    os.replace(tmp, path)
```

بيعمل إيه خطوة خطوة: (1) يتأكد إن المحتوى سليم قبل أي كتابة؛ (2) يكتب المحتوى كله في ملف مؤقت جنبه؛ (3) `os.replace` بيبدل الملف القديم بالجديد في خطوة واحدة على مستوى نظام التشغيل. النتيجة: يا إما الملف

القديم كامل، يا إما الجديد كامل. ده مهم جداً لملف واحد بيتحكم في كل التقسيم.

التقسيم (للتسجيلات اللي ملهاش مجموعة بس)

- `conditions(sessions, by_id)`: بيجمع، لكل (kind, group)، الـ conditions الموجودة في كل مجموعة.
- `train_conditions(...)`: بيدي الـ conditions (اليوم + الإضاءة) اللي تسجيل TEST لازم يتجنبها: اللي بتاعة تسجيلات train من نفس الـ group، ولو take كمان اللي بتاعة train sessions للحركات بتاعته.
- `choose_set(rec, conds, quota)`: بيطبق القواعد. الـ train مسموح بس لو condition التسجيل مش مستخدم في تسجيل test من نفس الـ group؛ والـ test مسموح بس لو الـ condition معروف ومش مستخدم في train. بعدين بيملا الحصص الدنيا بالترتيب train ثم test ثم validation؛ وبعد كده المجموعة اللي أبعد حاجة عن نسبتها هي اللي بتاخده.

```
allowed = {
    "train": c is None or c not in set(mine["test"]),       # never train on a test day + light
    "test": c is not None and c not in train_conditions(rec.kind, rec.group, rec.signs, conds),
    "validation": True,
}
n = {s: len(mine[s]) for s in SETS}
for s in FILL_ORDER:                                        # minimum quotas: train, test, validation
    if allowed[s] and n[s] < quota[s]:
        return s
```

بيعمل إيه خطوة خطوة:

1. `c` هو condition التسجيل الجديد (اليوم + الإضاءة).
2. `allowed["train"]`: ينفع يروح train لو الـ condition مش معروف، أو لو مفيش تسجيل test من نفس الحركة اتعمل في نفس اليوم ونفس الإضاءة.
3. `allowed["test"]`: ينفع يروح test بس لو الـ condition معروف، ومش موجود في ظروف تسجيلات الـ train لنفس الحركة.
4. `allowed["validation"]`: دايماً مسموح.
5. `n` عدد التسجيلات الموجودة في كل مجموعة للحركة دي.
6. يلف على الترتيب train ثم test ثم validation: أول مجموعة مسموحة ولسه ما وصلتش لحصتها تاخد التسجيل.

مثال عملي: عندنا حركة `dog` فيها 3 تسجيلات train كلهم `night` يوم 30-10-2026. اتسجل `dog` جديد `day` يوم 01-11-2026. الـ train وصل لحصته (3)، فبيبص على test: الـ condition `("2026-11-01", "day")` مش في ظروف الـ train، فمسموح، والـ test لسه 0 من 1، فالتسجيل يروح test. لو كان اتسجل `night` يوم 30-10-2026 كمان، كان test مش مسموح، فكان هيروح validation.

- `update(existing, recordings, ignored, quotas)`: بينسخ كل entry موجود زي ما هو ("التعديلات اليدوية بتكسب")، يدي مجموعات للجديد من الأقدم للأحدث، يرتب الـ entries، ويرجع `(new splits, ids assigned now)`.
- `assign(splits, recordings, rid, set_name)`: بيحط تسجيل واحد في مجموعة بالإيد (`assigned_by:manual`) ويرجع المجموعة اللي كان فيها قبل كده. (ملحوظة: الدالة دي بتحدّث `assigned_on` بس، ما بتحدّثش

`updated` بتاع الملف.)

المراجعة والطباعة

- `review(splits, recordings)`: بيرجع صفوف وتحذيرات لكل group: حصص ناقصة، تسجيل test بيشارك يوم + إضاءة مع بيانات التدريب ("test numbers will look better than reality")، تسجيلات من غير يوم أو إضاءة معروفة، و entries الملف بتاعها اختفى.
- `table(...)`: بيبني النص اللي `python evaluate.py splits` بيطبعه.

ليه "الـ test من يوم أو إضاءة تانية"؟ تسجيل test اتعمل في نفس القعدة مع تسجيلات التدريب بيقيس قد إيه الموديل فاكر القعدة دي، مش قدرته إنه يشتغل في ظروف جديدة.

ملحوظة عن الـ docstring بتاع `splits.py`: مكتوب فيه إن التشغيل بيتعمل بـ `python evaluate.py splits`. ده كان صح وقت ما اتكتب، بس قرار المشروع الحالي إن الأمر ده ما يتشغلش على البيانات الحقيقية (شوفي فوق). المكتبة نفسها لسه هي اللي بتستخدمها الأدوات اليدوية الآمنة.

الملف `train.py`

مسؤول عن إيه: يدرب الـ landmark classifier ويحفظه كـ `models/gesture_model.joblib`، ومعاه `models/training_report.txt` وصورة confusion matrix.

ليه اتعمل: عشان التدريب يبقى أمر واحد، بنفس الطريقة كل مرة، ويحترم التقسيم بالتسجيل، ويحفظ مع الموديل "هو اتعلم من إيه" عشان الـ leakage يتمنع أوتوماتيك.

مكانه في السيستم: بياخد الملفات من `data/landmarks/` والتقسيم من `data/splits.json`، ويطلع الموديل اللي `shinobi/classifier.py` بيحمله في التطبيق. والأدوات `tools/grouped_cv.py` و `tools/evaluate_pipeline.py` بيعيدوا استخدام دواله.

التشغيل: `python train.py`. الاختيارات:

الاختيار	الافتراضي	معناه
`--data`	`data/landmarks` (`config.LANDMARKS_DIR`)	فولدر التسجيلات
`--out`	`models/gesture_model.joblib`	مكان حفظ الموديل
`--step`	2	استخدم كل n فريم (2 = فريم آه وفريم لأ)
`--test-person NAME`	مفيش	كمان سيب شخص واحد بره التدريب خالص
`--splits`	`splits.json` جنب فولدر البيانات	ملف التقسيم

لو `data/splits.json` موجود (وهو موجود)، بيتستخدم `train_with_splits`. من غيره، بيشتغل الوضع القديم "all sessions".

ملحوظة مهمة من الكود: `training_report.txt` و `confusion_validation.png` بيتكتبوا **في نفس فولدر ملف `--out`** (`out_path.parent`). يعني لو شغلتي `train.py --out C:\path\outside\repo\candidate.joblib`، التقرير والصورة هيتكتبوا جنب الـ candidate بره المشروع، والموديل المتركب وتقريره في `models/` مش هيتلمسوا.

تحميل الفريمات

```
def load_sessions(data_dir, step, only=None):
    for path in sorted(Path(data_dir).glob("*/*.npz")):
        if only is not None and path.stem not in only:
            continue                                  # other files are not even opened
        d = np.load(path)
        meta = json.loads(str(d["meta"]))
        meta["label"] = config.LEGACY_LABELS.get(meta["label"], meta["label"])
        if meta["label"] not in config.GESTURES:
            continue                                  # e.g. old "chidori" / "fire"
        keep = d["present"].any(axis=1)               # frames where a hand was found
        idx = np.flatnonzero(keep)[::step]            # every step-th frame
```

بيعمل إيه خطوة خطوة:

1. يلف على كل ملف `.npz` في فولدرات الـ labels بترتيب ثابت.
2. لو اتدّاله `only` (قايمة ids مسموحة)، أي ملف مش فيها **ما بيتفتحش أصلاً**. كده ملفات الـ test عمرها ما بتتقري.
3. يقرا الـ meta، ويترجم الـ label القديم (`rasengan` بيبقى `rasengan_palm`).
4. لو الـ label مش من الحركات الحالية (زي `chidori` و `fire`) يطبع `skip ...` ويكمل.
5. `keep`: الفريمات اللي فيها إيد واحدة على الأقل.
6. `idx`: من الفريمات دي ياخد كل `step` فريم (افتراضي فريم آه وفريم لأ).

ليه؟ الفريمات اللي مفيهاش إيد ملهاش معنى للـ classifier (في التطبيق ما بيتسألش أصلاً لو مفيش إيد). والفريمات اللي جنب بعض شبه متطابقة، فاستخدام كلهم بيبطّأ التدريب من غير فايدة. بيرجع قايمة sessions، كل واحدة dictionary فيه الـ meta و `id` والـ arrays المختارة.

الـ Features والـ Augmentation

`build(sessions, rng)` بيحسب لكل فريم تلات feature vectors بـ `shinobi.features.extract`:

- الفريم **الأصلي**؛
- نسخة **معكوسة** (`features.mirror`: اليمين والشمال اتبدلوا)، فالحركة اللي اتعلمها والإيد اليمين فوق تشتغل برضه معكوسة؛
- نسخة فيها **رعشة (jitter)**: Gaussian noise (ضوضاء عشوائية بتوزيع طبيعي) قدرها 0.004 على إحداثيات الصورة و 0.002 متر على إحداثيات الـ world، على الإيدين الموجودين بس، فالموديل يستحمل الرعشة الصغيرة بتاعة MediaPipe.

```
noise_img = rng.normal(0, 0.004, img.shape) * pres[:, None, None]
noise_wld = rng.normal(0, 0.002, wld.shape) * pres[:, None, None]
X_jitter.append(extract(img + noise_img, wld + noise_wld, pres, s["aspect"]))
```

معنى القيم: `rng.normal(0, 0.004, ...)` يعني أرقام عشوائية متوسطها صفر وانحرافها 0.004 (يعني تقريباً 0.4% من عرض الصورة). الضرب في `pres[:, None, None]` بيخلي الضوضاء صفر على خانة الإيد اللي مش موجودة، عشان ما نخترعش إيد مش موجودة.

كمان بيرجع رقم الـ label، والـ session id (اللي هو الـ group في الـ cross-validation) والشخص. و `augmented(idx, ...)` بيرص صفوف الأصلي + المعكوس + اللي فيه رعشة للفريمات المختارة:

```
def augmented(idx, X, X_mirror, X_jitter, y):
    """Training set = originals + mirrored + jittered copies of the same frames."""
    return np.vstack([X[idx], X_mirror[idx], X_jitter[idx]]), np.concatenate([y[idx]] * 3)
```

يعني لو عندك 1000 فريم تدريب، الموديل بيتدرب على 3000 صف. الـ augmentation بيتعمل **على جزء التدريب بس** في أي تقسيم؛ الفريمات اللي بتتقيّم دايماً أصلية (`features_only`). وإلا كان ممكن نسخة معكوسة من فريم test تبقى قاعدة في التدريب، وده leakage.

الموديلين المرشحين

```
def make_models():
    return {
        "random_forest": RandomForestClassifier(n_estimators=300, min_samples_leaf=2,
                                                class_weight="balanced", n_jobs=-1, random_state=0),
        "mlp": make_pipeline(StandardScaler(),
                             MLPClassifier(hidden_layer_sizes=(128, 64), alpha=1e-3, max_iter=400,
                                           early_stopping=True, random_state=0)),
    }
```

معنى كل قيمة:

- `random_forest` : `n_estimators=300` يعني 300 شجرة. `min_samples_leaf=2` يعني كل ورقة في الشجرة لازم يكون فيها مثالين على الأقل (بيقلل الحفظ الأعمى). `class_weight="balanced"` بيخلي الحركات اللي فريماتها قليلة تتحسب بنفس وزن الحركات اللي فريماتها كتير. `n_jobs=-1` يعني يستخدم كل أنوية الـ CPU. `random_state=0` يعني النتيجة تتكرر بالظبط كل مرة.
- `mlp` : `StandardScaler` بيخلي كل feature متوسطها صفر وانحرافها 1، لأن الشبكات العصبية بتتدرب وحش لو الأرقام على مقاييس مختلفة. `hidden_layer_sizes=(128, 64)` طبقتين مخفيتين. `alpha=1e-3` قوة الـ regularization (عقاب للأوزان الكبيرة عشان الموديل ما يحفظش). `max_iter=400` أقصى عدد دورات تدريب. `early_stopping=True` يعني يقف لوحده لما التحسن يقف (بيسيب جزء صغير من بيانات التدريب داخلياً عشان يحكم).

مين بيستخدم `make_models` : `train.py` نفسه، و `tools/grouped_cv.py`، و `tools/evaluate_pipeline.py loro` (بياخد الـ `random_forest` بس).

`train_with_splits` : الموديل بيتختار ويتدرب إزاي

1. يقرا `data/splits.json` بـ `splits.load` (لو الملف بايظ كل حاجة بتقف: "Nothing was trained."). و `sessions_by_set` بيرتب الملفات لـ train و validation و test و ignored و unassigned **من اسم الملف بس**. ملفات الـ test بتتعد بس وعمرها ما بتتحمل؛ والملفات اللي ملهاش مجموعة بيطلع عنها تحذير وما بتتستخدمش.
2. يحمّل sessions الـ TRAIN والـ VALIDATION. مع `--test-person` ، sessions الشخص ده بتتشال من الاتنين وتتحط على جنب.
3. يرفض يدرب لو `idle` مش في الـ train: الـ idle بيعلم الموديل إيه اللي ما يتفاعلش معاه.

4. يختار نوع الموديل (تلات حالات، في الجدول تحت).
5. الموديل النهائي بيتدرب على TRAIN بس (مع الـ augmentation). الـ validation بيفضل اختبار أمين وبيتطبع عنه تقرير (`validation_block`) بيكتب classification report و (`confusion_validation.png`).
6. يحفظ الـ bundle ومعاه قوايم ids الـ train والـ validation، ويكتب `training_report.txt`.

إزاي الخطوة 4 بتختار:

الشرط	اللي بيحصل	`selection`
الـ validation فيه كل حركة موجودة في الـ train	الموديلين بيتدربوا على الـ train؛ اللي الـ validation macro-F1 بتاعه أعلى بيكسب	`validation`
غير كده، وكل حركة ليها على الأقل 2 train sessions	`StratifiedGroupKFold` cross-validation بالـ session، على الـ train بس	`cv`
غير كده	بيطبع تحذير وبيستخدم الافتراضي (Random Forest)	`default`

`StratifiedGroupKFold` يعني: قسّم لـ folds بحيث (1) كل session كاملة تفضل في fold واحد (Group)، و (2) نسب الحركات في كل fold تبقى قريبة من بعض قد ما يقدر (Stratified). وعدد الـ folds = `min(5, min_sessions)`.

```
best = max(results, key=results.get) if results else next(iter(make_models()))
```

السطر ده معناه: لو فيه نتايج مقارنة، خد الموديل صاحب أعلى macro-F1؛ لو مفيش (حالة `default`)، خد أول موديل في القاموس، اللي هو `random_forest`.

الموديل المتركب اتعمل في الحالة التالتة: يوم 2026-10-31 الـ validation كان فيه `idle` و palm بس، ومعظم الحركات كان ليها train session واحدة، فاستخدم Random Forest كافتراضي. وده باين حرفياً في `models/training_report.txt`: "WARNING: the models cannot be compared fairly yet ... Using the default model".

ملحوظة مهمة عن المستقبل القريب: النهارده الـ train فيه 43 تسجيل وكل حركة ليها على الأقل 3 تسجيلات train، بس الـ validation لسه فيه `idle` و palm بس. يعني لو حد شغل `train.py` دلوقتي، هيدخل الحالة التانية (`cv`) ويختار الموديل بالـ cross-validation **بالتسجيل** (مش بالظروف زي `tools/grouped_cv.py`). وده السبب إن تقرير Phase 3 قال إن تدريب الـ MLP محتاج "اختيار صريح صغير" في `train.py`، ووقت كتابة الكلام ده ما كانش فيه option يجبره على موديل معين. **تحديث:** اتضاف بعد كده `--model-type` (`random_forest` أو `mlp`)، ومن غيره `train.py` بيشتغل زي الأول بالظبط.

Macro-F1 هو متوسط الـ F1 بتاع الحركات، كل حركة بنفس الوزن. بيتستخدم لأن الـ accuracy العادية كانت هتبقى محكومة بالحركات اللي فريماتها كتير (`idle`، palm).

الحفظ

```
joblib.dump({"model": model, "model_name": best, "labels": labels,
             "feature_version": FEATURE_VERSION, "cv_macro_f1": results if selection == "cv" else {},
             "selection": selection, "selection_macro_f1": results, "validation_macro_f1": val_f1,
             "splits_file": str(splits_path),
             "train_sessions": sorted(s["id"] for s in train),
             "validation_sessions": sorted(s["id"] for s in val),
             "held_out_person": args.test_person,
             "created": datetime.now().isoformat(timespec="seconds")}, out_path)
```

بيعمل إيه: بيحفظ الموديل ومعاه كل المعلومات اللي تشرح هو اتعمل إزاي. أهم سطرين `train_sessions` و `validation_sessions`: دول القوايم اللي `evaluate.py` بيستخدمها عشان يرفض يقيّم الموديل على حاجة اتعلم منها. ولما يخلص بيطبع: "their names are stored in the model so evaluate.py can refuse to test on them".

الوضع من غير splits

`main()` من غير ملف splits بيبني كل الـ sessions، يطلب على الأقل 2 sessions لكل حركة، يشغل `StratifiedGroupKFold` بـ `groups = session`، يحتفظ بالـ out-of-fold predictions (التخمينات على الجزء اللي ما اتدربش عليه في كل fold)، يحفظ confusion PNG لكل موديل (`confusion_<name>.png`)، يختار أعلى macro-F1 ويدربه على كل حاجة (ما عدا `--test-person`). ده السلوك القديم، متساب للاكتمال. والموديل اللي بيطلع من الوضع ده **ما فيهوش** `train_sessions`، فـ `evaluate.py` بيرفض يستخدمه (وده متختبر).

الروابط

`train.py` بيعمل import لـ `splits` و `shinobi.features`. و `tools/grouped_cv.py` و `tools/evaluate_pipeline.py` بيعيدوا استخدام `load_sessions` و `build` و `make_models` و `augmented` و `save_confusion`، فكل دراسة بتدرب بنفس طريقة `train.py` بالظبط. والتطبيق بيحمّل الـ bundle من خلال `shinobi/classifier.py` (الـ `GestureClassifier`).

ملحوظة عن رسايل `train.py`: الـ docstring في أول الملف بيقول إن `data/splits.json` "made by `python evaluate.py splits`"، ولما يلاقي تسجيلات ملهاش مجموعة بيقول "to give them a set, then train again run `python evaluate.py splits`". النهارده التسجيل الوحيد اللي ملوش مجموعة هو الـ snake المرفوض، فالتحذير ده هيظهر في أي تدريب. **ما تنفذيش النصيحة دي**: التسجيل ده مرفوض عن قصد، والإضافة بتتعمل بالأدوات اليدوية بس.

مثال عملي

لو صاحبة المشروع وافقت إننا نجرب نموذج مرشح من غير ما نلمس الموديل المتركب:
`.venv\Scripts\python.exe train.py --out C:\path\outside\repo\candidate.joblib`. ده هيقرا `data/splits.json`، يحمّل الـ 43 train والـ 3 validation (ويعد الـ 22 test من غير ما يفتحهم)، يختار بالـ `cv`، يدرب الموديل النهائي على الـ 43، ويكتب `candidate.joblib` و `training_report.txt` و `confusion_validation.png` في الفولدر اللي بره المشروع.

الملف `evaluate.py`

مسؤول عن إيه: يقيس SHINOBI AI بيتعرف على الحركات والـ jutsu قد إيه كويس، على تسجيلات عمره ما اتعلم منها، باستخدام كود التطبيق نفسه (ولا حاجة متعادة كتابتها من الأول)، ويكتب تقرير مقري.

ليه اتعمل: عشان أي رقم نقوله عن المشروع يبقى جاي من نفس الكود اللي بيشتغل في التطبيق، على بيانات ما اتشافتش، وبطريقة تتكرر. ومعاه حماية تلقائية ضد الـ leakage.

مين بيعتمد عليه: `tools/make_test_set.py` (بيشغله كـ subprocess)، و `tools/grouped_cv.py` و `tools/evaluate_pipeline.py` (بيستخدموا دوال منه)، والاختبارات في `tests/test_evaluate.py`.

فيه أمرين فرعيين:

- `python evaluate.py splits` : بيدي مجموعات للتسجيلات الجديدة (`cmd_splits`). **ما تشغليهوش على بيانات المشروع** (شوفي `data/splits.json` فوق).
- `python evaluate.py run` : بيقيّم التسجيلات ويكتب `reports/<YYYY-MM-DD_HHMMSS>/summary.md` و `metrics.json` و `confusion_*.png` (`cmd_run`).

اختيارات `splits` (للمعرفة بس): `--data`، `--splits`، `--quota` (مثلاً `3,1,1`)، `--take-quota`، `--assign ID=SET` (ينفع تتكرر)، `--list`، `--dry-run`. حتى مع `--assign`، الأمر بيعمل `update` الأول، يعني بيدي مجموعة لكل تسجيل ملوش، فـ `--assign` لوحده **مش آمن** على البيانات دي. و `--dry-run` بس هو اللي ما بيكتبش.

اختيارات `run` :

الاختيار	الافتراضي	معناه
`--sets`	`test`	أو `validation`، `train`، قايمة بفواصل، أو `all`
`--sessions`	الكل	ids معينة بس (بفواصل)
`--recognizers`	`landmarks,yolox,fused`	أنهي recognizers يتقاسوا
`--model`	`models/gesture_model.joblib`	أو `none`
`--exclusions`	`data/evaluation_exclusions.json` لو موجود	أو `none`
`--skip-start`	0.5	ثواني ما بتتحسبش في أول كل session
`--yolox-delay`	0.06	ثواني لحد ما نتيجة YOLOX تبقى جاهزة
`--out`	`reports`	فولدر التقارير
`--title`، `--note`	مفيش	عنوان وملاحظات في أول `summary.md`

التلات recognizers

- **landmarks:** الـ GestureClassifier لوحده على النقط المتسجلة، على الفريمات اللي فيها إيد بس.
- **yolox:** YOLOX لوحده على فريمات الفيديو المحفوظة، من خلال الـ `SealWorker` بتاع التطبيق نفسه. إجابة الفريم هي أحسن detection لختم وصل لـ `SEAL_MIN_SCORE` (0.70)، وإلا `idle` (`yolox_sign`). وللحركات اللي YOLOX ملوش class ليها (`idle`، `clone_seal`، `rasengan_palm`) الإجابة الصح بتبقى `idle` (`yolox_truth`).

- **fused:** التسجيل كله بيتعاد تشغيله في الوقت (replay) من خلال `SignFusion` و `ActionController`، بنفس الاستدعاءات اللي في `shinobi/engine.py`. ده اللي التطبيق كان هيقرره فعلاً.

ليه تلاتة؟ عشان لو النتيجة النهائية (fused) واطية، نقدر نعرف المشكلة فين: هل الـ landmark model ضعيف، ولا YOLOX، ولا طريقة الدمج نفسها. وده بالظبط اللي الـ diagnosis استخدمه.

الدوال الأساسية

- `load_recording`، `video_file`، `read_video`: بيقروا تسجيل واحد والفيديو بتاعه.
- `saved_video_frames(timestamps, fps=15)`: بيعيد تطبيق قاعدة أدوات التسجيل ("اكتب فريم فيديو لما يعدي على الأقل 1/15 ثانية") على الـ timestamps؛ و `align_video` بيستخدمها، ولو العدد مش مطابق بيرجع لتوزيع الفريمات بالتساوي على الوقت (وبيتكتب في التقرير "approximate").
- `hands_from_arrays(...)`: بيعيد بناء الـ `Hand` objects اللي الـ tracker كان هيرجعها، بالبكسل بتاع فريم الفيديو (الـ handedness والـ score مش متخزنين ومش محتاجينهم).
- `SealRunner`: غلاف حوالين الـ `SealWorker` الحقيقي: بيبعت فريم واحد ويستنى النتيجة بتاعته بالظبط، فالـ replay هو اللي بيحدد التوقيت، مش ساعة الكمبيوتر.
- `seal_schedule(frame_times, rate_hz=SEAL_RATE_HZ, delay=0.06)`: بيحاكي إمتى thread الـ YOLOX في التطبيق كان هيشتغل: بحد أقصى `SEAL_RATE_HZ` مرة في الثانية على أحدث فريم، وكل نتيجة تبقى قابلة للاستخدام بعد 0.06 ثانية من الفريم بتاعها (وده متوسط زمن التشغيل اللي اتقاس، الـ median).
- `landmark_probs(...)`: بيشغل الـ classifier على كل فريم فيه إيد وبيقيس الوقت لكل فريم.
- `replay(times, hands, probs, lm_labels, seal_runs)`: بيدخل كل فريم في `fusion.update(...)` و `controller.update(...)` وبيسجل الحركة بعد الدمج، ومصدرها، والحركات اللي اتأكدت، والـ jutsu اللي اشتغلت، والتوقيتات.
- `frame_metrics(truth, pred, labels)`: بيحسب accuracy، و balanced accuracy (متوسط الـ recall)، و macro-F1، و precision / recall / F1 لكل class. الـ precision بيبقى None لـ class عمره ما اتقال؛ والمتوسطات بتستخدم بس الـ classes الموجودة في الإجابات الصح.
- `confusion`، `latency` (n، mean، median، p90، max)، `take_outcome` (`success`، `missed`، `wrong jutsu`، `fired twice`، أو للـ takes السلبية `ok` / `false activation`).

`frame_metrics` بالتفصيل

```
for c in classes:
    tp = int(np.sum((truth == c) & (pred == c)))
    support, predicted = int(np.sum(truth == c)), int(np.sum(pred == c))
    precision = tp / predicted if predicted else None
    recall = tp / support if support else None
    f1 = None
    if support:
        f1 = 2 * precision * recall / (precision + recall) if tp else 0.0
```

بيعمل إيه خطوة خطوة لكل class `c`:

1. `tp` (true positives): عدد الفريمات اللي كانت فعلاً `c` والموديل قال `c`.
2. `support`: عدد الفريمات اللي كانت فعلاً `c`. و `predicted`: عدد المرات اللي الموديل قال فيها `c`.

3. `precision = tp / predicted`، ولو الموديل عمره ما قال `c` تبقى `None` (مش صفر، عشان ما نخترعش رقم).
4. `recall = tp / support`.
5. `f1` بيتحسب بس لو الـ class موجود فعلاً في الإجابات الصح؛ ولو `tp` صفر يبقى 0.0.

وبعدين الـ macro-F1 = متوسط الـ F1 للـ classes اللي ليها `support` بس. ليه؟ عشان class ملوش ولا فريم في التسجيلات دي ما يدخلش في المتوسط ويشده لتحت من غير سبب.

`seal_schedule` بالتفصيل

```
period = 1.0 / rate_hz
times = list(frame_times)
runs, wake, last = [], -np.inf, -1
while True:
    k = bisect_right(times, wake) - 1
    if k <= last:                                   # nothing new yet: wait for the next frame
        k = last + 1
        if k >= len(times):
            break
        start = times[k]
    else:
        start = wake
    runs.append((k, start + delay))
    last = k
    wake = start + max(period, delay)               # one run at a time, at most rate_hz
return runs
```

بيعمل إيه: بيحاكي thread الـ YOLOX في التطبيق. كل ما "يصحى"، بياخد أحدث فريم وصل لحد الوقت ده. لو مفيش فريم جديد، يستنى لحد الفريم الجاي. كل تشغيل بيتسجل كـ (رقم الفريم، الوقت اللي النتيجة هتبقى جاهزة فيه = البداية + 0.06). وبعدين يصحى تاني بعد `max(period, delay)`، يعني بحد أقصى 10 مرات في الثانية ومرة واحدة في نفس الوقت. ليه ده مهم؟ لأن في التطبيق الحي YOLOX ما بيشتغلش على كل فريم، وبيوصل متأخر شوية. لو التقييم شغّله على كل فريم فوراً، كان هيطلع أحسن من الحقيقة. في تقرير 2026-11-01 الـ replay شغّل YOLOX 8.93 مرة في الثانية (median)، لأن الفيديو المحفوظ حوالي 10 فريم في الثانية.

حارس الـ leakage

```
def model_status(path):
    ...
    bundle = joblib.load(path)
    if "train_sessions" not in bundle:
        info["reason"] = ("it was trained without data/splits.json, so it may have learned from ANY "
                          "recording, test ones included. Re-train ... and evaluate again.")
        return info, None
```

بيعمل إيه: أي موديل ما فيهوش قايمة التسجيلات اللي اتدرب عليها **ما بيتستخدمش خالص**، لأننا مش نقدر نضمن إنه ما شافش الـ test. وجوه `evaluate_recording`، لو التسجيل موجود في `train_sessions` بتاعة الموديل، أرقام الـ landmarks والـ fused بتتخطى للتسجيل ده. و `cmd_run` كمان بيحذر لو `splits.json` اتعدل بعد التدريب (تسجيل كان train وبقى test، أو تسجيل validation بقى test).

```
leak = classifier is not None and rec.id in model["train_sessions"]
use_model = classifier is not None and not leak
if leak:
    summary["model_note"] = "the landmark model was trained on this recording: landmarks + fused
skipped"
```

السطرين دول هما قلب الحماية: `leak` بيبقى True لو التسجيل كان من بيانات التدريب، وساعتها `use_model` بيبقى False ومفيش أي رقم بيطلع من الموديل على التسجيل ده.

`evaluate_recording` بيعمل إيه لتسجيل واحد

1. يحمّل النقط، والـ timestamps، والـ aspect. الفريمات اللي قبل `--skip-start` (0.5 ثانية) ما بتتحسبش في الـ single-sign sessions: الإيدين بيبقوا في مكانهم من الثانية 0، بس الـ fusion محتاج `SEAL_CONFIRM` نتيجة YOLOX قبل ما يقدر يسمي ختم.
2. Landmarks: يخمّن كل فريم فيه إيد ويضيف (الإجابة الصح، التخمين) للمخزن.
3. YOLOX: يقرا الفيديو، يظبط المحاذاة، يشغل YOLOX على كل فريم فيديو، ويحسب الفريمات اللي بعد الـ skip.
4. Fused: replay بنتايج YOLOX على جدول التطبيق؛ يسجل أنهي حركات الـ controller أكدها، وإمتى الحركة الصح اتأكدت أول مرة، والتأكيدات الغلط، وأي jutsu اشتغل (مسك حركة واحدة عمره ما المفروض يشغل jutsu).
5. للـ takes: النتيجة (هل الـ jutsu الصح اشتغل مرة واحدة بالظبط، ومفيش حاجة في السلبي) والـ latency بعد آخر حركة.

التجميع والتقرير

`frame_level_results` بيحسب المقاييس والـ confusion لكل مجموعة ولكل recognizer. التسجيلات المستبعدة بتتقيّم في مخزن منفصل، فـ `frame_level` هو "valid recordings only" و `frame_level_all_recordings` فيه الكل. `aggregate_takes` بيدي نسب التفعيل لكل jutsu والتفعيلات الغلط؛ و `aggregate_streams` بيعد الـ jutsu اللي اشتغلت أثناء single-sign sessions (المفروض 0).

`_caveats` بيضيف حدود صريحة أوتوماتيك: حركات ملهاش تسجيل test، حركات كل تسجيلات الـ test بتاعتها مستبعدة، test بيشارك يوم + إضاءة مع الـ train، حركات ليها تسجيل واحد بس اتقيّم ("smoke test"، يعني اختبار مبدئي مش قياس)، كل التسجيلات من ظرف أو اتنين، وإن YOLOX بيشوف الـ mp4 بـ 480p مش فريم الكاميرا الحي 720p. وكمان إن الإيدين جايين من النقط المحفوظة وقت التسجيل (MediaPipe ما بيتعادش تشغيله).

`write_report` بيكتب `metrics.json` و `summary.md` (أقسامه: إيه اللي اتقاس، النتايج في لمحة، تفاصيل لكل مجموعة و recognizer بجداول لكل حركة و confusion matrices، كل تسجيل، مستوى الـ jutsu، التحذيرات، إزاي تقري التقرير)، وصور الـ confusion (`save_confusion_png`: درجات لون أزرق واحد، اللون = نسبة الصف، والإجابة الصح عليها إطار). فولدر التقرير عمره ما بيتكتب فوقه (بيتضاف `_1`، `_2` في الآخر).

```
stamp = datetime.now().strftime("%Y-%m-%d_%H%M%S")
out_dir = Path(args.out) / stamp
n = 1
while out_dir.exists():
    out_dir = Path(args.out) / f"{stamp}_{n}"
    n += 1
```

مثال عملي

`evaluate.py run` من غير أي اختيارات: بيقرا الـ 22 تسجيل test، يطبق الاستبعادات الخمسة، يشغل التلات recognizers، ويكتب فولدر جديد زي `reports/2026-11-02_101500/`. في الـ console هيطبع سطر لكل مجموعة و recognizer، مرة "valid recordings only" ومرة بعلامة "all recordings, with invalid labels". وهيطلع في التحذيرات إن `me_day_20261101_140757` ملوش مجموعة؛ ده متوقع ومقصود.

تنبيه مهم من قواعد المشروع: الـ 22 test القديمة بقت **diagnostic holdout** (اتبص عليها أثناء الـ diagnosis)، فأي تشغيل عليها دلوقتي بيدي رقم تشخيصي، مش رقم نهائي. والرقم النهائي الحقيقي محتاج test set جديد خالص.

أدوات التسجيل والتقسيم في فولدر `tools/`

الأدوات اللي جاية دي اتعملت بعد ما اكتشفنا إن `collect_data.py` لوحده مش كفاية: كان ممكن يحفظ تسجيل مفيهوش الحركة أصلاً. فالترتيب المنطقي: أداة مراجعة (`recording_review.py`)، أداتين تسجيل موجّهة بيستخدموها، وأداتين بيحطوا التسجيلات في المجموعات بالإيد وبأمان.

الملف `tools/recording_review.py`

مسؤول عن إيه: "مراجعة الست فريمات" (six-frame review) اللي بتظهر بعد كل تسجيل موجّه، عشان المستخدمة تقرر تحتفظ بيه ولا لأ. ولا حاجة بتتحفظ قبل الإجابة.

مين بيستخدمه: `tools/record_training_session.py` و `tools/record_test_session.py`.

الثوابت: `SHOTS = 6`، وزراير الحفظ `KEEP_KEYS` (ENTER بكوديه 13 و 10، و `k` و `K`)، والإعادة `REDO_KEYS` (`r` و `R`)، والإيقاف `QUIT_KEYS` (`q` و `Q` و Esc بكود 27)، وألوان الإيدين `HAND_COLORS` (أخضر للخانة 0، ماجنتا للخانة 1).

- `pick(n_video, shots=6)`: ست أرقام فريمات فيديو متوزعة من 5% لـ 95% من التسجيل.
- `thumbnail(frame, image_lm, present, width)`: فريم فيديو أصغر، ومرسوم عليه الإيدين اللي MediaPipe لقاها (أخضر للخانة 0، ماجنتا للخانة 1).
- `sheet(video_frames, video_idx, frames, title, lines, picture, size)`: شاشة المراجعة: العنوان والملاحظات فوق، والرسمة المرجعية على اليمين، والست صور الصغيرة في صفين كل صف 3، وعلى كل واحدة الوقت بتاعها، وشرح الزرايز تحت.
- `ask(window, screen)`: يستنى زرار: ENTER أو K يرجع `keep`، و R يرجع `redo`، و Q أو Esc يرجع `quit`؛ ولو الشباك اتقفل بيتحسب `quit`.

```
def pick(n_video, shots=SHOTS):
    """Indices of `shots` video frames spread over the recording (5% .. 95% of it)."""
    if n_video <= 0:
        return []
    return sorted({int(round(x)) for x in np.linspace(0.05 * (n_video - 1), 0.95 * (n_video - 1),
shots)})
```

بيعمل إيه: `np.linspace` بيطلع 6 أرقام متساوية المسافات بين 5% و 95% من طول الفيديو، وبعدين يقربهم لأرقام صحيحة ويشيل المكرر ويرتبهم. مثال: فيديو فيه 200 فريم يدي تقريباً 10، 46، 82، 117، 153، 189. ليه مش من 0% لـ 100%؟ لأن أول لحظة وآخر لحظة غالباً فيهم الإيد لسه داخلة أو خارجة. وده بيغطي التسجيل كله، فلو الحركة اختفت في النص (زي تسجيل الـ ram المستبعد) هتبان.

ليه بنعرض إيدين MediaPipe مش تخمين الـ AI؟ قرار الاحتفاظ بتسجيل ما ينفعش يعتمد على الموديل اللي هيتدرب أو يتختبر بيه، وإلا هنختار بس التسجيلات اللي الموديل بيعرفها، والنتايج هتبقى متحيزة. والمراجعة دي موجودة أصلاً لأن تسجيلين test فيهم مشاكل (ram و tiger) اتحفظوا بنسخة قديمة من أداة التسجيل كانت بتتأكد بس إن "إيد اتشافت".

الملف `tools/record_training_session.py`

مسؤول عن إيه: تسجيل موجّه لبيانات TRAINING و VALIDATION على تلات sessions بحد أقصى (68 تسجيل في الخطة)، مع blocks لأماكن الكاميرا ومراجعة الست فريمات.

ليه اتعمل: التشخيص بيّن إن الموديل اتدرب على ظروف قليلة جداً (تسجيل واحد لكل ختم، ومعظمه بالليل وبكاميرا في مستوى الوش). فالأداة دي بتفرض تنوع مخطط في مكان الكاميرا والإضاءة، وبتمنع الأخطاء القديمة.

التشغيل:

الأمر	بيعمل إيه
`python tools/record_training_session.py --session 1 --plan`	يطبع الخطة بس، من غير كاميرا
`python tools/record_training_session.py --session 1`	يسجل
`python tools/record_training_session.py --session 1 --resume reports/training_sessions/<log>.json`	يكمّل session اتوقفت

اختيارات تانية: `--person` (افتراضي `me`)، و `--lighting` (إجباري للـ session 3)، و `--seconds`.

الخطة

- `WEAK = ["ram", "snake", "tiger", "dog", "clone_seal", "rasengan_palm"]` بيتسجلوا في كل block؛ و `STRONG = ["monkey", "boar", "horse"]` في بعض الـ blocks بس؛ وكل block بيخلص بتسجيل `idle` واحد بنوع idle خاص بيه (علامة OK، قبضات ومخالب، الإشارة ولمس الوش، استخدام الموبايل، إيدين مرتاحة، أختام نص مكتملة ...؛ ولا مرة كف مفتوح الصوابع متباعدة).
- `CAMERAS` بيوصف ست أماكن للكاميرا: `A` (في مستوى الوش، قدامك)، `A-far` (قاعدة لورا)، `A-side` (متلفّة 15–20 درجة)، `B` (تحت، بتبص لفوق)، `B-near` (تحت وقريبة)، `C` (30 درجة على الجنب).

- `SESSIONS`: الـ session 1 = train، ضوء النهار، blocks `A` ، `B` ، `A-far`؛ الـ session 2 = train، لمبة بالليل، blocks `A` ، `C` ، `B-near`؛ الـ session 3 = validation، إضاءة تانية، blocks `A-side` ، `B`. في المجموع 51 train + 17 validation = 68.
- `EXTRA_HINTS`: الـ `clone_seal` لازم الإيدين الاتنين يفضلوا جوه الصورة؛ والـ `rasengan_palm` "ONE hand only"، بالتبادل اليمين (block فردي) والشمال (block زوجي).

حساب الـ 68 بالتفصيل (اتأكدنا منه من الكود):

الـ session	الـ blocks	التسجيلات
1 (train، day)	`A` بالأقوياء: 6 + 3 + 10 = idle؛ `B` بالأقوياء: 10؛ `A-far` من غيرهم: 6 + 7 = idle	27
2 (train، night)	`A` من غيرهم: 7؛ `C` بالأقوياء: 10؛ `B-near` من غيرهم: 7	24
3 (validation)	`A-side` بالأقوياء: 10؛ `B` من غيرهم: 7	17

`plan(session)` بيرجع الخطوات مترقمة. و `needed_hands(label)` بيرجع أقل نسبة فريمات فيها إيدين: 0.6 للحركات، و 0.3 للـ idle.

التسلسل في `main()`

1. يعمل أو يكمّل الـ log `reports/training_sessions/session<N>_<start>.json` (فيه `saved` ، `redone` ، `not_saved` ، `finished`).
2. قبل كل block، `setup()` بيعرض فين تحطي الكاميرا والإضاءة، ويستنى SPACE.
3. قبل كل تسجيل، عد تنازلي 6 ثواني بيمشي **بس طول ما الإيدين ظاهرين** (و SPACE بيوقفه مؤقتاً). ده بيحل المشكلة القديمة إن أداة التسجيل كانت بتسبق المستخدمة.
4. يسجل 20 ثانية (نقط + فيديو 15 fps، وبيفتكر كل فريم فيديو تبع أنهي فريم نقط).
5. لو الإيدين ظهروا في فريمات قليلة أوي، التسجيل ما بيتحفظش أبداً وبيتسجل تاني.
6. غير كده، مراجعة الست فريمات: keep يبقى `save_new(...)` من `record_test_session.py` (عمره ما بيكتب فوق ملف، وبيضيف meta زيادة زي الكاميرا والمجموعة المخططة والـ session والـ block)؛ redo يسجل تاني؛ quit يوقف.

```
left, paused, last = float(PREP_SECONDS), False, time.perf_counter()
while True:
    frame, hands, key = frame_and_hands()
    if key == ord(" ") and time.perf_counter() > grace:
        paused = not paused
    now = time.perf_counter()
    waiting = not hands
    if not paused and not waiting:
        left -= now - last
    last = now
    if left <= 0:
        break
```

بيعمل إيه خطوة خطوة: `left` بيبدأ 6 ثواني. في كل فريم: لو SPACE اتداس (بعد فترة سماح `SPACE_GRACE` = 0.6 ثانية عشان ضغطة الـ setup ما تتحسبش)، يقلب حالة الإيقاف. `waiting` بيبقى True لو مفيش إيدين. الوقت

بينقص **بس** لو مش متوقف ومش مستني إيدين. أول ما يوصل صفر، التسجيل يبدأ. النتيجة: لو المستخدمة مسكت موبايلها أو بعدت، العد بيقف لوحده لحد ما ترجع.

ولا حاجة بتتحط في مجموعة هنا. بعد الـ session، `tools/assign_recordings.py` بيحط التسجيلات المحفوظة في مجموعات بالـ id. الـ session 1 بس هي اللي اتسجلت (27 تسجيل يوم 01-11-2026، من 16:28 لـ 17:49)؛ والمستخدمة لغت الـ 2 sessions و 3. ومن الـ log (`reports/training_sessions/session1_20261101_155047.json`): الـ session بدأت 15:50:47، `finished: true`، 27 `saved`، و 24 مرة اختارت فيها المستخدمة تعيد التسجيل (`redone`)، و 2 ما اتحفظوش لأن الإيدين ما ظهروش كفاية (`not_saved`). ده بيوري إن المراجعة كانت شغالة فعلاً وبتتستخدم.

الملف `tools/record_test_session.py`

مسؤول عن إيه: تسجيل موجّه لـ test session **مستقلة**، المستخدمة فيها بتعمل الحركات وبس.

التشغيل: `python tools/record_test_session.py --person me --lighting day` (و `--resume LOG` بيكمّل session اتوقفت).

- `plan()`: لفتين، كل لفة: idle، ثم الـ 9 حركات، ثم idle، يعني 22 تسجيل: كل حركة مرتين وأربع أنواع idle مختلفة (الإيدين تحت، علامة OK، حركات نص مكتملة، حركات عشوائية).
- `min_hands(label, idle_index)`: 0.6 للحركات، 0.3 للـ idle من 2 لـ 4، و 0 لأول idle (الإيدين تحت).
- `save_new(label, person, lighting, frames, video_frames, aspect, extra)`: نفس شكل الملفات والأسماء بتاع `collect_data.py`، بس بيرمي error لو الملف موجود قبل كده، وبيفتح الـ `.npz` بـ mode `"xb"` (بيفشل لو الملف ظهر في اللحظة دي). الـ meta الزيادة مسموحة بس ما تقدرش تغير المفاتيح الأساسية. و `big(...)` بيرسم كلام كبير. الاتنين بيستخدمهم أداة تسجيل التدريب.
- قبل أول خطوة عد تنازلي 10 ثواني (`START_SECONDS`) "اقفي قدام الكاميرا"؛ وبعدين لكل خطوة: العد اللي بيمشي بس مع ظهور الإيدين، 20 ثانية تسجيل، فحص الإيدين، ومراجعة الست فريمات.
- `--resume` بيعيد فحص الخطوات اللي اتحفظت قبل كده: تسجيل محفوظ فيه إيدين قليلة أوي بيتعلّم عليه `rejected` في الـ log (عشان `make_test_set.py --exclude-from` يقدر يسيبه بره) وبيتسجل تاني.

```
with open(npz, "xb") as f:                                  # "x": fails if the file appeared
    np.savez_compressed(
        f,
        ...
        meta=json.dumps({**(extra or {}), "label": label, "session": session_id, "person": person,
                         "lighting": lighting, "aspect": aspect}),
    )
```

بيعمل إيه: الـ mode `"xb"` معناه "اعمل ملف جديد بالـ binary، وافشل لو موجود". والـ `{**(extra or {}), "label": ...}` معناه: حط الـ meta الزيادة الأول، وبعدين المفاتيح الأساسية فوقيها، فلو `extra` فيه `label` غلط، الـ label الصح هو اللي هيكسب. ليه؟ عشان أداة التسجيل عمرها ما تمسح تسجيل قديم بالغلط، وعشان المعلومات الأساسية ما تتلخبطش.

الـ log بيتكتب في `reports/test_sessions/<start>.json`. الـ log الموجود `20261101_140445.json` تبع الـ test session بتاعة 01-11-2026 (22 تسجيل). أول خطوات الـ session دي اتسجلت بنسخة أقدم من غير خطوة

المراجعة؛ ومن هنا جم تسجيلات الـ ram والـ tiger اللي فيهم مشاكل.

اللي لقيناه في الـ log ده بالظبط: الـ session بدأت 14:04:45، `finished: true` ، 22 `saved` ، اتعملها `resumed` مرة الساعة 14:12:57، وفيها `rejected` واحد: الـ snake `me_day_20261101_140757` (413 فريم، الإيدين 0.0) بالسبب "hands not seen enough: not part of the test set". ودي بالظبط التسجيل الوحيد اللي ملوش مجموعة في `splits.json` .

ملحوظة صغيرة من الكود: الـ docstring بيقول إن التسجيل المرفوض "tools/make_test_set.py --exclude leaves it out"، بس الاختيار اللي بيقرا الـ log فعلياً اسمه `--exclude-from` (و `--exclude` بياخد ids بإيدك). الاتنين بيوصلوا لنفس النتيجة.

الملف `tools/assign_recordings.py`

مسؤول عن إيه: يحط التسجيلات اللي اتحفظت في training session في train أو validation، بالـ id، بالإيد، وبعد dry run آمن الأول.

ليه اتعمل: عشان البديل الآمن لـ `python evaluate.py splits` . بيضيف بس التسجيلات اللي المستخدمة راجعتها وحفظتها، في المجموعة اللي كانت مخططة ليها، ومن غير ما يلمس أي حاجة تانية.

التشغيل: `python tools/assign_recordings.py --log reports/training_sessions/session1_<start>.json` (dry run، يعني تجربة على الناشف من غير كتابة) ونفس الأمر بـ `--apply` (يكتب). و `--log` ينفع يتكرر.

`check_plan(logs, splits, recordings, data_dir, excluded)` بيبني قايمة (id، label، set) وبيرفض أي تسجيل لو:

- متكرر مرتين؛
- مجموعته المخططة مش train أو validation (الأداة دي عمرها ما تقدر تحط حاجة في test)؛
- ليه مجموعة بالفعل، أي مجموعة (الـ entries الموجودة عمرها ما بتتحرك؛ والـ test set عمره ما بيتلمس)؛
- موجود في `data/evaluation_exclusions.json` ؛
- ملف النقط بتاعه مش موجود، أو الـ label على الديسك مختلف عن الـ log، أو الفيديو بتاعه مش موجود.

لو أي حاجة اترفضت، ولا حاجة بتتكتب. غير كده الـ dry run بيطبع الخطة والأعداد الناتجة لكل حركة (/ train validation / test). ومع `--apply` بيضيف بـ `splits.assign` (manual)، يحفظ، يعيد التحميل، ويتأكد إن **كل entry قديم ما اتغيرش ولا حرف** وإن كل entry جديد في مجموعته المخططة؛ لو لأ، بيقف برسالة "CHECK FAILED ... Restore data/splits.json from the backup".

```
before = json.loads(json.dumps(splits["sessions"]))
...
for rid, _, target, _ in plan:
    S.assign(splits, recordings, rid, target)
S.save(S.check(splits), splits_path)
after = S.load(splits_path)["sessions"]
changed = [rid for rid, entry in before.items() if after.get(rid) != entry]
wrong = [rid for rid, _, target, _ in plan if after.get(rid, {}).get("set") != target]
if changed or wrong:
    raise SystemExit(...)
```

بيعمل إيه خطوة خطوة: (1) ياخد نسخة عميقة من كل الـ entries قبل أي تعديل (`json.loads(json.dumps(...))` طريقة بسيطة لعمل نسخة مستقلة)؛ (2) يضيف الجديد؛ (3) يحفظ؛ (4) يقرا الملف من الديسك تاني؛ (5) يقارن كل entry قديم بنسخته قبل كده، ويتأكد إن كل جديد في مكانه. ليه بيقرا من الديسك تاني بدل ما يثق في اللي في الذاكرة؟ عشان يتأكد من اللي اتكتب فعلاً، مش من اللي كان ناوي يكتبه.

كده بالظبط الـ 27 تسجيل بتوع الـ 1 session دخلوا train (الـ entries من 41 لـ 68؛ والـ train من 16 لـ 43).

الملف `tools/make_test_set.py`

مسؤول عن إيه: يحول تسجيلات session جديدة لـ TEST set ويقيّمها.

التشغيل: `python tools/make_test_set.py --since "2026-11-01 12:00"` . الاختيارات: `--dry-run` ، `--no-eval` ، `--min-per-sign N` (تحذير التغطية، الافتراضي 2)، `--exclude ID,ID` ، `--exclude-from LOG` (يسيب بره الـ ids اللي الـ log الـ test session علّم عليها `rejected`).

الخطوات:

1. `scan` لكل التسجيلات ويحتفظ باللي اتعملت في أو بعد `--since` (`when(rec)` من التاريخ والوقت اللي في الـ id).
2. `trained_on()` بيقرا `train_sessions` و `validation_sessions` بتوع الموديل المتركب؛ أي تسجيل منهم بيترفض (REFUSED) (عمره ما يبقى بيانات test).
3. يشيل الـ ids المستبعدة (بتفضل من غير مجموعة، فـ `train.py` كمان بيتجاهلها).
4. يحط الباقي في `test` بالإيد (`manual`)، فالحصص عمرها ما تقدر تبعتهم للتدريب، ويحفظ.
5. يطبع كل تسجيل ومعاه نسبة الفريمات اللي فيها إيدين (`hands_share`)، وجدول تغطية لكل حركة.
6. لو مش `--dry-run` ولا `--no-eval` ، بيشغل `evaluate.py run --sets test --sessions <الـ ids دي> --` `title "Independent test set: ..."` كـ subprocess.

ده اللي عمل الـ test set بتاع الـ 22 تسجيل والتقرير `reports/2026-11-01_144049/` .

ملاحظات صغيرة من الكود: الـ docstring بيقول "Keys 1-9 choose a sign" لما بيوصف `collect_data.py` ، والحقيقة إن `collect_data.py` بيستخدم 0–9 (و 0 هو `idle` ، والـ docstring نفسه بيقول كده بعدها بسطر). و `trained_on()` بيدور على `config.MODEL_PATH` ولو مش موجود بيستخدم `models/gesture_model.joblib` ؛ وبما إن `config` فيه `GESTURE_MODEL_PATH` مش `MODEL_PATH` ، بيستخدم المسار الاحتياطي، وهو نفس الملف، فالنتيجة صح.

تحذير: الأداة دي بتنقل لـ test **كل** تسجيل اتعمل بعد `--since` (حتى لو كان ليه مجموعة تانية قبل كده، ما عدا اللي الموديل اتدرب عليه). عشان كده لازم دايماً `--dry-run` الأول، و `--since` يتحدد بدقة على بداية الـ test session.

الملف `tools/grouped_cv.py`

مسؤول عن إيه: cross-validation مقسم بالظروف (condition-grouped) للـ landmark model على الـ TRAIN set بس (عمره ما بيستخدم validation ولا test). بيقدّر الموديل هيعمل إيه مع وضع كاميرا ما شافهوش قبل كده.

ليه اتعمل: لأن الـ validation set فيه `idle` و palm بس، فما ينفعش نختار بيه بين الموديلات للأختام. والحل: نستخدم الـ train نفسه بس بطريقة أمينة، كل fold بيتختبر على ظروف تصوير ما اتدربش عليها خالص.

التشغيل: `python tools/grouped_cv.py` (اختيارات `--step` (افتراضي 2)، `--out` (افتراضي `reports`)، `--model` (الموديل المتركب للـ baseline)).

الـ groups (ظروف تسجيل كاملة):

الـ group	أنهي تسجيلات TRAIN
G1 oct30	كل تسجيل train اتعمل يوم 2026-10-30 (الإعداد القديم)
G2 A	2026-11-01، الكاميرا في مستوى الوش (A = `camera` meta)
G3 B	2026-11-01، الكاميرا تحت وبتبص لفوق
G4 A-far	2026-11-01، مستوى الوش، قاعدة لورا

```
def group_of(meta, date):
    if date == "2026-10-30":
        return "G1 oct30"
    if date == "2026-11-01" and meta.get("camera") in CAMERA_GROUP:
        return CAMERA_GROUP[meta["camera"]]
    raise SystemExit(f"recording {meta.get('session')} ({date}) has no condition group: nothing was
run")
```

بيعمل إيه: بيحدد كل تسجيل تبع أنهي group من تاريخه والـ `camera` اللي في الـ meta. ولو لقى تسجيل ملوش group بيقف خالص بدل ما يخمّن. ده معناه إن الأداة دي **مكتوبة على مقاس البيانات الحالية** (التواريخ متكتبة جواها): لو اتضافت تسجيلات train جديدة من يوم تاني، الأداة هتقف لحد ما حد يضيف group جديد.

لكل موديل من `train.make_models()`، أربع folds: يدرب على 3 groups (بـ `train.augmented`، يعني mirror + jitter على جزء التدريب بس) ويتوقع احتمالات على الفريمات الأصلية بتاعة الـ group الرابع. التوقعات بتتجمع out of fold (يعني كل فريم اتوقع عليه موديل ما شافهوش).

`score(...)` بيطلع لكل موديل: macro-F1 و accuracy مجمّعين، و precision / recall / F1 لكل class، والـ confusion matrix، و macro-F1 و recall لكل group، و accuracy لكل تسجيل، والتطبيق كان هيقبل إيه في مسار الـ landmarks. و `landmark_gate(probs, labels, hands)` بيطبق قواعد التطبيق تاني:

```
def landmark_gate(probs, labels, hands):
    i = int(np.argmax(probs))
    sign, p = labels[i], float(probs[i])
    if sign not in config.LANDMARK_SIGNS or p < config.LANDMARK_MIN_PROB:
        return None
    top2 = np.sort(probs)[-2:]
    if float(top2[1] - top2[0]) < config.LANDMARK_MIN_MARGIN:
        return None
    if sign == "rasengan_palm" and not open_palm(hands, config.PALM_MIN_THUMB_GAP):
        return None
    return sign
```

بيعمل إيه خطوة خطوة: (1) ياخد الحركة صاحبة أعلى احتمال؛ (2) لازم تكون من `LANDMARK_SIGNS` واحتمالها على الأقل 0.60؛ (3) الفرق بينها وبين اللي بعدها على الأقل 0.30؛ (4) لو هي `rasengan_palm`، لازم فحص الكف المفتوح `open_palm(hands, PALM_MIN_THUMB_GAP)` ينجح. لو أي شرط وقع يرجع `None` (يعني "مفيش حركة"). ليه

ده مهم؟ لأن الموديل ممكن يبقى صح في الـ argmax بس التطبيق يرفضه بسبب الـ thresholds (وده بالظبط اللي حصل مع `clone_seal` في التشخيص). فبنقيس اللي المستخدمة هتشوفه فعلاً. ومن ده بيطلع نسبة فريمات `clone_seal` / palm اللي اتقبلت، والـ false triggers (فريمات idle أو أختام اتقبلت كأنها palm أو clone).

الـ baseline: الموديل المتركب بيتقيّم على G2–G4، الـ groups اللي عمره ما شافها (وبيقف لو الموديل المتركب اتدرب على تسجيل من يوم 1-Nov). المخرجات: `reports/<time>_grouped_cv/metrics.json` و `confusion_<model>.png` (ومنهم `confusion_installed_model_G2-G4.png`). ولا حاجة بتتدرب بجد ولا بتتركب: الموديلات موجودة في الذاكرة بس.

الملف `tools/evaluate_pipeline.py`

مسؤول عن إيه: قياسات إضافية لـ `reports/ai_evaluation/`، فوق `evaluate.py`، وكلها بكود التطبيق نفسه. اتكتب لما ماكانش فيه test set لسه (16 test، 0 validation، 3 train). كل جزء بيكتب `reports/ai_evaluation/parts/<part>.json`.

التشغيل: `python tools/evaluate_pipeline.py <part>` والـ part واحد من:

- `negatives --cache DIR`: كليبات عمرها ما اتدربت لإيدين مش بتعمل أي حركة من حركات التطبيق (مخلب = `chidori` القديم، علامة OK = `fire` القديم، كفين مفتوحين في `data/_discarded`) ومعاهم تسجيلات الـ idle اللي في الـ validation. بيطلع كل recognizer بيقول إيه في كل فريم، وأنهي حركات التطبيق أكدها، وكام jutsu اشتغل في Live و Practice mode. و `rule_of_three` بيطلع الأحداث في الساعة، ولو ما حصلش ولا حدث، الحد الأعلى بثقة 95% = 3 / ساعات التعرض (قاعدة إحصائية: لو ما شفتش حدث في n محاولة، الاحتمال الحقيقي على الأغلب أقل من 3/n).
- `loro`: leave-one-recording-out للـ landmark classifier (يعني سيب تسجيل واحد بره كل مرة): الـ Random Forest بتاع `train.py` بيتدرب في الذاكرة من غير كل تسجيل بالدور. الحركة اللي ليها تسجيل واحد بس مستحيل تطلع صح لما تسجيلها يتساب بره، فالصفوف دي بتظهر بس بتتشال من المقاييس.
- `sequences --cache DIR`: أداءات "متلزقة" (spliced): شبابيك متقطعة من تسجيلات الحركة الواحدة، بترتيب الـ jutsu، وبينهم فراغات idle من 0.4 لـ 1.2 ثانية) من خلال fusion + controller: نسبة التفعيل، الـ jutsu الغلط، الـ latency؛ والأربع jutsu ورا بعض؛ وسلاسل حركات عشوائية طويلة لعد التفعيلات اللي ما كانتش مقصودة. محتاجينها لأن مفيش takes موجّهة.
- `montecarlo`: الـ `SequenceMatcher` لوحده بيتغذى بحركة عشوائية كل 1.8–3.6 ثانية لمدة 200 ساعة محاكاة (Live و Practice)، ومقارنته بقاعدة المطابقة القديمة.
- `fps --url ws://127.0.0.1:8003/ws --fixtures DIR`: بيبعت fixtures كاميرا بصيغة `.y4m` للسيرفر الحقيقي عبر الـ WebSocket زي المتصفح بالظبط، ويسجل التوقيت لكل مرحلة.
- `figures`: رسومات PNG (`confusion_landmarks_loro.png`، `fps_stages.png`).
- `summary`: يجمع كل حاجة في `reports/ai_evaluation/metrics.json`.

دوال مساعدة أساسية: `Clip` و `split_clips` / `negative_clips` / `load_clip` (نتايج YOLOX بتتحفظ cache كـ pickles في `--cache`)، و `Stream` (خط زمني متبني من شبابيك كليبات)، و `controller_for(mode, jutsu)` (Live = كل الـ jutsu بـ `SEQUENCE_MAX_GAP`؛ Practice = jutsu واحد، من غير حد وقت)، و `run_stream` (زي

`evaluate.replay`). السكريبت **بيرفض فولدر cache جوه الـ repository**، لأن الكليبات المتخزنة فيها صورة المستخدمة.

```
if ROOT in cache.parents or cache == ROOT:
    raise SystemExit("refusing to cache clips of the user inside the repository")
```

تحذير مهم لقيناه في الكود (مش مذكور في النسخة الإنجليزي): الدالة `split_clips()` مكتوب في الـ docstring بتاعها "The 19 recordings of data/splits.json"، بس الكود فعلياً بياخد **كل تسجيل ليه مجموعة** في `splits.json`. وقت كتابة الأداة كان ده 19 (16 train + 3 validation). **النهارده ده 68، ومنهم الـ 22 تسجيل test.** يعني لو شغلتي `tools/evaluate_pipeline.py loro` أو `sequences` النهارده:

- `loro` هيعمل leave-one-recording-out على الـ 68 كلهم، يعني هيدرب Random Forest في الذاكرة على تسجيلات **test** ويقيس عليها. ده لمس للـ test set ومخالف لقواعد التقييم في المشروع (حتى لو مفيش موديل بيتركب).
- `sequences` هيقطع شبابيك من تسجيلات test كمان.
- والنص المكتوب في الـ JSON الناتج (زي "there is no test set" و "16 train, 3 validation, 0 test") هيبقى غلط.

عشان كده: **ما تشغليش `loro` ولا `sequences` بشكلهم الحالي** لحد ما الأداة تتعدل إنها تفلتر على train (أو train + validation) بس. الجزء `negatives` آمن نسبياً (بيستخدم الكليبات السلبية + تسجيلات `validation` من نوع `idle` بس)، و `montecarlo` ما بيلمسش بيانات خالص. الأرقام الموجودة في `reports/ai_evaluation/` اتعملت قبل وجود الـ test set، فهي سليمة.

الملف `tools/check_voices.py`

مسؤول عن إيه: يفحص الـ 8 كليبات صوت الشخصيات في `assets/audio/voice/` (`<jutsu>/activation.wav` و `<jutsu>/name.wav` لكل من fire_style و chidori و shadow_clone و rasengan) قبل ما يتستخدموا.

التشغيل: `python tools/check_voices.py`. الـ exit code بيبقى 0 بس لما الـ 8 كلهم موجودين وفي المواصفات.

لكل كليب `check(path, part)` بيقول: مش موجود، أو بيتأكد إنه WAV بـ mono-16، bit PCM، و 44.1 أو 48 kHz؛ طوله 0.25–2.0 ثانية (activation) أو 0.25–2.5 ثانية (name)؛ بحد أقصى 30 ms سكوت قبل الصوت (الإشارة فوق −40 dBFS)؛ الـ peak بحد أقصى −1 dBFS؛ والـ loudness −16 ± 2 LUFS.

الثابت في الكود	القيمة	المعنى
`MAX_LENGTH`	`{"activation": 2.0, "name": 2.5}`	أقصى طول بالثواني
`MIN_LENGTH`	0.25	أقل طول
`MAX_LEAD`	0.03	أقصى سكوت قبل الصوت (ثانية)
`RATES`	`(44100, 48000)`	معدلات العينات المسموحة
`PEAK_MAX`	1.0-	أعلى peak بالـ dBFS
`LUFS_TARGET` ، `LUFS_TOLERANCE`	16.0-، 2.0	الـ loudness المطلوب والسماحية

`k_weighting(rate)` بيصمم الفلترين المبدئيين بتوع ITU-R BS.1770 لأي sample rate، و `integrated_lufs(x, rate)` بيحسب الـ integrated loudness بالـ gating (blocks 400 ms، تداخل 75%، gate مطلق −70 LUFS، gate نسبي −10 LU). (LUFS وحدة لقياس "علو الصوت كما تسمعه الودن"، مش مجرد أعلى نقطة.) والملفات التانية في الفولدر بتتعرض كـ "not used"؛ بالظبط: الكود بيعرض الملفات اللي **مش** `.wav` (وما عدا `README.md`).

ليه ده مهم: السيرفر بيقرا أطوال الكليبات عشان يوقّت التفعيل (voice ثم charge ثم name ثم release). والـ docstring كمان بيقول إن التسجيلات المرخصة بس هي اللي مكانها هناك.

الملف `tools/build_bold_signs.py`

مسؤول عن إيه: يعمل رسومات الحركات بخط عريض اللي تطبيق الويب بيعرضها في كروت السلاسل.

التشغيل: `python tools/build_bold_signs.py`. بيقرا `gesture_references/_sources/signs/*.png` (رسومات خطوط سودا رفيعة، من Schlum على Wikimedia Commons، برخصة CC BY-SA 4.0) وبيكتب `gesture_references/_sources/signs_bold/` (نفس الأسماء والمقاسات، بخلفية شفافة) ومعاهم `CREDITS.txt`. الأصول ما بتتغيرش.

- `stroke_width(gray)`: بيقيس متوسط (median) عرض الخط بـ distance transform (ضعف المسافة على الخطوط المركزية للخط)، بدقة 3 أضعاف.
- `display_scale(w, h)`: بيقدّر الرسمة بتتصغر قد إيه في الكروت (متوسط هندسي على مقاسين للكروت).
- `bolden(gray, factor, min_width)`: بيعرّض كل خط بنسبة عرضه هو حوالين خطه المركزي (التجاعيد الرفيعة بتاخد عرض أدنى)، بيخلي حشوة الضوافر الرمادي أغمق شوية، وبيخلي الخلفية البيضا شفافة.

المعامل بيتختار لكل رسمة لوحدها بحيث الحواف تبقى تقريباً 1.6 CSS pixel على الشاشة، فكل الرسومات تبان بنفس السُمك جنب بعض.

تنبيه: تشغيل الأداة دي **بيكتب فوق** الملفات الموجودة في `gesture_references/_sources/signs_bold/`. هي مش بتلمس البيانات ولا الموديلات، بس ما تتشغلش إلا لو فيه سبب.

الملف `tools/build_gesture_references.py`

مسؤول عن إيه: يبني الكروت المرجعية في `gesture_references/`: لكل فولدر jutsu `step_1.png` ... `step_4.png` (كارت لكل حركة) و `sequence_reference.png` (الأربع خطوات).

التشغيل: `python tools/build_gesture_references.py`.

- ترتيب الخطوات جاي من `config.JUTSU` (نفس الجدول اللي التطبيق بيستخدمه)؛ وكل النصوص (أسماء الحركات، الأسماء اليابانية، التلميحات، العناوين، الـ tags، الـ credits) جاية من `shinobi/catalog.py`، نفس النصوص اللي تطبيق الويب بياخدها من `/api/jutsu`.
- `draw_palm(size)`: بيرسم الـ Rasengan palm المخصوص من وضع إيد واحد اتسجل بـ MediaPipe (`gesture_references/_sources/rasengan_palm_landmarks.json`)، لأن الوضع ده ملوش رسمة منشورة: "كبسولات" للصوابع، ومضلع للكف، وجزء من الدراع، مرسومين مرتين (أشكال غامقة متكبرة، وبعدين أشكال بيضا) فيفضل بس outline نضيف. و `sign_art` بيستخدمه للـ palm، ورسومات Wikimedia للأختام.

- `step_card(...)`: بيرسم كارت 1280×1000: header ملون فيه عنوان الـ jutsu، وعداد الخطوات ونقط التقدم؛ اسم الحركة واسمها الياباني؛ badges (OFFICIAL SEAL / CUSTOM POSE، BOTH HANDS / ONE HAND)؛ الرسمة؛ تعليمات HOW TO / WHERE / HOLD / NEXT؛ وسطر الـ credit.
- `sequence_sheet(...)`: بيرسم لوحة 2500×1000 فيها أربع لوحات للخطوات، وأسهم، وصندوق "activates"، وملحوظة إن الترتيب مهم.
- دوال مساعدة: `font`، `bold`، `regular`، `japanese` (خطوط Windows: Segoe UI و Yu Gothic)، `wrap` (لف الكلام على السطور)، `badge`، `fit`.

`tests/test_catalog.py` بيرسم الكروت دي في فولدر مؤقت ويتأكد إنها متطابقة بايت ببايت مع الكروت المحفوظة في المشروع.

تنبيه: تشغيل الأداة نفسها (مش الاختبار) **بيكتب فوق** الكروت في `gesture_references/`. ولأنها بتعتمد على خطوط Windows، تشغيلها على جهاز خطوطه مختلفة ممكن يطلع صور مختلفة شوية، وساعتها الاختبار هيفشل.

فولدر `reports/`: كل فولدر فيه إيه

الفولدر	هو إيه
`reports/2026-10-31_024126/`	أول تشغيل تجريبي (pilot) لـ `evaluate.py` (validation + train، من غير landmark model، ومن غير تسجيلات test). بيثبت إن الأداة شغالة؛ مش نتيجة.
`reports/ai_evaluation/`	تقييم الـ AI اللي اتكتب قبل ما يبقى فيه أي test set: `README.md` (التقرير، بدرجات قوة الدليل)، و `metrics.json`، و `parts/*.json` من `tools/evaluate_pipeline.py`، وتشغيلين لـ `evaluate.py` (`evaluate_py/` بالموديل المتركب، و `evaluate_py_yolox_path/` من غيره)، و `confusion_landmarks_loro.png`، و `fps_stages.png`
`reports/test_sessions/20261101_140445.json`	log الـ test session المستقلة (22 تسجيل)
`reports/2026-11-01_144049/`	**تقييم 01-11-2026**: `evaluate.py run` على الـ 22 تسجيل test الجداد (عملهم `tools/make_test_set.py`)، بالـ Random Forest المتركب؛ فيه `summary.md` و `metrics.json` وصور confusion للـ landmarks والـ yolox والـ fused. التشغيل ده اتعمل قبل ما الاستبعادات تتعمل، فهو شامل الـ 22 كلهم.

الفولدر	هو إيه
`reports/2026-11-01_diagnosis/`	**التشخيص** لسبب إن التقييم ده طلع واطي: `report.md` ، و `scripts/` (`diagnose.py` ، `sheets.py` ، `ablation.py`) و `evidence/` (contact sheets لكل حركة، وشرايط فريمات كثيفة لأول ram وأول tiger، و `diagnosis.json` ، و `ablation.json` ، و `timeline.json`). تشخيصي بس؛ ولا حاجة اتظبطت على الـ test set.
`reports/training_sessions/session1_20261101_155047.json`	الـ log الـ training session الموجّهة رقم 1 (27 تسجيل محفوظ)
`reports/2026-11-01_session1_plan/`	**خطة الـ session 1**: فحص الـ 27 تسجيل الجداد، وتجربة التقسيم على الناشف (dry run)، وخطة التدريب المعدلة (grouped cross-validation، اختيار الموديل، حمايات الـ leakage)
`reports/2026-11-01_191152_grouped_cv/`	**Phase 3**: `summary.md` **مقارنة الـ grouped CV +** (Phase 2: condition-grouped cross-validation)، و `phase3_comparison.md` ضد Random Forest) (MLP)، و `metrics.json` من `tools/grouped_cv.py` ، وصور confusion للـ RF والـ MLP والموديل المتركب على G2–G4

ملحوظة: تجربة الموديلات الجديدة (RF ضد MLP كموديلات تجريبية) اللي شغالة بالتوازي النهارده ممكن تضيف فولدر تقرير جديد. النتايج دي لسه مستنية مراجعة صاحبة المشروع، ومش مكتوبة هنا عن قصد.

فولدر `tests/` : كل ملف اختبار بيتأكد من إيه

Testing (الاختبارات الأوتوماتيكية) يعني كود صغير بيشغّل أجزاء من المشروع ويتأكد إنها بتعمل المطلوب. كل اختبارات Python بتشتغل بـ `pytest` (أداة بتدور على الدوال اللي اسمها بيبدأ بـ `test_` وتشغلها وتقولك نجحت ولا فشلت).

قواعد عامة في الاختبارات:

- الاختبارات اللي بتكتب ملفات بتستخدم `tmp_path` بتاع pytest (فولدر مؤقت جديد كل مرة)، فعمرها ما بتلمس `data/` أو `models/` الحقيقيين.
- بعض الاختبارات بتقرا تسجيلات حقيقية من `data/videos/` (قراية بس).
- الاختبارات اللي محتاجة الـ landmark model المدرب بيتعمل لها skip لو `models/gesture_model.joblib` مش موجود.
- `tests/__init__.py` فاضي؛ وظيفته بس إنه يخلي `tests` package عشان الملفات تقدر تعمل import من بعض (مثلاً `from tests.test_splits import write_session`).

اتأكدنا بـ `pytest --collect-only` (بيعد الاختبارات من غير ما يشغلها): **224 اختبار** في 12 ملف. الـ `parametrize` (تشغيل نفس الاختبار بكذا قيمة) بيخلي عدد الاختبارات أكبر من عدد الدوال.

الملف	عدد الاختبارات
`tests/test_activation.py`	18
`tests/test_catalog.py`	4
`tests/test_core.py`	17
`tests/test_engine.py`	11
`tests/test_evaluate.py`	16
`tests/test_events.py`	28
`tests/test_fusion.py`	18
`tests/test_recording_tools.py`	16
`tests/test_sequences.py`	33
`tests/test_server.py`	20
`tests/test_shadow_clone.py`	31
`tests/test_splits.py`	12

(ده عدد الاختبارات اللي اتجمعت، مش نتيجة تشغيل. ما شغلناش الاختبارات في الشرح ده.)

الملف `tests/test_core.py`

قطع البناء الأساسية، بإيدين صناعية (synthetic، يعني متولدة بالكود مش متسجلة):

- الـ features: حجم الـ feature vector وحالة غياب الإيد التانية؛ الـ features ما بتتأثرش بمكان وحجم إحداثيات الـ world؛ العكس مرتين بيرجع الأصل (`test_mirror_twice_is_identity`)؛
- الـ GestureStabilizer: محتاج مسكة (hold) وبيشتغل مرة واحدة؛ بيتجاهل القفزات اللي في فريم واحد؛ الـ smoothing محسوب بالثانية مش بالفريم (وبيساوي الـ EMA القديم اللي بالفريم عند المعدل المرجعي بتاعه)؛ بيرجع جاهز تاني بعد ما الحركة تتساب؛
- الـ SequenceMatcher والـ ActionController: السلسلة الكاملة ليها الأولوية وبتنتهي بالوقت؛ الـ cooldown؛ حركة واحدة مش بداية سلسلة بتشغل حاجة؛
- كل effect بيترسم من غير crash (اختبار لكل effect عن طريق `parametrize`)؛
- `train.py` من أوله لآخره على بيانات صناعية في فولدر مؤقت: الـ label القديم `rasengan` بيتدرب كـ `rasengan_palm`، والقديم `fire` بيتخطى، والـ classifier الناتج الـ labels بتاعته `idle, tiger, rasengan_palm`؛
- الـ "fast forest" بتاع الـ classifier (بيمشي على الشجر بنفسه لفريم واحد) بيدي احتمالات sklearn بالظبط.

```
subprocess.run([sys.executable, "train.py", "--data", str(tmp_path / "data"), "--out", str(out),
                "--step", "1"], check=True, cwd=config.ROOT)
from shinobi.classifier import GestureClassifier
clf = GestureClassifier(out)
assert clf.labels == ["idle", "tiger", "rasengan_palm"]
```

بيعمل إيه: الاختبار بيعمل فولدر مؤقت فيه أربع 3 sessions، labels (`idle`، `rasengan`، `tiger`، `fire`) لكل واحد، ويشغّل `train.py` الحقيقي كبرنامج منفصل، والـ `--out` في الفولدر المؤقت. بما إن الفولدر المؤقت مفيهوش

`splits.json`، `train.py` بيشتغل بالوضع القديم. وبعدين يتأكد إن الموديل الناتج فيه بالظبط `idle` و `tiger` و `rasengan_palm` (يعني `rasengan` اتترجم و `fire` اتشال). الموديل الحقيقي ما بيتلمسش.

الملف `tests/test_splits.py`

بيختبر `splits.py`، و `python evaluate.py splits`، و `train.py` مع الـ splits، كله في فولدرات مؤقتة. فيه دالة مساعدة `write_session` بتعمل تسجيل صغير بشكل `collect_data.py`. بيتأكد من:

- كل تسجيل في مجموعة واحدة بالظبط والـ labels القديمة بتتترجم؛
- تسجيل متكرر مرتين أو بمجموعة مش معروفة بيترفض؛
- ملفين بنفس الـ id بيترفضوا؛
- التقسيمات ثابتة والتعديلات اليدوية بتكسب؛
- التسجيلات الجديدة بس هي اللي بتاخد مجموعة؛
- الـ test جاي من يوم أو إضاءة تانية وعمره ما بيشاركها مع الـ train؛
- نفس الإضاءة في يوم تاني بتتحسب مختلفة؛
- ترتيب الملء train ثم test ثم validation؛
- الـ takes بتتجنب اليوم والإضاءة اللي الحركات بتاعتها اتدربت فيهم؛
- الـ review بيحذر من المجموعات الناقصة ومن test من يوم التدريب؛
- أمر الـ splits بيكتب ملف الـ splits بس؛
- و `train.py` بيستخدم sessions الـ train بس وعمره ما بيفتح بتوع الـ test.

مهم: الاختبار ده بيشغّل `evaluate.py splits` **في فولدر مؤقت** ببيانات صناعية، فده آمن تماماً. التحذير من الأمر ده خاص ببيانات المشروع الحقيقية بس.

الملف `tests/test_evaluate.py`

بيختبر `evaluate.py`:

- الـ frame metrics على مثال معروف النتيجة؛
- توقعات مثالية وتوقعات فاضية؛
- confusion matrix صفوفها غير أعمدتها؛
- ملخص الـ latency؛
- نتايج الـ takes؛
- إن `saved_video_frames` ماشي على قاعدة أدوات التسجيل؛
- الإيدين بيتعاد بناؤهم ببكسل الفيديو؛
- YOLOX لوحده بيتجاهل الـ classes اللي مش مستخدمة والـ scores الضعيفة؛
- جدول الختم محدود المعدل وبياخد أحدث فريم؛
- الـ replay بيشغل الـ jutsu مرة واحدة زي التطبيق؛
- موديل اتدرب من غير splits ما بيتستخدمش؛

- تشغيل كامل بيكتب تقرير ومش بيكتب أي حاجة في `data/`؛
- سجل الاستبعادات بيتفحص؛
- التسجيلات المستبعدة بتتقيّم بس بتتحسب لوحدها؛
- استبعادات بره التشغيل أو مقفولة ما بتغيرش حاجة؛
- سجل استبعادات بايظ بيوقف التشغيل.

الملف `tests/test_recording_tools.py`

دوال أدوات التسجيل الموجّهة:

- خطة التدريب فيها الأعداد المتفق عليها (6 train / 2 validation لكل حركة ضعيفة ولـ idle، و 3 / 1 لكل حركة قوية، و 68 في المجموع)؛
- الـ validation session لوحدها وكل block ليه كاميرا معروفة؛
- الـ palm دايماً "ONE hand only" والـ idle عمره ما بيطلب كف مفتوح؛
- `--plan` مش محتاج كاميرا (الـ Camera بتتبدل بدالة بتفشّل الاختبار لو اتنادت)؛
- الـ session 3 محتاجة tag إضاءة؛
- المراجعة بتختار ست فريمات متوزعة، وبترسم الإيدين بمقاس الكاميرا، وبتربط الزراير بـ keep / redo / quit، وشباك مقفول ما بيحفظش حاجة؛
- `save_new` بيحتفظ بالـ meta الزيادة بس عمره ما بيكتب فوق ملف؛
- `assign_recordings`: الـ dry run ما بيكتبش حاجة، و `--apply` بيسيب الـ entries القديمة زي ما هي، وكل حالة رفض (موجود في test، مستبعد، مفيش فيديو، label غلط، المجموعة المخططة test، مفيش ملف، متكرر مرتين) ما بتكتبش حاجة.

الملف `tests/test_sequences.py`

سلاسل الـ jutsu ذات الأربع خطوات:

- كل jutsu ليه أربع حركات معروفة وأول حركة خاصة بيه؛
- كل سلسلة بتشغل الـ jutsu بتاعها بس؛
- الصور المرجعية موجودة لكل خطوة؛
- أي ترتيب تاني لنفس الحركات ما بيشغلش حاجة؛
- السلاسل الناقصة ما بتشتغلش؛
- وقفة أطول من أقصى فجوة بتعيد السلسلة من الأول؛
- حركة غلط بتحافظ على التقدم، والتفعيل بيصفّر كل حاجة؛
- آخر حركة في jutsu ما بتبدأش jutsu تاني؛
- الإعدادات المتعارضة (نفس الحركة مرتين ورا بعض، jutsu مستخبي جوه jutsu تاني) بترفض؛
- الحركات الممسوكة بتشغل مرة واحدة بالظبط، حتى بعد ما الإيد تختفي لحظة؛
- تكرار أول أو آخر حركة ما بيضرش؛

- الـ Practice mode ملوش حد وقت وبيحتفظ بالخطوات بعد محاولات غلط كتير؛
- الـ Free (Live) mode بيتابع كل الـ jutsu مرة واحدة، كل واحد بساعته؛
- وكمان: layout الـ timeline بتاع `record_sequences.py`، والـ takes السلبية المتولدة عمرها ما بتشغل حاجة، و take محفوظ فيه النقط والـ labels بتاعة الفريمات.

الملف `tests/test_fusion.py`

YOLOX detector و SignFusion:

- الـ detector ما بيرجعش حاجة على فريم فاضي، وبيلاقي كل ختم في الرسمة المرجعية بتاعته؛
- الـ non-maximum suppression (لما فيه كذا box فوق بعض، يسيب الأحسن) بيسيب أحسن box؛
- الـ worker اللي في الخلفية بيبلّغ عن الـ latency؛
- ختم YOLOX محتاج نتيجتين ورا بعض على إيد؛
- box بعيد عن الإيدين بيترفض؛
- من غير أي إيد محتاج score أعلى؛
- النتايج القديمة بتتجاهل؛
- الأختام اللي مفيش jutsu بيستخدمها بتتجاهل؛
- الـ landmark model مسؤول عن `clone_seal` و palm وعمره ما بيغطي على أختام YOLOX؛
- الـ palm محتاج إيد مفتوحة مش علامة OK (وعلامة OK في آخر خطوة عمرها ما بتشغل Rasengan)؛
- حركة landmark مش متأكدة = مفيش حركة؛
- التعارضات بيكسبها الـ score الأعلى وبتتعد؛
- Chidori من YOLOX بيشتغل مرة واحدة؛
- Rasengan بيخلط الموديلين وبيشتغل مرة واحدة؛
- ترتيب غلط ما بيشغلش حاجة.

الملف `tests/test_engine.py`

`shinobi/engine.py` و `shinobi/main.py` من غير شباك، باستخدام فريمات من تسجيل idle حقيقي من `data/videos/idle/` (بالظبط `me_night_20261030_221934.mp4`، قراية بس):

- أي مقاس فريم (عرضي، طولي) والـ Shadow Clone شغال؛
- تغيير المقاس في نص Shadow Clone؛
- النتيجة فيها اللي الـ HUD والسيرفر محتاجينه؛
- الـ reset الـ Practice mode وأرقام التفعيل؛
- حد الوقت في الـ Free mode والـ timeout؛
- ولا حركة بتتحسب لحد ما الـ engine يبقى جاهز؛
- النقط بتترسم بس لما تتطلب؛
- و `main.py` بيشتغل من غير شاشة بكاميرا مزيفة، وزرار 1 بيبدأ الـ Shadow Clone.

الملف `tests/test_events.py`

`shinobi/events.py`، اللي بيحول حالات الـ controller والـ effect لـ events في بروتوكول الويب:

- مفيش حاجة حصلت، مفيش حاجة بتتبعت؛
- حركة وبعدين event خطوة؛
- آخر حركة بتبقى event jutsu؛
- قواعد الحركة الغلط في Free و Practice mode؛
- التكرار بياخد event حركة بس؛
- الـ timeouts بتتبلغ مرة واحدة للـ jutsu اللي متقدم أكتر (وعمرها ما بتتبلغ في Practice)؛
- الـ reset وتغيير الـ mode بينسوا السلسلة من غير events؛
- دورة حياة الـ effect (stopping وبعدين end، والاستبدال)؛
- event الـ clones بس لما عدد النسخ الظاهرة يبقى أكبر من أي وقت فات؛
- واختبارات مع الـ ActionController الحقيقي (أخطاء الـ Live بتحافظ على التقدم، الخطوات اللي تحت الحد بشوية بتشتغل، الـ Practice ما بيصفّرش حاجة بعد 30 ثانية أو 10 دقايق، ونتايج YOLOX الثابتة بتشغل Chidori مرة واحدة).

الملف `tests/test_activation.py`

مراحل تفعيل الـ voice :(`effects/base.py`) jutsu ثم charge ثم name ثم release، متوقتة على كليبات الصوت المتركبة:

- من غير كليبات، التحضير قصير؛
- بالكليبات، المراحل بتاخد طول ملفات الـ WAV؛
- قايمة الأصوات WAV بس ومعاها الأطوال؛
- كل effect بيتحضر وبعدين بيتطلق في وقته؛
- الـ effects اللي بتبدأ باليد (الكيبورد) ملهاش تحضير وما بتبلغش عن مراحل؛
- مستوى الشحن بيعلى؛
- الـ Shadow Clone بيظهر بس بعد نداء الاسم؛
- الإيقاف أثناء التحضير بيختفي تدريجياً من غير release؛
- effects الإيد بتستحمل اختفاء الإيد طول مدتها، بحد أدنى متظبط؛
- كل مرحلة بتتبلغ مرة واحدة وبالترتيب حتى بعد فريم بطيء.

الملف `tests/test_shadow_clone.py`

الـ Shadow Clone بـ 10 نسخ بالظبط (والعدد ده نهائي في المشروع: 10 نسخ + المستخدمة = 11 شخص، وعمره ما بيتقلل عشان الـ FPS):

- entry واحد لكل نسخة، متوزعين بتوازن، وكل التأخيرات مختلفة؛

- قوايم النسخ الغلط بترفض، مش بتتقصر؛
- كل النسخ بترسم من فريمات حية والشخص بيفضل قدام؛
- جسم كل نسخة بيفضل جوه الفريم وكل الـ 10 رؤوس باينين أياً كان المستخدمة واقفة فين؛
- النسخ بتروح مكان ما فيه مساحة؛
- مستخدمة ماشية بتحرك النسخ في دخان من غير رعشة؛
- مساحة قليلة بتحتفظ بكل نسخة وبتقول كده؛
- الطبقة الموازية متأخرة فريم واحد والمستخدمة فاضلة قدام؛
- راس متقطوعة من الحافة اللي فوق بتاخد حافة ناعمة؛
- النسخ بتظهر وتختفي بدخان؛
- السلسلة بتبدأ الـ effect بكل النسخ؛
- الـ segmentation mask ما فيهاش NaN على حيطة فاتحة سادة.

الملف `tests/test_catalog.py`

`shinobi/catalog.py`، النسخة الوحيدة من نصوص الكروت والويب:

- كل حركة و jutsu ليها النصوص بتاعتها؛
- الـ kanji هو اللي في التعليمات؛
- عناصر الـ credits فيها بالظبط أربع مفاتيح (`text`، `url`، `license`، `license_url`)؛
- والكروت المرجعية اللي بترسم من الـ catalog (في فولدر مؤقت) متطابقة بايت ببايت مع المحفوظة (ولو Pillow مش موجود، الاختبار ده بيتعمل له skip).

الملف `tests/test_server.py`

`shinobi/server.py`:

- الـ REST endpoints: `/api/info`، الـ catalog ماشي على الـ config وكل صورة بتتقدم، الـ credits وكل صوت بيتقدم، قايمة كليبات الصوت، والـ frontend fallback ومساعدة الـ build؛
- الـ binary frame pack / unpack (تحويل الفريم لبايتات والعكس) بيرجع زي ما هو؛
- بروتوكول الـ WebSocket: hello، ping، mode، options، reset؛ الفريم بيروح ويرجع مع العكس والتصغير؛ فريمات كتير مستنية بتتخطى بالترتيب؛ الفريمات البايظة بتتخطى؛ "busy" لما الـ sessions تزيد عن الحد الأقصى والخانة بتفضى بعد الفصل؛ الـ crash بيقفل الـ session دي بس.
- اختبارات الـ end-to-end بتبعت تسجيلات المستخدمة الحقيقية عبر الـ WebSocket في الوقت الحقيقي، وبتتأكد إن الـ pipeline الحقيقي بيشغل Fire Style، و Chidori في Practice، ولا حاجة لترتيب غلط، وإعادة المحاولة بعد حركة غلط، والـ session اللي بتعمل الأختام بس هي اللي بتاخد الـ jutsu، و Shadow Clone و Rasengan (الاتنين دول محتاجين الـ landmark model، وإلا skip)، ورسم الـ Shadow Clone عبر الـ WebSocket.

ملحوظة أمانة من الكود نفسه: تسجيل الـ `clone_seal` اللي الاختبار ده بيستخدمه (`clone_seal/me_night_20261030_214750.mp4`) هو نفس التسجيل اللي الموديل المتركب **اتدرب عليه**، وتسجيل الـ `rasengan_palm` هو تسجيل الـ validation. التعليق في الكود بيقول صراحة إن الاختبارات دي بتثبت إن التوصيل شغال (wiring)، مش إن التعرف بيشتغل على تسجيلات جديدة.

اختبارات المتصفح (Browser tests) باختصار

غير اختبارات Python، فيه نوعين اختبارات للواجهة (الشرح الكامل في الجزء التالت):

- **Frontend unit tests** بـ vitest: `cd frontend; npm test`.
- **Browser end-to-end tests** في `frontend/e2e/`: سكريبتات Node بتفتح Edge حقيقي بكاميرا مزيفة (ملفات `.y4m`)، مع الـ backend الحقيقي وسيرفر Vite. السكريبتات: `live-jutsu.mjs` (بياخد اسم jutsu، الافتراضي `fire_style`)، `live-wrong-order.mjs`، `practice.mjs`، `sequence-rules.mjs`، `two-browsers.mjs`، `reconnect.mjs`، `responsive.mjs`، `music-flow.mjs`، `audio-timeline.mjs`، ومعاهم `lib.mjs` (الدوال المشتركة) و `make_fixtures.py` (بيعمل ملفات الكاميرا المزيفة).

الـ fixtures فيها وش المستخدمة، فـ `make_fixtures.py --out` إجباري ولازم يكون **بره الـ repository**، والسكريبتات بترفض تشتغل من غير متغير البيئة `SHINOBI_FIXTURES`. الأوامر بالظبط في القسم الجاي.

الحالة الحالية للموديل والتقييم (بصراحة)

القسم ده بيلخص ملفات التقارير بأمانة. ولا رقم هنا اتغير عن اللي في التقارير. وكل البيانات من شخص واحد ("me")، وكاميرا ويب واحدة، وأوضة واحدة. ده أهم حد لازم تفتكريه وإنتي بتقري أي رقم.

أعداد البيانات

المجموعة	التسجيلات	جاية منين
train	43	16 من 30-10-2026 (تسجيل واحد لكل ختم، palm 3، idle 4، monkey 2؛ معظمهم بالليل) + 27 من الـ session الموجّهة 1 يوم 01-11-2026 (ضوء نهار، كاميرات A و B و A-far، كلهم اتراجعوا بالست فريمات)
validation	3	palm 2 + idle 1، من 30-10-2026؛ مفيش ولا ختم، فما يقدرش يختار موديل لـ 8 من الـ 10 حركات
test	22	الـ test session المستقلة يوم 01-11-2026 (14:04–14:24): كل حركة مرتين، idle 4؛ **5 منهم مستبعدين** من النتايج، فالسليم 17

Ignored: 8 تسجيلات بـ labels قديمة. من غير مجموعة: تسجيل snake واحد مرفوض. الـ 2 sessions و 3 من خطة التدريب المستخدمة لغتهم، فمفيش تسجيلات جديدة هتتعمل دلوقتي.

تقييم 01-11-2026 (`reports/2026-11-01_144049`)

بالـ Random Forest المتركب، على الـ 22 تسجيل test كلهم (قبل الاستبعادات):

الـ recognizer	التسجيلات	الفريمات	accuracy	macro-F1	balanced accuracy
landmarks	21	6450	62.7%	0.615	61.9%

الـ recognizer	التسجيلات	الفريمات	accuracy	macro-F1	balanced accuracy
yolox	22	3903	69.3%	0.604	58.0%
fused (قرار التطبيق)	22	7147	49.4%	0.462	45.2%

يعني: الـ (macro-F1 0.615) 62.7% landmarks accuracy، و YOLOX لوحده 69.3% (0.604)، وقرار التطبيق بعد الدمج 49.4% (0.462). نقط الضعف كانت snake و tiger و ram و dog (من ناحية YOLOX)، و `clone_seal` (اترفض بسبب الـ thresholds) و `rasengan_palm` (اتقال عليه idle).

إزاي تقري الجدول ده: الـ fused أوطى من الاتنين لوحدهم. ده معناه إن المشكلة مش بس في الموديلات، لكن كمان في طريقة الدمج والـ thresholds، وده اللي التشخيص طلّعه.

نتايج التشخيص (`reports/2026-11-01_diagnosis/report.md`)

1. تلات تسجيلات test مش بتوري الـ label بتاعها (tiger 140826: مفيش Tiger؛ Ram :140631 ram بس لمدة ~9 ثواني؛ idle 141753: كفوف مفتوحة بدل علامة OK). أول اتنين جم من النسخة القديمة من أداة التسجيل اللي كانت بتسبق المستخدمة.
2. تسجيلين الـ palm في الـ test اتعملوا بإيدين، بينما التعريف وكل تسجيلات الـ palm في التدريب بإيد واحدة. الـ classifier بيقرا الكفين على إنهم idle. (دول الاتنين + التلاتة اللي فوق = الـ 5 استبعادات.)
3. إعداد الكاميرا اتغير: التدريب (Oct 30) في مستوى الوش؛ الـ (Nov 1) test تحت ومايلة لفوق، مع ضوء شباك قوي. الـ scores Ram / Tiger / Snake / Dog بتاعة YOLOX بتنهار هناك، رغم إن YOLOX عمره ما اتدرب على بيانات المشروع، يعني الظروف هي اللي اتغيرت، مش الموديل بس.
4. الـ fusion بيسمع لموديل واحد بس لكل حركة. على تسجيلات الـ test اللي اتعملت صح، الـ landmark model كان صح في 91% Ram، و 75% Tiger، و 95–100% Dog، في الوقت اللي YOLOX كان فيه 0–27%؛ والإجابة الصح بتاعة الـ landmarks بتترمي.
5. الـ Clone Seal بيترفض أساساً بسبب `LANDMARK_MIN_PROB = 0.60` (هو أعلى class في 95% من الفريمات بس الاحتمال $p \approx 0.47$)، مع إن الإيد اللي فوق متقطوعة من حافة الفريم، وتسجيل تدريب واحد بس.
6. بيانات قليلة أوي عشان نختار أي حاجة بعدل: تسجيل تدريب واحد لكل حركة، و validation فيه idle و palm بس.

التوصيات كانت: مراجعة الست فريمات في أداة التسجيل (اتعملت)، بيانات تدريب أكتر تنوعاً (الـ 1 session اتسجلت)، إعادة التدريب على الـ train بس، مقارنة أشكال الـ fusion على الـ validation (أبداً على الـ test)، وتقييم نهائي واحد على الـ holdout من غير تغيير.

خطة الـ 1 session (`reports/2026-11-01_session1_plan/report.md`)

الـ 27 تسجيل الجداد كان فيهم إيدين في 100% من الفريمات، و palms بإيد واحدة، وعلامة OK حقيقية. وحتى قبل التدريب عليهم، بيّنوا فشل في الأختام من ناحية YOLOX (dog بكاميرا واطية 6%، و boar بكاميرا واطية 0%، و snake والمستخدمة قاعدة لورا 0%)، وده حاجة موديل landmarks أحسن لوحده مش هيقدر يصلحها من غير تغيير في الـ fusion. الـ dry run بتاع `assign_recordings.py` قسّم 27، ورفض 0، وحافظ على الـ 41 entry القديمة زي ما هي؛ وبعدين اتنفذ (train من 16 لـ 43). الخطة: اختيار الموديل بـ condition-grouped cross-validation على الـ TRAIN (لأن الـ validation مفيهوش أختام)، الـ thresholds والـ fusion يفضلوا زي ما هم، backup للموديل

المتركب قبل أي استبدال، وأرقام الـ holdout النهائية تتقال على إنها "best available, not fully independent" (لأن الـ holdout اتسجل في نفس اليوم ونفس الأوضة زي الـ session 1، والـ palm ملوش ولا تسجيل test سليم خالص).

الـ Grouped Cross-Validation ومقارنة Phase 3

من `reports/2026-11-01_191152_grouped_cv/summary.md` و `phase3_comparison.md` (43 تسجيل، 7621 TRAIN فريم، groups من G1 لـ G4، cross-validation، **مش test**):

	Random Forest	MLP
macro-F1، مجمّع	0.856	0.889
accuracy، مجمّع	0.838	0.871
macro-F1 على G2–G4 (الموديل المتركب هناك: 0.639)	0.877	0.901
macro-F1 للـ folds، المتوسط ± الانحراف المعياري	0.840 ± 0.051	0.859 ± 0.098
أسوأ fold	0.789 (G1)	0.775 (G4)
وقت التوقع لكل فريم، median	0.41 ms	1.20 ms
فريمات `clone_seal` اللي قواعد التطبيق قبلتها	99.9%	100%
فريمات `rasengan_palm` اللي اتقبلت	59%	82%
فريمات idle اتقبلت غلط كـ palm	0.18% (2)	0.9% (12)

تفاصيل الـ groups من `summary.md`:

الـ group	التسجيلات	الفريمات	ملاحظات
G1	16	2535	كل حاجة من 30-10-2026
G2	10	1772	1-Nov، الكاميرا في مستوى الوش
G3	10	2017	1-Nov، الكاميرا تحت
G4	7	1297	1-Nov، قاعدة لورا؛ مفيش monkey / boar / horse

و macro-F1 لكل group اتساب بره (G1 / G2 / G3 / G4): Random Forest 0.789 / 0.902 / 0.859 / 0.809، و MLP 0.834 / 0.826 / 1.000 / 0.775.

إزاي تقري الأرقام دي: التدريب على ظروف أكتر بيساعد جامد على الظروف اللي ما اتشافتش (من 0.64 لـ 0.88–0.90 على G2–G4). الـ MLP أحسن في المتوسط وبيصلح Boar بالكاميرا الواطية والـ palms القديمة؛ والـ RF أثبت (الانحراف 0.051 مقابل 0.098) وعنده فريمات idle أقل اتقبلت كـ palm. الاتنين بيفشلوا في Snake والمستخدمة قاعدة لورا (G4) و Tiger بتاع الإعداد القديم (G1)؛ والـ precision بتاع idle واطي (حوالي 0.6) لأن فريمات الحركات اللي الموديل مش متأكد منها بتقع على idle. الـ Clone Seal اتحل في مسار الـ landmarks في الظروف دي. ومعظم الخانات (group × class) فيها تسجيل واحد بس، فالأرقام بتتهز كتير.

توصية Phase 3 كانت الـ MLP، بس **بعد** (أ) موافقتك، و (ب) option صغير صريح في `train.py` يدرب الموديل ده (النهارده `train.py` كان هيختار بالـ cross-validation بالتسجيل بتاعه هو)، و (ج) replay للـ MLP بعد إعادة تدريبه على تسجيلات الـ idle من خلال stabilizer + fusion عشان نتأكد إنه عمره ما بيأكد Rasengan palm غلط؛ وإلا الـ RF هو البديل الآمن. فيه backup للموديل الحالي (`SHINOBI-AI_backup_20261101_190821_phase2_model`).

المشروع واقف فين النهارده

- **الموديل الـ production الحالي هو** (`models/gesture_model.joblib` ، md5 `ed330ca7…`) **MLP**، اتدرب على الـ 43 تسجيل train بـ `train.py --model-type mlp`، واتركب يوم 2026-11-01 بموافقة صاحبة المشروع. التقرير في `reports/model_experiment_20261101/deployment.md` . الـ Random Forest القديم (md5 `be848524…`) محفوظ في `models/previous_production_20261101/` وفي الـ backup على الديسكتوب.
- نتايج الـ grouped CV على الـ 43 train: **RF 0.856 / MLP 0.889 macro-F1** (cross-validation، مش test).
- تجربة الموديلات (RF ضد MLP) خلصت ونتايجها في القسم الأخير من الجزء ده، والـ MLP هو اللي اتركب.
- إعادة التدريب على الـ 43 تسجيل TRAIN، واختيار الموديل، وأي تغيير في الـ fusion كلهم **مستنيين موافقتك**.
- التقييم النهائي المفروض يتعمل مرة واحدة، بعد ما كل حاجة تتثبت، على الـ 17 تسجيل holdout السليمين، والنتايج تتعرض بالاستبعادات الخمسة ومن غيرها. وحسب قواعد التقييم في المشروع، الـ holdout القديم ده بقى دلوقتي **diagnostic holdout** (اتبص عليه أثناء التشخيص)؛ والرقم النهائي الحقيقي محتاج **test set جديد** يتسجل في ظروف جديدة. والضبط بيتعمل على الـ validation أو cross-validation على الـ train بس، أبداً على الـ test.

جدول: إيه اللي implemented وإيه اللي tested وإيه اللي experimental وإيه اللي لسه

الحاجة	الحالة
التسجيل بـ `collect_data.py`	implemented ومستخدم (أول 16 test + validation + train)
أدوات التسجيل الموجّهة + مراجعة الست فريمات	implemented ومتختبرة؛ الـ test session (جزئياً) والـ 1 session اتسجلوا بيها
`record_sequences.py` (takes سلاسل)	implemented ومتختبر، **ما اتستخدمش** (مفيش takes)
التقسيم بالتسجيل + الـ splits + الاستبعادات	implemented ومتختبر ومستخدم
`train.py` (وضع الـ splits)	implemented ومتختبر؛ الموديل المتركب منه (2026-10-31)
option في `train.py` لاختيار موديل معين	**اتضاف** (`--model-type random_forest` أو `--model-type mlp`) يوم 2026-11-01 عشان تجربة الموديلات
`evaluate.py run` مع حارس الـ leakage والاستبعادات	implemented ومتختبر؛ اتشغل على الـ 22 test قبل الاستبعادات
تقييم بعد الاستبعادات على الـ 17 السليمين	**لسه ما اتعملش** (مستني تثبيت كل حاجة)
`tools/grouped_cv.py`	implemented ومستخدم (Phase 2)؛ مالوش اختبارات مخصوصة في `tests/`
`tools/evaluate_pipeline.py`	implemented ومستخدم قبل وجود الـ test set؛ `loro` و `sequences` **مش آمنين النهارده** (بيشملوا الـ test)
موديل مدرب على الـ 43 train	**تجريبي فقط** (التجربة الجارية)، النتايج مستنية المراجعة، مش متركب
تغيير الـ fusion (مثلاً سماع الـ landmarks للأختام)	**لسه** (توصية بس)
الـ 2 sessions و 3 (ليل، validation)	**اتلغت**
test set جديد مستقل للرقم النهائي	**لسه** (مطلوب)

حدود لازم تتقال بصراحة

- شخص واحد، كاميرا واحدة، أوضة واحدة: مفيش أي رقم بيقول التطبيق هيشتغل إزاي مع ناس تانية.
- الـ validation فيه 3 تسجيلات بس ومفيهوش ولا ختم، فما بيقدرش يختار أو يظبط حاجة للأختام.
- معظم الخانات في الـ grouped CV فيها تسجيل واحد، فالفرق بين 0.856 و 0.889 ممكن يتغير مع بيانات أكتر.
- الـ palm ملوش ولا تسجيل test سليم (الاتنين اتستبعدوا).
- الـ test القديم اتسجل في نفس يوم ونفس أوضة الـ session 1، فحتى بعد الاستبعادات هو "best available, not fully independent".
- YOLOX في التقييم بيشوف فيديو 480p، مش الفريم الحي 720p.

إزاي تشغلي كل حاجة

كل الأوامر لـ Windows PowerShell من فولدر المشروع `C:\Users\user\Desktop\SHINOBI-AI`، باستخدام الـ virtual environment بتاع المشروع (`.venv`: بيئة بايثون معزولة فيها المكتبات بتاعة المشروع بس).

التجهيز (Setup)

```
py -3.12 -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
cd frontend; npm install; cd ..
```

بيعمل إيه: السطر الأول بيعمل الـ virtual environment بـ Python 3.12. التاني بينزّل مكتبات بايثون بالإصدارات المثبتة. التالت بيدخل فولدر الواجهة وينزّل مكتبات JavaScript ويرجع. (لو scipy أو Pillow ناقصين لأي سبب: `.venv\Scripts\python.exe -m pip install scipy pillow`.)

تشغيل التطبيق

```
.venv\Scripts\python.exe -m shinobi.server                # web server on http://localhost:8000
.venv\Scripts\python.exe -m shinobi.server --port 8765    # another port
.venv\Scripts\python.exe -m shinobi.main                  # the OpenCV window version (keys 1-4, 0,
Q)
```

السيرفر عنده كمان `--host` (افتراضي `127.0.0.1`) و `--max-sessions` (افتراضي 2). نسخة شباك OpenCV لسه موجودة لحد ما الـ AI يستقر، والهدف النهائي تطبيق الويب.

تطوير الواجهة والـ build

```
.venv\Scripts\python.exe -m shinobi.server                # terminal 1: backend on 8000
cd frontend; npm run dev                                  # terminal 2: Vite on
http://localhost:5173
cd frontend; npm run build                                # production build into frontend\dist
cd frontend; npm run typecheck                            # TypeScript check only
cd frontend; npm run lint                                 # oxlint
```

بعد `npm run build`، السيرفر لوحده بيقدم التطبيق على http://localhost:8000.

الاختبارات

```
.venv\Scripts\python.exe -m pytest -q                       # all Python tests
.venv\Scripts\python.exe -m pytest -q tests\test_splits.py  # one module
.venv\Scripts\python.exe -m pytest -v -k cooldown           # tests whose name contains "cooldown"
cd frontend; npm test                                       # vitest (frontend unit tests)
```

اختبارات المتصفح من أولها لآخرها (Edge حقيقي بكاميرا مزيفة، backend حقيقي، سيرفر Vite للتطوير). الـ fixtures فيها وشك، فاكتبيها **بره** الـ repository:

```
.venv\Scripts\python.exe frontend\e2e\make_fixtures.py --out C:\path\outside\repo\fixtures
.venv\Scripts\python.exe -m shinobi.server                  # terminal 1
cd frontend; npm run dev                                    # terminal 2
cd frontend; $env:SHINOBI_FIXTURES = "C:\path\outside\repo\fixtures"; node e2e/live-jutsu.mjs
fire_style
```

سكريبتات e2e تانية في `frontend/e2e/`: `live-jutsu.mjs chidori`، `live-wrong-order.mjs`، `practice.mjs`، `sequence-rules.mjs`، `two-browsers.mjs`، `reconnect.mjs`، `responsive.mjs`، `music-flow.mjs`، `audio-timeline.mjs`. (و `make_fixtures.py` عنده كمان `--only` لو عايزة fixtures معينة بس. و `SHINOBI_WEB` بيغير عنوان التطبيق، افتراضي `http://localhost:5173`.)

تسجيل البيانات

```
.venv\Scripts\python.exe collect_data.py --person me --lighting day
.venv\Scripts\python.exe record_sequences.py --person me --lighting day
.venv\Scripts\python.exe tools\record_training_session.py --session 1 --plan
.venv\Scripts\python.exe tools\record_training_session.py --session 1
.venv\Scripts\python.exe tools\record_training_session.py --session 1 --resume
reports\training_sessions\<log>.json
.venv\Scripts\python.exe tools\record_test_session.py --person me --lighting day
```

ولا واحد من دول بيحط التسجيل في مجموعة. التسجيلات الجديدة بتفضل متجاهلة لحد الخطوة الجاية.

إضافة التسجيلات للمجموعات (بالـ id، والـ dry run الأول)

```
.venv\Scripts\python.exe tools\assign_recordings.py --log reports\training_sessions\<log>.json
.venv\Scripts\python.exe tools\assign_recordings.py --log reports\training_sessions\<log>.json --apply
.venv\Scripts\python.exe tools\make_test_set.py --since "YYYY-MM-DD HH:MM" --dry-run
.venv\Scripts\python.exe tools\make_test_set.py --since "YYYY-MM-DD HH:MM" --exclude-from
reports\test_sessions\<log>.json
```

ما تشغليش `python evaluate.py splits` على البيانات الحقيقية: بيدي مجموعات أوتوماتيك للتسجيلات اللي ملهاش. وخلي بالك إن `make_test_set.py` من غير `--dry-run` و `--no-eval` بيشغّل `evaluate.py run` على الـ test لوحده بعد التقسيم.

التدريب (بموافقتك بس: بيستبدل الموديل المتركب)

```
.venv\Scripts\python.exe train.py
.venv\Scripts\python.exe train.py --out C:\path\outside\repo\candidate.joblib   # train without
replacing the installed model
.venv\Scripts\python.exe tools\grouped_cv.py                                    # condition-grouped CV,
nothing installed
```

السطر الأول بيكتب فوق `models/gesture_model.joblib` و `models/training_report.txt` و `models/confusion_validation.png`. التاني بيكتب الموديل وتقريره وصورته في الفولدر اللي بره المشروع بس. والتالت ما بيركبش أي موديل، بس بيكتب فولدر تقرير جديد في `reports/`.

التقييم

```
.venv\Scripts\python.exe evaluate.py run                        # TEST set, exclusions applied ->
reports\<time>\
.venv\Scripts\python.exe evaluate.py run --sets validation      # numbers for tuning, NOT a final
result
.venv\Scripts\python.exe evaluate.py run --sessions ID1,ID2 --recognizers fused
.venv\Scripts\python.exe evaluate.py run --model none           # YOLOX + hands only
.venv\Scripts\python.exe tools\evaluate_pipeline.py loro
.venv\Scripts\python.exe tools\evaluate_pipeline.py negatives --cache C:\path\outside\repo\cache
```

تحذيرات مرتبطة بالأوامر دي (من الكود):

- `evaluate.py run` على الـ test النهارده بيدي رقم **تشخيصي** (الـ holdout القديم اتبص عليه). استخدميه بحساب.
- `tools\evaluate_pipeline.py loro` (و `sequences`) بشكلهم الحالي **هيشملوا الـ 22 تسجيل test** لأن `split_clips()` بياخد كل اللي ليه مجموعة. ما يتشغلوش لحد ما الأداة تتعدل تفلتر على train/validation بس. الأمر موجود في القايمة لأنه موجود في الكود، مش لأنه آمن دلوقتي.
- `negatives` محتاج `--cache` بره الـ repository (وإلا بيرفض).

الـ Assets

```
.venv\Scripts\python.exe tools\check_voices.py
.venv\Scripts\python.exe tools\build_bold_signs.py
.venv\Scripts\python.exe tools\build_gesture_references.py
```

الأول بيقرا بس. التاني والتالت **بيكتبوا فوق** صور في `gesture_references/` (الرسومات العريضة والكروت المرجعية)؛ شغليهم بس لو غيرتي الرسومات المصدر أو نصوص الـ catalog، وبعدها شغلي `pytest -q tests\test_catalog.py`.

الفروقات اللي لقيناها بين الكلام والكود

القايمة دي بتجمع كل حتة الكلام المكتوب (النسخة الإنجليزي من الشرح، أو الـ docstrings والرسايل جوه الكود نفسه، أو الـ README) ما كانش مطابق للكود أو للبيانات الحالية. في كل الحالات الشرح ده ماشي على الكود.

1. **نصيحة `python evaluate.py splits` جوه الكود:** الـ docstrings في `train.py` و `splits.py` و `evaluate.py`، ورسالة التحذير في `train.py` وفي `evaluate.py run` عن التسجيلات اللي ملهاش مجموعة، كلهم بيقولوا شغّل `python evaluate.py splits`. ده مخالف لقاعدة المشروع الحالية. وبسبب الـ snake المرفوض، التحذير ده بيظهر في كل تدريب وكل تقييم النهارده. الصح: تجاهله.
2. **`tools/evaluate_pipeline.py` بقى بيلمس الـ test:** `split_clips()` بيقول في الـ docstring "19 recordings"، والنصوص اللي بيكتبها في الـ JSON بتقول "there is no test set"، بس الكود بياخد كل تسجيل ليه مجموعة، وده النهارده 68 منهم 22 test. النسخة الإنجليزي من الشرح كانت بتحط `loro` في أوامر التشغيل العادية من غير تحذير.
3. **`scipy` و Pillow:** مش في `requirements.txt` (زي ما الإنجليزي قال)، بس اتأكدنا إنهم متركبين فعلاً في الـ `.venv` (scipy 1.18.1، Pillow 13.0.0) كـ dependencies غير مباشرة لـ scikit-learn و matplotlib.
4. **`README.md`** بيقول `classifier (MLP / RF)`، والمتركب فعلياً RF بس؛ والـ quick start بيشغّل `train.py` كأنه خطوة عادية، والنهارده ده بيستبدل الموديل المتركب ومحتاج موافقة.
5. **`updated` في `data/splits.json`** لسه `2026-10-31T02:03:19` رغم إن 49 تسجيل اتضافوا بالإيد بعدها، لأن `splits.assign` ما بيحدّثش الحقل ده.
6. **`tools/make_test_set.py`:** الـ docstring بيقول "Keys 1-9 choose a sign" عن `collect_data.py` (الحقيقة 0–9)؛ و `trained_on()` بيدور على `config.MODEL_PATH` اللي مش موجود وبيرجع للمسار الافتراضي (نفس الملف، فالنتيجة صح). وكمان الأداة بتنقل لـ test أي تسجيل بعد `--since` حتى لو كان في train، ما عدا اللي الموديل المتركب اتعلم منه؛ عشان كده الـ dry run ضروري.
7. **`tools/record_test_session.py`:** الـ docstring بيقول `make_test_set.py --exclude` للمرفوضين، والاختيار اللي بيقرا الـ log اسمه `--exclude-from`.
8. **`tools/check_voices.py`:** الإنجليزي قال "Other files in the folder are listed as not used"؛ الكود بيعرض بس الملفات اللي مش `.wav` (وما عدا `README.md`)، وأي `.wav` زيادة ما بيتعرضش.
9. **مكان تقرير التدريب:** `train.py` بيكتب `training_report.txt` و `confusion_validation.png` جنب ملف `--out`، مش في `models/` دايماً. ده بيخلي `--out` بره المشروع آمن تماماً (الإنجليزي ما وضحش ده).
10. **`train.py` النهارده هيختار بـ `cv`:** الإنجليزي شرح الحالات التلاتة، بس ما قالش إن مع الـ 43 train (كل حركة ≥ 3 تسجيلات) والـ validation الحالي، التشغيل الجاي هيدخل حالة `cv` بالتسجيل، ومكانش فيه option لاختيار موديل بعينه، لحد ما اتضاف `--model-type` في تجربة الموديلات.
11. **`tools/grouped_cv.py`** فيه تواريخ ثابتة (`2026-10-30`، `2026-11-01`) وأسماء كاميرات؛ أي تسجيل train من يوم تاني هيوقفه. ده مش غلط، بس معناه إن الأداة على مقاس البيانات الحالية.
12. **`tests/test_server.py`:** اختبارات الـ e2e بتاعة `clone_seal` بتستخدم تسجيل الموديل اتدرب عليه، والكود نفسه بيقول إنها بتثبت التوصيل مش التعرف.
13. **أدوات الـ assets** (`build_bold_signs.py` و `build_gesture_references.py`) بتكتب فوق ملفات في `gesture_references/`؛ الإنجليزي حطهم في الأوامر من غير ما يقول كده.
14. **إضافات اتأكدنا منها ومش في الإنجليزي:** عدد الاختبارات 224 في 12 ملف؛ log الـ 1 session فيه 24 `redone` و 2 `not_saved`؛ log الـ test session فيه `resumed` مرة و `rejected` واحد (الـ snake)؛ وفولدر `reports/` لحد وقت كتابة الشرح ده ما فيهوش تقرير للتجربة الجارية.

ملخص الجزء في سطور

- البيانات = تسجيلات 20 ثانية، كل واحد `.npz` (نقط خام) + `.mp4` (مراجعة). 77 ملف في 13 فولدر.
- التقسيم بالتسجيل الكامل في `data/splits.json` : 43 test، 22 validation، 3 train (5 مستبعدين)، 8 ignored، 1 مرفوض من غير مجموعة. والإضافة بالأدوات اليدوية بس.
- `train.py` بيدرب على الـ train بس، بـ mirror + jitter على التدريب بس، وبيحفظ مع الموديل قايمة اللي اتعلم منه.
- `evaluate.py run` بيقيس التلات recognizers بكود التطبيق، وبيرفض أي leakage، وبيطلع الأرقام بالاستبعادات ومن غيرها.
- الموديل الـ **MLP** = production على 43 تسجيل (اتركب 01-11-2026). الـ grouped CV: RF 0.856 / MLP 0.889 macro-F1. والرقم النهائي المستقل لسه محتاج test set جديد.

تحديث: نتايج تجربة الموديلات (01-11-2026)، والـ MLP اتركب بعدها

اتدرب موديلين تجريبيين على الـ 43 تسجيل train بس: `train.py --model-type random_forest` و `--model-type mlp`، واتحفظوا في `models/experiments/`. وقت التجربة الموديل الشغال في الموقع ما اتغيرش، وبعد مراجعة النتايج صاحبة المشروع وافقت والـ MLP اتركب. فيه backup للقديم في `SHINOBI-AI_backup_20261101_224739_before_model_experiment`. التلات موديلات اتقيموا بنفس `evaluate.py run -- sets test,validation`، والـ 5 تسجيلات المستبعدة برة الأرقام. التقرير الكامل في `reports/model_experiment_20261101/comparison.md`.

	الموديل القديم (16 RF، تسجيل)	RF تجريبي (43)	MLP تجريبي (43)
test: macro-F1 على الـ landmark model لوحده	0.712	0.820	0.845
test: macro-F1 على الـ قرار الموقع (fused)	0.515	0.622	0.622
Clone Seal في قرار الموقع (recall)	6%	100%	100%
jutsu اتطلقت غلط (false activations)	0	0	0
وقت الموديل للفريم الواحد (median)	ms 0.80	ms 0.84	ms 1.32

إزاي تقري الجدول ده:

- الـ 7 أختام قرارها من YOLOX، فأرقامها في قرار الموقع واحدة في التلات موديلات. المكسب الحقيقي في Clone Seal.
- الـ test القديم مش مستقل تماماً: اتسجل في نفس اليوم والأوضة زي تسجيلات الـ 1 session، فأرقام الموديلين الجداد عليه متفائلة شوية. ومفيهوش ولا تسجيل palm سليم.
- الموديلين الجداد أضعف من القديم في Tiger لوحده (تسجيل واحد بس)، بس ده ما بيأثرش على الموقع لإن YOLOX هو اللي بيقرر Tiger.
- التوصية: MLP (أحسن في الـ landmark macro-F1 وفي الـ grouped CV)، و RF لو الثبات أهم. **الـ MLP اتركب فعلاً يوم 01-11-2026 بموافقتك** (تفاصيل التركيب والاختبارات في `reports/model_experiment_20261101/deployment.md`).